Introduction au Deep Learning

Des Fondements Mathématiques aux Réseaux de Neurones

Cours de Mathématiques Appliquées

21 mars 2026

Table des matières

1 Introduction **7**

2 Le Neurone Unique : Fondements **7**
2.1 Un Problème Concret : Distinguer Gorilles et Chimpanzés 7
2.1.1 Le Contexte 7
2.1.2 Visualisation des Données 8
2.1.3 La Frontière de Décision Linéaire 8
2.2 Le Modèle Linéaire 9
2.3 La Fonction d'Activation Sigmoïde 10
2.4 Application à Notre Exemple : Des Distances aux Probabilités 11

3 La Fonction de Coût : Approche Probabiliste **13**
3.1 Modélisation Probabiliste : Loi de Bernoulli 13
3.2 La Vraisemblance 14
3.3 La Log-Vraisemblance 17
3.4 Fonction de Coût (Loss Function) 17
3.5 Extension : la régression et l'erreur quadratique 18

4 L'Optimisation : Descente de Gradient **20**
4.1 Principe Général 20
4.2 Calcul des Gradients : Règle de la Chaîne 22
4.2.1 Structure de Calcul 23
4.2.2 Calcul de $\frac{\partial L}{\partial w_1}$ 23
4.2.3 Gradients Complets 25

5 Vectorisation **25**
5.1 Motivation 25
5.2 Notations Matricielles 26
5.3 Forward Propagation Vectorisée 26
5.4 Fonction de Coût Vectorisée 27
5.5 Gradients Vectorisés 28

6 Extension : Une Couche de Neurones **30**
6.1 Architecture 30
6.2 Vectorisation de la Couche 30

7 Réseaux Multi-Couches (Deep Neural Networks) 32
7.1 Architecture . . . 32
7.2 Forward Propagation . . . 33
7.3 Backpropagation : Calcul Rigoureux des Gradients . . . 34
7.3.1 Principe Général . . . 34
7.3.2 Couche de Sortie . . . 34
7.3.3 Formules de Backpropagation Générales . . . 35
7.4 Algorithme Complet . . . 39
7.5 La Puissance des Réseaux Multicouches : Frontières Non-Linéaires . . . 40
7.5.1 Exemple : Classification de Cercles Concentriques . . . 40

8 Classification Multi-Classes : de la Sigmoïde à la Softmax 41
8.1 Exemple motivant : Gorille, Chimpanzé, Orang-outan ($K = 3$) . . . 41
8.2 De la Bernoulli à la catégorielle : le bon cadre probabiliste . . . 43
8.3 Softmax : définition, propriétés, interprétations . . . 43
8.4 Perte (NLL) : de la vraisemblance à la cross-entropie catégorielle . . . 47
8.5 Gradient clef : $dZ = A - Y$ (analogie parfaite avec le binaire) . . . 50
8.6 Géométrie des décisions : $K - 1$ hyperplans . . . 55
8.7 Identifiabilité, jauge et convexité (modèle linéaire) . . . 55
8.8 Implémentation stable et vectorisée . . . 56
8.9 Exemples numériques détaillés (mêmes notations que ci-dessus) . . . 56
8.10 Application au dataset des primates : visualisation des frontières . . . 57
8.11 Déséquilibre de classes, régularisation, lissage d'étiquettes . . . 58
8.11.1 Pondération de classes : gérer le déséquilibre . . . 59
8.11.2 Label smoothing : assouplir les cibles . . . 61
8.12 Température et calibration . . . 63
8.12.1 Application 1 : Calibration des probabilités . . . 64
8.12.2 Application 2 : Knowledge distillation (distillation de connaissances) 65
8.12.3 Application 3 : Génération de texte et sampling . . . 65
8.12.4 Application 4 : Exploration vs exploitation en RL . . . 66
8.13 Multi-classes *vs* multi-label : bien distinguer . . . 66
8.14 Algorithme complet (rappel) avec sortie softmax . . . 67

9 Fonctions d'activation cachées : exprimer et stabiliser le réseau 68
9.1 Le piège de la composition linéaire . . . 68
9.2 Critères mathématiques pour une activation cachée . . . 69
9.3 Panorama des activations classiques . . . 69
9.3.1 Sigmoïde et tanh : les activations historiques . . . 69
9.3.2 ReLU et variantes : fonctions par morceaux affines . . . 70
9.3.3 Activations lissées modernes . . . 71
9.4 Analyse pratique et diagnostic . . . 72

10 Initialisation des paramètres : conserver la variance et les gradients 72
10.1 Le problème de l'initialisation : constat empirique . . . 72
10.2 Modélisation probabiliste du signal . . . 73
10.3 Condition de variance constante (Forward) . . . 75
10.4 Propagation arrière : préserver les gradients . . . 75
10.5 Le compromis Glorot/Xavier . . . 76
10.6 Initialisation de He pour les activations ReLU . . . 77

10.7 Procédure pratique et vérification 77

11 Normalisation interne : Batch Normalization 79
11.1 Motivation : la dérive de la distribution interne 79
11.2 Définition formelle de Batch Normalization 81
11.2.1 Normalisation scalaire par neurone 81
11.3 Rôle des paramètres apprenables γ et β 82
11.4 Formulation vectorisée et vérification dimensionnelle 83
11.5 Rétropropagation complète à travers BatchNorm 84
11.5.1 Graphe de calcul 84
11.5.2 Gradient de β et γ 85
11.5.3 Gradient de $\hat{Z}$ 85
11.5.4 Gradient de σ^2 86
11.5.5 Gradient de μ 86
11.5.6 Gradient de Z (gradient final) 87
11.5.7 Récapitulatif et algorithme complet 89
11.6 Inférence : moyennes et variances mobiles 90
11.6.1 Le problème de l'inférence 90
11.6.2 Moyennes mobiles exponentielles 90
11.7 Placement dans l'architecture et bonnes pratiques 92
11.7.1 Avant ou après l'activation ? 92
11.7.2 Exigences sur la taille du mini-batch 92
11.7.3 Effet régularisant 93
11.8 Variantes de la normalisation 93
11.9 Illustration et stabilisation de l'entraînement 94
11.10 Résumé : le pipeline complet d'une couche avec BatchNorm 95

12 Optimiseurs avancés : inertie et adaptativité 95
12.1 Motivation : les limites de SGD dans les vallées étroites 95
12.2 Moyenne mobile exponentielle (EMA) 97
12.3 SGD avec momentum 99
12.3.1 Définition et analogie physique 99
12.3.2 Analyse de convergence sur une quadratique 100
12.4 Momentum de Nesterov 102
12.5 Pas adaptatifs : RMSProp 102
12.6 Adam : Adaptive Moment Estimation 103
12.6.1 Définition complète 103
12.6.2 Preuve complète de la correction de biais 104
12.6.3 Adam vs. SGD+Momentum : interprétation géométrique 105
12.6.4 AdamW : découplage du weight decay 105
12.7 Trajectoires de convergence 107
12.8 Comparaison et guide pratique 108

13 Régularisation : maîtriser la complexité du modèle 109
13.1 Préliminaire : séparation des données et protocole d'évaluation 109
13.2 Le problème du sur-apprentissage 110
13.2.1 Compromis biais-variance 110
13.2.2 Caractérisation formelle du sur-apprentissage 111
13.3 Régularisation L2 (Ridge / Weight Decay) 112

13.3.1 Définition formelle . . . 112
13.3.2 Calcul du gradient de la pénalité . . . 112
13.3.3 Règle de mise à jour : weight decay . . . 113
13.3.4 Interprétation géométrique : optimisation sous contrainte . . . 113
13.3.5 Pourquoi ne pas régulariser les biais ? . . . 114
13.3.6 Exemple numérique . . . 114
13.4 Régularisation L1 (Lasso) et sparsité . . . 115
13.4.1 Définition . . . 115
13.4.2 Sous-gradient et mise à jour . . . 115
13.4.3 Pourquoi L1 produit de la sparsité . . . 116
13.5 Dropout : régularisation par ensemble implicite . . . 117
13.5.1 Définition formelle . . . 117
13.5.2 Conservation de l'espérance . . . 118
13.5.3 Effet sur la variance . . . 118
13.5.4 Interprétation comme ensemble de sous-réseaux . . . 119
13.6 Early stopping : arrêt précoce . . . 120
13.6.1 Principe et définition . . . 120
13.6.2 Lien avec la régularisation L2 . . . 121
13.7 Augmentation de données et Mixup . . . 122
13.8 Synthèse et comparaison . . . 123

14 Réseaux Convolutifs (CNN) 124
14.1 Motivation : pourquoi les MLP ne suffisent pas pour les images . . . 124
14.1.1 L'explosion des paramètres . . . 124
14.1.2 La perte de structure spatiale . . . 125
14.2 L'opération de convolution discrète . . . 125
14.2.1 Définition . . . 125
14.2.2 Exemple numérique complet . . . 126
14.2.3 Formule de la taille de sortie . . . 126
14.2.4 Propriétés fondamentales . . . 127
14.3 D'une convolution à une couche convolutive . . . 128
14.3.1 Entrée multi-canaux et filtres multiples . . . 128
14.3.2 Comptage des paramètres . . . 128
14.4 Padding et Stride . . . 129
14.4.1 Padding . . . 129
14.4.2 Stride . . . 129
14.5 Pooling : réduction spatiale . . . 130
14.5.1 Rétropropagation à travers le max pooling . . . 130
14.6 Rétropropagation dans les couches convolutives . . . 131
14.6.1 Notations . . . 131
14.6.2 Gradient par rapport aux poids . . . 131
14.6.3 Gradient par rapport à l'entrée . . . 132
14.6.4 Gradient par rapport au biais . . . 132
14.7 Architecture type : de LeNet à la conception moderne . . . 133
14.7.1 LeNet-5 : l'architecture fondatrice . . . 133
14.7.2 Patterns de conception modernes . . . 133
14.8 Équivariance et invariance . . . 134
14.8.1 Théorème d'équivariance . . . 134

14.8.2 De l'équivariance à l'invariance 135

15 Modèles séquentiels : RNN et LSTM 136
15.1 Motivation : les limites des architectures fixes 136
15.2 Des mots aux vecteurs : tokenisation et embeddings 137
15.3 Le réseau récurrent simple (RNN) 138
15.3.1 Définition formelle 138
15.3.2 Dépliement temporel et partage de paramètres 138
15.3.3 Exemple numérique complet 139
15.4 Rétropropagation à travers le temps (BPTT) 140
15.4.1 Formulation du gradient 140
15.4.2 BPTT tronquée 140
15.5 Le problème des gradients qui disparaissent (et explosent) 141
15.5.1 Analyse formelle 141
15.5.2 Exemple numérique 142
15.5.3 Gradient clipping 142
15.6 LSTM : Long Short-Term Memory 143
15.6.1 Motivation : une autoroute pour le gradient 143
15.6.2 Définition formelle 143
15.6.3 Flux du gradient dans le LSTM 144
15.6.4 Exemple numérique 144
15.6.5 Nombre de paramètres 145
15.7 GRU : une simplification efficace 145
15.8 Architectures et applications 146
15.8.1 Configurations entrée-sortie 146
15.8.2 RNN bidirectionnels 147
15.8.3 RNN profonds (empilés) 147

16 Mécanisme d'Attention et Transformers 148
16.1 Motivation : les limites des RNN pour les longues séquences 148
16.2 Attention : le mécanisme fondamental 148
16.2.1 Requêtes, clés et valeurs 148
16.2.2 Exemple numérique complet 149
16.3 Scaled Dot-Product Attention 150
16.3.1 Formulation matricielle 150
16.3.2 Justification du facteur $\sqrt{d_k}$ 150
16.4 Multi-Head Attention 151
16.4.1 Coût de calcul : multi-têtes vs. tête unique 152
16.5 Encodage positionnel 153
16.5.1 Le problème de la permutation 153
16.5.2 Encodage positionnel sinusoïdal 154
16.5.3 Propriété fondamentale : expressibilité des positions relatives 154
16.5.4 Exemple numérique 155
16.6 Le bloc Transformer 155
16.6.1 Architecture d'un bloc encodeur 155
16.6.2 Connexions résiduelles 156
16.6.3 Layer Normalization 156
16.6.4 Décompte des paramètres 157

16.7 Masquage causal pour la génération 158
16.8 L'architecture Transformer complète 159
16.8.1 Encodeur-décodeur (Vaswani et al., 2017) 159
16.8.2 Variantes modernes 160
16.9 Analyse de complexité 160
16.9.1 Comparaison avec les RNN 161

17 Synthèse finale : du neurone unique au Transformer 162
17.1 Le parcours accompli 162
17.2 Les trois piliers du deep learning 164
17.2.1 Pilier 1 : Modélisation 164
17.2.2 Pilier 2 : Optimisation 164
17.2.3 Pilier 3 : Généralisation 165
17.3 Algorithme complet d'entraînement moderne 165
17.4 Diagnostic pratique 167
17.5 Récapitulatif des formules clés 168

1 Introduction

Ce cours a pour objectif de construire progressivement la compréhension mathématique du deep learning, en partant des concepts les plus simples pour arriver aux architectures modernes. Nous commencerons par un neurone unique effectuant de la classification binaire, puis généraliserons à des réseaux de taille arbitraire, avant d'aborder des architectures plus avancées (convolutions, RNN, transformers).

Intuition

Le deep learning repose sur trois piliers fondamentaux :

- **Modélisation** : construire une fonction paramétrique
- **Optimisation** : trouver les meilleurs paramètres via la descente de gradient
- **Généralisation** : s'assurer que le modèle fonctionne sur de nouvelles données

2 Le Neurone Unique : Fondements

2.1 Un Problème Concret : Distinguer Gorilles et Chimpanzés

2.1.1 Le Contexte

Imaginons que vous êtes paléontologue et que vous découvrez des ossements fossilisés de primates. Comment déterminer s'ils appartiennent à un gorille ou à un chimpanzé ? Une approche naturelle consiste à mesurer les dimensions caractéristiques des os et à chercher des patterns qui différencient les deux espèces.

Prenons deux mesures simples :

- x_1 : la longueur du radius (os de l'avant-bras) en cm
- x_2 : la longueur de l'humérus (os du bras) en cm

Nous disposons d'une base de données de spécimens dont l'espèce est connue :

- Gorilles : $y = 1$ (en bleu sur les graphiques)
- Chimpanzés : $y = 0$ (en orange sur les graphiques)

2.1.2 Visualisation des Données

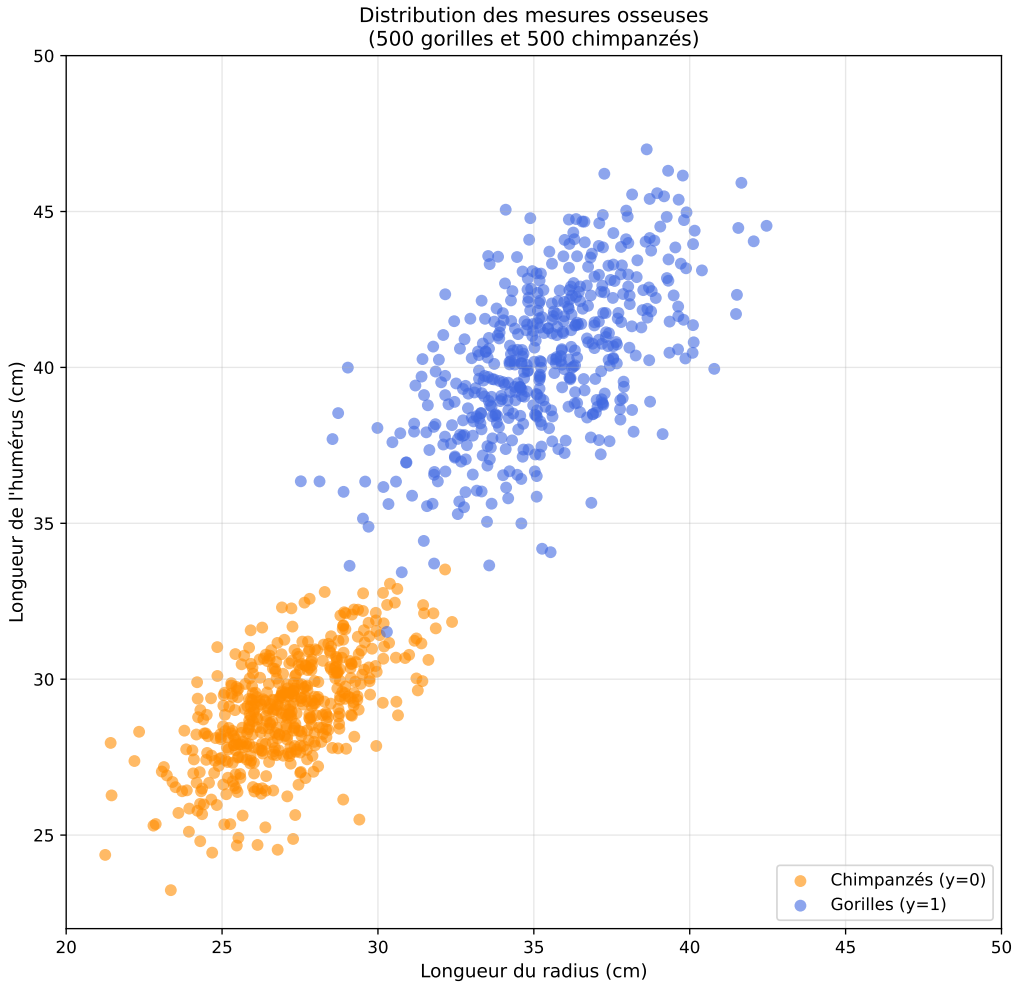

Figure 1 : Distribution des mesures osseuses pour 500 gorilles et 500 chimpanzés. On observe clairement deux nuages de points distincts : les chimpanzés ont des os plus courts que les gorilles.

En observant la Figure 1, une observation saute aux yeux : les deux populations forment des nuages de points bien séparés ! Les chimpanzés (orange) ont tendance à avoir des os plus courts (en bas à gauche) tandis que les gorilles (bleu) ont des os plus longs (en haut à droite).

Intuition

Cette séparation naturelle suggère qu'il existe une droite qui pourrait séparer efficacement les deux classes. L'idée du neurone unique est précisément de trouver cette frontière de manière automatique !

2.1.3 La Frontière de Décision Linéaire

Si nous traçons une droite qui sépare au mieux les deux populations, nous obtenons quelque chose comme ceci :

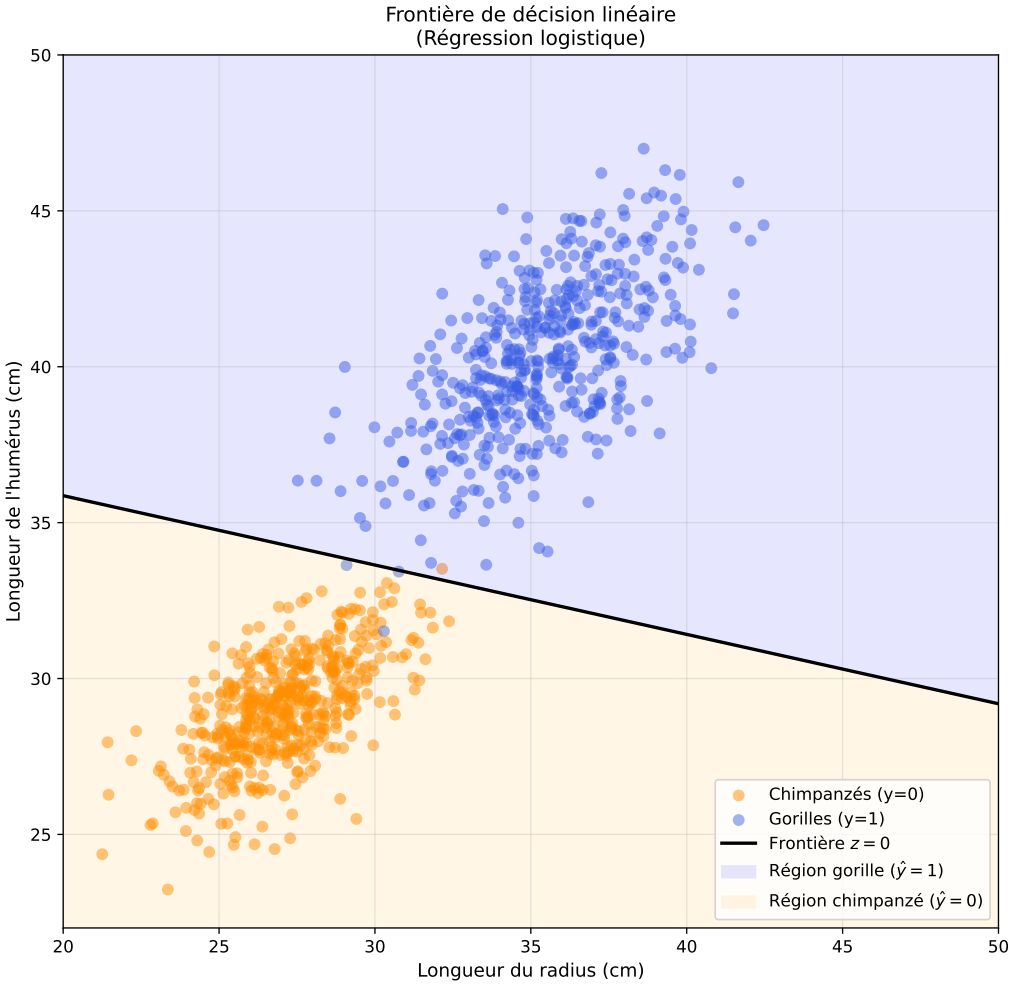

Figure 2 : Une droite sépare l'espace en deux régions : celle où nous prédisons "chimpanzé" (en bas) et celle où nous prédisons "gorille" (en haut).

Comment interpréter cette droite ?

— Points **au-dessus** de la droite $\Rightarrow$ prédiction "gorille" ($\hat{y} = 1$)

— Points **en-dessous** de la droite $\Rightarrow$ prédiction "chimpanzé" ($\hat{y} = 0$)

— Points **sur** la droite $\Rightarrow$ incertitude maximale

Remarque 2.1. Une simple règle binaire (au-dessus/en-dessous) ne capture pas notre degré de confiance dans la prédiction. Intuitivement, un point très éloigné de la frontière devrait nous donner plus de confiance qu'un point proche. Nous allons voir comment transformer cette intuition en probabilités grâce au modèle mathématique du neurone.

2.2 Le Modèle Linéaire

Formalisons maintenant mathématiquement ce que nous venons d'observer. Considérons le problème général de prédire une sortie y à partir de deux caractéristiques (x_1, x_2).

Définition 2.1 (Neurone linéaire)**.** Un neurone linéaire est une fonction qui combine linéairement ses entrées :

$$z(x_1, x_2) = w_1 x_1 + w_2 x_2 + b \tag{1}$$

où :

— $w_1, w_2 \in \mathbb{R}$ sont les **poids** (weights) du neurone
— $b \in \mathbb{R}$ est le **biais** (bias)
— $z \in \mathbb{R}$ est appelé la **pré-activation**

Intuition

Géométriquement, l'équation $z = 0$ (i.e., $w_1x_1 + w_2x_2 + b = 0$) définit une droite dans le plan (x_1, x_2) qui sépare l'espace en deux régions :

— Région où $z > 0$: un côté de la droite
— Région où $z < 0$: l'autre côté de la droite

Cette droite est appelée **frontière de décision**.

Le vecteur $\vec{w} = \begin{pmatrix} w_1 \\ w_2 \end{pmatrix}$ est perpendiculaire à la frontière de décision et indique la "direction" de la classe positive. Le biais b permet de translater cette droite sans changer son orientation.

Justification Mathématique

Pourquoi cette forme linéaire ?
L'équation générale d'une droite dans le plan (x_1, x_2) est : $ax_1 + bx_2 + c = 0$. En remplaçant $a \to w_1$, $b \to w_2$, et $c \to b$ (le biais), on retrouve exactement notre équation $w_1x_1 + w_2x_2 + b = 0$.
La valeur z représente alors la distance signée (à un facteur d'échelle près) d'un point (x_1, x_2) à cette droite :

— Si $z > 0$: le point est du côté positif (direction de $\vec{w}$)
— Si $z < 0$: le point est du côté négatif
— Si $z = 0$: le point est exactement sur la droite

2.3 La Fonction d'Activation Sigmoïde

Pour transformer $z \in \mathbb{R}$ (qui peut prendre n'importe quelle valeur) en une probabilité (valeur entre 0 et 1), nous appliquons une fonction non-linéaire.

Définition 2.2 (Fonction sigmoïde)**.** La fonction sigmoïde (ou logistique) est définie par :

$$\sigma : \mathbb{R} \to (0, 1), \quad \sigma(z) = \frac{1}{1 + e^{-z}} \tag{2}$$

Proposition 2.1 (Propriétés fondamentales de la sigmoïde)**.**

1. $\sigma(z) \in (0, 1)$ pour tout $z \in \mathbb{R}$ (strictement)
2. $\lim_{z\to+\infty} \sigma(z) = 1$ et $\lim_{z\to-\infty} \sigma(z) = 0$
3. $\sigma(0) = \frac{1}{2}$
4. $\sigma(-z) = 1 - \sigma(z)$ (symétrie par rapport au point $(0, 0.5)$)
5. $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ (dérivée très simple !)

Démonstrations. Propriété 1 : Pour tout $z \in \mathbb{R}$, on a $e^{-z} > 0$, donc $1 + e^{-z} > 1$, d'où :

$$0 < \frac{1}{1+e^{-z}} < 1 \tag{3}$$

Propriété 2 :

$$\lim_{z \to +\infty} \sigma(z) = \lim_{z \to +\infty} \frac{1}{1+e^{-z}} = \frac{1}{1+0} = 1 \tag{4}$$

$$\lim_{z \to -\infty} \sigma(z) = \lim_{z \to -\infty} \frac{1}{1+e^{-z}} = \frac{1}{1+\infty} = 0 \tag{5}$$

Propriété 3 :

$$\sigma(0) = \frac{1}{1+e^{0}} = \frac{1}{1+1} = \frac{1}{2} \tag{6}$$

Propriété 4 : Calculons $\sigma(-z)$:

$$\sigma(-z) = \frac{1}{1+e^{-(-z)}} = \frac{1}{1+e^{z}} \tag{7}$$

$$= \frac{1}{1+e^{z}} \cdot \frac{e^{-z}}{e^{-z}} = \frac{e^{-z}}{e^{-z}+1} \tag{8}$$

$$= \frac{1+e^{-z}-1}{1+e^{-z}} = 1 - \frac{1}{1+e^{-z}} = 1 - \sigma(z) \tag{9}$$

Propriété 5 : Calculons la dérivée en utilisant la règle du quotient.
Posons $\sigma(z) = \frac{1}{1+e^{-z}} = (1+e^{-z})^{-1}$:

$$\sigma'(z) = -1 \cdot (1+e^{-z})^{-2} \cdot (-e^{-z}) = \frac{e^{-z}}{(1+e^{-z})^2} \tag{10}$$

$$= \frac{1}{1+e^{-z}} \cdot \frac{e^{-z}}{1+e^{-z}} \tag{11}$$

$$= \frac{1}{1+e^{-z}} \cdot \frac{1+e^{-z}-1}{1+e^{-z}} \tag{12}$$

$$= \frac{1}{1+e^{-z}} \cdot \left(1 - \frac{1}{1+e^{-z}}\right) \tag{13}$$

$$= \sigma(z) \cdot (1 - \sigma(z)) \tag{14}$$

□

Remarque 2.2. La sortie du neurone est alors :

$$a = \sigma(z) = \sigma(w_1 x_1 + w_2 x_2 + b) \tag{15}$$

où $a \in (0,1)$ est appelée l'**activation** du neurone. On interprète a comme $P(y = 1|x_1, x_2)$, la probabilité que y vaille 1 étant données les caractéristiques (x_1, x_2).

2.4 Application à Notre Exemple : Des Distances aux Probabilités

Revenons maintenant à notre problème de classification gorilles/chimpanzés. Nous pouvons désormais comprendre comment le neurone transforme les mesures osseuses en probabilités.

Supposons que notre entraînement a trouvé les paramètres optimaux : w_1, w_2, et b. Pour un nouveau squelette avec des mesures (x_1, x_2), le neurone calcule :

$$z = w_1 x_1 + w_2 x_2 + b \quad \text{(distance signée à la frontière)} \tag{16}$$

$$P(\text{gorille}) = \sigma(z) = \frac{1}{1 + e^{-z}} \tag{17}$$

Intuition

La valeur z est proportionnelle à la **distance signée** du point à la frontière de décision :

- $z > 0$: le point est du côté "gorille" $\Rightarrow P(\text{gorille}) > 0.5$
- $z < 0$: le point est du côté "chimpanzé" $\Rightarrow P(\text{gorille}) < 0.5$
- $z = 0$: le point est exactement sur la frontière $\Rightarrow P(\text{gorille}) = 0.5$
- Plus $|z|$ est grand, plus la probabilité s'approche de 0 ou 1

Visualisons maintenant concrètement ce phénomène sur plusieurs points tests :
Analysons les prédictions du neurone sur la Figure 3 :

- Point rouge ($d_\perp = 5.00$ cm du côté gorille) :
  - z est très positif $\Rightarrow \sigma(z) \approx 1.00$
  - Conclusion : quasi-certitude que c'est un gorille
- Point vert ($d_\perp = 2.04$ cm du côté gorille) :
  - z est positif modéré $\Rightarrow \sigma(z) = 0.90$
  - Conclusion : forte confiance, probablement un gorille
- Point violet ($d_\perp = 2.01$ cm du côté chimpanzé) :
  - z est négatif modéré $\Rightarrow \sigma(z) = 0.10$
  - Conclusion : forte confiance, probablement un chimpanzé
- Point marron ($d_\perp = 4.99$ cm du côté chimpanzé) :
  - z est très négatif $\Rightarrow \sigma(z) \approx 0.00$
  - Conclusion : quasi-certitude que c'est un chimpanzé

Attention

Remarquez la symétrie : les points verts et violets sont presque à la même distance de la frontière (≈ 2 cm) mais de part et d'autre. Leurs probabilités sont complémentaires : 0.90 et 0.10. C'est une conséquence directe de la propriété $\sigma(-z) = 1 - \sigma(z)$ démontrée précédemment.

Exemple 2.1 (Calcul explicite)**.** Supposons qu'après entraînement, nous avons trouvé $w_1 = 0.5$, $w_2 = 0.8$, $b = -30$.

Pour un squelette avec radius $x_1 = 35$ cm et humérus $x_2 = 45$ cm :

$$z = 0.5 \times 35 + 0.8 \times 45 - 30 = 17.5 + 36 - 30 = 23.5 \tag{18}$$

$$P(\text{gorille}) = \sigma(23.5) = \frac{1}{1 + e^{-23.5}} \approx 0.999999999 \tag{19}$$

Le neurone est extrêmement confiant : c'est un gorille !

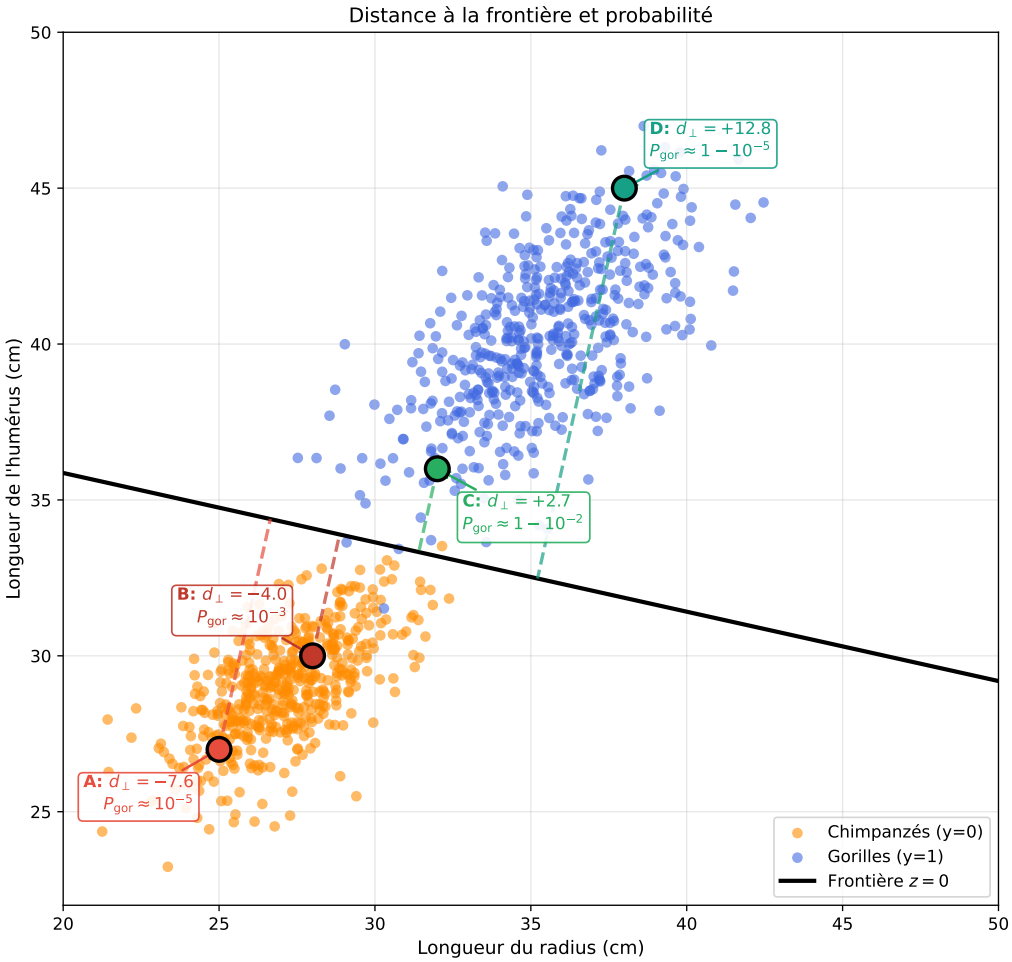

Figure 3 : La distance orthogonale $d_\perp$ d'un point à la frontière, via la sigmoïde, détermine la probabilité d'appartenir à la classe gorille. Plus on s'éloigne de la frontière, plus la probabilité tend vers 0 ou 1.

3 La Fonction de Coût : Approche Probabiliste

3.1 Modélisation Probabiliste : Loi de Bernoulli

Pour un problème de classification binaire, nous avons :

— $y \in \{0, 1\}$: la classe réelle (variable aléatoire)

— $a = \sigma(z) \in (0, 1)$: la probabilité prédite que $y = 1$

Définition 3.1 (Modèle probabiliste de Bernoulli)**.** Nous supposons que la variable aléatoire y suit une loi de Bernoulli de paramètre a :

$$P(Y = 1|x) = a = \sigma(w_1x_1 + w_2x_2 + b) \tag{20}$$

$$P(Y = 0|x) = 1 - a \tag{21}$$

Justification Mathématique

Pourquoi utiliser une loi de Bernoulli ?
La loi de Bernoulli est la distribution de probabilité naturelle pour modéliser une expérience binaire (succès/échec, 0/1). Elle est caractérisée par un unique paramètre $p \in [0, 1]$ (ici, a) qui représente la probabilité de succès.
Dans notre cas :

— "Succès" = l'animal est un gorille ($y = 1$)
— "Échec" = l'animal est un chimpanzé ($y = 0$)
— Le paramètre a dépend des observations $x = (x_1, x_2)$ via le neurone

Proposition 3.1 (Forme compacte de la loi de Bernoulli)**.** Les deux équations ci-dessus peuvent s'écrire sous une forme compacte unique :

$$P(Y = y|x) = a^y \cdot (1 - a)^{1-y}, \quad \text{pour } y \in \{0, 1\} \tag{22}$$

Vérification. Vérifions que cette formule compacte reproduit bien les deux cas :
Cas 1 : Si $y = 1$:

$$P(Y = 1|x) = a^1 \cdot (1 - a)^{1-1} = a \cdot (1 - a)^0 = a \cdot 1 = a \quad \checkmark \tag{23}$$

Cas 2 : Si $y = 0$:

$$P(Y = 0|x) = a^0 \cdot (1 - a)^{1-0} = 1 \cdot (1 - a)^1 = 1 - a \quad \checkmark \tag{24}$$

La formule compacte est donc correcte et équivalente aux deux équations séparées. □

Justification Mathématique

Pourquoi cette forme compacte est-elle utile ?
Cette notation permet d'écrire une formule unique qui fonctionne pour toutes les valeurs de y (0 ou 1). C'est particulièrement pratique pour :

1. Écrire la vraisemblance d'un ensemble de données (voir plus bas)
2. Calculer des dérivées
3. Implémenter le code (une seule formule au lieu de deux cas)

3.2 La Vraisemblance

Intuition

Nous avons maintenant un modèle probabiliste : pour chaque squelette $x^{(i)}$ avec des mesures $(x_1^{(i)}, x_2^{(i)})$, notre neurone prédit une probabilité $a_i = \sigma(w_1 x_1^{(i)} + w_2 x_2^{(i)} + b)$ que ce soit un gorille.
Mais comment choisir les "bons" paramètres (w_1, w_2, b) ? L'idée naturelle est de choisir ceux qui **rendent les données observées les plus probables**. C'est exactement ce que fait le principe du maximum de vraisemblance.

Revenons à notre exemple des primates. Supposons que dans notre base de données, nous avons observé 3 spécimens :

— Spécimen 1 : mesures $x^{(1)} = (35, 45)$ cm, classification réelle : gorille ($y^{(1)} = 1$)
— Spécimen 2 : mesures $x^{(2)} = (25, 30)$ cm, classification réelle : chimpanzé ($y^{(2)} = 0$)
— Spécimen 3 : mesures $x^{(3)} = (40, 48)$ cm, classification réelle : gorille ($y^{(3)} = 1$)

Pour un choix de paramètres (w_1, w_2, b) donné, notre neurone calcule pour chaque spécimen :

$$a_1 = \sigma(w_1 \cdot 35 + w_2 \cdot 45 + b) \quad \text{(probabilité prédite que le spécimen 1 soit un gorille)} \tag{25}$$

$$a_2 = \sigma(w_1 \cdot 25 + w_2 \cdot 30 + b) \quad \text{(probabilité prédite que le spécimen 2 soit un gorille)} \tag{26}$$

$$a_3 = \sigma(w_1 \cdot 40 + w_2 \cdot 48 + b) \quad \text{(probabilité prédite que le spécimen 3 soit un gorille)} \tag{27}$$

Question clé : Quelle est la probabilité que notre modèle attribue à la séquence d'observations *réellement observée* (gorille, chimpanzé, gorille) ?

En utilisant la loi de Bernoulli, la probabilité d'observer chaque classification réelle est :

$$P(y^{(1)} = 1|x^{(1)}) = a_1 \quad \text{(on a observé un gorille, le modèle lui donne probabilité } a_1\text{)} \tag{28}$$

$$P(y^{(2)} = 0|x^{(2)}) = 1 - a_2 \quad \text{(on a observé un chimpanzé, le modèle lui donne probabilité } 1 - a_2\text{)} \tag{29}$$

$$P(y^{(3)} = 1|x^{(3)}) = a_3 \quad \text{(on a observé un gorille, le modèle lui donne probabilité } a_3\text{)} \tag{30}$$

Si les exemples sont indépendants (l'espèce du spécimen 1 n'influence pas celle du spécimen 2), alors la probabilité conjointe est le produit :

$$\mathcal{L}(w_1, w_2, b) = a_1 \cdot (1 - a_2) \cdot a_3 \tag{31}$$

Cette quantité s'appelle la **vraisemblance** (likelihood).

Exemple 3.1 (Comparaison de deux modèles)**.** Supposons deux choix de paramètres différents :

Modèle A : (w_1^A, w_2^A, b^A) donne $a_1 = 0.95$, $a_2 = 0.30$, $a_3 = 0.92$

$$\mathcal{L}_A = 0.95 \times (1 - 0.30) \times 0.92 = 0.95 \times 0.70 \times 0.92 \approx 0.612 \tag{32}$$

Modèle B : (w_1^B, w_2^B, b^B) donne $a_1 = 0.60$, $a_2 = 0.80$, $a_3 = 0.55$

$$\mathcal{L}_B = 0.60 \times (1 - 0.80) \times 0.55 = 0.60 \times 0.20 \times 0.55 \approx 0.066 \tag{33}$$

Le modèle A a une vraisemblance beaucoup plus élevée : il attribue une probabilité bien plus forte aux classifications réellement observées ! Intuitivement, le modèle A "explique mieux" les données.

Justification Mathématique

Pourquoi maximiser la vraisemblance est-il sensé ?
La vraisemblance $\mathcal{L}(w_1, w_2, b)$ mesure à quel point notre modèle (avec ces paramètres) considère les données observées comme probables :

- Si $\mathcal{L}$ est proche de 1 : le modèle attribue une forte probabilité aux observations réelles $\Rightarrow$ les paramètres sont cohérents avec les données
- Si $\mathcal{L}$ est proche de 0 : le modèle considère les observations réelles comme très improbables $\Rightarrow$ les paramètres sont inadéquats

Le **principe du maximum de vraisemblance** consiste à choisir les paramètres qui maximisent $\mathcal{L}$, c'est-à-dire ceux qui rendent les données observées *les plus probables*.

Disposons d'un ensemble de données d'entraînement : $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^m$ où m est le nombre d'exemples.

Définition 3.2 (Vraisemblance)**.** En supposant les exemples i.i.d., la vraisemblance des données pour (w_1, w_2, b) est

$$\mathcal{L}(w_1, w_2, b) = \prod_{i=1}^{m} P\big(y^{(i)} \mid x^{(i)}; w_1, w_2, b\big)\,, \tag{34}$$

où $a_i = \sigma(w_1 x_1^{(i)} + w_2 x_2^{(i)} + b)$.

Justification Mathématique

Pourquoi le produit ?
L'hypothèse d'indépendance des exemples signifie que la probabilité conjointe se factorise :

$$P(y^{(1)}, y^{(2)}, \ldots, y^{(m)} | x^{(1)}, \ldots, x^{(m)}) = \prod_{i=1}^{m} P(y^{(i)} | x^{(i)}) \tag{35}$$

C'est une hypothèse standard en machine learning : connaître le résultat d'un exemple ne nous donne pas d'information sur les autres exemples.

En utilisant la forme compacte de Bernoulli :

$$\mathcal{L}(w_1, w_2, b) = \prod_{i=1}^{m} a_i^{y^{(i)}} \cdot (1 - a_i)^{1-y^{(i)}} \tag{36}$$

Attention

Problème numérique : Le produit de nombreuses probabilités (valeurs dans $[0, 1]$) tend vers 0 très rapidement.
Exemple : Si $m = 1000$ et que chaque probabilité vaut environ 0.5, alors :

$$\mathcal{L} \approx 0.5^{1000} \approx 10^{-301} \tag{37}$$

Cette valeur est bien en-dessous de la précision machine (underflow) !
Solution : Passer au logarithme, qui transforme les produits en sommes.

3.3 La Log-Vraisemblance

Définition 3.3 (Log-vraisemblance)**.** La log-vraisemblance est le logarithme népérien de la vraisemblance :

$$\ell(w_1, w_2, b) = \log \mathcal{L}(w_1, w_2, b) \tag{38}$$

Proposition 3.2 (Expression de la log-vraisemblance)**.**

$$\begin{aligned}
\ell(w_1, w_2, b) &= \log \left(\prod_{i=1}^{m} a_i^{y^{(i)}} \cdot (1 - a_i)^{1-y^{(i)}} \right) && (39)\\
&= \sum_{i=1}^{m} \log \left(a_i^{y^{(i)}} \cdot (1 - a_i)^{1-y^{(i)}} \right) && (40)\\
&= \sum_{i=1}^{m} \left[\log(a_i^{y^{(i)}}) + \log((1 - a_i)^{1-y^{(i)}}) \right] && (41)\\
&= \sum_{i=1}^{m} \left[y^{(i)} \log(a_i) + (1 - y^{(i)}) \log(1 - a_i) \right] && (42)
\end{aligned}$$

Justification Mathématique

Passage ligne par ligne :
Ligne 1 $\to$ 2 : Propriété du logarithme : $\log(\prod x_i) = \sum \log(x_i)$
Ligne 2 $\to$ 3 : Propriété du logarithme : $\log(ab) = \log(a) + \log(b)$
Ligne 3 $\to$ 4 : Propriété du logarithme : $\log(x^k) = k \log(x)$
Ainsi : $\log(a_i^{y^{(i)}}) = y^{(i)} \log(a_i)$ et $\log((1 - a_i)^{1-y^{(i)}}) = (1 - y^{(i)}) \log(1 - a_i)$

Remarque 3.1. Le logarithme est une fonction strictement croissante. Par conséquent :

$$\arg\max_{w_1, w_2, b} \mathcal{L}(w_1, w_2, b) = \arg\max_{w_1, w_2, b} \ell(w_1, w_2, b) \tag{43}$$

Maximiser la vraisemblance est équivalent à maximiser la log-vraisemblance. On préfère travailler avec ℓ pour éviter les problèmes numériques.

3.4 Fonction de Coût (Loss Function)

Définition 3.4 (Binary Cross-Entropy Loss)**.** En apprentissage machine, on préfère minimiser une fonction de coût plutôt que maximiser une vraisemblance. On définit donc :

$$L(w_1, w_2, b) = -\frac{1}{m} \ell(w_1, w_2, b) = -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \log(a_i) + (1 - y^{(i)}) \log(1 - a_i) \right] \tag{44}$$

Cette fonction est appelée **binary cross-entropy loss** (ou log loss).

Justification Mathématique

Explications des transformations :

1. Le signe négatif $(-)$: Transforme un problème de maximisation en minimisation :
$$\arg\max_w \ell(w) = \arg\min_w(-\ell(w)) \tag{45}$$
2. La division par m $(\frac{1}{m})$: Donne une moyenne plutôt qu'une somme. Avantages :
   — Indépendance par rapport à la taille du dataset (permet de comparer des modèles entraînés sur des datasets de tailles différentes)
   — Stabilité numérique (évite que L devienne trop grand avec beaucoup de données)
   — Interprétation : coût moyen par exemple

Remarque importante : La division par m ne change pas l'emplacement du minimum :

$$\arg\min_w \sum_{i=1}^{m} f_i(w) = \arg\min_w \frac{1}{m}\sum_{i=1}^{m} f_i(w) \tag{46}$$

car $\frac{1}{m}$ est une constante positive.

Intuition

Interprétation de la binary cross-entropy :
Pour un exemple donné, le coût $-[y\log(a) + (1-y)\log(1-a)]$ mesure la "surprise" ou l'"erreur" du modèle :

— Si $y = 1$ (gorille) et $a \approx 1$: $-\log(a) \approx 0$ (faible coût, bonne prédiction)
— Si $y = 1$ et $a \approx 0$: $-\log(a) \to +\infty$ (coût très élevé, mauvaise prédiction)
— Si $y = 0$ (chimpanzé) et $a \approx 0$: $-\log(1-a) \approx 0$ (faible coût)
— Si $y = 0$ et $a \approx 1$: $-\log(1-a) \to +\infty$ (coût très élevé)

Le modèle est donc fortement pénalisé quand il est confiant mais se trompe !

3.5 Extension : la régression et l'erreur quadratique

Intuition

Jusqu'ici, nous avons étudié la **classification** : prédire une étiquette discrète $y \in \{0, 1\}$ (gorille ou chimpanzé). Mais les réseaux de neurones peuvent aussi prédire des **valeurs continues** : un prix, une température, une distance, etc. On parle alors de **régression**.
La question est : quelle fonction de coût utiliser lorsque la sortie $y \in \mathbb{R}$ est continue ?

Définition 3.5 (Neurone en régression)**.** En régression, la fonction d'activation de sortie est l'**identité** : on n'applique pas de sigmoïde. La prédiction du neurone est directement :

$$\hat{y} = z = w^T x + b \tag{47}$$

où $\hat{y} \in \mathbb{R}$ est la valeur prédite.

Justification Mathématique

Pourquoi ne pas utiliser la sigmoïde ?
La sigmoïde compresse la sortie dans l'intervalle $(0, 1)$, ce qui est adapté pour une probabilité. En régression, on veut pouvoir prédire n'importe quelle valeur réelle (par exemple, un prix de 250 € ou une température de $-5\,°\mathrm{C}$). L'activation identité laisse la sortie libre de prendre toute valeur dans $\mathbb{R}$.

Définition 3.6 (Erreur quadratique moyenne (MSE))**.** Pour un ensemble de données $\{(x^{(i)}, y^{(i)})\}_{i=1}^{m}$ avec $y^{(i)} \in \mathbb{R}$, la fonction de coût **Mean Squared Error** (MSE) est :

$$L(w, b) = \frac{1}{m} \sum_{i=1}^{m} \left(\hat{y}^{(i)} - y^{(i)}\right)^2 \tag{48}$$

où $\hat{y}^{(i)} = w^T x^{(i)} + b$ est la prédiction du modèle pour l'exemple i.

Proposition 3.3 (Gradient de la MSE)**.** Le gradient de la MSE par rapport aux poids est :

$$\frac{\partial L}{\partial w} = \frac{2}{m} \sum_{i=1}^{m} \left(\hat{y}^{(i)} - y^{(i)}\right) x^{(i)} \tag{49}$$

Ce gradient a la même structure que celui de la cross-entropy : c'est une somme de termes (erreur) $\times$ (entrée), mais sans la sigmoïde.

Justification Mathématique

Puisque $\hat{y}^{(i)} = w^T x^{(i)} + b$, on a $\frac{\partial \hat{y}^{(i)}}{\partial w} = x^{(i)}$. Par la règle de dérivation en chaîne :

$$\frac{\partial}{\partial w} \left(\hat{y}^{(i)} - y^{(i)}\right)^2 = 2\left(\hat{y}^{(i)} - y^{(i)}\right) \cdot x^{(i)} \tag{50}$$

Il suffit ensuite de sommer sur les m exemples et de diviser par m.

Remarque 3.2. Les deux cadres (classification et régression) partagent une structure élégante :

Tâche	**Classification binaire**	**Régression**
Activation	Sigmoïde $\sigma(z)$	Identité $\hat{y} = z$
Coût	Binary cross-entropy	MSE
Gradient ($\propto$)	$\sum(a^{(i)} - y^{(i)})x^{(i)}$	$\sum(\hat{y}^{(i)} - y^{(i)})x^{(i)}$

Dans les deux cas, le gradient a la forme **erreur** $\times$ **entrée** : la mise à jour des poids est proportionnelle à l'écart entre la prédiction et la réalité, pondéré par la valeur de l'entrée.

Attention

Le choix de la fonction de coût est dicté par la **nature de la sortie** :

- **Classes discrètes** ($y \in \{0, 1\}$) $\longrightarrow$ cross-entropy (fondée sur le maximum de vraisemblance de Bernoulli)
- **Valeurs continues** ($y \in \mathbb{R}$) $\longrightarrow$ MSE, ou des variantes comme la MAE (Mean Absolute Error) ou la perte de Huber (robuste aux valeurs aberrantes)

Utiliser une MSE pour de la classification, ou une cross-entropy pour de la régression, mènerait à un entraînement inefficace voire instable.

4 L'Optimisation : Descente de Gradient

4.1 Principe Général

Notre objectif est de trouver les paramètres (w_1, w_2, b) qui minimisent la fonction de coût L :

$$(w_1^*, w_2^*, b^*) = \arg\min_{w_1, w_2, b} L(w_1, w_2, b) \tag{51}$$

Définition 4.1 (Gradient)**.** Le gradient de L par rapport aux paramètres est le vecteur des dérivées partielles :

$$\nabla L = \begin{pmatrix} \frac{\partial L}{\partial w_1} \\ \frac{\partial L}{\partial w_2} \\ \frac{\partial L}{\partial b} \end{pmatrix} \in \mathbb{R}^3 \tag{52}$$

Théorème 4.1 (Direction de plus forte descente)**.** Le gradient ∇L pointe dans la direction de plus forte croissance de la fonction L. Par conséquent, $-\nabla L$ pointe dans la direction de plus forte décroissance.

Justification Mathématique

Ce théorème provient du développement de Taylor au premier ordre :

$$L(w + \epsilon v) \approx L(w) + \epsilon \nabla L(w)^T v \tag{53}$$

Pour minimiser L, on veut que $\nabla L(w)^T v < 0$. Le choix $v = -\nabla L(w)$ donne :

$$\nabla L(w)^T(-\nabla L(w)) = -\|\nabla L(w)\|^2 < 0 \tag{54}$$

C'est la direction qui minimise le plus rapidement L (parmi toutes les directions de norme unitaire).

Attention

La descente de gradient garantit de diminuer la fonction de coût à chaque itération (si α est assez petit, voir la section suivante), mais cela ne garantit pas d'atteindre le minimum global ! En effet, l'algorithme pourrait se bloquer dans un minimum local (un point où le gradient est nul, mais qui n'est pas le point le plus bas possible).
Qu'est-ce qu'une fonction convexe ?
Une fonction $f : \mathbb{R}^n \to \mathbb{R}$ est convexe si pour tous points x, y et tout $\lambda \in [0, 1]$:

$$f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y) \tag{55}$$

Une fonction convexe a la forme d'une "cuvette" sans vallées secondaires (par exemple, $f(x) = x^2$, $x \in \mathbb{R}$).
Propriété fondamentale : Pour une fonction convexe différentiable, *tout minimum local est un minimum global.*
Notre cas : classification binaire avec sigmoïde
Pour notre neurone unique avec binary cross-entropy, la fonction de coût

$$L(w_1, w_2, b) = -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \log(a_i) + (1 - y^{(i)}) \log(1 - a_i) \right] \tag{56}$$

où $a_i = \sigma(w_1 x_1^{(i)} + w_2 x_2^{(i)} + b)$, est **convexe** en (w_1, w_2, b) !
Pourquoi est-ce convexe ?

- La composition $z \mapsto -\log(\sigma(z))$ est convexe en z
- z est linéaire en (w_1, w_2, b)
- La composition de fonctions convexes avec des fonctions affines préserve la convexité

Attention pour les réseaux profonds : Dès que nous empilerons plusieurs couches de neurones, la fonction de coût deviendra **non-convexe**. Il y aura alors potentiellement de nombreux minima locaux. En pratique, les minima locaux trouvés par la descente de gradient sont souvent de très bonne qualité, et l'architecture des réseaux profonds permet de surmonter ce problème théorique.

Définition 4.2 (Algorithme de descente de gradient)**.** L'algorithme de descente de gradient met à jour itérativement les paramètres dans la direction opposée au gradient :

$$w_1^{(t+1)} = w_1^{(t)} - \alpha \left. \frac{\partial L}{\partial w_1} \right|_{w^{(t)}} \tag{57}$$

$$w_2^{(t+1)} = w_2^{(t)} - \alpha \left. \frac{\partial L}{\partial w_2} \right|_{w^{(t)}} \tag{58}$$

$$b^{(t+1)} = b^{(t)} - \alpha \left. \frac{\partial L}{\partial b} \right|_{w^{(t)}} \tag{59}$$

où :

- t est l'itération courante
- $\alpha > 0$ est le **taux d'apprentissage** (learning rate)
- Les dérivées sont évaluées aux paramètres courants $w^{(t)} = (w_1^{(t)}, w_2^{(t)}, b^{(t)})$

Attention

Choix du learning rate α :

- Trop grand ($\alpha \gg 1$) : divergence, oscillations
- Trop petit ($\alpha \ll 1$) : convergence très lente
- Typiquement : $\alpha \in [10^{-4}, 10^{-1}]$ selon le problème

4.2 Calcul des Gradients : Règle de la Chaîne

Calculons maintenant explicitement les gradients. Nous devons appliquer soigneusement la règle de la chaîne (chain rule) du calcul différentiel.

Intuition

La règle de la chaîne est un outil fondamental du calcul différentiel qui permet de calculer la dérivée d'une fonction composée.
Idée intuitive : Si une grandeur y dépend de u, qui lui-même dépend de x, alors le taux de variation de y par rapport à x est le produit des taux de variations intermédiaires.
Par exemple, si la température augmente rapidement (variation de y par rapport à u) et que l'altitude augmente lentement (variation de u par rapport à x), alors la température augmente modérément en fonction de l'altitude (variation de y par rapport à x).

Définition 4.3 (Règle de la chaîne - cas univarié)**.** Si $y = f(u)$ et $u = g(x)$, alors la dérivée de la composition $y = f(g(x))$ par rapport à x est :

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx} \tag{60}$$

Exemple 4.1 (Illustration simple)**.** Soit $y = u^2$ avec $u = 3x + 1$. Calculons $\frac{dy}{dx}$:

$$\frac{dy}{du} = 2u \tag{61}$$

$$\frac{du}{dx} = 3 \tag{62}$$

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx} = 2u \cdot 3 = 6u = 6(3x + 1) = 18x + 6 \tag{63}$$

Vérification directe : $y = (3x + 1)^2 = 9x^2 + 6x + 1$, donc $\frac{dy}{dx} = 18x + 6$ ✓

Définition 4.4 (Règle de la chaîne - cas multivarié)**.** Si L dépend de variables intermédiaires $a_1, \ldots, a_m$, qui elles-mêmes dépendent d'une variable w, alors :

$$\frac{\partial L}{\partial w} = \sum_{i=1}^{m} \frac{\partial L}{\partial a_i} \cdot \frac{\partial a_i}{\partial w} \tag{64}$$

On somme les contributions de toutes les variables intermédiaires.

Justification Mathématique

Pourquoi cette somme ?
Si L dépend de plusieurs variables $a_1, \ldots, a_m$, et que chaque a_i dépend de w, alors modifier w affecte L à travers *tous* les a_i simultanément. La variation totale de L est donc la somme des variations induites par chaque chemin.
Par exemple, dans notre neurone, la fonction de coût

$$L = -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \log(a^{(i)}) + (1 - y^{(i)}) \log(1 - a^{(i)}) \right] \tag{65}$$

dépend de *toutes* les activations $a^{(1)}, \ldots, a^{(m)}$. Chaque activation $a^{(i)}$ dépend du poids w_1 (via $z^{(i)} = w_1 x_1^{(i)} + w_2 x_2^{(i)} + b$). Donc modifier w_1 affecte L à travers les m chemins :

$$w_1 \to z^{(1)} \to a^{(1)} \to L, \quad w_1 \to z^{(2)} \to a^{(2)} \to L, \quad \ldots, \quad w_1 \to z^{(m)} \to a^{(m)} \to L \tag{66}$$

La dérivée totale $\frac{\partial L}{\partial w_1}$ est donc la **somme** des contributions de chaque exemple.

4.2.1 Structure de Calcul

Notre fonction de coût a la structure compositionnelle suivante :

$$w_1, w_2, b \to z \to a \to L \tag{67}$$

Plus précisément :

$$z^{(i)} = w_1 x_1^{(i)} + w_2 x_2^{(i)} + b \tag{68}$$

$$a^{(i)} = \sigma(z^{(i)}) = \frac{1}{1 + e^{-z^{(i)}}} \tag{69}$$

$$L = -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \log(a^{(i)}) + (1 - y^{(i)}) \log(1 - a^{(i)}) \right] \tag{70}$$

4.2.2 Calcul de $\frac{\partial L}{\partial w_1}$

Proposition 4.1 (Règle de la chaîne pour $\frac{\partial L}{\partial w_1}$)**.** Pour calculer $\frac{\partial L}{\partial w_1}$, nous devons passer par les variables intermédiaires. Pour chaque exemple i :

$$\frac{\partial L}{\partial w_1} = \sum_{i=1}^{m} \frac{\partial L}{\partial a^{(i)}} \cdot \frac{\partial a^{(i)}}{\partial z^{(i)}} \cdot \frac{\partial z^{(i)}}{\partial w_1} \tag{71}$$

Justification Mathématique

Cette formule découle directement de la règle de la chaîne multivariée. Puisque L dépend de toutes les activations $a^{(1)}, \ldots, a^{(m)}$ (via la somme), et que chaque $a^{(i)}$ dépend de son propre $z^{(i)}$, qui lui-même dépend de w_1, nous devons sommer les contributions de tous les exemples.

Calculons maintenant chaque terme de la chaîne :

Étape 1 : Calcul de $\frac{\partial L}{\partial a^{(i)}}$
Rappel : $L = -\frac{1}{m}\sum_{j=1}^{m}\left[y^{(j)}\log(a^{(j)}) + (1-y^{(j)})\log(1-a^{(j)})\right]$
En dérivant par rapport à $a^{(i)}$ (un seul terme de la somme dépend de $a^{(i)}$) :

$$\frac{\partial L}{\partial a^{(i)}} = -\frac{1}{m}\frac{\partial}{\partial a^{(i)}}\left[y^{(i)}\log(a^{(i)}) + (1-y^{(i)})\log(1-a^{(i)})\right] \tag{72}$$

$$= -\frac{1}{m}\left[y^{(i)}\cdot\frac{1}{a^{(i)}} + (1-y^{(i)})\cdot\frac{1}{1-a^{(i)}}\cdot(-1)\right] \tag{73}$$

$$= -\frac{1}{m}\left[\frac{y^{(i)}}{a^{(i)}} - \frac{1-y^{(i)}}{1-a^{(i)}}\right] \tag{74}$$

En mettant au même dénominateur :

$$\frac{\partial L}{\partial a^{(i)}} = -\frac{1}{m}\left[\frac{y^{(i)}(1-a^{(i)}) - (1-y^{(i)})a^{(i)}}{a^{(i)}(1-a^{(i)})}\right] \tag{75}$$

$$= -\frac{1}{m}\left[\frac{y^{(i)} - y^{(i)}a^{(i)} - a^{(i)} + y^{(i)}a^{(i)}}{a^{(i)}(1-a^{(i)})}\right] \tag{76}$$

$$= -\frac{1}{m}\left[\frac{y^{(i)} - a^{(i)}}{a^{(i)}(1-a^{(i)})}\right] \tag{77}$$

Étape 2 : Calcul de $\frac{\partial a^{(i)}}{\partial z^{(i)}}$
Nous avons démontré précédemment que $\sigma'(z) = \sigma(z)(1-\sigma(z))$, donc :

$$\frac{\partial a^{(i)}}{\partial z^{(i)}} = \frac{\partial \sigma(z^{(i)})}{\partial z^{(i)}} = \sigma(z^{(i)})(1-\sigma(z^{(i)})) = a^{(i)}(1-a^{(i)}) \tag{78}$$

Étape 3 : Calcul de $\frac{\partial z^{(i)}}{\partial w_1}$
Rappel : $z^{(i)} = w_1 x_1^{(i)} + w_2 x_2^{(i)} + b$

$$\frac{\partial z^{(i)}}{\partial w_1} = \frac{\partial}{\partial w_1}(w_1 x_1^{(i)} + w_2 x_2^{(i)} + b) = x_1^{(i)} \tag{79}$$

Étape 4 : Combinaison (règle de la chaîne)

$$\frac{\partial L}{\partial w_1} = \sum_{i=1}^{m}\frac{\partial L}{\partial a^{(i)}}\cdot\frac{\partial a^{(i)}}{\partial z^{(i)}}\cdot\frac{\partial z^{(i)}}{\partial w_1} \tag{80}$$

$$= \sum_{i=1}^{m}\left[-\frac{1}{m}\cdot\frac{y^{(i)} - a^{(i)}}{a^{(i)}(1-a^{(i)})}\right]\cdot\left[a^{(i)}(1-a^{(i)})\right]\cdot x_1^{(i)} \tag{81}$$

$$= \sum_{i=1}^{m}\left[-\frac{1}{m}(y^{(i)} - a^{(i)})\right]\cdot x_1^{(i)} \tag{82}$$

$$= -\frac{1}{m}\sum_{i=1}^{m}(y^{(i)} - a^{(i)})x_1^{(i)} \tag{83}$$

$$= \frac{1}{m}\sum_{i=1}^{m}(a^{(i)} - y^{(i)})x_1^{(i)} \tag{84}$$

Attention

Simplification cruciale !
Les termes $a^{(i)}(1 - a^{(i)})$ au numérateur et au dénominateur se simplifient. C'est une propriété remarquable de la combinaison sigmoïde + binary cross-entropy : le gradient a une forme très simple !

4.2.3 Gradients Complets

Par des calculs similaires (en changeant simplement la dernière étape), on obtient :

Proposition 4.2 (Gradients pour le neurone unique)**.**

$$\frac{\partial L}{\partial w_1} = \frac{1}{m}\sum_{i=1}^{m}(a^{(i)} - y^{(i)})x_1^{(i)} \tag{85}$$

$$\frac{\partial L}{\partial w_2} = \frac{1}{m}\sum_{i=1}^{m}(a^{(i)} - y^{(i)})x_2^{(i)} \tag{86}$$

$$\frac{\partial L}{\partial b} = \frac{1}{m}\sum_{i=1}^{m}(a^{(i)} - y^{(i)}) \tag{87}$$

Justification Mathématique

Pour $\frac{\partial L}{\partial w_2}$:

$$\frac{\partial z^{(i)}}{\partial w_2} = x_2^{(i)} \tag{88}$$

Le reste du calcul est identique.
Pour $\frac{\partial L}{\partial b}$:

$$\frac{\partial z^{(i)}}{\partial b} = 1 \tag{89}$$

Le reste du calcul est identique, et on obtient simplement $(a^{(i)} - y^{(i)})$ sans multiplication par une caractéristique.

Remarque 4.1. La forme $(a^{(i)} - y^{(i)})$ est extrêmement élégante : c'est simplement l'erreur de prédiction ! Le gradient est proportionnel à l'erreur, pondéré par la valeur de la caractéristique correspondante.

Intuition :

- Si $a^{(i)} > y^{(i)}$: on a surestimé, on doit diminuer z (donc diminuer les poids si $x > 0$)
- Si $a^{(i)} < y^{(i)}$: on a sous-estimé, on doit augmenter z (donc augmenter les poids si $x > 0$)

5 Vectorisation

5.1 Motivation

En pratique, nous travaillons avec des milliers (voire millions) d'exemples et de caractéristiques. L'implémentation avec des boucles for serait extrêmement lente. La vectorisation permet d'exploiter les bibliothèques d'algèbre linéaire optimisées qui utilisent :

— Le parallélisme CPU
— Le parallélisme GPU
— Des optimisations de cache mémoire

Cela permet également de simplifier grandement les notations mathématiques (voir section ci-dessous) et le code source.

5.2 Notations Matricielles

Définition 5.1 (Matrice des données)**.** On organise les données en une matrice $X \in \mathbb{R}^{n \times m}$ où :

— n = nombre de caractéristiques (features)
— m = nombre d'exemples
— La colonne i contient l'exemple i : $x^{(i)} = \begin{pmatrix} x_1^{(i)} \\ \vdots \\ x_n^{(i)} \end{pmatrix}$

$$X = \begin{pmatrix} | & | & & | \\ x^{(1)} & x^{(2)} & \cdots & x^{(m)} \\ | & | & & | \end{pmatrix} = \begin{pmatrix} x_1^{(1)} & x_1^{(2)} & \cdots & x_1^{(m)} \\ x_2^{(1)} & x_2^{(2)} & \cdots & x_2^{(m)} \\ \vdots & \vdots & \ddots & \vdots \\ x_n^{(1)} & x_n^{(2)} & \cdots & x_n^{(m)} \end{pmatrix} \tag{90}$$

Attention

Convention importante ! Chaque colonne = un exemple. Cette convention (column-major) est standard en machine learning car elle permet des opérations matricielles efficaces.

Définition 5.2 (Vecteurs de paramètres et de labels)**.**

$$W = \begin{pmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{pmatrix} \in \mathbb{R}^{n \times 1} \quad \text{(vecteur colonne)} \tag{91}$$

$$Y = \begin{pmatrix} y^{(1)} & y^{(2)} & \cdots & y^{(m)} \end{pmatrix} \in \mathbb{R}^{1 \times m} \quad \text{(vecteur ligne)} \tag{92}$$

5.3 Forward Propagation Vectorisée

Proposition 5.1 (Calculs vectorisés)**.** Les calculs pour tous les exemples peuvent s'écrire matriciellement :

$$Z = W^T X + b \in \mathbb{R}^{1 \times m} \tag{93}$$

$$A = \sigma(Z) \in \mathbb{R}^{1 \times m} \tag{94}$$

où σ est appliqué élément par élément (element-wise).

Justification Mathématique

Vérification dimensionnelle :

$$W^T \in \mathbb{R}^{1\times n} \quad \text{(transposée de } W\text{)} \tag{95}$$
$$X \in \mathbb{R}^{n\times m} \tag{96}$$
$$W^T X \in \mathbb{R}^{1\times m} \quad \text{(produit matriciel : } (1\times n)\times(n\times m) = (1\times m)\text{)} \tag{97}$$

Vérification composante par composante :
La composante $(1, i)$ de $Z = W^T X$ est :

$$Z_i = [W^T X]_i = \sum_{k=1}^{n} W_k X_{k,i} = \sum_{k=1}^{n} w_k x_k^{(i)} \tag{98}$$
$$= w_1 x_1^{(i)} + w_2 x_2^{(i)} + \cdots + w_n x_n^{(i)} \tag{99}$$
$$= z^{(i)} \quad \checkmark \tag{100}$$

C'est bien notre formule du neurone pour l'exemple i !
Le biais b : En Python, l'addition $W^T X + b$ utilise le **broadcasting** : le scalaire b est automatiquement ajouté à chaque élément du vecteur ligne $W^T X$.
Broadcasting expliqué : Lorsqu'on additionne des tenseurs de dimensions différentes, NumPy/PyTorch étend automatiquement les dimensions plus petites pour correspondre aux plus grandes. Par exemple :

- Scalaire + Vecteur : $b + \begin{pmatrix} z_1 & z_2 & \cdots & z_m \end{pmatrix} = \begin{pmatrix} b+z_1 & b+z_2 & \cdots & b+z_m \end{pmatrix}$
- Vecteur colonne + Matrice : Un vecteur colonne $\in \mathbb{R}^{n\times 1}$ est répliqué sur toutes les colonnes d'une matrice $\in \mathbb{R}^{n\times m}$

Cette fonctionnalité évite d'écrire des boucles explicites et accélère considérablement les calculs.

$$Z = W^T X + b = \begin{pmatrix} z^{(1)} & z^{(2)} & \cdots & z^{(m)} \end{pmatrix} \tag{101}$$

$$A = \sigma(Z) = \begin{pmatrix} a^{(1)} & a^{(2)} & \cdots & a^{(m)} \end{pmatrix} = \begin{pmatrix} \sigma(z^{(1)}) & \sigma(z^{(2)}) & \cdots & \sigma(z^{(m)}) \end{pmatrix} \tag{102}$$

5.4 Fonction de Coût Vectorisée

La fonction de coût peut s'écrire vectoriellement (avec produit élément par élément $\odot$) :

Proposition 5.2 (Binary cross-entropy vectorisée)**.**

$$L = -\frac{1}{m}\left[Y \cdot \log(A)^T + (1-Y)\cdot \log(1-A)^T\right] \tag{103}$$

où :

- $\log(A)$ est appliqué élément par élément
- $(1-Y)$ et $(1-A)$ sont des soustractions élément par élément
- $\cdot$ représente le produit scalaire (dot product)

Justification Mathématique

Développons cette expression :

$$L = -\frac{1}{m}\left[Y \cdot \log(A)^T + (1-Y)\cdot \log(1-A)^T\right] \tag{104}$$

$$= -\frac{1}{m}\left[\begin{pmatrix} y^{(1)} & \cdots & y^{(m)} \end{pmatrix} \cdot \begin{pmatrix} \log(a^{(1)}) \\ \vdots \\ \log(a^{(m)}) \end{pmatrix} + \cdots\right] \tag{105}$$

$$= -\frac{1}{m}\left[\sum_{i=1}^{m} y^{(i)}\log(a^{(i)}) + \sum_{i=1}^{m}(1-y^{(i)})\log(1-a^{(i)})\right] \tag{106}$$

$$= -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log(a^{(i)}) + (1-y^{(i)})\log(1-a^{(i)})\right] \quad \checkmark \tag{107}$$

On retrouve bien notre formule originale !

5.5 Gradients Vectorisés

Proposition 5.3 (Backward propagation vectorisée)**.** Les gradients s'expriment vectoriellement comme :

$$\frac{\partial L}{\partial W} = \frac{1}{m}X(A-Y)^T \in \mathbb{R}^{n\times 1} \tag{108}$$

$$\frac{\partial L}{\partial b} = \frac{1}{m}\sum_{i=1}^{m}(a^{(i)} - y^{(i)}) = \frac{1}{m}\mathbf{1}^T(A-Y)^T \in \mathbb{R} \tag{109}$$

où $\mathbf{1} = \begin{pmatrix} 1 & 1 & \cdots & 1 \end{pmatrix}^T \in \mathbb{R}^{m\times 1}$ est le vecteur de uns.

Justification Mathématique

Vérification dimensionnelle pour $\frac{\partial L}{\partial W}$:

$$X \in \mathbb{R}^{n\times m} \tag{110}$$

$$(A-Y)^T \in \mathbb{R}^{m\times 1} \quad \text{(transposée de deux vecteurs lignes)} \tag{111}$$

$$X(A-Y)^T \in \mathbb{R}^{n\times 1} \quad \text{(produit : } (n\times m)\times(m\times 1) = (n\times 1)\text{)} \quad \checkmark \tag{112}$$

Le résultat a bien la même dimension que W !
Vérification composante par composante :
La composante k de $\frac{\partial L}{\partial W}$ est :

$$\left[\frac{\partial L}{\partial W}\right]_k = \frac{1}{m}[X(A-Y)^T]_k \tag{113}$$

$$= \frac{1}{m}\sum_{i=1}^{m} X_{k,i}\cdot(A_i - Y_i) \tag{114}$$

$$= \frac{1}{m}\sum_{i=1}^{m} x_k^{(i)}(a^{(i)} - y^{(i)}) \tag{115}$$

$$= \frac{\partial L}{\partial w_k} \quad \checkmark \tag{116}$$

On retrouve bien notre formule non-vectorisée !
Pour le biais : La somme $\sum_{i=1}^{m}(a^{(i)} - y^{(i)})$ peut s'écrire comme le produit scalaire $\mathbf{1}^T(A-Y)^T$, mais en pratique on utilise simplement `np.sum(A - Y)` en Python.

Attention

Vérification systématique des dimensions !
C'est la clé pour débugger les erreurs de vectorisation. Avant d'implémenter une formule vectorisée :

1. Écrivez les dimensions de chaque terme
2. Vérifiez que les produits matriciels sont valides
3. Vérifiez que le résultat a la bonne dimension
4. Testez sur un petit exemple (2-3 exemples) et comparez avec la version non-vectorisée

6 Extension : Une Couche de Neurones

Intuition

Pourquoi plusieurs neurones en parallèle ?
Un neurone unique avec fonction sigmoïde ne peut créer qu'une **seule** frontière de décision linéaire. Or, de nombreux problèmes réels nécessitent des frontières de décision **complexes et non linéaires**.
L'idée clé : en combinant **plusieurs neurones en parallèle**, chacun avec ses propres poids, chaque neurone "découpe" l'espace d'entrée selon une direction différente. La combinaison de ces sorties permet d'approximer des fonctions beaucoup plus complexes.
Exemple concret : Pour séparer deux classes dans un plan, une seule droite ne suffit pas toujours. Mais plusieurs droites combinées peuvent créer des régions polygonales qui séparent efficacement les deux groupes.

6.1 Architecture

Passons maintenant à l neurones en parallèle, tous connectés aux mêmes entrées $x_1, \ldots, x_n$.

Définition 6.1 (Couche de neurones)**.** Une couche de l neurones effectue l transformations linéaires indépendantes suivies d'activations :

Pour chaque neurone $j = 1, \ldots, l$:

$$z_j = \sum_{k=1}^{n} w_{jk} x_k + b_j = w_{j1}x_1 + w_{j2}x_2 + \cdots + w_{jn}x_n + b_j \tag{117}$$

$$a_j = \sigma(z_j) \tag{118}$$

Intuition

Chaque neurone "regarde" les mêmes entrées mais avec des poids différents, donc chacun extrait une caractéristique différente des données. La couche produit l activations qui seront utilisées par la suite.

6.2 Vectorisation de la Couche

Définition 6.2 (Matrices de poids et de biais)**.** On définit :

$$W = \begin{pmatrix} w_{11} & w_{12} & \cdots & w_{1n} \\ w_{21} & w_{22} & \cdots & w_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{l1} & w_{l2} & \cdots & w_{ln} \end{pmatrix} \in \mathbb{R}^{l \times n}, \quad b = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_l \end{pmatrix} \in \mathbb{R}^{l \times 1} \tag{119}$$

où la **ligne** j de W contient les poids du neurone j.

Attention

Convention importante ! La ligne j de W correspond au neurone j. Chaque ligne contient les poids connectant toutes les entrées à ce neurone.

Proposition 6.1 (Forward propagation pour une couche - un exemple)**.** Pour un exemple $x \in \mathbb{R}^{n\times 1}$:

$$Z = Wx + b \in \mathbb{R}^{l\times 1} \tag{120}$$
$$A = \sigma(Z) \in \mathbb{R}^{l\times 1} \tag{121}$$

Justification Mathématique

Vérification dimensionnelle :

$$W \in \mathbb{R}^{l\times n} \tag{122}$$
$$x \in \mathbb{R}^{n\times 1} \tag{123}$$
$$Wx \in \mathbb{R}^{l\times 1} \quad (\text{produit} : (l \times n) \times (n \times 1) = (l \times 1)) \tag{124}$$

Vérification composante par composante :
La composante j de $Z = Wx + b$ est :

$$Z_j = [Wx]_j + b_j = \sum_{k=1}^{n} W_{jk}x_k + b_j \tag{125}$$
$$= \sum_{k=1}^{n} w_{jk}x_k + b_j \tag{126}$$
$$= z_j \quad \checkmark \tag{127}$$

C'est bien la pré-activation du neurone j !

Proposition 6.2 (Forward propagation pour une couche - m exemples)**.** Pour m exemples (matrice $X \in \mathbb{R}^{n\times m}$) :

$$Z = WX + b \in \mathbb{R}^{l\times m} \tag{128}$$
$$A = \sigma(Z) \in \mathbb{R}^{l\times m} \tag{129}$$

où b est broadcasté : on ajoute b (vecteur colonne) à chaque colonne de WX.

Justification Mathématique

Vérification dimensionnelle :

$$W \in \mathbb{R}^{l \times n} \tag{130}$$
$$X \in \mathbb{R}^{n \times m} \tag{131}$$
$$WX \in \mathbb{R}^{l \times m} \quad (\text{produit} : (l \times n) \times (n \times m) = (l \times m)) \tag{132}$$

Interprétation :

- Chaque **ligne** de Z correspond à un neurone (ligne j = sorties du neurone j pour tous les exemples)
- Chaque **colonne** de Z correspond à un exemple (colonne i = sorties de tous les neurones pour l'exemple i)

L'élément Z_{ji} représente la pré-activation du neurone j pour l'exemple i :

$$Z_{ji} = \sum_{k=1}^{n} w_{jk} x_k^{(i)} + b_j \tag{133}$$

7 Réseaux Multi-Couches (Deep Neural Networks)

Intuition

Pourquoi empiler plusieurs couches ?
Une seule couche de neurones peut séparer l'espace d'entrée par des hyperplans, mais reste limitée à des combinaisons linéaires suivies d'une activation. Pour capturer des patterns véritablement complexes (reconnaissance d'objets, compréhension du langage), nous avons besoin de **compositions de transformations non linéaires**.
L'idée de la profondeur :

- Couche 1 : détecte des caractéristiques simples (bords, textures basiques)
- Couche 2 : combine ces caractéristiques simples en motifs plus complexes (coins, formes)
- Couches profondes : construisent des représentations abstraites (parties d'objets, concepts)
- Couche finale : prend une décision basée sur ces représentations de haut niveau

Cette hiérarchie de représentations est la clé du succès du deep learning : chaque couche apprend automatiquement des features pertinentes sans intervention humaine.

7.1 Architecture

Un réseau profond est composé de L couches successives, où la sortie de chaque couche devient l'entrée de la suivante. C'est une composition de fonctions.

Définition 7.1 (Notations pour les réseaux profonds)**.** Pour la couche $l = 1, \ldots, L$:

- $n^{[l]}$: nombre de neurones dans la couche l (dimension de sortie)

— $n^{[l-1]}$: nombre de neurones dans la couche précédente (dimension d'entrée)
— $W^{[l]} \in \mathbb{R}^{n^{[l]} \times n^{[l-1]}}$: matrice de poids
— $b^{[l]} \in \mathbb{R}^{n^{[l]} \times 1}$: vecteur de biais
— $Z^{[l]} \in \mathbb{R}^{n^{[l]} \times m}$: pré-activations pour m exemples
— $A^{[l]} \in \mathbb{R}^{n^{[l]} \times m}$: activations pour m exemples

Convention : $A^{[0]} = X \in \mathbb{R}^{n^{[0]} \times m}$ (l'entrée du réseau, où $n^{[0]} = n$ est le nombre de features)

Justification Mathématique

Pourquoi $W^{[l]} \in \mathbb{R}^{n^{[l]} \times n^{[l-1]}}$?
La couche l a :
— Entrée : $A^{[l-1]} \in \mathbb{R}^{n^{[l-1]} \times m}$ (sortie de la couche précédente)
— Sortie : $A^{[l]} \in \mathbb{R}^{n^{[l]} \times m}$

Pour que $Z^{[l]} = W^{[l]} A^{[l-1]} + b^{[l]}$ ait la bonne dimension :

$$\underbrace{Z^{[l]}}_{n^{[l]} \times m} = \underbrace{W^{[l]}}_{n^{[l]} \times n^{[l-1]}} \underbrace{A^{[l-1]}}_{n^{[l-1]} \times m} + \underbrace{b^{[l]}}_{n^{[l]} \times 1} \tag{134}$$

Le produit matriciel est valide : $(n^{[l]} \times n^{[l-1]}) \times (n^{[l-1]} \times m) = (n^{[l]} \times m)$ ✓

7.2 Forward Propagation

Proposition 7.1 (Forward propagation générale)**.** Pour chaque couche $l = 1, \ldots, L$:

$$Z^{[l]} = W^{[l]} A^{[l-1]} + b^{[l]} \in \mathbb{R}^{n^{[l]} \times m} \tag{135}$$

$$A^{[l]} = \sigma(Z^{[l]}) \in \mathbb{R}^{n^{[l]} \times m} \tag{136}$$

avec $A^{[0]} = X$.

Intuition

Le réseau effectue une série de transformations :

$$X = A^{[0]} \xrightarrow{\text{couche } 1} A^{[1]} \xrightarrow{\text{couche } 2} A^{[2]} \xrightarrow{\text{couche } 3} \cdots \xrightarrow{\text{couche } L} A^{[L]} = \hat{Y} \tag{137}$$

Chaque couche transforme sa représentation d'entrée en une représentation de sortie, potentiellement de dimension différente. Les couches intermédiaires extraient des caractéristiques de plus en plus abstraites.

Exemple 7.1 (Réseau à 3 couches)**.** Soit un réseau avec :
— Entrée : $n^{[0]} = 2$ features (ex : radius et humérus)
— Couche 1 : $n^{[1]} = 4$ neurones (couche cachée 1)
— Couche 2 : $n^{[2]} = 3$ neurones (couche cachée 2)
— Couche 3 : $n^{[3]} = 1$ neurone (sortie binaire)

Forward propagation :

$$Z^{[1]} = W^{[1]}X + b^{[1]}, \quad A^{[1]} = \sigma(Z^{[1]}) \quad \text{où } W^{[1]} \in \mathbb{R}^{4\times 2} \tag{138}$$

$$Z^{[2]} = W^{[2]}A^{[1]} + b^{[2]}, \quad A^{[2]} = \sigma(Z^{[2]}) \quad \text{où } W^{[2]} \in \mathbb{R}^{3\times 4} \tag{139}$$

$$Z^{[3]} = W^{[3]}A^{[2]} + b^{[3]}, \quad A^{[3]} = \sigma(Z^{[3]}) \quad \text{où } W^{[3]} \in \mathbb{R}^{1\times 3} \tag{140}$$

La prédiction finale est $\hat{Y} = A^{[3]} \in \mathbb{R}^{1\times m}$.

7.3 Backpropagation : Calcul Rigoureux des Gradients

7.3.1 Principe Général

La rétropropagation (backpropagation) consiste à appliquer systématiquement la règle de la chaîne pour calculer les gradients en remontant depuis la sortie vers l'entrée.

Définition 7.2 (Notations pour les gradients)**.** On note :

$$dZ^{[l]} = \frac{\partial L}{\partial Z^{[l]}} \in \mathbb{R}^{n^{[l]}\times m} \tag{141}$$

$$dA^{[l]} = \frac{\partial L}{\partial A^{[l]}} \in \mathbb{R}^{n^{[l]}\times m} \tag{142}$$

$$dW^{[l]} = \frac{\partial L}{\partial W^{[l]}} \in \mathbb{R}^{n^{[l]}\times n^{[l-1]}} \tag{143}$$

$$db^{[l]} = \frac{\partial L}{\partial b^{[l]}} \in \mathbb{R}^{n^{[l]}\times 1} \tag{144}$$

Attention

Chaque gradient a exactement la même dimension que la variable correspondante. C'est un principe fondamental du calcul différentiel.

7.3.2 Couche de Sortie

Proposition 7.2 (Gradient de la couche de sortie)**.** Pour la dernière couche L avec binary cross-entropy :

$$dZ^{[L]} = A^{[L]} - Y \in \mathbb{R}^{n^{[L]}\times m} \tag{145}$$

Démonstration. Rappel de la fonction de coût :

$$L = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log(a^{[L](i)}) + (1-y^{(i)})\log(1-a^{[L](i)})\right] \tag{146}$$

Pour un neurone de sortie (supposons $n^{[L]} = 1$ pour simplifier), nous avons démontré précédemment :

$$\frac{\partial L}{\partial z^{[L](i)}} = \frac{\partial L}{\partial a^{[L](i)}} \cdot \frac{\partial a^{[L](i)}}{\partial z^{[L](i)}} \tag{147}$$

$$= \left[-\frac{1}{m}\cdot\frac{y^{(i)} - a^{[L](i)}}{a^{[L](i)}(1-a^{[L](i)})}\right] \cdot \left[a^{[L](i)}(1-a^{[L](i)})\right] \tag{148}$$

$$= \frac{1}{m}(a^{[L](i)} - y^{(i)}) \tag{149}$$

En notation vectorisée pour tous les exemples :

$$dZ^{[L]} = \frac{1}{m}(A^{[L]} - Y) \tag{150}$$

Remarque importante sur le facteur $\frac{1}{m}$:

Pour simplifier l'implémentation, on omet souvent le facteur $\frac{1}{m}$ dans $dZ^{[L]}$ et on l'incorpore directement dans les formules de mise à jour des poids. Concrètement :

Version avec facteur dans dZ :

$$dZ^{[L]} = \frac{1}{m}(A^{[L]} - Y), \quad dW^{[l]} = dZ^{[l]}(A^{[l-1]})^T, \quad db^{[l]} = dZ^{[l]}\mathbf{1} \tag{151}$$

Version simplifiée (équivalente, plus courante en pratique) :

$$dZ^{[L]} = A^{[L]} - Y, \quad dW^{[l]} = \frac{1}{m}dZ^{[l]}(A^{[l-1]})^T, \quad db^{[l]} = \frac{1}{m}dZ^{[l]}\mathbf{1} \tag{152}$$

Les deux approches donnent les mêmes gradients finaux pour W et b. Nous utiliserons la seconde dans la suite du cours car elle est plus standard dans les frameworks modernes (PyTorch, TensorFlow). □

7.3.3 Formules de Backpropagation Générales

Théorème 7.1 (Backpropagation pour la couche l)**.** Supposons que nous avons calculé $dZ^{[l]}$ (le gradient par rapport aux pré-activations de la couche l). Alors :

$$dW^{[l]} = \frac{1}{m}dZ^{[l]}(A^{[l-1]})^T \in \mathbb{R}^{n^{[l]} \times n^{[l-1]}} \tag{153}$$

$$db^{[l]} = \frac{1}{m}\sum_{\text{colonnes}} dZ^{[l]} = \frac{1}{m}dZ^{[l]}\mathbf{1} \in \mathbb{R}^{n^{[l]} \times 1} \tag{154}$$

$$dA^{[l-1]} = (W^{[l]})^T dZ^{[l]} \in \mathbb{R}^{n^{[l-1]} \times m} \tag{155}$$

$$dZ^{[l-1]} = dA^{[l-1]} \odot \sigma'(Z^{[l-1]}) \in \mathbb{R}^{n^{[l-1]} \times m} \tag{156}$$

où :

- $\odot$ est le produit élément par élément (Hadamard product)
- $\sigma'(Z^{[l-1]}) = A^{[l-1]} \odot (1 - A^{[l-1]})$ pour la sigmoïde
- $\mathbf{1} = \begin{pmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{pmatrix} \in \mathbb{R}^{m \times 1}$

Démonstration de chaque formule. 1. Démonstration de (153) : $dW^{[l]} = \frac{1}{m}dZ^{[l]}(A^{[l-1]})^T$

Rappel : $Z^{[l]} = W^{[l]}A^{[l-1]} + b^{[l]}$

Nous voulons calculer $\frac{\partial L}{\partial W^{[l]}}$. Par la règle de la chaîne :

$$\frac{\partial L}{\partial W_{ij}^{[l]}} = \sum_{k=1}^{m} \frac{\partial L}{\partial Z_{ik}^{[l]}} \cdot \frac{\partial Z_{ik}^{[l]}}{\partial W_{ij}^{[l]}} \tag{157}$$

Calculons $\frac{\partial Z_{ik}^{[l]}}{\partial W_{ij}^{[l]}}$:

$$Z_{ik}^{[l]} = \sum_{p=1}^{n^{[l-1]}} W_{ip}^{[l]} A_{pk}^{[l-1]} + b_i^{[l]} \tag{158}$$

$$\frac{\partial Z_{ik}^{[l]}}{\partial W_{ij}^{[l]}} = A_{jk}^{[l-1]} \tag{159}$$

(Le seul terme qui dépend de $W_{ij}^{[l]}$ est celui où $p = j$)
Donc :

$$\frac{\partial L}{\partial W_{ij}^{[l]}} = \sum_{k=1}^{m} \frac{\partial L}{\partial Z_{ik}^{[l]}} \cdot A_{jk}^{[l-1]} \tag{160}$$

$$= \sum_{k=1}^{m} dZ_{ik}^{[l]} \cdot A_{jk}^{[l-1]} \tag{161}$$

$$= \sum_{k=1}^{m} dZ_{ik}^{[l]} \cdot (A^{[l-1]})_{kj}^{T} \tag{162}$$

C'est exactement l'élément (i, j) du produit matriciel $dZ^{[l]}(A^{[l-1]})^T$ (sans le facteur $\frac{1}{m}$).
En incluant le facteur de moyenne :

$$dW^{[l]} = \frac{1}{m} dZ^{[l]}(A^{[l-1]})^T \quad \checkmark \tag{163}$$

Vérification dimensionnelle :

$$\underbrace{dW^{[l]}}_{n^{[l]} \times n^{[l-1]}} = \frac{1}{m} \underbrace{dZ^{[l]}}_{n^{[l]} \times m} \underbrace{(A^{[l-1]})^T}_{m \times n^{[l-1]}} \tag{164}$$

$(n^{[l]} \times m) \times (m \times n^{[l-1]}) = (n^{[l]} \times n^{[l-1]})$ ✓

2. Démonstration de (154) : $db^{[l]} = \frac{1}{m} \sum_{\text{colonnes}} dZ^{[l]}$
Par la règle de la chaîne :

$$\frac{\partial L}{\partial b_i^{[l]}} = \sum_{k=1}^{m} \frac{\partial L}{\partial Z_{ik}^{[l]}} \cdot \frac{\partial Z_{ik}^{[l]}}{\partial b_i^{[l]}} \tag{165}$$

$$= \sum_{k=1}^{m} dZ_{ik}^{[l]} \cdot 1 \quad \left(\text{car } \frac{\partial (W^{[l]} A^{[l-1]} + b^{[l]})_i}{\partial b_i^{[l]}} = 1\right) \tag{166}$$

$$= \sum_{k=1}^{m} dZ_{ik}^{[l]} \tag{167}$$

C'est la somme de la ligne i de $dZ^{[l]}$ (équivalent à sommer sur les colonnes pour chaque ligne).

En incluant le facteur de moyenne et en notation vectorielle :

$$db^{[l]} = \frac{1}{m} dZ^{[l]} \mathbf{1} \quad \checkmark \tag{168}$$

où $\mathbf{1} \in \mathbb{R}^{m\times 1}$ est le vecteur de uns.
3. Démonstration de (155) : $dA^{[l-1]} = (W^{[l]})^T dZ^{[l]}$
Par la règle de la chaîne :

$$\frac{\partial L}{\partial A_{jk}^{[l-1]}} = \sum_{i=1}^{n^{[l]}} \frac{\partial L}{\partial Z_{ik}^{[l]}} \cdot \frac{\partial Z_{ik}^{[l]}}{\partial A_{jk}^{[l-1]}} \tag{169}$$

$$= \sum_{i=1}^{n^{[l]}} dZ_{ik}^{[l]} \cdot W_{ij}^{[l]} \quad (\text{car } Z_{ik}^{[l]} = \sum_p W_{ip}^{[l]} A_{pk}^{[l-1]} + b_i^{[l]}) \tag{170}$$

$$= \sum_{i=1}^{n^{[l]}} (W^{[l]})_{ji}^T \cdot dZ_{ik}^{[l]} \tag{171}$$

C'est exactement l'élément (j,k) du produit matriciel $(W^{[l]})^T dZ^{[l]}$:

$$dA^{[l-1]} = (W^{[l]})^T dZ^{[l]} \quad \checkmark \tag{172}$$

Vérification dimensionnelle :

$$\underbrace{dA^{[l-1]}}_{n^{[l-1]}\times m} = \underbrace{(W^{[l]})^T}_{n^{[l-1]}\times n^{[l]}} \underbrace{dZ^{[l]}}_{n^{[l]}\times m} \tag{173}$$

$(n^{[l-1]} \times n^{[l]}) \times (n^{[l]} \times m) = (n^{[l-1]} \times m)$ ✓
4. Démonstration de (296) : $dZ^{[l-1]} = dA^{[l-1]} \odot \sigma'(Z^{[l-1]})$
Rappel : $A^{[l-1]} = \sigma(Z^{[l-1]})$, donc $A_{jk}^{[l-1]} = \sigma(Z_{jk}^{[l-1]})$
Par la règle de la chaîne :

$$\frac{\partial L}{\partial Z_{jk}^{[l-1]}} = \frac{\partial L}{\partial A_{jk}^{[l-1]}} \cdot \frac{\partial A_{jk}^{[l-1]}}{\partial Z_{jk}^{[l-1]}} \tag{174}$$

$$= dA_{jk}^{[l-1]} \cdot \sigma'(Z_{jk}^{[l-1]}) \tag{175}$$

$$= dA_{jk}^{[l-1]} \cdot A_{jk}^{[l-1]}(1 - A_{jk}^{[l-1]}) \tag{176}$$

En notation vectorielle (élément par élément) :

$$dZ^{[l-1]} = dA^{[l-1]} \odot A^{[l-1]} \odot (1 - A^{[l-1]}) \quad \checkmark \tag{177}$$

□

Justification Mathématique

Intuition de la backpropagation :

1. $dZ^{[l]}$: On commence avec le gradient par rapport aux pré-activations de la couche courante
2. $dW^{[l]}, db^{[l]}$: On calcule les gradients par rapport aux paramètres de cette couche (pour la mise à jour)
3. $dA^{[l-1]}$: On propage le gradient vers la couche précédente (à travers les poids)
4. $dZ^{[l-1]}$: On applique la dérivée de l'activation pour obtenir le gradient par rapport aux pré-activations de la couche précédente
5. On répète pour chaque couche en remontant vers l'entrée

Le terme $(W^{[l]})^T$ dans (155) "distribue" les gradients de la couche l vers la couche $l-1$ en fonction des connexions (poids).

7.4 Algorithme Complet

Algorithme : Entraînement d'un Réseau de Neurones Profond

Entrées :

- Dataset : $X \in \mathbb{R}^{n^{[0]} \times m}$, $Y \in \mathbb{R}^{1 \times m}$
- Architecture : nombre de couches L, tailles $(n^{[1]}, \ldots, n^{[L]})$
- Hyperparamètres : learning rate α, nombre d'itérations T

Initialisation : Initialiser aléatoirement $W^{[l]} \sim \mathcal{N}(0, \sqrt{2/n^{[l-1]}})$, $b^{[l]} = 0$ pour $l = 1, \ldots, L$

Pour $t = 1, \ldots, T$:

1. **Forward propagation :**
   Poser $A^{[0]} = X$
   Pour $l = 1, \ldots, L$:
   $Z^{[l]} = W^{[l]} A^{[l-1]} + b^{[l]}$
   $A^{[l]} = \sigma(Z^{[l]})$
2. **Calcul du coût :**
   $L = -\frac{1}{m} \sum_{i=1}^{m} [y^{(i)} \log(a^{[L](i)}) + (1 - y^{(i)}) \log(1 - a^{[L](i)})]$
3. **Backward propagation :**
   Initialiser : $dZ^{[L]} = A^{[L]} - Y$
   Pour $l = L, L-1, \ldots, 1$ (en ordre décroissant) :
   $dW^{[l]} = \frac{1}{m} dZ^{[l]} (A^{[l-1]})^T$
   $db^{[l]} = \frac{1}{m} dZ^{[l]} \mathbf{1}$
   Si $l > 1$:
   $dA^{[l-1]} = (W^{[l]})^T dZ^{[l]}$
   $dZ^{[l-1]} = dA^{[l-1]} \odot A^{[l-1]} \odot (1 - A^{[l-1]})$
4. **Mise à jour des paramètres :**
   Pour $l = 1, \ldots, L$:
   $W^{[l]} \leftarrow W^{[l]} - \alpha \, dW^{[l]}$
   $b^{[l]} \leftarrow b^{[l]} - \alpha \, db^{[l]}$

Retourner : Les paramètres optimisés $(W^{[1]}, b^{[1]}, \ldots, W^{[L]}, b^{[L]})$

Attention

Vérifications de Dimensions - Tableau Récapitulatif
Pour la couche l avec m exemples :

Variable	Dimension	Description
$W^{[l]}$	$n^{[l]} \times n^{[l-1]}$	Poids
$b^{[l]}$	$n^{[l]} \times 1$	Biais
$Z^{[l]}, A^{[l]}$	$n^{[l]} \times m$	Pré-activations, Activations
$dW^{[l]}$	$n^{[l]} \times n^{[l-1]}$	Gradient des poids (même que $W^{[l]}$)
$db^{[l]}$	$n^{[l]} \times 1$	Gradient du biais (même que $b^{[l]}$)
$dZ^{[l]}, dA^{[l]}$	$n^{[l]} \times m$	Gradients (mêmes que $Z^{[l]}, A^{[l]}$)

Règle d'or : Chaque gradient a exactement la même dimension que la variable correspondante !

7.5 La Puissance des Réseaux Multicouches : Frontières Non-Linéaires

Jusqu'à présent, nous avons vu comment empiler plusieurs couches de neurones et calculer les gradients efficacement. Mais pourquoi utiliser plusieurs couches ?

Intuition

Un neurone unique ne peut créer qu'une frontière de décision linéaire. Cela limite considérablement les problèmes qu'il peut résoudre.
En empilant plusieurs couches avec des non-linéarités, le réseau peut apprendre des **frontières arbitrairement complexes** : courbes, cercles, spirales, formes discontinues...

7.5.1 Exemple : Classification de Cercles Concentriques

Considérons un problème impossible pour un neurone unique : distinguer deux classes organisées en cercles concentriques.

— Classe 0 (bleu) : points à l'extérieur du cercle

— Classe 1 (orange) : points à l'intérieur du cercle

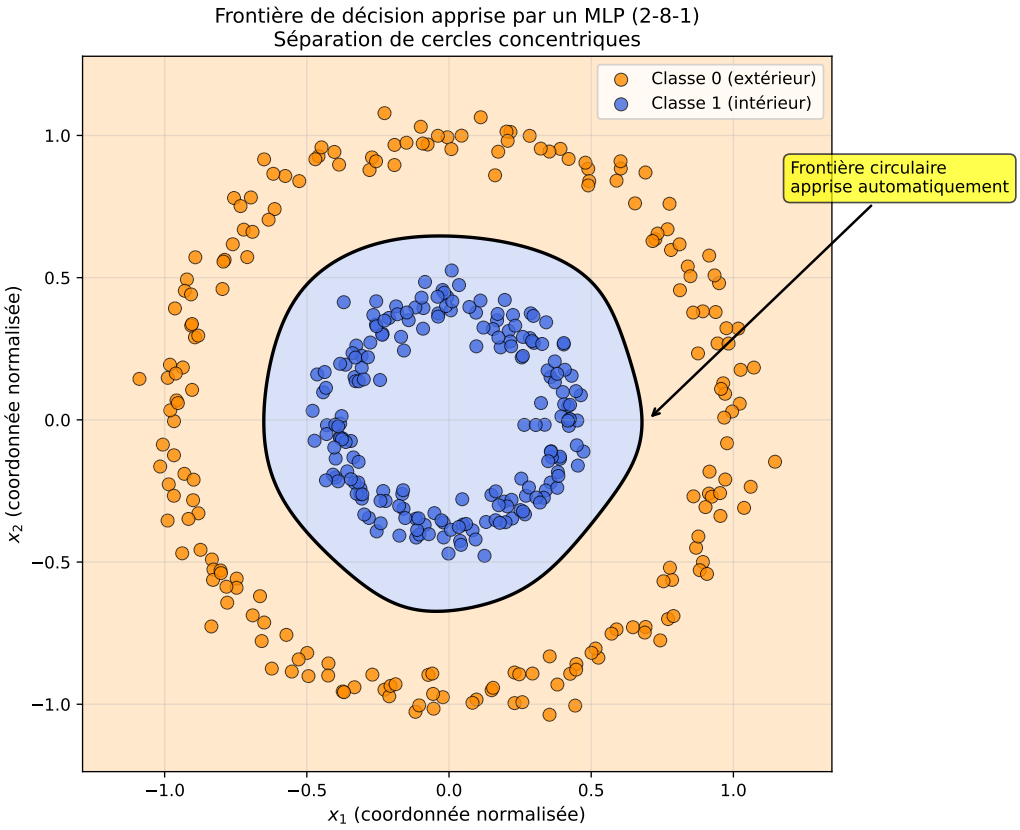

Figure 4 : Frontière de décision apprise par un réseau multicouches (MLP) avec sigmoïdes. Le réseau a appris une frontière circulaire (elliptique) qui sépare parfaitement les deux classes, une tâche impossible pour un neurone unique.

Transition : du cas binaire au multi-classes

Dans tout ce qui précède, la sortie du réseau était binaire : $a = \sigma(z) \in (0, 1)$, interprété comme $P(Y = 1 \mid x)$ et $1 - a = P(Y = 0 \mid x)$. Nous passions d'un score linéaire $z = w^T x + b$ à une probabilité via la sigmoïde, et l'inférence/optimisation reposaient sur la loi de Bernoulli et la *binary cross-entropy*.

Or, quantité de problèmes réels sont multi-classes exclusifs : une unique étiquette $Y \in \{1, \ldots, K\}$ doit être prédite (par exemple *gorille, chimpanzé, orang-outan*). La question naturelle est donc : *comment généraliser rigoureusement le cadre sigmoïde à $K \geq 3$ classes, avec une interprétation probabiliste propre et des gradients simples ?* C'est précisément le rôle de la **softmax** et de la **cross-entropie catégorielle** que nous introduisons maintenant.

8 Classification Multi-Classes : de la Sigmoïde à la Softmax

8.1 Exemple motivant : Gorille, Chimpanzé, Orang-outan ($K = 3$)

Reprenons nos deux mesures osseuses (x_1, x_2) (radius, humérus) mais élargissons la base à trois espèces :

$$Y \in \{\text{gorille}, \ \text{chimpanzé}, \ \text{orang-outan}\}.$$

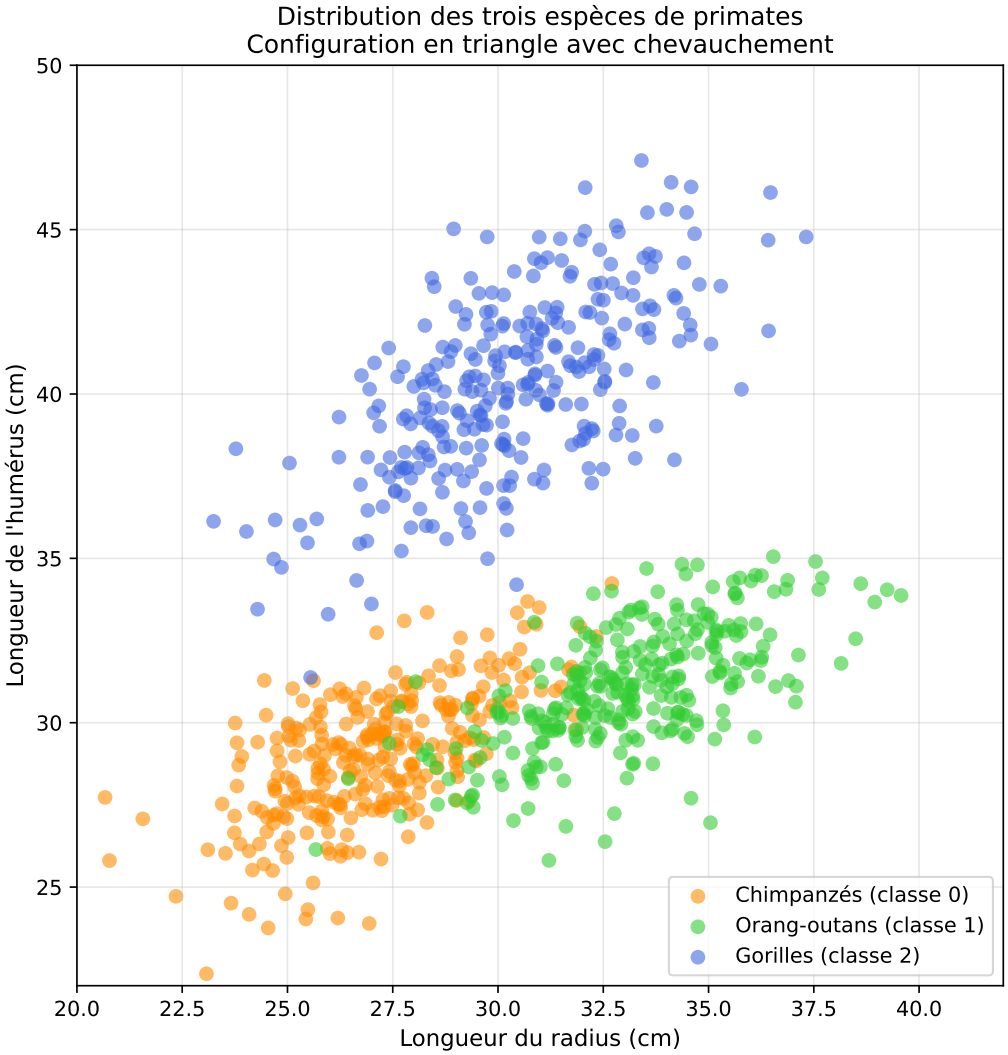

Figure 5 : Distributions des proportions Humérus-Radius pour trois espèces de primates. Les trois populations forment des nuages de points dans le plan (x_1, x_2).

Empiriquement, les trois nuages forment des régions (partiellement) séparables dans le plan (x_1, x_2), comme illustré sur la Figure 5. Notre but est de construire, pour tout x, un *vecteur de probabilités*

$$a(x) = (a_1(x), a_2(x), a_3(x))^T \in [0,1]^3, \qquad \sum_{k=1}^{3} a_k(x) = 1,$$

interprété comme $a_k(x) = P(Y = k \mid x)$.

Attention

[Pourquoi *pas* K sigmoïdes indépendantes ?] Une tentation naturelle est de poser $a_k = \sigma(z_k)$ avec K sorties indépendantes et d'assigner la classe $\arg\max_k a_k$. Cette approche pose plusieurs problèmes majeurs :
Problème 1 - Pas de normalisation : On n'a **aucune contrainte** $\sum_k a_k = 1$. Par exemple, on pourrait obtenir :

$$(a_1, a_2, a_3) = (0.7, 0.6, 0.5) \quad \text{avec} \sum a_k = 1.8$$

Impossible à interpréter comme des probabilités mutuellement exclusives ! Quelle est "vraiment" la probabilité de la classe 1 ?
Problème 2 - Incohérence des log-odds : Avec des sigmoïdes indépendantes, les rapports de probabilités entre paires de classes ne sont pas cohérents. Par exemple, si $P(1)/P(2) = 2$ et $P(2)/P(3) = 3$, on devrait avoir $P(1)/P(3) = 6$, mais rien ne garantit cette cohérence avec des sigmoïdes séparées.
Problème 3 - Interprétation probabiliste brisée : Le modèle ne maximise pas une vraie vraisemblance d'une distribution catégorielle. On perd le fondement théorique.
Conclusion : Pour des classes *mutuellement exclusives*, il faut un *couplage explicite* entre les sorties via normalisation : c'est le rôle de la softmax.
Note : Si les classes ne sont PAS mutuellement exclusives (multi-label : une image peut contenir à la fois un chat ET un chien), alors des sigmoïdes indépendantes sont appropriées.

8.2 De la Bernoulli à la catégorielle : le bon cadre probabiliste

Dans le cas binaire, $Y \in \{0, 1\}$ et $P(Y = 1 \mid x) = a$ (loi de Bernoulli). En multi-classes exclusives, la distribution naturelle est la **loi catégorielle** de paramètre $a \in \Delta^{K-1}$ (simplexe) :

$$P(Y = k \mid x) = a_k, \qquad \sum_{k=1}^{K} a_k = 1, \quad a_k \geq 0.$$

Il nous faut donc une application $z \mapsto a$ qui transforme un vecteur arbitraire de scores $z = (z_1, \ldots, z_K)$ en une *distribution* a. La généralisation de la sigmoïde (qui, en binaire, normalise deux masses e^z et 1) est la **softmax**.

8.3 Softmax : définition, propriétés, interprétations

Définition 8.1 (Softmax)**.** Pour $z \in \mathbb{R}^K$,

$$a_k = \text{softmax}(z)_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad k = 1, \ldots, K.$$

Alors $a \in (0, 1)^K$ et $\sum_k a_k = 1$.

Proposition 8.1 (Invariance et log-odds linéaires)**.** La fonction softmax possède deux propriétés fondamentales :

1. Invariance par translation : Pour tout $c \in \mathbb{R}$,

$$\text{softmax}(z + c\mathbf{1}) = \text{softmax}(z)$$

où $\mathbf{1} = (1, 1, \ldots, 1)^T$ est le vecteur de uns.

2. Log-odds linéaires : Le logarithme du rapport de probabilités entre deux classes est égal à la différence de leurs scores :

$$\log \frac{a_k}{a_j} = z_k - z_j$$

pour toutes classes $k, j \in \{1, \ldots, K\}$.

3. Conséquence pour un modèle linéaire : Si $z = Wx + b$ (modèle linéaire), alors

$$\log \frac{a_k}{a_j} = (w_k - w_j)^T x + (b_k - b_j)$$

est une fonction **linéaire** de x.

Démonstration. Propriété 1 : Invariance par translation

Calculons $\text{softmax}(z + c\mathbf{1})$ où $c \in \mathbb{R}$ est une constante arbitraire :

$$\begin{aligned}
\text{softmax}_k(z + c\mathbf{1}) &= \frac{e^{z_k + c}}{\sum_{j=1}^K e^{z_j + c}} && (178)\\
&= \frac{e^{z_k} \cdot e^c}{\sum_{j=1}^K e^{z_j} \cdot e^c} \quad (\text{propriété } e^{a+b} = e^a \cdot e^b) && (179)\\
&= \frac{e^c \cdot e^{z_k}}{e^c \cdot \sum_{j=1}^K e^{z_j}} && (180)\\
&= \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}} \quad (\text{simplification par } e^c) && (181)\\
&= \text{softmax}_k(z) && (182)
\end{aligned}$$

Interprétation : Ajouter la même constante à tous les scores ne change pas les probabilités finales. Seules les *différences relatives* entre scores comptent.

Propriété 2 : Log-odds linéaires

Par définition de la softmax :

$$a_k = \frac{e^{z_k}}{\sum_{i=1}^K e^{z_i}}, \quad a_j = \frac{e^{z_j}}{\sum_{i=1}^K e^{z_i}} \tag{183}$$

Calculons le rapport $\frac{a_k}{a_j}$:

$$\begin{aligned}
\frac{a_k}{a_j} &= \frac{\frac{e^{z_k}}{\sum_{i=1}^K e^{z_i}}}{\frac{e^{z_j}}{\sum_{i=1}^K e^{z_i}}} && (184)\\
&= \frac{e^{z_k}}{\sum_{i=1}^K e^{z_i}} \cdot \frac{\sum_{i=1}^K e^{z_i}}{e^{z_j}} && (185)\\
&= \frac{e^{z_k}}{e^{z_j}} \quad (\text{simplification des dénominateurs}) && (186)\\
&= e^{z_k - z_j} \quad (\text{propriété } \frac{e^a}{e^b} = e^{a-b}) && (187)
\end{aligned}$$

En prenant le logarithme des deux côtés :

$$\log \frac{a_k}{a_j} = \log(e^{z_k - z_j}) = z_k - z_j \quad \checkmark \tag{188}$$

Interprétation : Le log-odds (logarithme du rapport de probabilités) dépend uniquement de la différence des scores, pas de leur valeur absolue.

Propriété 3 : Linéarité dans un modèle linéaire
Si $z = Wx + b$ où $W \in \mathbb{R}^{K\times n}$ et $b \in \mathbb{R}^K$, alors :

$$z_k = w_k^T x + b_k \quad (\text{où } w_k \text{ est la } k\text{-ème ligne de } W) \tag{189}$$
$$z_j = w_j^T x + b_j \tag{190}$$

D'après la propriété 2 :

$$\log \frac{a_k}{a_j} = z_k - z_j \tag{191}$$
$$= (w_k^T x + b_k) - (w_j^T x + b_j) \tag{192}$$
$$= w_k^T x - w_j^T x + b_k - b_j \tag{193}$$
$$= (w_k - w_j)^T x + (b_k - b_j) \quad \checkmark \tag{194}$$

Ceci est une fonction **affine** (donc linéaire au sens large) de x, exactement comme en régression logistique binaire ! □

Exemple 8.1 (Calcul numérique)**.** Soit $z = (2, 5, 3)$ pour $K = 3$ classes. Calculons les probabilités softmax et vérifions les propriétés :
Calcul de softmax :

$$e^{z_1} = e^2 \approx 7.39, \quad e^{z_2} = e^5 \approx 148.41, \quad e^{z_3} = e^3 \approx 20.09 \tag{195}$$
$$\sum_j e^{z_j} \approx 7.39 + 148.41 + 20.09 = 175.89 \tag{196}$$
$$a_1 \approx \frac{7.39}{175.89} \approx 0.042, \quad a_2 \approx \frac{148.41}{175.89} \approx 0.844, \quad a_3 \approx \frac{20.09}{175.89} \approx 0.114 \tag{197}$$

Vérifions : $a_1 + a_2 + a_3 \approx 0.042 + 0.844 + 0.114 = 1.000$ ✓
Vérification de l'invariance : Ajoutons $c = 10$ à tous les scores : $z' = (12, 15, 13)$

$$e^{z'_1 - 15} = e^{-3} \approx 0.050, \quad e^{z'_2 - 15} = e^0 = 1, \quad e^{z'_3 - 15} = e^{-2} \approx 0.135 \tag{198}$$
$$\sum_j e^{z'_j - 15} \approx 0.050 + 1 + 0.135 = 1.185 \tag{199}$$
$$a'_1 \approx \frac{0.050}{1.185} \approx 0.042, \quad a'_2 \approx \frac{1}{1.185} \approx 0.844, \quad a'_3 \approx \frac{0.135}{1.185} \approx 0.114 \tag{200}$$

On retrouve exactement les mêmes probabilités ! ✓
Vérification des log-odds : Calculons $\log \frac{a_2}{a_1}$ de deux façons :

$$\text{Méthode 1 :} \quad \log \frac{a_2}{a_1} = \log \frac{0.844}{0.042} = \log(20.1) \approx 3.00 \tag{201}$$
$$\text{Méthode 2 :} \quad z_2 - z_1 = 5 - 2 = 3.00 \tag{202}$$

Les deux méthodes donnent le même résultat ! ✓

Intuition

Que signifient ces propriétés concrètement ?
1. L'invariance par translation signifie que la softmax ne regarde que les *écarts relatifs* entre scores. Si tous les neurones de la dernière couche "tirent fort" (scores élevés) ou "tirent faiblement" (scores faibles), cela n'a aucune importance : seules les *différences* comptent.
Exemple intuitif : Trois candidats à une élection obtiennent respectivement 100, 150, et 120 voix. Les proportions sont $(0.27, 0.41, 0.32)$. Si on multiplie tous les scores par 10 (1000, 1500, 1200 voix), les proportions restent identiques ! C'est exactement ce que dit l'invariance.
2. Les log-odds linéaires établissent un lien direct entre les scores z et les rapports de probabilités, sans passer par la normalisation. C'est comme en régression logistique binaire où $\log \frac{P(Y=1)}{P(Y=0)} = z$.
Pourquoi est-ce puissant ? Si le modèle est linéaire ($z = Wx + b$), alors les frontières de décision entre chaque paire de classes sont des **hyperplans**, et toutes ces frontières sont **cohérentes** (elles se coupent en des lieux géométriquement sensés). Avec des sigmoïdes indépendantes, on perd cette cohérence globale.
3. La géométrie relative : L'invariance traduit que la softmax vit dans un espace projectif : on peut toujours choisir une "classe de référence" (par exemple, fixer $z_K = 0$ ou $b_K = 0$) sans perte de généralité. C'est ce couplage entre toutes les sorties, via le dénominateur commun $\sum_j e^{z_j}$, qui distingue fondamentalement la softmax des sigmoïdes indépendantes.

Attention

[Stabilité numérique : `log-sum-exp`] Pour éviter les débordements (overflow) et sous-débordements (underflow), on centre toujours z par $c = \max_j z_j$:

$$a_k = \frac{e^{z_k - c}}{\sum_j e^{z_j - c}}, \qquad \log \sum_j e^{z_j} = c + \log \sum_j e^{z_j - c}.$$

Ainsi $e^{z_j - c} \in (0, 1]$ et la somme est bornée par K.
Exemple du problème : Si $z = (1000, 1001, 1002)$, alors $e^{1002} \approx 10^{435}$ dépasse la capacité de représentation des float64 ($\approx 10^{308}$). Avec centrage par $c = 1002$: $z' = (-2, -1, 0)$ et $e^0 = 1$, $e^{-1} \approx 0.37$, $e^{-2} \approx 0.14$ sont tous calculables. Cette astuce est **essentielle** en pratique.

8.4 Perte (NLL) : de la vraisemblance à la cross-entropie catégorielle

Intuition

Nous avons maintenant un modèle qui prédit un vecteur de probabilités $a = (a_1, \ldots, a_K)$ pour chaque exemple x. Comment choisir les paramètres (W, b) qui donnent les meilleures prédictions ?
Comme dans le cas binaire, nous allons utiliser le **principe du maximum de vraisemblance** : choisir les paramètres qui rendent les observations réelles les plus probables.
Rappel du cas binaire : Pour un exemple avec vraie classe $y \in \{0, 1\}$, la probabilité que le modèle attribue à cette observation était $a^y(1-a)^{1-y}$. En multiclasse, l'idée est similaire mais généralisée.

Reprenons notre exemple des trois primates. Supposons que nous observons un squelette avec mesures $x^{(i)}$ et que la vraie espèce est "gorille" (classe $k = 2$). Notre modèle calcule :

$$a_1^{(i)} = 0.10 \quad \text{(probabilité prédite pour chimpanzé)} \tag{203}$$

$$a_2^{(i)} = 0.75 \quad \text{(probabilité prédite pour gorille)} \tag{204}$$

$$a_3^{(i)} = 0.15 \quad \text{(probabilité prédite pour orang-outan)} \tag{205}$$

Question : Quelle probabilité le modèle attribue-t-il à l'observation réelle (gorille) ?
Réponse : C'est simplement $a_2^{(i)} = 0.75$, la probabilité de la classe vraie.
Plus généralement, si la vraie classe est k, la probabilité que le modèle attribue à cette observation est a_k. Pour m exemples indépendants, la vraisemblance est :

$$\mathcal{L}(W, b) = \prod_{i=1}^{m} a_{y^{(i)}}^{(i)} \tag{206}$$

où $y^{(i)} \in \{1, \ldots, K\}$ est la vraie classe de l'exemple i.
Représentation one-hot : Pour faciliter les calculs (surtout les dérivées), on encode la vraie classe $y^{(i)}$ en vecteur **one-hot** :

$$y^{(i)} = \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \\ \vdots \\ 0 \end{pmatrix} \leftarrow \text{la composante de la vraie classe vaut 1, les autres 0} \tag{207}$$

Par exemple, si l'exemple i est un gorille (classe 2) parmi 3 classes, on a $y^{(i)} = (0, 1, 0)^T$.
Astuce mathématique : Avec le codage one-hot, on peut écrire :

$$a_{y^{(i)}}^{(i)} = \prod_{k=1}^{K} \left(a_k^{(i)}\right)^{y_k^{(i)}} \tag{208}$$

Vérification : Si $y^{(i)} = (0, 1, 0)^T$, alors :

$$\prod_{k=1}^{3} \left(a_k^{(i)}\right)^{y_k^{(i)}} = (a_1^{(i)})^0 \cdot (a_2^{(i)})^1 \cdot (a_3^{(i)})^0 = a_2^{(i)} \quad \checkmark \tag{209}$$

Seul le terme correspondant à la vraie classe "survit" !

Définition 8.2 (Log-vraisemblance et fonction de perte)**.** Avec K classes, $W \in \mathbb{R}^{K\times n}$, $b \in \mathbb{R}^{K\times 1}$, $z = Wx + b$, $a = \text{softmax}(z)$. Pour m exemples :

Vraisemblance :

$$\mathcal{L}(W, b) = \prod_{i=1}^{m}\prod_{k=1}^{K}\left(a_k^{(i)}\right)^{y_k^{(i)}} \tag{210}$$

Log-vraisemblance :

$$\ell(W, b) = \log \mathcal{L}(W, b) = \sum_{i=1}^{m}\sum_{k=1}^{K} y_k^{(i)} \log a_k^{(i)} \tag{211}$$

Fonction de perte (Negative Log-Likelihood - NLL) :

$$L(W, b) = -\frac{1}{m}\ell(W, b) = -\frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{K} y_k^{(i)} \log a_k^{(i)} \tag{212}$$

Cette perte s'appelle la **cross-entropie catégorielle** (categorical cross-entropy).

Notation matricielle : Pour $X \in \mathbb{R}^{n\times m}$, $Z = WX + b \in \mathbb{R}^{K\times m}$, $A = \text{softmax}(Z)$ (colonne par colonne), et $Y \in \{0, 1\}^{K\times m}$ (one-hot) :

$$\ell(W, b) = \sum_{i=1}^{m}\sum_{k=1}^{K} Y_{ki} \log A_{ki}, \qquad L(W, b) = -\frac{1}{m}\ell(W, b)$$

Exemple 8.2 (Calcul explicite avec 3 classes)**.** Supposons $K = 3$ classes et $m = 2$ exemples :

- Exemple 1 : vraie classe = 2 (gorille), $y^{(1)} = (0, 1, 0)^T$
- Exemple 2 : vraie classe = 1 (chimpanzé), $y^{(2)} = (1, 0, 0)^T$

Prédictions du modèle :

- Exemple 1 : $a^{(1)} = (0.10, 0.75, 0.15)^T$
- Exemple 2 : $a^{(2)} = (0.80, 0.15, 0.05)^T$

Calcul de la log-vraisemblance :

$$\begin{aligned}
\ell(W, b) &= \sum_{i=1}^{2}\sum_{k=1}^{3} y_k^{(i)} \log a_k^{(i)} && (213)\\
&= [0 \cdot \log(0.10) + 1 \cdot \log(0.75) + 0 \cdot \log(0.15)] && (214)\\
&\quad + [1 \cdot \log(0.80) + 0 \cdot \log(0.15) + 0 \cdot \log(0.05)] && (215)\\
&= \log(0.75) + \log(0.80) && (216)\\
&\approx -0.288 + (-0.223) = -0.511 && (217)
\end{aligned}$$

Fonction de perte :

$$L(W, b) = -\frac{1}{2} \times (-0.511) = 0.256 \tag{218}$$

Interprétation : Seules les probabilités prédites pour les *vraies classes* comptent dans la perte ! Le modèle est pénalisé si $a_k^{(i)}$ est faible quand $y_k^{(i)} = 1$.

Justification Mathématique

[Pourquoi cette fonction de perte ?] 1. Principe du maximum de vraisemblance
Comme dans le cas binaire, maximiser la log-vraisemblance $\ell(W, b)$ revient à minimiser $L(W, b) = -\ell(W, b)/m$. Les paramètres optimaux sont ceux qui rendent les données observées les plus probables.
2. Optimalité bayésienne : minimiser la divergence de Kullback-Leibler
En théorie de l'information, la **divergence de Kullback-Leibler** (KL) mesure la "distance" entre deux distributions de probabilités. Pour une entrée x donnée, soit :

— $p(k|x)$: la vraie distribution de probabilité (inconnue en pratique, mais encodée dans les données)

— $q(k|x) = a_k(x)$: la distribution prédite par notre modèle

La divergence KL est définie par :

$$\mathrm{KL}(p\|q) = \sum_{k=1}^{K} p(k|x) \log \frac{p(k|x)}{q(k|x)} = \sum_{k=1}^{K} p(k|x) \log p(k|x) - \sum_{k=1}^{K} p(k|x) \log q(k|x) \tag{219}$$

Le premier terme $H(p) = -\sum_k p(k|x) \log p(k|x)$ est l'**entropie** de la vraie distribution (constante, indépendante du modèle).
Le second terme $-\sum_k p(k|x) \log q(k|x)$ est la **cross-entropie** entre p et q.
Donc :

$$\mathrm{KL}(p\|q) = H(p) + \mathbb{E}_{k\sim p}[-\log q(k|x)] \tag{220}$$

Minimiser $\mathrm{KL}(p\|q)$ revient à minimiser la cross-entropie (puisque $H(p)$ est constante).
3. "Proper scoring rule"
La cross-entropie a une propriété remarquable : elle est **minimale uniquement quand** $q = p$. Autrement dit, si on sait que la vraie distribution est p, la meilleure prédiction (au sens de minimiser la cross-entropie) est de prédire exactement $q = p$. Cette propriété garantit que le modèle est incité à prédire les **vraies probabilités**, pas juste la classe la plus probable. C'est crucial pour avoir des prédictions bien calibrées (utile en médecine, finance, etc.).
4. Lien avec la théorie de l'information
La quantité $-\log a_k$ peut être interprétée comme la "surprise" ou le "coût de codage" pour encoder la classe k si le modèle prédit a_k. Plus a_k est proche de 0, plus la surprise est grande ($-\log(0.01) \approx 4.6$ bits vs $-\log(0.99) \approx 0.01$ bits).
La cross-entropie est donc l'espérance de cette surprise sur toutes les observations.

Attention

[Cas particuliers et interprétation] Prédiction parfaite : Si pour chaque exemple i, le modèle prédit $a^{(i)}_{y^{(i)}} = 1$ (probabilité 1 sur la vraie classe), alors :

$$L = -\frac{1}{m}\sum_{i=1}^{m}\log(1) = 0 \tag{221}$$

La perte est nulle ! C'est le minimum absolu (impossible en pratique sur de vraies données).
Prédiction uniforme : Si le modèle prédit $a_k = 1/K$ pour toutes les classes (hasard complet), alors :

$$L = -\frac{1}{m}\sum_{i=1}^{m}\log(1/K) = \log(K) \tag{222}$$

Pour $K = 3$: $L \approx 1.10$. C'est la perte d'un modèle qui tire au hasard.
Mauvaise prédiction : Si le modèle prédit $a^{(i)}_{y^{(i)}} \to 0$ (probabilité très faible sur la vraie classe), alors $-\log(a^{(i)}_{y^{(i)}}) \to +\infty$. La perte explose ! Le modèle est fortement pénalisé pour les erreurs de confiance.
Comparaison avec l'accuracy : L'accuracy (pourcentage de prédictions correctes) ne regarde que si $\arg\max_k a_k = y$. La cross-entropie est plus exigeante : elle pénalise aussi les prédictions correctes mais peu confiantes (par exemple, prédire $(0.4, 0.35, 0.25)$ quand la vraie classe est 1).

8.5 Gradient clef : $dZ = A - Y$ (analogie parfaite avec le binaire)

Intuition

Dans le cas binaire (sigmoïde + binary cross-entropy), nous avions obtenu un résultat remarquablement simple pour le gradient :

$$\frac{\partial L}{\partial z} = a - y$$

C'est-à-dire : le gradient est simplement l'erreur de prédiction (différence entre la prédiction a et la vraie valeur y).
Miracle : Avec la softmax et la cross-entropie catégorielle, nous obtenons *exactement le même résultat* en multiclasse !

$$\frac{\partial L}{\partial z} = a - y$$

Cette simplicité n'est pas un hasard : c'est le résultat de la **compatibilité** entre la fonction d'activation (softmax) et la fonction de perte (cross-entropie). Ces deux fonctions sont conçues pour "bien travailler ensemble".

Ce résultat simple a plusieurs conséquences pratiques :

— **Efficacité computationnelle** : Pas besoin de calculer des dérivées complexes, juste une soustraction !

— **Stabilité numérique** : Pas de divisions risquées ou d'exponentielles dans le gradient
— **Interprétation claire** : Le gradient pointe dans la direction qui réduit l'erreur de prédiction
— **Généralité** : La formule fonctionne pour tout $K \geq 2$

Théorème 8.1 (Gradient de la NLL pour softmax)**.** Pour un exemple $z \mapsto a = \text{softmax}(z)$ et cible one-hot $y \in \{0,1\}^K$, la dérivée de la perte par rapport aux scores z est :

$$\frac{\partial}{\partial z}\Big(-\sum_{k=1}^{K} y_k \log a_k\Big) = a - y$$

Forme vectorisée pour m exemples :

$$dZ^{[L]} = A^{[L]} - Y, \quad dW^{[L]} = \frac{1}{m} dZ^{[L]}(A^{[L-1]})^T, \quad db^{[L]} = \frac{1}{m} dZ^{[L]}\mathbf{1}$$

où $Z^{[L]} \in \mathbb{R}^{K\times m}$, $A^{[L]} \in \mathbb{R}^{K\times m}$, et $Y \in \{0,1\}^{K\times m}$.

Avant de démontrer ce théorème, nous devons comprendre la **jacobienne de la softmax**, car la fonction softmax est vectorielle : elle prend un vecteur $z \in \mathbb{R}^K$ et retourne un vecteur $a \in \mathbb{R}^K$.

Définition 8.3 (Jacobienne de la softmax)**.** La softmax est une fonction $\mathbb{R}^K \to \mathbb{R}^K$ définie par :

$$a_i = \text{softmax}_i(z) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

La **jacobienne** de la softmax est la matrice $J \in \mathbb{R}^{K\times K}$ des dérivées partielles :

$$J_{ij} = \frac{\partial a_i}{\partial z_j}$$

Chaque composante a_i dépend de *tous* les z_j (à cause du dénominateur commun), donc la jacobienne n'est pas diagonale.

Proposition 8.2 (Calcul de la jacobienne de la softmax)**.** Pour $i, j \in \{1, \ldots, K\}$:

$$\frac{\partial a_i}{\partial z_j} = a_i(\delta_{ij} - a_j) = \begin{cases} a_i(1-a_i) & \text{si } i = j \\ -a_i a_j & \text{si } i \neq j \end{cases}$$

où δ_{ij} est le symbole de Kronecker ($\delta_{ij} = 1$ si $i = j$, 0 sinon).
Forme matricielle : $J = \text{diag}(a) - aa^T$

Démonstration de la jacobienne. Rappel : $a_i = \frac{e^{z_i}}{\sum_{\ell=1}^{K} e^{z_\ell}}$
Notons $S = \sum_{\ell=1}^{K} e^{z_\ell}$ le dénominateur commun. Alors $a_i = \frac{e^{z_i}}{S}$.
Cas 1 : Dérivée diagonale ($i = j$)
Calculons $\frac{\partial a_i}{\partial z_i}$ en utilisant la règle du quotient :

$$\frac{\partial a_i}{\partial z_i} = \frac{\partial}{\partial z_i}\left(\frac{e^{z_i}}{S}\right) \tag{223}$$

$$= \frac{e^{z_i} \cdot S - e^{z_i} \cdot \frac{\partial S}{\partial z_i}}{S^2} \quad \text{(règle du quotient)} \tag{224}$$

Calculons $\frac{\partial S}{\partial z_i}$:

$$\frac{\partial S}{\partial z_i} = \frac{\partial}{\partial z_i}\left(\sum_{\ell=1}^{K} e^{z_\ell}\right) = e^{z_i} \tag{225}$$

Donc :

$$\begin{aligned}
\frac{\partial a_i}{\partial z_i} &= \frac{e^{z_i} \cdot S - e^{z_i} \cdot e^{z_i}}{S^2} && (226)\\
&= \frac{e^{z_i}(S - e^{z_i})}{S^2} && (227)\\
&= \frac{e^{z_i}}{S} \cdot \frac{S - e^{z_i}}{S} && (228)\\
&= a_i \cdot \left(1 - \frac{e^{z_i}}{S}\right) && (229)\\
&= a_i(1 - a_i) \quad \checkmark && (230)
\end{aligned}$$

Cas 2 : Dérivée hors-diagonale ($i \neq j$)
Calculons $\frac{\partial a_i}{\partial z_j}$ pour $i \neq j$:

$$\begin{aligned}
\frac{\partial a_i}{\partial z_j} &= \frac{\partial}{\partial z_j}\left(\frac{e^{z_i}}{S}\right) && (231)\\
&= \frac{0 \cdot S - e^{z_i} \cdot \frac{\partial S}{\partial z_j}}{S^2} \quad (e^{z_i} \text{ ne dépend pas de } z_j) && (232)\\
&= \frac{-e^{z_i} \cdot e^{z_j}}{S^2} && (233)\\
&= -\frac{e^{z_i}}{S} \cdot \frac{e^{z_j}}{S} && (234)\\
&= -a_i a_j \quad \checkmark && (235)
\end{aligned}$$

Forme matricielle :
La matrice jacobienne J a pour coefficients :

$$J_{ij} = a_i \delta_{ij} - a_i a_j = a_i(\delta_{ij} - a_j)$$

On peut écrire $J = \text{diag}(a) - aa^T$ où :

— $\text{diag}(a)$ est la matrice diagonale avec a sur la diagonale

— aa^T est le produit externe de a avec lui-même (matrice de rang 1)

□

Exemple 8.3 (Jacobienne numérique pour $K = 3$)**.** Soit $z = (1, 2, 0.5)$ donnant $a \approx (0.186, 0.506, 0.308)^T$ après softmax.

Termes diagonaux :

$$\begin{aligned}
J_{11} &= a_1(1 - a_1) = 0.186 \times (1 - 0.186) \approx 0.151 && (236)\\
J_{22} &= a_2(1 - a_2) = 0.506 \times (1 - 0.506) \approx 0.250 && (237)\\
J_{33} &= a_3(1 - a_3) = 0.308 \times (1 - 0.308) \approx 0.213 && (238)
\end{aligned}$$

Termes hors-diagonale :

$$J_{12} = J_{21} = -a_1 a_2 = -0.186 \times 0.506 \approx -0.094 \tag{239}$$
$$J_{13} = J_{31} = -a_1 a_3 = -0.186 \times 0.308 \approx -0.057 \tag{240}$$
$$J_{23} = J_{32} = -a_2 a_3 = -0.506 \times 0.308 \approx -0.156 \tag{241}$$

Matrice complète :

$$J \approx \begin{pmatrix} 0.151 & -0.094 & -0.057 \\ -0.094 & 0.250 & -0.156 \\ -0.057 & -0.156 & 0.213 \end{pmatrix}$$

Observation : La jacobienne est symétrique ($J = J^T$) !

Nous avons maintenant tous les outils pour démontrer le théorème principal.

Démonstration du théorème : $\frac{\partial L}{\partial z} = a - y$. Pour un exemple, la perte est :

$$L = -\sum_{k=1}^{K} y_k \log a_k$$

où $y \in \{0,1\}^K$ est one-hot (un seul $y_t = 1$, les autres sont 0).
Étape 1 : Dérivée de L par rapport à a
Calculons $\frac{\partial L}{\partial a_i}$:

$$\frac{\partial L}{\partial a_i} = \frac{\partial}{\partial a_i}\left(-\sum_{k=1}^{K} y_k \log a_k\right) \tag{242}$$
$$= -y_i \cdot \frac{1}{a_i} \tag{243}$$
$$= -\frac{y_i}{a_i} \tag{244}$$

En notation vectorielle : $\frac{\partial L}{\partial a} = -\frac{y}{a}$ (division élément par élément).
Étape 2 : Règle de la chaîne
Par la règle de la chaîne multivariée :

$$\frac{\partial L}{\partial z_j} = \sum_{i=1}^{K} \frac{\partial L}{\partial a_i} \cdot \frac{\partial a_i}{\partial z_j} = \sum_{i=1}^{K} \left(-\frac{y_i}{a_i}\right) \cdot J_{ij} \tag{245}$$

En notation matricielle : $\frac{\partial L}{\partial z} = J^T \frac{\partial L}{\partial a}$
Étape 3 : Substitution

$$\frac{\partial L}{\partial z_j} = \sum_{i=1}^{K} \left(-\frac{y_i}{a_i} \right) \cdot a_i(\delta_{ij} - a_j) \tag{246}$$

$$= \sum_{i=1}^{K} (-y_i)(\delta_{ij} - a_j) \tag{247}$$

$$= \sum_{i=1}^{K} (-y_i \delta_{ij} + y_i a_j) \tag{248}$$

$$= -y_j + a_j \sum_{i=1}^{K} y_i \tag{249}$$

$$= -y_j + a_j \cdot 1 \quad (\text{car } y \text{ est one-hot : } \sum_{i=1}^{K} y_i = 1) \tag{250}$$

$$= a_j - y_j \quad \checkmark \tag{251}$$

En notation vectorielle : $\frac{\partial L}{\partial z} = a - y$
Méthode alternative (plus directe pour comprendre) :
Soit t l'indice de la vraie classe (tel que $y_t = 1$, et $y_k = 0$ pour $k \neq t$).
La perte devient :

$$L = -\sum_{k=1}^{K} y_k \log a_k = -y_t \log a_t = -\log a_t$$

Calculons $\frac{\partial L}{\partial z_j} = \frac{\partial(-\log a_t)}{\partial z_j}$:
Par la règle de la chaîne :

$$\frac{\partial(-\log a_t)}{\partial z_j} = -\frac{1}{a_t} \cdot \frac{\partial a_t}{\partial z_j}$$

Cas 1 : Si $j = t$ (dérivée par rapport au score de la vraie classe) :

$$\frac{\partial L}{\partial z_t} = -\frac{1}{a_t} \cdot a_t(1 - a_t) = -(1 - a_t) = a_t - 1 = a_t - y_t \quad \checkmark \tag{252}$$

Cas 2 : Si $j \neq t$ (dérivée par rapport au score d'une autre classe) :

$$\frac{\partial L}{\partial z_j} = -\frac{1}{a_t} \cdot (-a_t a_j) = a_j = a_j - 0 = a_j - y_j \quad \checkmark \tag{253}$$

Dans les deux cas : $\frac{\partial L}{\partial z_j} = a_j - y_j$ □

Exemple 8.4 (Calcul numérique du gradient)**.** Soit $K = 3$ classes, vraie classe $t = 2$ (gorille), donc $y = (0, 1, 0)^T$.
Prédictions : $a = (0.15, 0.70, 0.15)^T$
Calcul du gradient :

$$\frac{\partial L}{\partial z} = a - y = \begin{pmatrix} 0.15 \\ 0.70 \\ 0.15 \end{pmatrix} - \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0.15 \\ -0.30 \\ 0.15 \end{pmatrix} \tag{254}$$

Interprétation :

— $\frac{\partial L}{\partial z_1} = 0.15 > 0$: diminuer z_1 (chimpanzé) réduirait la perte
— $\frac{\partial L}{\partial z_2} = -0.30 < 0$: augmenter z_2 (gorille, vraie classe) réduirait la perte
— $\frac{\partial L}{\partial z_3} = 0.15 > 0$: diminuer z_3 (orang-outan) réduirait la perte

Le gradient pousse à augmenter le score de la vraie classe et à diminuer les scores des autres classes !

Attention

[Vectorisation pour m exemples] Pour m exemples en parallèle :

— $Z^{[L]} \in \mathbb{R}^{K\times m}$: scores pour toutes les classes et tous les exemples
— $A^{[L]} = \text{softmax}(Z^{[L]}) \in \mathbb{R}^{K\times m}$: probabilités (softmax colonne par colonne)
— $Y \in \{0,1\}^{K\times m}$: vraies classes en one-hot

Le gradient vectorisé est simplement :

$$dZ^{[L]} = A^{[L]} - Y \in \mathbb{R}^{K\times m}$$

Chaque colonne i contient le gradient pour l'exemple i : $(dZ^{[L]})_{:,i} = a^{(i)} - y^{(i)}$
Les gradients des poids et biais s'obtiennent ensuite par :

$$dW^{[L]} = \frac{1}{m} dZ^{[L]} (A^{[L-1]})^T \in \mathbb{R}^{K\times n^{[L-1]}} \tag{255}$$

$$db^{[L]} = \frac{1}{m} dZ^{[L]} \mathbf{1} \in \mathbb{R}^{K\times 1} \tag{256}$$

où $\mathbf{1} \in \mathbb{R}^{m\times 1}$ est le vecteur de uns (somme sur les exemples).

Remarque 8.1 (Réduction binaire K=2)**.** $a_1 = \dfrac{e^{z_1}}{e^{z_1}+e^{z_2}} = \dfrac{1}{1+e^{-(z_1-z_2)}} = \sigma\big((w_1 - w_2)^T x + (b_1 - b_2)\big)$. Le modèle softmax se réduit exactement à la régression logistique binaire sur la différence de logits.

8.6 Géométrie des décisions : $K-1$ hyperplans

La règle $\hat{y} = \arg\max_k a_k = \arg\max_k z_k$ (softmax est croissante en chaque composante) induit des frontières $z_k = z_j$:

$$(w_k - w_j)^T x + (b_k - b_j) = 0.$$

Les régions de décision sont donc délimitées par des **hyperplans affines** (droites dans $\mathbb{R}^2$), comme en binaire, mais *en éventail* pour toutes les paires (k, j). Chaque frontière est exactement le lieu où les classes k et j sont à égalité de probabilité.

8.7 Identifiabilité, jauge et convexité (modèle linéaire)

Attention

[Jauge (invariance par translation)] Comme a est invariant à $z \mapsto z + c\mathbf{1}$, les paramètres (W, b) ne sont pas identifiables au décalage près. On *fixe une jauge* (par ex. $w_K = 0$, $b_K = 0$) ou $\sum_k w_k = 0$, $\sum_k b_k = 0$.

Proposition 8.3 (Convexité)**.** Dans la *régression softmax* (sortie linéaire $z = Wx + b$), $L(W, b)$ est convexe en (W, b). Le Hessien est somme (sur les exemples) de termes $X(\mathrm{diag}(a) - aa^T)X^T$, positifs semi-définis.

8.8 Implémentation stable et vectorisée

Proposition 8.4 (Formules vectorisées (K classes, m exemples))**.**

$$Z^{[L]} = W^{[L]}A^{[L-1]} + b^{[L]}, \quad A^{[L]} = \mathrm{softmax}(Z^{[L]}) \text{ (colonne par colonne, centrée)},$$

$$L = -\frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{K} Y_{ki}\log A_{ki}, \qquad dZ^{[L]} = A^{[L]} - Y, \quad dW^{[L]} = \frac{1}{m}dZ^{[L]}(A^{[L-1]})^T, \quad db^{[L]} = \frac{1}{m}dZ^{[L]}\mathbf{1}.$$

Attention

[Vérification des dimensions] $W^{[L]} \in \mathbb{R}^{K\times n^{[L-1]}}$, $b^{[L]} \in \mathbb{R}^{K\times 1}$, $Z^{[L]}, A^{[L]}, Y \in \mathbb{R}^{K\times m}$, $dW^{[L]}$ a la même taille que $W^{[L]}$, $db^{[L]}$ la même que $b^{[L]}$.

8.9 Exemples numériques détaillés (mêmes notations que ci-dessus)

Exemple 1 (un seul exemple). Prenons K=3, n=2,

$$W = \begin{pmatrix} 0.4 & 0.1 \\ -0.2 & 0.3 \\ 0.05 & -0.4 \end{pmatrix}, \quad b = \begin{pmatrix} 0.3 \\ 0.1 \\ -0.2 \end{pmatrix}, \quad x = \begin{pmatrix} 3.2 \\ 4.1 \end{pmatrix}.$$

Alors

$$z = Wx + b = \begin{pmatrix} 1.99 \\ 0.69 \\ -1.68 \end{pmatrix}, \qquad a = \mathrm{softmax}(z) \approx \begin{pmatrix} 0.7704 \\ 0.2100 \\ 0.0196 \end{pmatrix}.$$

Si la classe vraie est $t = 2$ (i.e. $y = (0, 1, 0)^T$),

$$L^{(1)} = -\log a_2 \approx 1.5608, \qquad dZ = a - y \approx \begin{pmatrix} 0.7704 \\ -0.7900 \\ 0.0196 \end{pmatrix},$$

$$dW = dZ\, x^T \approx \begin{pmatrix} 2.4653 & 3.1587 \\ -2.5281 & -3.2392 \\ 0.0628 & 0.0805 \end{pmatrix}, \quad db = dZ.$$

On retrouve la structure **erreur** $\times$ **entrée** (comme en binaire), mais pour chaque classe.

Exemple 2 (mini-batch à $m = 2$). Avec un second exemple $x' = (3.6, 3.7)^T$ dont la classe vraie est $t' = 1$, on obtient (avec la moyenne sur le batch)

$$A \approx \begin{pmatrix} 0.7704 & 0.8164 \\ 0.2100 & 0.1616 \\ 0.0196 & 0.0221 \end{pmatrix}, \quad L_{\text{batch}} \approx 0.8819,$$

$$dW \approx \begin{pmatrix} 0.9021 & 1.2396 \\ -0.9733 & -1.3207 \\ 0.0712 & 0.0811 \end{pmatrix}, \qquad db \approx \begin{pmatrix} 0.2934 \\ -0.3142 \\ 0.0209 \end{pmatrix}.$$

8.10 Application au dataset des primates : visualisation des frontières

Appliquons maintenant l'algorithme complet de classification multiclasse avec softmax à notre dataset des trois espèces de primates (chimpanzé, gorille, orang-outan) introduit au début de cette section.

Intuition

Nous avons maintenant tous les outils nécessaires :

- **Modèle** : Un réseau avec softmax en sortie ($K = 3$ classes)
- **Fonction de perte** : Cross-entropie catégorielle
- **Optimisation** : Descente de gradient avec backpropagation
- **Gradient simple** : $dZ^{[L]} = A^{[L]} - Y$

Rappelons que notre dataset contient les mesures de longueur d'humérus (x_1) et de radius (x_2) pour les trois espèces. Après entraînement d'un réseau multicouches avec sortie softmax, nous obtenons les frontières de décision suivantes :

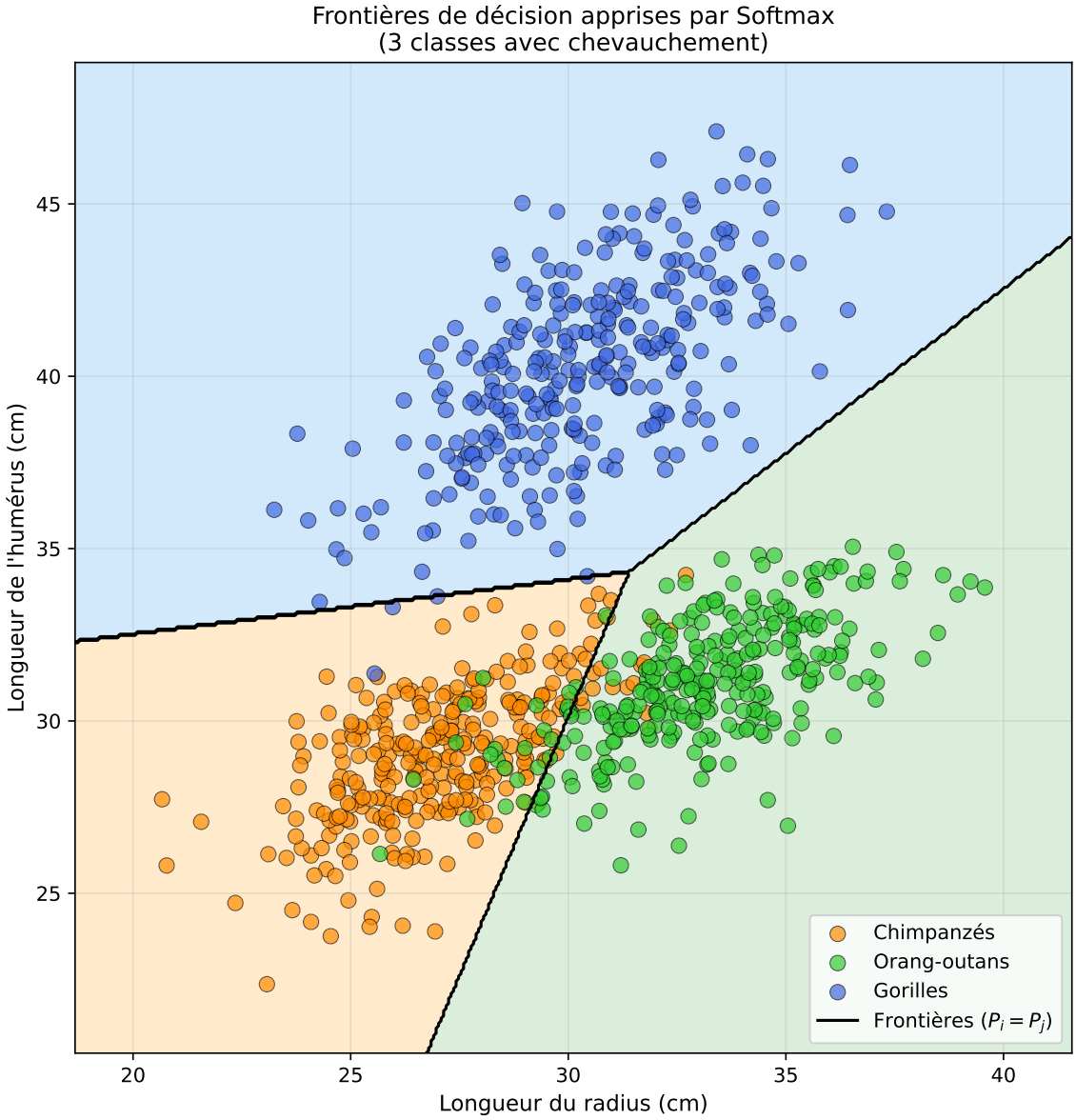

Figure 6 : Frontières de décision apprises par le classificateur softmax sur le dataset des primates. Les trois régions colorées correspondent aux prédictions du modèle : bleu pour chimpanzé, rouge pour gorille, vert pour orang-outan. Les frontières sont déterminées par les lieux où deux classes ont la même probabilité prédite.

8.11 Déséquilibre de classes, régularisation, lissage d'étiquettes

Dans la pratique, les données réelles posent souvent des défis spécifiques qui nécessitent d'adapter la fonction de perte standard. Nous présentons ici trois techniques importantes pour améliorer la robustesse et la généralisation des modèles.

8.11.1 Pondération de classes : gérer le déséquilibre

Intuition

Imaginons un problème de détection de maladies rares où :

- Classe 0 (sain) : 9500 exemples (95%)
- Classe 1 (malade type A) : 400 exemples (4%)
- Classe 2 (malade type B) : 100 exemples (1%)

Un modèle "paresseux" pourrait apprendre à prédire systématiquement "sain" et obtenir 95% d'accuracy, sans jamais détecter les malades ! Ce n'est évidemment pas acceptable.
Problème : La fonction de perte standard traite tous les exemples de manière égale. Les 9500 exemples "sain" dominent complètement l'apprentissage, et le modèle n'est pas suffisamment incité à bien classer les classes rares.
Solution : Pondérer la perte pour donner plus d'importance aux classes rares.

Définition 8.4 (Pondération de classes (class weighting))**.** Pour gérer des fréquences de classes très inégales, on introduit des poids $\alpha_k > 0$ pour chaque classe k dans la fonction de perte :

$$L_{\text{pond}}(W, b) = -\frac{1}{m} \sum_{i=1}^{m} \sum_{k=1}^{K} \alpha_k Y_{ki} \log A_{ki}$$

Choix typique des poids : Inversement proportionnel à la fréquence

$$\alpha_k = \frac{m}{K \cdot n_k}$$

où n_k est le nombre d'exemples de la classe k.

Gradient modifié : Pour l'exemple i de classe vraie $t^{(i)}$:

$$dZ^{(i)} = \alpha_{t^{(i)}}(a^{(i)} - y^{(i)})$$

Exemple 8.5 (Calcul des poids pour données déséquilibrées)**.** Reprenons notre exemple médical avec $m = 10000$ exemples et $K = 3$ classes :

- Classe 0 : $n_0 = 9500$ exemples
- Classe 1 : $n_1 = 400$ exemples
- Classe 2 : $n_2 = 100$ exemples

Calcul des poids :

$$\alpha_0 = \frac{10000}{3 \times 9500} \approx 0.35 \tag{257}$$

$$\alpha_1 = \frac{10000}{3 \times 400} \approx 8.33 \tag{258}$$

$$\alpha_2 = \frac{10000}{3 \times 100} \approx 33.33 \tag{259}$$

Interprétation :

- Une erreur sur un exemple de classe 2 (la plus rare) pèse $\frac{33.33}{0.35} \approx 95$ fois plus qu'une erreur sur la classe 0 !

— Le modèle est maintenant fortement incité à bien classer les classes rares

— Les poids "rééquilibrent" artificiellement l'importance des classes

Impact sur le gradient : Si le modèle prédit mal un exemple de classe 2, le gradient sera multiplié par $\alpha_2 \approx 33.33$, poussant le modèle à corriger fortement son erreur.

Justification Mathématique

Pourquoi cette pondération fonctionne-t-elle ?
La perte pondérée peut s'interpréter comme une vraisemblance sur un ensemble "rééchantillonné" où chaque classe apparaîtrait avec la même fréquence. En pratique, cela revient à :

— Sur-échantillonner les classes rares (les compter plusieurs fois)

— Sous-échantillonner les classes fréquentes (en ignorer certains exemples)

Mais au lieu de modifier physiquement le dataset (ce qui poserait des problèmes de mémoire et de variance), on modifie simplement les gradients via les poids α_k.
Variante : poids focaux (focal loss) Pour les déséquilibres extrêmes, on peut utiliser une variante plus sophistiquée qui pénalise davantage les exemples "difficiles" (faible probabilité prédite sur la vraie classe).

Attention

[Précautions avec la pondération] Surpondération : Des poids trop élevés peuvent rendre le modèle instable ou le faire sur-apprendre sur les classes rares.
Choix des poids : La formule $\alpha_k = m/(K \cdot n_k)$ est un bon point de départ, mais souvent on ajuste empiriquement (par validation croisée).
Métriques d'évaluation : Avec des classes déséquilibrées, l'accuracy globale est trompeuse. Préférer :

— Accuracy par classe (recall)

— F1-score (moyenne harmonique de précision et recall)

— Matrice de confusion

— AUC-ROC (aire sous la courbe ROC)

Nous reviendrons sur ces métriques dans le cours sur l'évaluation des modèles.

Remarque 8.2 (Régularisation)**.** En pratique, la softmax combinée à la cross-entropie peut pousser les poids vers des valeurs extrêmes pour produire des probabilités proches de 0 ou 1. Les techniques de régularisation (pénalité L2, dropout, etc.) permettent de contrôler ce phénomène. Elles seront traitées en détail dans la section 13.

8.11.2 Label smoothing : assouplir les cibles

Intuition

Avec la cross-entropie standard, les cibles sont "dures" : $y = (0, 0, 1, 0, \ldots)$ (one-hot). Le modèle est incité à pousser $a_k \to 1$ pour la vraie classe et $a_j \to 0$ pour les autres. Mais dans la softmax, cela nécessite que $z_k \to +\infty$ et $z_j \to -\infty$: les poids peuvent croître indéfiniment !

Problèmes :

- **Sur-confiance :** Le modèle devient trop confiant (prédit 0.999 alors que 0.7 suffirait)
- **Mauvaise calibration :** Les probabilités prédites ne reflètent pas l'incertitude réelle
- **Poids grands :** Nécessite de gros poids, risque de sur-apprentissage

Idée : Assouplir les cibles pour dire "je suis sûr que c'est la classe k, mais pas à 100%".

Définition 8.5 (Label smoothing / lissage d'étiquettes)**.** Pour un hyperparamètre $\varepsilon \in [0, 1)$ (typiquement $\varepsilon = 0.1$), on remplace les cibles one-hot Y par des cibles "lissées" $\tilde{Y}$:

$$\tilde{Y} = (1 - \varepsilon)Y + \frac{\varepsilon}{K}\mathbf{1}\mathbf{1}^T$$

où $\mathbf{1}\mathbf{1}^T \in \mathbb{R}^{K \times m}$ est la matrice de uns.

Pour un exemple avec vraie classe t :

$$\tilde{y}_k = \begin{cases} 1 - \varepsilon + \frac{\varepsilon}{K} = 1 - \varepsilon\frac{K-1}{K} & \text{si } k = t \\ \frac{\varepsilon}{K} & \text{si } k \neq t \end{cases}$$

Gradient modifié :

$$dZ = A - \tilde{Y}$$

Le lissage évite des cibles trop dures, améliore la généralisation et surtout la **calibration** (probabilités prédites plus fiables).

Exemple 8.6 (Calcul avec label smoothing)**.** Soit $K = 3$ classes, vraie classe $t = 2$, et $\varepsilon = 0.1$.

Sans lissage : $y = (0, 1, 0)^T$

Avec lissage :

$$\tilde{y}_1 = \frac{0.1}{3} \approx 0.033 \tag{260}$$

$$\tilde{y}_2 = 1 - 0.1 + \frac{0.1}{3} = 0.9 + 0.033 \approx 0.933 \tag{261}$$

$$\tilde{y}_3 = \frac{0.1}{3} \approx 0.033 \tag{262}$$

Vérification : $0.033 + 0.933 + 0.033 = 1.0$ ✓

Impact sur le gradient : Si le modèle prédit $a = (0.05, 0.85, 0.10)$:

$$\text{Sans lissage :} \quad dZ = (0.05, -0.15, 0.10) \tag{263}$$

$$\text{Avec lissage :} \quad dZ = (0.05 - 0.033, 0.85 - 0.933, 0.10 - 0.033) = (0.017, -0.083, 0.067) \tag{264}$$

Le gradient sur la vraie classe est moins fort (en valeur absolue), le modèle n'est pas poussé aussi agressivement vers la cible "parfaite". On remarque que les optimisations évitant le surapprentissage ont souvent cet effet d'atténuation des gradients extrêmes et de lissage (léger) des prédictions.

Justification Mathématique

Pourquoi le label smoothing améliore-t-il la généralisation ?

1. Régularisation implicite : Le lissage décourage les poids très grands (puisque $z_k \to \infty$ n'est plus nécessaire)
2. Pénalisation de la sur-confiance : Le modèle ne peut plus "tricher" en étant arbitrairement confiant
3. Meilleure calibration : Les probabilités prédites reflètent mieux l'incertitude réelle. Par exemple, si la vraie classe est ambiguë dans les données, le modèle peut prédire 0.8 au lieu de 0.99
4. Connexion avec la KL-divergence : Le lissage peut s'interpréter comme minimiser la KL-divergence vers une distribution uniforme, évitant les distributions trop "piquées"

Interprétation probabiliste : On peut voir ε comme modélisant le "bruit d'étiquetage" : avec probabilité ε, l'étiquette observée est incorrecte et est en fait une classe tirée uniformément au hasard.
Résultats empiriques : Le label smoothing bien appliqué améliore souvent :

- La performance en test (top-1 accuracy)
- La calibration (ECE - Expected Calibration Error)
- La robustesse aux adversarial examples

Attention

[Choix de ε et limitations] Choix typique : $\varepsilon = 0.1$ fonctionne bien pour beaucoup de problèmes.

- ε trop petit (< 0.01) : effet négligeable
- ε trop grand (> 0.3) : peut dégrader les performances (cibles trop floues)

Incompatibilité avec certaines métriques : Le label smoothing peut légèrement dégrader la log-likelihood sur l'ensemble de test (puisqu'on optimise pour des cibles lissées), mais améliore généralement l'accuracy et la calibration.
Variante adaptative : Certains travaux proposent d'adapter ε par classe ou par exemple (plus de lissage pour les classes ambiguës).
Combinaison : Le label smoothing peut être combiné avec la pondération de classes et la régularisation ℓ_2 pour un effet cumulatif.

8.12 Température et calibration

Intuition

La softmax standard ($T = 1$) peut produire des probabilités très "confiantes" : par exemple, $(0.95, 0.03, 0.02)$. Mais que se passe-t-il si nous voulons :

- Assouplir les prédictions pour avoir des probabilités moins extrêmes ?
- Accentuer les différences pour avoir des prédictions encore plus tranchées ?
- Calibrer le modèle après entraînement pour mieux refléter l'incertitude réelle ?

Le paramètre de température T permet de contrôler la "netteté" des distributions de probabilités sans modifier les poids du réseau.

Définition 8.6 (Softmax avec température)**.** La softmax paramétrée par une température $T > 0$ est définie par :

$$a_k(T) = \frac{e^{z_k/T}}{\sum_{j=1}^{K} e^{z_j/T}}, \quad T > 0$$

Cas particuliers :

- $T = 1$: softmax standard
- $T > 1$: distribution **adoucie** (plus uniforme, plus entropique)
- $T < 1$: distribution **piquée** (plus concentrée sur la classe maximale)
- $T \to 0^+$: limite vers one-hot (tout le poids sur $\arg\max_k z_k$)
- $T \to +\infty$: limite vers distribution uniforme ($a_k \to 1/K$)

Exemple 8.7 (Impact de la température sur les probabilités)**.** Soit $K = 3$ classes avec scores $z = (2, 5, 3)$. Calculons $a(T)$ pour différentes températures :

Température basse : $T = 0.5$ (distribution piquée)

$$z/T = (4, 10, 6) \tag{265}$$
$$e^{z/T} \approx (54.6, 22026, 403.4) \tag{266}$$
$$a(0.5) \approx (0.002, 0.976, 0.018) \tag{267}$$

La classe 2 domine encore plus (97.6% vs 84.4% avec $T = 1$) !

Température standard : $T = 1$

$$e^z \approx (7.4, 148.4, 20.1) \tag{268}$$
$$a(1) \approx (0.042, 0.844, 0.114) \tag{269}$$

Distribution de référence (softmax classique).

Température très élevée : $T = 10$ (presque uniforme)

$$z/T = (0.2, 0.5, 0.3) \tag{270}$$
$$e^{z/T} \approx (1.22, 1.65, 1.35) \tag{271}$$
$$a(10) \approx (0.289, 0.391, 0.320) \tag{272}$$

Les trois classes ont des probabilités proches (proche de $1/3 = 0.333$).

Visualisation de l'effet de T :

Pour $z = (2, 5, 3)$, voici comment évoluent les probabilités avec T :

T	a_1	a_2	a_3
0.1	0.000	1.000	0.000
0.5	0.002	0.976	0.018
1.0	0.042	0.844	0.114
2.0	0.139	0.630	0.232
5.0	0.259	0.453	0.288
10.0	0.289	0.391	0.320
∞	0.333	0.333	0.333

Justification Mathématique

Interprétation thermodynamique (physique statistique) :
La softmax avec température provient de la distribution de Boltzmann en mécanique statistique :

$$p_k \propto e^{-E_k/(k_B T)}$$

où E_k est l'énergie de l'état k et T la température physique.
En posant $E_k = -z_k$ (énergie négative = score), on retrouve la softmax. Dans ce contexte :

- Température élevée ($T \to \infty$) : système chaotique, états équiprobables (distribution uniforme)
- Température basse ($T \to 0$) : système gelé dans l'état de plus basse énergie (état d'énergie minimale, score maximal)

Lien avec l'entropie :
L'entropie de Shannon d'une distribution a est :

$$H(a) = -\sum_{k=1}^{K} a_k \log a_k$$

Plus T augmente, plus $H(a(T))$ augmente (distribution plus "incertaine"). L'entropie est maximale pour la distribution uniforme.

8.12.1 Application 1 : Calibration des probabilités

Temperature scaling (calibration post-entraînement) :

Après l'entraînement, on peut ajuster T sur un ensemble de validation pour améliorer la calibration :

1. Entraîner le modèle normalement avec $T = 1$
2. Sur l'ensemble de validation, chercher la température T^* qui minimise la NLL ou maximise la calibration
3. À l'inférence, utiliser T^* pour calculer les probabilités finales

Métrique de calibration : Expected Calibration Error (ECE)

$$\text{ECE} = \sum_{b=1}^{B} \frac{|B_b|}{N} \left|\text{acc}(B_b) - \text{conf}(B_b)\right| \tag{273}$$

où on regroupe les prédictions par bins de confiance. Un modèle parfaitement calibré a ECE = 0.

Exemple pratique : Pour un ResNet-50 sur ImageNet, on trouve souvent $T^* \approx 1.5$ après entraînement, ce qui améliore significativement la calibration sans changer l'accuracy.

8.12.2 Application 2 : Knowledge distillation (distillation de connaissances)

Intuition

La distillation consiste à entraîner un petit modèle (étudiant) à imiter un grand modèle (professeur) déjà entraîné. L'objectif : transférer les connaissances du gros modèle vers un modèle plus léger et rapide.
Pourquoi ne pas juste entraîner l'étudiant sur les étiquettes dures ?
Les probabilités du professeur contiennent plus d'information que les étiquettes one-hot ! Par exemple :

- Étiquette dure : chat $= (0, 0, 1, 0)$ (chat, chien, tigre, lion)
- Professeur : $(0.02, 0.03, 0.87, 0.08)$ révèle que "tigre" et "lion" sont proches de "chat", pas "chien"

Procédure de distillation avec température :

1. Phase d'entraînement du professeur : Entraîner le gros modèle normalement avec $T = 1$
2. Phase de distillation :
   - Choisir une température élevée $T_{\text{distill}} > 1$ (typiquement $T \in [2, 20]$)
   - Pour chaque exemple x :
     - Calculer les probabilités "douces" du professeur : $a_{\text{prof}}(T_{\text{distill}})$
     - Calculer les probabilités "douces" de l'étudiant : $a_{\text{stud}}(T_{\text{distill}})$
     - Minimiser la divergence KL entre les deux distributions douces
3. Perte de distillation :

$$L_{\text{distill}} = (1 - \lambda)L_{\text{hard}} + \lambda \cdot T^2 \cdot \text{KL}(a_{\text{prof}}(T) \| a_{\text{stud}}(T))$$

   où L_{hard} est la cross-entropie avec les vraies étiquettes, et $\lambda \in [0, 1]$ équilibre les deux termes. Le facteur T^2 compense la diminution des gradients à haute température.

Pourquoi utiliser une température élevée pour la distillation ?

Avec $T = 1$, les probabilités du professeur sont souvent très piquées : $(0.98, 0.01, 0.01)$. L'étudiant apprend principalement l'argmax, pas les nuances.

Avec $T = 5$, les probabilités deviennent : $(0.65, 0.20, 0.15)$. Les probabilités des "mauvaises" classes révèlent des similarités importantes (tigre proche de chat, chien moins proche).

Résultats typiques : Un petit modèle (étudiant) entraîné par distillation peut atteindre 95% de la performance d'un grand modèle (professeur) tout en étant 10× plus rapide et 10× plus léger.

8.12.3 Application 3 : Génération de texte et sampling

Dans les modèles de langage (GPT, etc.), on génère du texte en échantillonnant à chaque étape le prochain token selon la distribution softmax sur le vocabulaire.

Température basse ($T < 1$) : génération déterministe

— Tokens avec scores élevés deviennent encore plus probables
— Génération plus prévisible, cohérente, mais potentiellement répétitive
— Exemple : réponses factuelles, code, traductions

Température élevée ($T > 1$) : génération créative

— Même les tokens avec scores faibles ont une chance d'être choisis
— Génération plus diverse, créative, mais potentiellement incohérente
— Exemple : écriture créative, brainstorming, exploration

8.12.4 Application 4 : Exploration vs exploitation en RL

En apprentissage par renforcement, la softmax avec température est utilisée pour la politique d'action :

— T élevé (début d'apprentissage) : exploration, l'agent essaie des actions variées
— T faible (fin d'apprentissage) : exploitation, l'agent choisit les meilleures actions connues
— Annealing de température : T décroît progressivement pendant l'entraînement

Attention

[Propriétés importantes et limitations] Invariance de l'argmax : Changer T ne modifie **pas** la classe prédite $\arg\max_k a_k(T) = \arg\max_k z_k$ (indépendant de T). Donc l'accuracy ne change pas, seules les probabilités changent.
Gradients avec température : Si on entraîne avec $T \neq 1$, les gradients sont :

$$\frac{\partial L}{\partial z} = \frac{1}{T}(a(T) - y)$$

Avec $T > 1$, les gradients sont plus petits (apprentissage plus lent).
Limitations :

— La température est un hyperparamètre global (même T pour tous les exemples)
— Pour une calibration optimale, des méthodes plus sophistiquées existent (Platt scaling, isotonic regression, etc.)
— La température ne peut pas corriger des erreurs systématiques du modèle (seulement ajuster les probabilités)

8.13 Multi-classes *vs* multi-label : bien distinguer

Attention

[Quand *ne pas* utiliser softmax] Si plusieurs étiquettes peuvent être vraies simultanément (ex. pathologies présentes/absentes), on est en **multi-label**. Alors on utilise K *sigmoïdes* indépendantes et on somme des binary cross-entropies :

$$L_{\text{multi-label}} = -\frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{K}\big(Y_{ki}\log A_{ki} + (1 - Y_{ki})\log(1 - A_{ki})\big).$$

8.14 Algorithme complet (rappel) avec sortie softmax

Entraînement d'un réseau (sortie softmax, K classes)

Entrées : $X \in \mathbb{R}^{n^{[0]} \times m}$, $Y \in \{0,1\}^{K \times m}$, architecture $\{n^{[l]}\}_{l=1}^{L}$ (avec $n^{[L]} = K$), hyperparamètres α, T.
Initialisation : $W^{[l]} \sim \mathcal{N}(0, \sqrt{2/n^{[l-1]}})$, $b^{[l]} = 0$ (ou He/Xavier adaptés à l'activation des couches cachées, nous verrons ces notions en détail prochainement).
Pour $t = 1..T$:

a) Forward :
   Pour $l = 1..L-1$:
   $Z^{[l]} = W^{[l]} A^{[l-1]} + b^{[l]}$
   $A^{[l]} = \phi(Z^{[l]})$ (e.g. ReLU)
   À la sortie ($l = L$) :
   $Z^{[L]} = W^{[L]} A^{[L-1]} + b^{[L]}$
   $A^{[L]} = \text{softmax}(Z^{[L]}/T)$ (centré colonne par colonne)

b) Perte :
   $L = -\frac{1}{m} \sum_{i,k} Y_{ki} \log A^{[L]}_{ki}$ (ou version pondérée/lissée selon le cas)

c) Backward :
   $dZ^{[L]} = A^{[L]} - Y$ (ou pondéré)
   $dW^{[L]} = \frac{1}{m} dZ^{[L]} (A^{[L-1]})^T$
   $db^{[L]} = \frac{1}{m} dZ^{[L]} \mathbf{1}$
   Pour $l = L-1..1$:
   $dA^{[l]} = (W^{[l+1]})^T dZ^{[l+1]}$
   $dZ^{[l]} = dA^{[l]} \odot \phi'(Z^{[l]})$
   $dW^{[l]} = \frac{1}{m} dZ^{[l]} (A^{[l-1]})^T$
   $db^{[l]} = \frac{1}{m} dZ^{[l]} \mathbf{1}$

d) Update :
   Pour $l = 1..L$:
   $W^{[l]} \leftarrow W^{[l]} - \alpha\, dW^{[l]}$
   $b^{[l]} \leftarrow b^{[l]} - \alpha\, db^{[l]}$ (ou Adam, momentum, etc.)

Transition : vers des couches cachées stables et expressives

Les sections précédentes nous ont conduit d'un neurone logistique jusqu'à la classification multi-classes, en passant par la descente de gradient et la rétropropagation entièrement vectorisée. Nous savons désormais construire un réseau profond, calculer ses gradients et interpréter ses sorties probabilistes.

Dès que l'on empile plusieurs couches, deux défis apparaissent immédiatement : (i) préserver l'expressivité du réseau en introduisant des non-linéarités adaptées et (ii) maintenir un flux d'information stable lors de la propagation avant comme arrière. Sans cela, le réseau stagne (*vanishing gradients*), explose (*exploding gradients*) ou se fige dans des

plateaux d'entraînement.

Les cinq sections qui suivent constituent un bloc cohérent dédié à la **stabilisation de l'apprentissage profond** :

1) choix et analyse des **fonctions d'activation cachées** ;
2) **initialisation** des poids pour conserver la variance des signaux ;
3) **normalisation interne** via la Batch Normalization ;
4) **optimisation avancée** pour accélérer et sécuriser la convergence ;
5) **régularisation** pour contrôler la généralisation.

Chaque thème est présenté avec le triptyque : mise en contexte, justification mathématique détaillée et guide pratique, afin de rester fidèle à l'esprit des parties 1 à 8.

9 Fonctions d'activation cachées : exprimer et stabiliser le réseau

9.1 Le piège de la composition linéaire

Avant d'introduire des activations non linéaires, clarifions ce que réalise exactement un empilement de couches affines pures. Cette mise en contexte montre que la profondeur, à elle seule, ne suffit pas à gagner en expressivité.

Proposition 9.1 (Composition affine)**.** Si, pour chaque couche l, $A^{[l]} = Z^{[l]}$ (pas d'activation) et $Z^{[l]} = W^{[l]}A^{[l-1]} + b^{[l]}$, alors le réseau entier réalise

$$A^{[L]} = \underbrace{\left(W^{[L]} \cdots W^{[1]}\right)}_{\tilde{W}} X + \tilde{b},$$

soit une application affine unique. Autrement dit, *la profondeur sans non-linéarité n'apporte aucun gain d'expressivité.*

Démonstration détaillée. Le raisonnement se fait par récurrence sur le nombre de couches L.

— **Cas $L = 1$.** On a $A^{[1]} = W^{[1]}X + b^{[1]}$, ce qui est clairement affine.

— **Hérédité.** Supposons qu'à l'étage $L-1$ il existe une matrice $\tilde{W}^{[L-1]}$ et un biais $\tilde{b}^{[L-1]}$ tels que $A^{[L-1]} = \tilde{W}^{[L-1]}X + \tilde{b}^{[L-1]}$. Alors

$$A^{[L]} = W^{[L]}A^{[L-1]} + b^{[L]} = W^{[L]}\tilde{W}^{[L-1]}X + W^{[L]}\tilde{b}^{[L-1]} + b^{[L]},$$

ce qui est encore affine avec $\tilde{W} = W^{[L]}\tilde{W}^{[L-1]}$ et $\tilde{b} = W^{[L]}\tilde{b}^{[L-1]} + b^{[L]}$.

La composition de deux applications affines étant affine, le résultat est acquis pour tout L. □

Justification Mathématique

Cette propriété explique pourquoi les activations interviennent : elles introduisent la cassure non linéaire indispensable pour approximer des fonctions complexes (cercles, spirales, ambiguïtés locales, etc.).

9.2 Critères mathématiques pour une activation cachée

Afin qu'une activation facilite vraiment l'entraînement, elle doit satisfaire plusieurs contraintes liées à l'optimisation et à la propagation de l'information.

Attention

[Checklist de conception] Une activation efficace vérifie généralement :

- non-linéarité marquée (cassures ou courbures) tout en restant monotone pour faciliter l'optimisation ;
- pente non nulle autour de 0 afin que les gradients circulent dès l'initialisation ;
- image bornée ou contrôlée pour éviter des sorties trop extrêmes qui saturent les couches suivantes ;
- symétrie autour de 0 (ou centrage) pour ne pas biaiser les pré-activations, ce qui accélère souvent l'apprentissage ;
- coût de calcul modéré (implémentation simple et dérivée explicite).

Justification Mathématique

Ces critères se traduisent directement dans la rétropropagation : $dZ^{[l]} = dA^{[l]} \odot g'(Z^{[l]})$. Si g' est quasi nul ou très variable, les gradients se dissipent ou explosent. Une activation adaptée maintient donc $g'(z)$ dans une zone contrôlée pour les valeurs typiques de $Z^{[l]}$ (souvent centrées en 0 avec variance modérée).

9.3 Panorama des activations classiques

Les activations présentées ci-dessous reviennent dans presque toutes les architectures modernes. Pour chacune, nous donnons le contexte d'apparition, la forme exacte et les démonstrations utiles.

9.3.1 Sigmoïde et tanh : les activations historiques

La sigmoïde apparaît naturellement en régression logistique : elle transforme une combinaison linéaire en probabilité en comprimant la droite réelle dans $(0, 1)$. La tangente hyperbolique en est une version recentrée autour de 0, mieux adaptée aux couches cachées.

Définition 9.1 (Sigmoïde et tangente hyperbolique)**.**

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \qquad \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}.$$

Justification Mathématique

Dans le cadre probabiliste, $\sigma(z)$ est la fonction qui relie le log-odds $\log\left(\frac{P(Y=1|x)}{P(Y=0|x)}\right)$ au score linéaire $z = w^\top x + b$. La fonction tanh correspond quant à elle à une sigmoïde symétrisée : $\tanh(z) = 2\sigma(2z) - 1$. Les deux rendent les activations bornées, ce qui facilite l'interprétation probabiliste ou les contraintes physiques (sortie entre -1 et 1).

Proposition 9.2 (Dérivées)**.**

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)), \qquad \tanh'(z) = 1 - \tanh^2(z).$$

Calcul détaillé. Pour σ, écrivons $\sigma(z) = (1 + e^{-z})^{-1}$. En dérivant et en utilisant la règle de la chaîne :

$$\sigma'(z) = -(1 + e^{-z})^{-2}(-e^{-z}) = \frac{e^{-z}}{(1 + e^{-z})^2} = \frac{1}{1 + e^{-z}}\left(1 - \frac{1}{1 + e^{-z}}\right) = \sigma(z)(1 - \sigma(z)).$$

Pour tanh, on note $\tanh(z) = \frac{\sinh(z)}{\cosh(z)}$. La dérivée est

$$\tanh'(z) = \frac{\cosh(z)\cosh(z) - \sinh(z)\sinh(z)}{\cosh^2(z)} = \frac{1}{\cosh^2(z)} = 1 - \tanh^2(z),$$

car $1 - \tanh^2(z) = \frac{\cosh^2(z) - \sinh^2(z)}{\cosh^2(z)} = \frac{1}{\cosh^2(z)}$. □

Attention

[Fenêtre utile très réduite] Pour $|z| \gg 0$, $\sigma'(z) \approx 0$ et $\tanh'(z) \approx 0$. Les gradients sont donc multipliés par une quantité presque nulle : c'est le *vanishing gradient*. Tanh étant centrée, elle réduit néanmoins les biais systématiques observés avec la sigmoïde.

Exemple 9.1 (Propagation d'un gradient)**.** Avec $z = 8$, $\sigma(z) \approx 0.9997$ et $\sigma'(z) \approx 3 \cdot 10^{-4}$. Après trois couches sigmoïdes, un gradient initial 0.5 devient $0.5 \times (3 \cdot 10^{-4})^3 \approx 1.4 \cdot 10^{-11}$. Cette démonstration numérique illustre la difficulté d'entraîner des réseaux profonds avec des activations saturantes.

Justification Mathématique

Ces fonctions restent pertinentes en sortie (probabilités ou valeurs centrées), mais dans les couches cachées il est préférable d'utiliser des activations à pente constante pour préserver les gradients.

9.3.2 ReLU et variantes : fonctions par morceaux affines

Les Rectified Linear Units (ReLU) ont été introduites pour remédier au vanishing gradient tout en conservant un calcul très simple. Elles conservent la partie positive du signal et annulent la partie négative.

Définition 9.2 (ReLU et variantes)**.**

$$\mathrm{ReLU}(z) = \max(0, z), \qquad \mathrm{Leaky}_\alpha(z) = \begin{cases} z, & z \geq 0 \\ \alpha z, & z < 0 \end{cases}, \quad 0 < \alpha \leq 0.2,$$

$$\mathrm{ELU}_\alpha(z) = \begin{cases} z, & z \geq 0 \\ \alpha(e^z - 1), & z < 0 \end{cases}.$$

Justification Mathématique

La ReLU agit comme un détecteur d'activation : tant que la pré-activation est négative, le neurone reste silencieux. Les variantes Leaky/ELU conservent une petite pente négative pour éviter les neurones *morts* qui ne se réactivent jamais.

Proposition 9.3 (Dérivées presque partout)**.**

$$\mathrm{ReLU}'(z) = \begin{cases} 1, & z > 0 \\ 0, & z < 0 \end{cases}, \qquad \mathrm{Leaky}'_\alpha(z) = \begin{cases} 1, & z > 0 \\ \alpha, & z < 0 \end{cases},$$

$$\mathrm{ELU}'_\alpha(z) = \begin{cases} 1, & z \geq 0 \\ \alpha e^z, & z < 0 \end{cases}.$$

Calcul des dérivées. Chaque fonction est définie morceau par morceau. Pour $z > 0$, les trois fonctions sont identiques à l'identité donc la dérivée vaut 1. Pour $z < 0$, il suffit de dériver l'expression correspondante : αz pour Leaky, $\alpha(e^z - 1)$ pour ELU. Le cas $z = 0$ est traité par sous-gradient (on prend une valeur comprise entre 0 et 1 pour ReLU). □

Exemple 9.2 (Répartition des activations)**.** Si les pré-activations $Z^{[l]}$ suivent une loi symétrique centrée, ~50% des neurones ReLU restent silencieux. Cette épargne d'activation agit comme une régularisation implicite qui améliore souvent la généralisation.

Attention

[Avantages et limites des ReLU]

- Pro : calcul très simple, gradients constants pour $z > 0$, comportement stable à grande profondeur.
- Con : pour $z < 0$, gradient nul (neurones morts). Leaky/ELU/GELU atténuent ce phénomène au prix d'un calcul légèrement plus coûteux.

9.3.3 Activations lissées modernes

Les architectures récentes (Transformers, EfficientNet) utilisent des activations lissées pour conserver une pente non nulle partout tout en introduisant une légère courbure.

Définition 9.3 (SiLU et GELU)**.**

$$\mathrm{SiLU}(z) = z \cdot \sigma(z) \quad \text{(aussi appelée Swish)}, \qquad \mathrm{GELU}(z) = z \cdot \Phi(z),$$

où Φ est la fonction de répartition d'une loi normale standard.

Justification Mathématique

SiLU correspond à l'activation optimale d'un neurone soumis à du bruit logistique (d'où la présence de σ). GELU provient d'une moyenne conditionnelle : on multiplie l'entrée par la probabilité qu'un bruit gaussien l'active. Ces activations conservent une dérivée lisse sur tout $\mathbb{R}$, ce qui améliore la stabilité en présence de normalisations.

Proposition 9.4 (Dérivée de SiLU)**.**

$$\mathrm{SiLU}'(z) = \sigma(z) + z\sigma'(z) = \sigma(z) + z\sigma(z)(1 - \sigma(z)).$$

Démonstration. On applique la règle du produit : $\mathrm{SiLU}(z) = z\sigma(z)$. Sa dérivée vaut

$$\mathrm{SiLU}'(z) = 1 \cdot \sigma(z) + z \cdot \sigma'(z) = \sigma(z) + z\sigma(z)(1 - \sigma(z)).$$

Elle reste strictement positive, garantissant un gradient non nul. □

9.4 Analyse pratique et diagnostic

Guide de choix des activations

- MLP classiques : ReLU par défaut ; passer à Leaky/ELU si l'on observe beaucoup de neurones inactifs.
- Réseaux très profonds ou Transformers : GELU ou SiLU pour bénéficier d'une transition lisse, compatible avec LayerNorm/BatchNorm.
- Sorties réelles centrées : utiliser tanh en sortie (auto-encodeurs) et ReLU/SiLU dans les couches cachées.
- Contraintes matérielles : ReLU/Leaky évitent les exponentielles. Sur GPU/TPU modernes, GELU reste abordable.

Attention

[Diagnostics pendant l'entraînement] Inspecter l'histogramme des activations $A^{[l]}$:

- si tout est positif et collé à 0, envisager Leaky/ELU ou augmenter la variance d'initialisation ;
- si la variance est gigantesque, réduire la variance d'initialisation ou insérer BatchNorm ;
- si l'on observe une saturation (sigmoïde/tanh), recentrer les données d'entrée et ajuster l'initialisation.

10 Initialisation des paramètres : conserver la variance et les gradients

10.1 Le problème de l'initialisation : constat empirique

Jusqu'à présent, nous avons supposé que nous pouvions entraîner des réseaux profonds en empilant des couches. En pratique, avec une initialisation naïve (par exemple $W_{ij} \sim \mathcal{N}(0,1)$ ou $W_{ij} \sim \mathcal{N}(0, 0.01)$), les réseaux de plus de 3-4 couches peinent à converger ou ne convergent pas du tout.

Exemple 10.1 (Expérience mentale : propagation d'un signal)**.** Considérons un réseau à 10 couches cachées de 500 neurones chacune, avec des entrées $x_j \sim \mathcal{N}(0,1)$ initialisées de manière standard. À la première passe avant (forward), calculons ce qui se passe à la couche 5.

Avec une initialisation $W_{ij}^{[l]} \sim \mathcal{N}(0,1)$:

- Couche 1 : $\mathrm{Var}(z^{[1]}) = 500 \times 1 \times 1 = 500$ (car 500 entrées)
- Après activation tanh : les valeurs sont saturées à ± 1 pour 99% des neurones

— Couche 2 : $\mathrm{Var}(z^{[2]}) \approx 500 \times 1 \times 1 = 500$ (même ordre)

En réalité, le phénomène est encore plus grave : si les entrées d'une couche ont une variance q, la variance de sortie est $n \times \sigma_w^2 \times q$. Avec $n = 500$ et $\sigma_w^2 = 1$, la variance croît exponentiellement avec la profondeur !

Intuition

Le problème de l'initialisation est celui d'un amplificateur en cascade :

— Si chaque étage amplifie le signal (gain > 1), la saturation est inévitable après quelques couches

— Si chaque étage atténue le signal (gain < 1), le signal disparaît dans le bruit numérique

— Il faut un gain exactement égal à 1 pour préserver l'information sur des centaines de couches

En deep learning, ce "gain" est contrôlé par la variance des poids σ_w^2.

Attention

[Conséquences pratiques] Une mauvaise initialisation se manifeste par :

— **Explosion des activations** : $z^{[l]} \in [-1000, 1000]$, les fonctions d'activation saturent (tanh à ± 1, sigmoïde à 0 ou 1), les gradients deviennent nuls

— **Disparition des activations** : $z^{[l]} \approx 0$ partout, les gradients sont nuls car $\sigma'(0) \approx 0$ pour sigmoïde/tanh

— **Explosion des gradients** : en backpropagation, les gradients croissent exponentiellement avec la profondeur (vanishing gradient inverse)

10.2 Modélisation probabiliste du signal

Pour analyser rigoureusement la propagation de la variance, nous faisons les hypothèses suivantes, standard en théorie des réseaux de neurones :

Définition 10.1 (Hypothèses de modélisation)**.** Pour une couche dense $Z^{[l]} = W^{[l]} A^{[l-1]} + b^{[l]}$, nous supposons :

1. Les entrées $A_j^{[l-1]}$ sont i.i.d. avec $\mathbb{E}[A_j^{[l-1]}] = 0$ et $\mathrm{Var}(A_j^{[l-1]}) = q^{[l-1]}$
2. Les poids $W_{ij}^{[l]}$ sont i.i.d., indépendants des entrées, avec $\mathbb{E}[W_{ij}^{[l]}] = 0$ et $\mathrm{Var}(W_{ij}^{[l]}) = \sigma_w^2$
3. Les biais $b_i^{[l]} = 0$ (initialisation standard)

Justification Mathématique

Pourquoi ces hypothèses ?

— Le centrage $\mathbb{E}[A] = 0$ est réaliste car les données sont souvent normalisées (soustraction de la moyenne) et les biais sont initialisés à 0

— L'indépendance est une approximation : en pratique, les entrées d'une couche sont les sorties de la couche précédente et sont donc corrélées, mais cette hypothèse suffit pour obtenir des schémas d'initialisation efficaces

— L'initialisation des poids à moyenne nulle est naturelle : on ne veut pas biaiser a priori les sorties dans une direction particulière

Proposition 10.1 (Propagation de la variance - Forward)**.** Sous les hypothèses ci-dessus, la variance des pré-activations de la couche l est :

$$\mathrm{Var}(Z_i^{[l]}) = n^{[l-1]}\sigma_w^2 q^{[l-1]} = n^{[l-1]}\sigma_w^2 \mathrm{Var}(A^{[l-1]}) \tag{274}$$

où $n^{[l-1]}$ est le nombre de neurones d'entrée (dimension de $A^{[l-1]}$).

Démonstration complète. Pour un neurone i de la couche l, la pré-activation est :

$$Z_i^{[l]} = \sum_{j=1}^{n^{[l-1]}} W_{ij}^{[l]} A_j^{[l-1]}$$

Étape 1 : Espérance Par linéarité de l'espérance et indépendance :

$$\mathbb{E}[Z_i^{[l]}] = \sum_{j=1}^{n^{[l-1]}} \mathbb{E}[W_{ij}^{[l]}]\mathbb{E}[A_j^{[l-1]}] = \sum_{j=1}^{n^{[l-1]}} 0 \times 0 = 0$$

Étape 2 : Variance Comme $\mathbb{E}[Z_i^{[l]}] = 0$, on a $\mathrm{Var}(Z_i^{[l]}) = \mathbb{E}[(Z_i^{[l]})^2]$. Développons :

$$(Z_i^{[l]})^2 = \left(\sum_{j=1}^{n^{[l-1]}} W_{ij}^{[l]} A_j^{[l-1]}\right)^2 = \sum_{j=1}^{n^{[l-1]}} (W_{ij}^{[l]} A_j^{[l-1]})^2 + \sum_{j \neq k} W_{ij}^{[l]} A_j^{[l-1]} W_{ik}^{[l]} A_k^{[l-1]}$$

Prenons l'espérance :

— Pour les termes diagonaux ($j = k$) :

$$\mathbb{E}[(W_{ij}^{[l]})^2 (A_j^{[l-1]})^2] = \mathbb{E}[(W_{ij}^{[l]})^2]\mathbb{E}[(A_j^{[l-1]})^2] = \sigma_w^2 q^{[l-1]}$$

car W et A sont indépendants, et $\mathrm{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2 = \mathbb{E}[X^2]$ pour une variable centrée.

— Pour les termes croisés ($j \neq k$) :

$$\mathbb{E}[W_{ij}^{[l]} A_j^{[l-1]} W_{ik}^{[l]} A_k^{[l-1]}] = \mathbb{E}[W_{ij}^{[l]}]\mathbb{E}[A_j^{[l-1]}]\mathbb{E}[W_{ik}^{[l]}]\mathbb{E}[A_k^{[l-1]}] = 0$$

par indépendance et centrage.

Il reste donc $n^{[l-1]}$ termes diagonaux identiques :

$$\mathrm{Var}(Z_i^{[l]}) = n^{[l-1]}\sigma_w^2 q^{[l-1]} \quad \checkmark$$

□

Exemple 10.2 (Vérification numérique)**.** Prenons $n^{[l-1]} = 100$, $\sigma_w^2 = 1$, et $A_j^{[l-1]} \sim \mathcal{N}(0,1)$ (donc $q^{[l-1]} = 1$).

Théorie : $\mathrm{Var}(Z_i^{[l]}) = 100 \times 1 \times 1 = 100$, donc $\sigma_Z = 10$.

Une simulation avec des tirages aléatoires (voir exemple `04_initialisation`) donne typiquement $\mathrm{Var}(Z) \approx 98.5$, proche de la prédiction théorique (Figure 8, droite). La légère différence provient de la variance d'échantillonnage.

10.3 Condition de variance constante (Forward)

Pour éviter l'explosion ou la disparition du signal, nous imposons que la variance soit préservée d'une couche à l'autre :

$$\mathrm{Var}(Z^{[l]}) = \mathrm{Var}(A^{[l-1]})$$

Pour une activation linéaire ou tanh/sigmoid autour de 0 (où $\phi(z) \approx z$), cela donne :

$$n^{[l-1]}\sigma_w^2 = 1 \quad \Rightarrow \quad \sigma_w^2 = \frac{1}{n^{[l-1]}}$$

Définition 10.2 (Initialisation Forward)**.** Pour préserver la variance en propagation avant, on initialise :

$$W_{ij}^{[l]} \sim \mathcal{N}\left(0, \frac{1}{n^{[l-1]}}\right) \quad \text{ou} \quad W_{ij}^{[l]} \sim \mathcal{U}\left[-\sqrt{\frac{3}{n^{[l-1]}}}, \sqrt{\frac{3}{n^{[l-1]}}}\right]$$

(Unif. de variance σ^2 sur $[-a, a]$: $a = \sqrt{3\sigma^2}$)

Attention

[Cas particulier : ReLU] Pour une activation ReLU, $\phi(z) = \max(0, z)$. Si Z est symétrique autour de 0 :

- La moitié des valeurs sont mises à 0 (partie négative)
- L'autre moitié passe inchangée
- Donc $\mathbb{E}[\mathrm{ReLU}(Z)] = \frac{1}{2}\mathbb{E}[|Z|]$ (non nul !) et $\mathrm{Var}(\mathrm{ReLU}(Z)) \approx \frac{1}{2}\mathrm{Var}(Z)$

Pour compenser cette perte de variance d'un facteur 2, il faut doubler la variance des poids :

$$\sigma_w^2 = \frac{2}{n^{[l-1]}} \quad \text{(Initialisation de He)}$$

10.4 Propagation arrière : préserver les gradients

La rétropropagation suit des équations similaires mais "en sens inverse". Rappelons la formule du gradient par rapport aux activations de la couche précédente :

$$dA^{[l-1]} = (W^{[l]})^T dZ^{[l]}$$

Proposition 10.2 (Flux du gradient - Backward)**.** Sous les mêmes hypothèses d'indépendance et de centrage pour les gradients $dZ_k^{[l]}$ (variance $r^{[l]}$), on a :

$$\mathrm{Var}(dA_j^{[l-1]}) = n^{[l]}\sigma_w^2 r^{[l]} \tag{275}$$

où $n^{[l]}$ est le nombre de neurones de sortie (dimension de $Z^{[l]}$).

Démonstration. La démonstration est identique à celle du forward. Pour un élément j :

$$dA_j^{[l-1]} = \sum_{k=1}^{n^{[l]}} W_{kj}^{[l]} dZ_k^{[l]}$$

Les termes croisés s'annulent par indépendance, il reste :

$$\mathrm{Var}(dA_j^{[l-1]}) = \sum_{k=1}^{n^{[l]}} \mathrm{Var}(W_{kj}^{[l]})\mathrm{Var}(dZ_k^{[l]}) = n^{[l]}\sigma_w^2 r^{[l]} \quad \checkmark$$

□

Pour que les gradients ne disparaissent ni n'explosent lors de la rétropropagation, il faut :

$$n^{[l]}\sigma_w^2 = 1 \quad \Rightarrow \quad \sigma_w^2 = \frac{1}{n^{[l]}}$$

10.5 Le compromis Glorot/Xavier

Nous avons deux conditions contradictoires :

- **Forward** demande : $\sigma_w^2 = \frac{1}{n^{[l-1]}}$ (nombre d'entrées)
- **Backward** demande : $\sigma_w^2 = \frac{1}{n^{[l]}}$ (nombre de sorties)

Intuition

Imaginez une autoroute à plusieurs voies :

- Le forward est le trafic allant de l'entrée vers la sortie (nombre de voies d'entrée n_{in})
- Le backward est le trafic retour (nombre de voies de sortie n_{out})
- On veut que le flux soit fluide dans les deux directions
- La solution de compromis est une "moyenne" entre les deux contraintes

Définition 10.3 (Initialisation de Glorot/Xavier (2010))**.** Pour satisfaire approximativement les deux contraintes, on utilise la moyenne harmonique :

$$\sigma_w^2 = \frac{2}{n^{[l-1]} + n^{[l]}} \tag{276}$$

Les variantes courantes sont :

- **Xavier/Glorot Normal** : $W_{ij} \sim \mathcal{N}\left(0, \frac{2}{n^{[l-1]}+n^{[l]}}\right)$
- **Xavier/Glorot Uniforme** : $W_{ij} \sim \mathcal{U}\left[-\sqrt{\frac{6}{n^{[l-1]}+n^{[l]}}}, \sqrt{\frac{6}{n^{[l-1]}+n^{[l]}}}\right]$

Justification Mathématique

Pourquoi la moyenne harmonique $\frac{2}{n_{in}+n_{out}}$ plutôt que $\frac{1}{n_{in}}$ ou $\frac{1}{n_{out}}$?
Cette forme minimise le maximum des ratios $\frac{\mathrm{Var(sortie)}}{\mathrm{Var(entrée)}}$ et $\frac{\mathrm{Var(gradient)}}{\mathrm{Var(gradient\ amont)}}$. C'est le choix qui équilibre le mieux les deux directions de propagation.
Si $n_{in} = n_{out} = n$, on retrouve $\sigma_w^2 = \frac{1}{n}$, qui satisfait parfaitement les deux contraintes.

10.6 Initialisation de He pour les activations ReLU

Pour les réseaux utilisant des ReLU (ou variantes comme LeakyReLU), l'analyse doit tenir compte du fait que la ReLU coupe la moitié du signal.

Proposition 10.3 (Variance avec ReLU)**.** Si $Z \sim \mathcal{N}(0, \sigma^2)$ et $A = \text{ReLU}(Z) = \max(0, Z)$, alors :

$$\mathbb{E}[A] = \frac{\sigma}{\sqrt{2\pi}}, \quad \text{Var}(A) = \frac{\sigma^2}{2}\left(1 - \frac{1}{\pi}\right) \approx \frac{\sigma^2}{2}$$

Pour une approximation rapide, on considère $\text{Var}(A) \approx \frac{1}{2}\text{Var}(Z)$ (la moitié de la variance est perdue car la moitié des valeurs sont mises à 0).

Définition 10.4 (Initialisation de He (2015))**.** Pour compenser la perte d'un facteur 2 due à la ReLU :

$$\sigma_w^2 = \frac{2}{n^{[l-1]}} \tag{277}$$

Variantes :

- **He Normal** : $W_{ij} \sim \mathcal{N}\left(0, \frac{2}{n^{[l-1]}}\right)$
- **He Uniforme** : $W_{ij} \sim \mathcal{U}\left[-\sqrt{\frac{6}{n^{[l-1]}}}, \sqrt{\frac{6}{n^{[l-1]}}}\right]$

Attention

[Quand utiliser quoi ?]

Activation	Initialisation	Variance
tanh, sigmoid, softsign	Glorot/Xavier	$\frac{2}{n_{in}+n_{out}}$
ReLU, LeakyReLU, SiLU	He	$\frac{2}{n_{in}}$
Linéaire (sortie)	Glorot	$\frac{2}{n_{in}+n_{out}}$

Note : Les biais sont toujours initialisés à $b_i = 0$ (sauf cas particuliers comme la forget gate des LSTM).

10.7 Procédure pratique et vérification

Protocole d'initialisation

1. Identifier l'activation de la couche : ReLU $\to$ He, tanh/sigmoid $\to$ Glorot
2. Calculer n_{in} (dimension entrée) et éventuellement n_{out} pour Glorot
3. Initialiser les poids selon la variance prescrite
4. Initialiser les biais à 0
5. **Vérifier** : après la première passe forward, inspecter les statistiques des activations de chaque couche

Exemple 10.3 (Vérification numérique)**.** Pour une couche dense avec $n_{in} = 784$ entrées et $n_{out} = 256$ sorties, activation ReLU, avec He Normal ($\sigma_w = \sqrt{2/784} \approx 0.0505$) :

- Entrées $X \sim \mathcal{N}(0, 1)$: $\text{Var}(X) = 1$
- Après multiplication : $\text{Var}(Z) \approx n_{in} \cdot \sigma_w^2 \cdot \text{Var}(X) = 784 \times 0.00255 \times 1 \approx 1.0$

— Après ReLU : $\mathrm{Var}(A) \approx 0.5$ (perte d'un facteur 2)

On observe bien que la variance est préservée en avant ($\mathrm{Var}(Z) \approx 1$), ce qui garantit que le signal ne sature pas et ne disparaît pas.

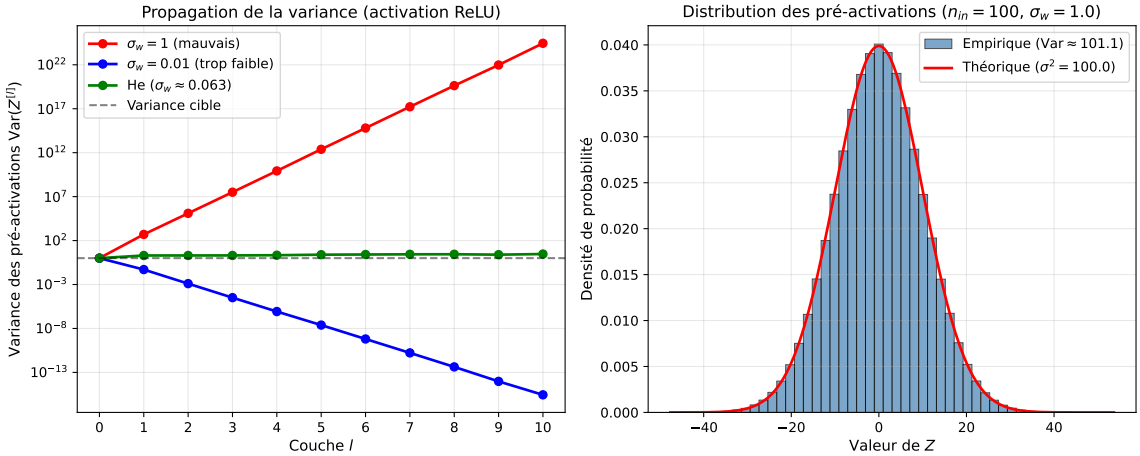

Figure 7 : **Gauche :** Propagation de la variance à travers 10 couches avec activation ReLU. L'initialisation He ($\sigma_w = \sqrt{2/n}$) maintient la variance stable autour de 1, tandis que $\sigma_w = 1$ provoque une explosion et $\sigma_w = 0.01$ une disparition du signal. **Droite :** Vérification empirique de la formule théorique $\mathrm{Var}(Z) = n_{in}\sigma_w^2$ avec $n_{in} = 100$ et $\sigma_w = 1$. La distribution empirique (histogramme) s'ajuste bien à la gaussienne théorique de variance 100.

Attention

[Diagnostic d'initialisation problématique] La Figure 8 illustre l'impact de trois initialisations différentes sur les distributions des activations.

Signes d'initialisation trop forte (σ_w trop grand) :

— Perte = NaN dès les premières itérations

— Histogramme des activations : valeurs extrêmes ($|z| > 10$)

— Taux de saturation élevé (pour tanh/sigmoid)

Signes d'initialisation trop faible (σ_w trop petit) :

— Perte stagne, pas d'apprentissage

— Tous les gradients ≈ 0

— Histogramme des activations : concentré en $[-0.01, 0.01]$

Solution : Ajuster σ_w^2 par facteur 2 (doubler ou diviser par 2) jusqu'à obtenir des activations de variance ≈ 1 en début d'entraînement.

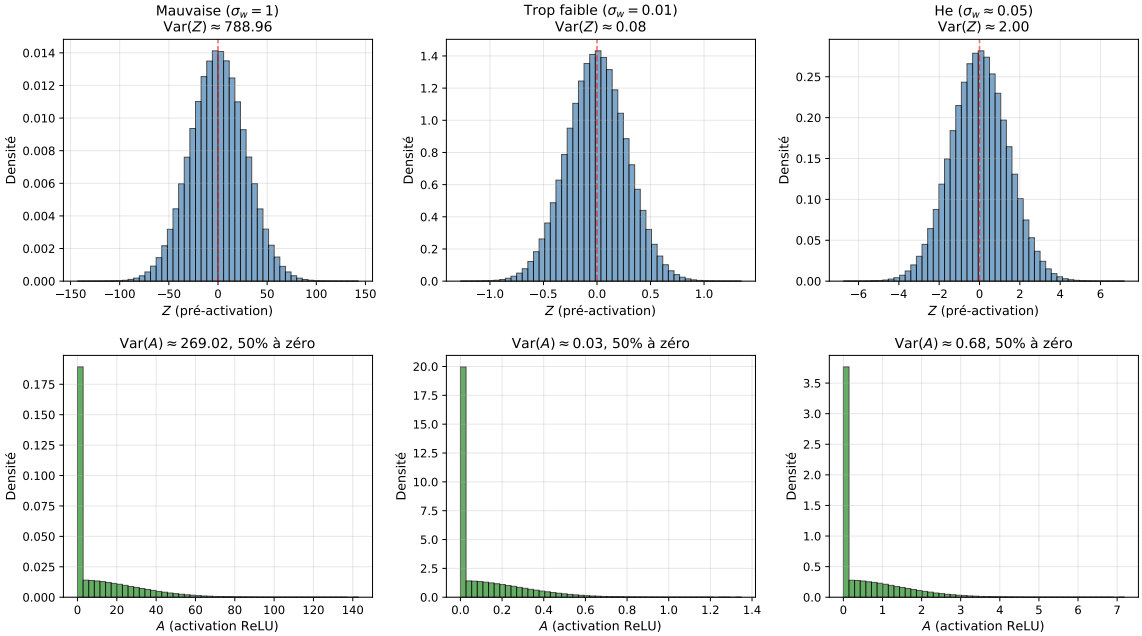

Figure 8 : Impact de l'initialisation sur les distributions des pré-activations (ligne du haut) et des activations ReLU (ligne du bas) pour une couche avec $n_{in} = 784$. **Gauche :** Avec $\sigma_w = 1$, les pré-activations ont une variance de ≈ 780 et les activations saturent (99% des valeurs positives sont à 1). **Centre :** Avec $\sigma_w = 0.01$, la variance est trop faible (≈ 0.08) et les activations sont quasi-nulles. **Droite :** L'initialisation He ($\sigma_w \approx 0.05$) donne une variance proche de 1 et une distribution équilibrée après ReLU.

Justification Mathématique

[Pourquoi ne pas initialiser à 0 ?] Si $W = 0$ partout, alors $Z = 0$ pour toutes les couches. Le gradient $\frac{\partial L}{\partial W}$ serait identique pour tous les poids sortant d'un même neurone (symétrie). Le réseau apprendrait exactement la même chose pour tous ses neurones : pas de spécialisation, pas d'apprentissage utile.
L'initialisation aléatoire brise cette symétrie et permet à chaque neurone d'apprendre des features différentes.

11 Normalisation interne : Batch Normalization

11.1 Motivation : la dérive de la distribution interne

Au chapitre précédent, nous avons vu comment l'initialisation (Glorot, He) permet de fixer la variance des activations *au premier pas* de l'entraînement. Mais que se passe-t-il après quelques milliers de mises à jour ? Les poids $W^{[l]}$ changent, et les distributions des pré-activations $Z^{[l]}$ changent en conséquence. Chaque couche doit alors s'adapter en permanence à une entrée dont la distribution évolue : c'est le phénomène d'*internal covariate shift* introduit par Ioffe et Szegedy (2015).

Intuition

Imaginez un orchestre où chaque musicien ajuste son volume en fonction de ce qu'il entend autour de lui. Si le trompettiste monte d'un cran, le violoniste doit compenser, ce qui amène le hautbois à s'ajuster, etc. L'orchestre finit par osciller sans jamais se stabiliser.
Dans un réseau profond, chaque couche est un "musicien" : la couche l apprend à transformer $A^{[l-1]}$ en quelque chose d'utile. Si la distribution de $A^{[l-1]}$ change constamment (parce que les couches précédentes mettent à jour leurs poids), la couche l poursuit une cible mouvante. L'entraînement en est ralenti, voire destabilisé.

Exemple 11.1 (Dérive numérique sur le problème gorilles/chimpanzés)**.** Reprenons notre réseau à 3 couches cachées ($n^{[1]} = 64$, $n^{[2]} = 32$, $n^{[3]} = 16$, sortie $n^{[4]} = 1$) avec activation ReLU et initialisation He. Supposons qu'à l'initialisation ($t = 0$), les statistiques des pré-activations sont bien contrôlées :

	Couche 1	Couche 2	Couche 3	Couche 4
$\mu(Z^{[l]})$ à $t = 0$	≈ 0	≈ 0	≈ 0	≈ 0
$\sigma(Z^{[l]})$ à $t = 0$	≈ 1.0	≈ 1.0	≈ 1.0	≈ 1.0
$\mu(Z^{[l]})$ à $t = 500$	-0.3	$+1.2$	$+3.8$	$+7.1$
$\sigma(Z^{[l]})$ à $t = 500$	1.4	2.9	8.5	24.3

Après 500 itérations de gradient, la moyenne et la variance des pré-activations dérivent, et cette dérive **s'amplifie avec la profondeur**. À la couche 4, la variance a été multipliée par 24 : les entrées de la sigmoïde de sortie sont dans la zone saturée, les gradients deviennent quasiment nuls, et l'apprentissage stagne.

Attention

[Signes d'instabilité liée à la dérive]

- Les gradients explosent dès que l'on augmente le learning rate.
- L'entraînement est très sensible à l'initialisation ou à l'ordre des mini-batchs.
- Les distributions de $Z^{[l]}$ dérivent fortement au fil des epochs : la moyenne s'éloigne de 0, la variance augmente (ou diminue) exponentiellement avec la profondeur.
- Les fonctions d'activation saturent (tanh, sigmoïde) ou deviennent excessivement creuses (ReLU avec variance trop grande).

Justification Mathématique

L'initialisation He ou Glorot fixe la condition $n^{[l-1]}\sigma_w^2 = 1$ (ou $= 2$ pour ReLU) qui garantit $\mathrm{Var}(Z^{[l]}) \approx \mathrm{Var}(A^{[l-1]})$ *à l'instant initial*. Mais cette condition porte sur les poids initiaux. Dès la première mise à jour $W \leftarrow W - \alpha\, dW$, il n'y a plus aucune garantie que la variance soit préservée. L'idée de Batch Normalization est de **forcer** cette normalisation à chaque pas, comme une contrainte structurelle du réseau.

11.2 Définition formelle de Batch Normalization

11.2.1 Normalisation scalaire par neurone

Définition 11.1 (Batch Normalization sur une couche dense)**.** Considérons la couche l d'un réseau profond. Après le calcul de la pré-activation linéaire :

$$Z^{[l]} = W^{[l]}A^{[l-1]} + b^{[l]} \in \mathbb{R}^{n^{[l]} \times m}$$

où m est la taille du mini-batch, Batch Normalization transforme $Z^{[l]}$ en quatre étapes. Pour chaque neurone $j = 1, \ldots, n^{[l]}$:

Étape 1 – Moyenne du mini-batch :

$$\mu_j = \frac{1}{m} \sum_{i=1}^{m} Z_j^{(i)} \tag{278}$$

Étape 2 – Variance du mini-batch :

$$\sigma_j^2 = \frac{1}{m} \sum_{i=1}^{m} \left(Z_j^{(i)} - \mu_j \right)^2 \tag{279}$$

Étape 3 – Normalisation :

$$\hat{Z}_j^{(i)} = \frac{Z_j^{(i)} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}} \tag{280}$$

Étape 4 – Mise à l'échelle et décalage (scale and shift) :

$$\widetilde{Z}_j^{(i)} = \gamma_j \hat{Z}_j^{(i)} + \beta_j \tag{281}$$

où :

— $\varepsilon > 0$ est une constante de stabilité numérique (typiquement $\varepsilon = 10^{-5}$),

— γ_j et β_j sont des **paramètres apprenables** (un par neurone), initialisés à $\gamma_j = 1$ et $\beta_j = 0$.

La sortie $\widetilde{Z}^{[l]}$ remplace $Z^{[l]}$ dans la suite de la couche : on applique ensuite l'activation $A^{[l]} = g(\widetilde{Z}^{[l]})$.

Remarque 11.1. La notation $Z_j^{(i)}$ désigne la pré-activation du neurone j pour l'exemple i du mini-batch. On a donc $Z_j^{(i)} = [Z^{[l]}]_{j,i}$ dans notre convention matricielle (ligne = neurone, colonne = exemple), cohérente avec les sections précédentes.

Exemple 11.2 (Vérification numérique sur un mini-batch de taille 4)**.** Prenons un neurone j avec les pré-activations suivantes pour un mini-batch de $m = 4$ exemples :

$$Z_j^{(1)} = 2.0, \quad Z_j^{(2)} = 4.0, \quad Z_j^{(3)} = 6.0, \quad Z_j^{(4)} = 8.0$$

Étape 1 : $\mu_j = \frac{1}{4}(2 + 4 + 6 + 8) = 5.0$
Étape 2 : $\sigma_j^2 = \frac{1}{4}\left[(2-5)^2 + (4-5)^2 + (6-5)^2 + (8-5)^2\right] = \frac{1}{4}(9 + 1 + 1 + 9) = 5.0$

Étape 3 : Avec $\varepsilon = 0$ pour simplifier, $\sqrt{\sigma_j^2} = \sqrt{5} \approx 2.236$:

$$\hat{Z}_j^{(1)} = \frac{2-5}{\sqrt{5}} \approx -1.342, \quad \hat{Z}_j^{(2)} \approx -0.447, \quad \hat{Z}_j^{(3)} \approx 0.447, \quad \hat{Z}_j^{(4)} \approx 1.342$$

Vérification : La moyenne des $\hat{Z}_j$ vaut $\frac{-1.342-0.447+0.447+1.342}{4} = 0$ et leur variance vaut $\frac{1.342^2+0.447^2+0.447^2+1.342^2}{4} = \frac{1.8+0.2+0.2+1.8}{4} = 1.0$ ✓

Étape 4 : Avec $\gamma_j = 2$ et $\beta_j = 3$:

$$\widetilde{Z}_j^{(1)} = 2 \times (-1.342) + 3 = 0.316, \quad \widetilde{Z}_j^{(2)} = 2.106, \quad \widetilde{Z}_j^{(3)} = 3.894, \quad \widetilde{Z}_j^{(4)} = 5.684$$

Le réseau a appris (via γ, β) à repositionner la distribution autour de 3 avec un écart-type de 2, plutôt que la distribution d'origine centrée en 5 avec un écart-type de $\sqrt{5} \approx 2.24$.

11.3 Rôle des paramètres apprenables γ et β

Après l'étape de normalisation (étape 3), les $\hat{Z}_j^{(i)}$ ont une moyenne nulle et une variance unitaire. On pourrait se demander : pourquoi ajouter γ et β ? La réponse est que sans ces paramètres, on **contraint** la représentation interne du réseau de manière trop restrictive.

Proposition 11.1 (Perte d'expressivité sans γ et β)**.** Sans les paramètres γ et β, la sortie de la couche BatchNorm est contrainte à vérifier, pour chaque neurone j :

$$\frac{1}{m}\sum_{i=1}^{m} \hat{Z}_j^{(i)} = 0 \quad \text{et} \quad \frac{1}{m}\sum_{i=1}^{m} \left(\hat{Z}_j^{(i)}\right)^2 = 1$$

quel que soit l'état des poids $W^{[l]}$. La couche perd donc deux degrés de liberté par neurone.

Démonstration. Par construction de $\hat{Z}_j^{(i)} = \frac{Z_j^{(i)} - \mu_j}{\sqrt{\sigma_j^2+\varepsilon}}$, calculons la moyenne sur le mini-batch :

$$\frac{1}{m}\sum_{i=1}^{m} \hat{Z}_j^{(i)} = \frac{1}{m}\sum_{i=1}^{m} \frac{Z_j^{(i)} - \mu_j}{\sqrt{\sigma_j^2+\varepsilon}} = \frac{1}{\sqrt{\sigma_j^2+\varepsilon}}\left(\frac{1}{m}\sum_{i=1}^{m} Z_j^{(i)} - \mu_j\right) = \frac{\mu_j - \mu_j}{\sqrt{\sigma_j^2+\varepsilon}} = 0$$

Pour la variance (en négligeant ε) :

$$\frac{1}{m}\sum_{i=1}^{m} \left(\hat{Z}_j^{(i)}\right)^2 = \frac{1}{m}\sum_{i=1}^{m} \frac{(Z_j^{(i)} - \mu_j)^2}{\sigma_j^2} = \frac{\sigma_j^2}{\sigma_j^2} = 1$$

Ces deux égalités sont **identiques** : elles sont vérifiées quels que soient les poids $W^{[l]}$ et les entrées $A^{[l-1]}$. L'optimiseur ne peut pas ajuster la moyenne ni la variance de $\hat{Z}_j$ en modifiant $W^{[l]}$, car la normalisation annule systématiquement ces changements. ✓ □

Intuition

Considérons un cas concret : supposons que, pour une tâche donnée, la distribution optimale des entrées d'une activation ReLU soit centrée en $\mu^* = 0.5$ avec un écart-type $\sigma^* = 2$. Sans γ et β, la normalisation force $\mu = 0$ et $\sigma = 1$, et le réseau ne peut pas atteindre cette configuration optimale.
Avec γ et β, il suffit que le réseau apprenne $\gamma_j = 2$ et $\beta_j = 0.5$ pour retrouver la distribution souhaitée : $\widetilde{Z}_j = 2\hat{Z}_j + 0.5$ a bien une moyenne de 0.5 et un écart-type de 2.
Cas extrême : si $\gamma_j = \sqrt{\sigma_j^2 + \varepsilon}$ et $\beta_j = \mu_j$, alors $\widetilde{Z}_j = \gamma_j \hat{Z}_j + \beta_j = Z_j$, et la couche BatchNorm devient l'identité. Le réseau peut donc "annuler" la normalisation s'il le juge utile.

Attention

[Le biais $b^{[l]}$ est redondant avec β] Puisque BatchNorm soustrait la moyenne μ_j (qui absorbe le biais $b_j^{[l]}$), puis ajoute β_j, le biais de la couche linéaire n'a plus d'effet sur la sortie. En pratique, on supprime $b^{[l]}$ dans les couches suivies de BatchNorm :

$$Z^{[l]} = W^{[l]} A^{[l-1]} \quad \text{(pas de biais)}$$

Le paramètre β de BatchNorm joue le rôle du biais.

11.4 Formulation vectorisée et vérification dimensionnelle

Passons maintenant à l'écriture matricielle complète, en cohérence avec les notations des sections précédentes.

Définition 11.2 (BatchNorm vectorisé – Forward)**.** Pour la couche l avec un mini-batch de m exemples, les matrices ont les dimensions suivantes :

$$Z^{[l]} = W^{[l]} A^{[l-1]} \in \mathbb{R}^{n^{[l]} \times m} \tag{282}$$

Étape 1 – Moyenne par neurone :

$$\mu = \frac{1}{m} Z^{[l]} \mathbf{1}_m \in \mathbb{R}^{n^{[l]} \times 1} \tag{283}$$

où $\mathbf{1}_m \in \mathbb{R}^{m \times 1}$ est le vecteur de uns.

Étape 2 – Variance par neurone :

$$\sigma^2 = \frac{1}{m} (Z^{[l]} - \mu \mathbf{1}_m^T) \odot (Z^{[l]} - \mu \mathbf{1}_m^T) \cdot \mathbf{1}_m \in \mathbb{R}^{n^{[l]} \times 1} \tag{284}$$

(somme des carrés des écarts, colonne par colonne, puis sommation sur les exemples)

Étape 3 – Normalisation :

$$\hat{Z} = (Z^{[l]} - \mu \mathbf{1}_m^T) \oslash \left(\sqrt{\sigma^2 + \varepsilon}\, \mathbf{1}_m^T \right) \in \mathbb{R}^{n^{[l]} \times m} \tag{285}$$

où $\oslash$ est la division élément par élément et $\sqrt{\cdot}$ s'applique composante par composante.

Étape 4 – Mise à l'échelle :

$$\widetilde{Z} = \gamma \odot \hat{Z} + \beta \mathbf{1}_m^T \in \mathbb{R}^{n^{[l]} \times m} \tag{286}$$

avec $\gamma, \beta \in \mathbb{R}^{n^{[l]} \times 1}$ (un paramètre par neurone, broadcasté sur les m colonnes).

La sortie $\widetilde{Z}$ est ensuite passée à l'activation : $A^{[l]} = g(\widetilde{Z})$.

Justification Mathématique

Vérifions les dimensions à chaque étape :

Étape 1 :

$$\underbrace{\mu}_{n^{[l]} \times 1} = \frac{1}{m} \underbrace{Z^{[l]}}_{n^{[l]} \times m} \underbrace{\mathbf{1}_m}_{m \times 1} \quad \text{OK} : (n^{[l]} \times m)(m \times 1) = (n^{[l]} \times 1) \checkmark$$

Étape 2 : Le terme $\mu \mathbf{1}_m^T$ a la dimension $(n^{[l]} \times 1)(1 \times m) = (n^{[l]} \times m)$, c'est le broadcasting de μ sur les m colonnes. La soustraction $Z^{[l]} - \mu \mathbf{1}_m^T$ est donc bien $(n^{[l]} \times m)$. Le produit de Hadamard donne $(n^{[l]} \times m)$, et la multiplication par $\mathbf{1}_m$ somme sur les colonnes : $(n^{[l]} \times m)(m \times 1) = (n^{[l]} \times 1)$ ✓

Étape 3 : Le dénominateur $\sqrt{\sigma^2 + \varepsilon}\, \mathbf{1}_m^T$ est de dimension $(n^{[l]} \times 1)(1 \times m) = (n^{[l]} \times m)$. La division élément par élément est donc bien définie : $(n^{[l]} \times m) \oslash (n^{[l]} \times m) = (n^{[l]} \times m)$ ✓

Étape 4 : $\gamma \odot \hat{Z}$ se lit avec broadcasting : $\gamma \in \mathbb{R}^{n^{[l]} \times 1}$ est broadcasté en $(n^{[l]} \times m)$, puis produit de Hadamard avec $\hat{Z} \in \mathbb{R}^{n^{[l]} \times m}$. Résultat : $(n^{[l]} \times m)$ ✓

Remarque 11.2. En pratique (NumPy, PyTorch), le broadcasting rend implicite l'écriture $\mu \mathbf{1}_m^T$. On écrit simplement `Z - mu` où `mu` a la forme `(n, 1)` et `Z` a la forme `(n, m)`, et le framework effectue automatiquement la réplication.

11.5 Rétropropagation complète à travers BatchNorm

C'est la partie la plus délicate de cette section. La normalisation introduit des dépendances entre les exemples du mini-batch (via μ et σ^2), ce qui rend la différentiation plus subtile que pour une couche linéaire standard.

11.5.1 Graphe de calcul

Pour structurer la rétropropagation, identifions la chaîne de dépendances :

$$Z \xrightarrow{\text{moyenne}} \mu \to \hat{Z} \xrightarrow{\gamma, \beta} \widetilde{Z}$$

$$Z \xrightarrow{\text{variance}} \sigma^2 \to \hat{Z}$$

Plus précisément, pour chaque neurone j (que nous omettons dans les indices pour alléger) :

$$\mu = f_1(Z^{(1)}, \ldots, Z^{(m)}) \tag{287}$$

$$\sigma^2 = f_2(Z^{(1)}, \ldots, Z^{(m)}, \mu) \tag{288}$$

$$\hat{Z}^{(i)} = f_3(Z^{(i)}, \mu, \sigma^2) \tag{289}$$

$$\widetilde{Z}^{(i)} = f_4(\hat{Z}^{(i)}, \gamma, \beta) \tag{290}$$

Le gradient $d\widetilde{Z} = \frac{\partial L}{\partial \widetilde{Z}} \in \mathbb{R}^{n^{[l]} \times m}$ est donné par la couche suivante (via la rétropropagation standard). Nous devons calculer $d\gamma$, $d\beta$, et dZ (le gradient par rapport aux pré-activations *avant* normalisation).

11.5.2 Gradient de β et γ

Proposition 11.2 (Gradients des paramètres apprenables)**.**

$$d\beta_j = \sum_{i=1}^{m} d\widetilde{Z}_j^{(i)} \qquad \text{c'est-à-dire} \qquad d\beta = d\widetilde{Z}\,\mathbf{1}_m \in \mathbb{R}^{n^{[l]} \times 1} \tag{291}$$

$$d\gamma_j = \sum_{i=1}^{m} d\widetilde{Z}_j^{(i)} \cdot \hat{Z}_j^{(i)} \qquad \text{c'est-à-dire} \qquad d\gamma = (d\widetilde{Z} \odot \hat{Z})\,\mathbf{1}_m \in \mathbb{R}^{n^{[l]} \times 1} \tag{292}$$

Démonstration. Rappelons que $\widetilde{Z}_j^{(i)} = \gamma_j \hat{Z}_j^{(i)} + \beta_j$. Par la règle de la chaîne :

Pour β_j :

$$\frac{\partial L}{\partial \beta_j} = \sum_{i=1}^{m} \frac{\partial L}{\partial \widetilde{Z}_j^{(i)}} \cdot \frac{\partial \widetilde{Z}_j^{(i)}}{\partial \beta_j}$$

Or $\frac{\partial \widetilde{Z}_j^{(i)}}{\partial \beta_j} = 1$ pour tout i (car β_j apparaît linéairement dans chaque $\widetilde{Z}_j^{(i)}$). Donc :

$$d\beta_j = \sum_{i=1}^{m} d\widetilde{Z}_j^{(i)} \quad \checkmark$$

Pour γ_j :

$$\frac{\partial L}{\partial \gamma_j} = \sum_{i=1}^{m} \frac{\partial L}{\partial \widetilde{Z}_j^{(i)}} \cdot \frac{\partial \widetilde{Z}_j^{(i)}}{\partial \gamma_j}$$

Or $\frac{\partial \widetilde{Z}_j^{(i)}}{\partial \gamma_j} = \hat{Z}_j^{(i)}$. Donc :

$$d\gamma_j = \sum_{i=1}^{m} d\widetilde{Z}_j^{(i)} \cdot \hat{Z}_j^{(i)} \quad \checkmark$$

Vérification dimensionnelle : $d\beta$ et $d\gamma$ sont dans $\mathbb{R}^{n^{[l]} \times 1}$, même dimension que β et γ ✓ □

11.5.3 Gradient de $\hat{Z}$

Proposition 11.3 (Gradient de la sortie normalisée)**.**

$$d\hat{Z}_j^{(i)} = d\widetilde{Z}_j^{(i)} \cdot \gamma_j \qquad \text{c'est-à-dire} \qquad d\hat{Z} = d\widetilde{Z} \odot \gamma \in \mathbb{R}^{n^{[l]} \times m} \tag{293}$$

(avec broadcasting de $\gamma \in \mathbb{R}^{n^{[l]} \times 1}$ sur les m colonnes)

Démonstration. De $\widetilde{Z}_j^{(i)} = \gamma_j \hat{Z}_j^{(i)} + \beta_j$, on tire directement :

$$\frac{\partial \widetilde{Z}_j^{(i)}}{\partial \hat{Z}_j^{(i)}} = \gamma_j$$

Donc $d\hat{Z}_j^{(i)} = d\widetilde{Z}_j^{(i)} \cdot \gamma_j$. En notation matricielle : $d\hat{Z} = d\widetilde{Z} \odot \gamma$ ✓ □

11.5.4 Gradient de σ^2

C'est ici que la dérivation devient plus subtile, car σ^2 intervient dans le dénominateur de $\hat{Z}$.

Proposition 11.4 (Gradient de la variance)**.**

$$d\sigma_j^2 = \sum_{i=1}^{m} d\hat{Z}_j^{(i)} \cdot (Z_j^{(i)} - \mu_j) \cdot \left(-\frac{1}{2}\right) (\sigma_j^2 + \varepsilon)^{-3/2} \tag{294}$$

Démonstration. Rappelons l'expression de $\hat{Z}_j^{(i)}$ en fonction de σ_j^2 :

$$\hat{Z}_j^{(i)} = (Z_j^{(i)} - \mu_j) \cdot (\sigma_j^2 + \varepsilon)^{-1/2}$$

Par la règle de la chaîne, σ_j^2 influence L à travers tous les $\hat{Z}_j^{(i)}$:

$$\frac{\partial L}{\partial \sigma_j^2} = \sum_{i=1}^{m} \frac{\partial L}{\partial \hat{Z}_j^{(i)}} \cdot \frac{\partial \hat{Z}_j^{(i)}}{\partial \sigma_j^2}$$

Calculons la dérivée partielle. On pose $u = \sigma_j^2 + \varepsilon$ et on dérive $\hat{Z}_j^{(i)} = (Z_j^{(i)} - \mu_j) \cdot u^{-1/2}$ par rapport à u :

$$\frac{\partial \hat{Z}_j^{(i)}}{\partial \sigma_j^2} = (Z_j^{(i)} - \mu_j) \cdot \left(-\frac{1}{2}\right) u^{-3/2} = (Z_j^{(i)} - \mu_j) \cdot \left(-\frac{1}{2}\right) (\sigma_j^2 + \varepsilon)^{-3/2}$$

En substituant :

$$d\sigma_j^2 = \sum_{i=1}^{m} d\hat{Z}_j^{(i)} \cdot (Z_j^{(i)} - \mu_j) \cdot \left(-\frac{1}{2}\right) (\sigma_j^2 + \varepsilon)^{-3/2} \quad \checkmark$$

□

11.5.5 Gradient de μ

Proposition 11.5 (Gradient de la moyenne)**.**

$$d\mu_j = \underbrace{\sum_{i=1}^{m} d\hat{Z}_j^{(i)} \cdot \frac{-1}{\sqrt{\sigma_j^2 + \varepsilon}}}_{\text{contribution via } \hat{Z}} + \underbrace{d\sigma_j^2 \cdot \frac{-2}{m} \sum_{i=1}^{m} (Z_j^{(i)} - \mu_j)}_{\text{contribution via } \sigma^2} \tag{295}$$

Démonstration. La moyenne μ_j influence la perte L par **deux chemins** dans le graphe de calcul :

1. Directement via $\hat{Z}_j^{(i)} = (Z_j^{(i)} - \mu_j)(\sigma_j^2 + \varepsilon)^{-1/2}$
2. Indirectement via $\sigma_j^2 = \frac{1}{m} \sum_i (Z_j^{(i)} - \mu_j)^2$

Chemin 1 – via $\hat{Z}$:

$$\left.\frac{\partial L}{\partial \mu_j}\right|_{\text{via } \hat{Z}} = \sum_{i=1}^{m} \frac{\partial L}{\partial \hat{Z}_j^{(i)}} \cdot \frac{\partial \hat{Z}_j^{(i)}}{\partial \mu_j}$$

Or $\frac{\partial \hat{Z}_j^{(i)}}{\partial \mu_j} = \frac{-1}{\sqrt{\sigma_j^2+\varepsilon}}$ (car on dérive le numérateur $Z_j^{(i)} - \mu_j$ par rapport à μ_j, à σ_j^2 fixé). Donc :

$$\left.\frac{\partial L}{\partial \mu_j}\right|_{\text{via } \hat{Z}} = \sum_{i=1}^{m} d\hat{Z}_j^{(i)} \cdot \frac{-1}{\sqrt{\sigma_j^2 + \varepsilon}}$$

Chemin 2 – via σ^2 :

$$\left.\frac{\partial L}{\partial \mu_j}\right|_{\text{via } \sigma^2} = \frac{\partial L}{\partial \sigma_j^2} \cdot \frac{\partial \sigma_j^2}{\partial \mu_j}$$

Calculons $\frac{\partial \sigma_j^2}{\partial \mu_j}$:

$$\sigma_j^2 = \frac{1}{m}\sum_{i=1}^{m}(Z_j^{(i)} - \mu_j)^2 \quad \Rightarrow \quad \frac{\partial \sigma_j^2}{\partial \mu_j} = \frac{1}{m}\sum_{i=1}^{m} 2(Z_j^{(i)} - \mu_j)(-1) = \frac{-2}{m}\sum_{i=1}^{m}(Z_j^{(i)} - \mu_j)$$

Donc :

$$\left.\frac{\partial L}{\partial \mu_j}\right|_{\text{via } \sigma^2} = d\sigma_j^2 \cdot \frac{-2}{m}\sum_{i=1}^{m}(Z_j^{(i)} - \mu_j)$$

En sommant les deux contributions : $d\mu_j = \left.\frac{\partial L}{\partial \mu_j}\right|_{\text{via } \hat{Z}} + \left.\frac{\partial L}{\partial \mu_j}\right|_{\text{via } \sigma^2}$ ✓ □

Remarque 11.3. Le second terme de $d\mu_j$ (via σ^2) contient $\sum_i (Z_j^{(i)} - \mu_j)$, qui est **exactement nul** par définition de la moyenne. Donc en pratique :

$$d\mu_j = \sum_{i=1}^{m} d\hat{Z}_j^{(i)} \cdot \frac{-1}{\sqrt{\sigma_j^2 + \varepsilon}}$$

Néanmoins, on conserve le second terme dans l'implémentation pour la stabilité numérique (les erreurs d'arrondi font que $\sum_i (Z_j^{(i)} - \mu_j)$ n'est pas exactement nul en arithmétique flottante).

11.5.6 Gradient de Z (gradient final)

Proposition 11.6 (Gradient par rapport aux pré-activations)**.**

$$dZ_j^{(i)} = \underbrace{\frac{d\hat{Z}_j^{(i)}}{\sqrt{\sigma_j^2 + \varepsilon}}}_{\text{via } \hat{Z}} + \underbrace{\frac{2}{m}\, d\sigma_j^2 \cdot (Z_j^{(i)} - \mu_j)}_{\text{via } \sigma^2} + \underbrace{\frac{1}{m}\, d\mu_j}_{\text{via } \mu} \tag{296}$$

Démonstration. Chaque $Z_j^{(i)}$ influence L par **trois chemins** :

1. Directement via $\hat{Z}_j^{(i)} = (Z_j^{(i)} - \mu_j)(\sigma_j^2 + \varepsilon)^{-1/2}$
2. Via $\sigma_j^2 = \frac{1}{m}\sum_k (Z_j^{(k)} - \mu_j)^2$ (car $Z_j^{(i)}$ participe à la variance)
3. Via $\mu_j = \frac{1}{m}\sum_k Z_j^{(k)}$ (car $Z_j^{(i)}$ participe à la moyenne)

Chemin 1 – via $\hat{Z}$ directement :

$$\frac{\partial \hat{Z}_j^{(i)}}{\partial Z_j^{(i)}} = \frac{1}{\sqrt{\sigma_j^2 + \varepsilon}}$$

(à μ_j et σ_j^2 fixés, seul le numérateur $Z_j^{(i)} - \mu_j$ dépend de $Z_j^{(i)}$, avec une dérivée de 1). Contribution :

$$\left.\frac{\partial L}{\partial Z_j^{(i)}}\right|_{\text{via } \hat{Z}} = d\hat{Z}_j^{(i)} \cdot \frac{1}{\sqrt{\sigma_j^2 + \varepsilon}}$$

Chemin 2 – via σ^2 :

$$\frac{\partial \sigma_j^2}{\partial Z_j^{(i)}} = \frac{1}{m} \cdot 2(Z_j^{(i)} - \mu_j) = \frac{2}{m}(Z_j^{(i)} - \mu_j)$$

Contribution :

$$\left.\frac{\partial L}{\partial Z_j^{(i)}}\right|_{\text{via } \sigma^2} = d\sigma_j^2 \cdot \frac{2}{m}(Z_j^{(i)} - \mu_j)$$

Chemin 3 – via μ :

$$\frac{\partial \mu_j}{\partial Z_j^{(i)}} = \frac{1}{m}$$

Contribution :

$$\left.\frac{\partial L}{\partial Z_j^{(i)}}\right|_{\text{via } \mu} = d\mu_j \cdot \frac{1}{m}$$

En sommant les trois chemins :

$$dZ_j^{(i)} = \frac{d\hat{Z}_j^{(i)}}{\sqrt{\sigma_j^2 + \varepsilon}} + \frac{2}{m}\, d\sigma_j^2 (Z_j^{(i)} - \mu_j) + \frac{1}{m}\, d\mu_j \quad \checkmark$$

□

11.5.7 Récapitulatif et algorithme complet

Algorithme : Backpropagation à travers BatchNorm

Entrée : $d\widetilde{Z} \in \mathbb{R}^{n^{[l]} \times m}$ (gradient venant de la couche suivante), ainsi que les quantités mémorisées lors du forward : $Z, \hat{Z}, \mu, \sigma^2, \gamma$.

Étape 1 – Gradients des paramètres :

$$d\beta = d\widetilde{Z}\,\mathbf{1}_m \in \mathbb{R}^{n^{[l]} \times 1} \tag{297}$$

$$d\gamma = (d\widetilde{Z} \odot \hat{Z})\,\mathbf{1}_m \in \mathbb{R}^{n^{[l]} \times 1} \tag{298}$$

Étape 2 – Gradient de $\hat{Z}$:

$$d\hat{Z} = d\widetilde{Z} \odot \gamma \in \mathbb{R}^{n^{[l]} \times m} \tag{299}$$

Étape 3 – Gradient de la variance :

$$d\sigma^2 = \left(d\hat{Z} \odot (Z - \mu \mathbf{1}_m^T)\right) \mathbf{1}_m \odot \left(-\tfrac{1}{2}\right) (\sigma^2 + \varepsilon)^{-3/2} \in \mathbb{R}^{n^{[l]} \times 1} \tag{300}$$

Étape 4 – Gradient de la moyenne :

$$d\mu = (d\hat{Z})\mathbf{1}_m \odot \frac{-1}{\sqrt{\sigma^2 + \varepsilon}} + d\sigma^2 \odot \frac{-2}{m}(Z - \mu \mathbf{1}_m^T)\mathbf{1}_m \in \mathbb{R}^{n^{[l]} \times 1} \tag{301}$$

Étape 5 – Gradient de Z :

$$dZ = \frac{d\hat{Z}}{\sqrt{\sigma^2 + \varepsilon}\,\mathbf{1}_m^T} + \frac{2}{m}\, d\sigma^2 \odot (Z - \mu \mathbf{1}_m^T) + \frac{1}{m}\, d\mu\, \mathbf{1}_m^T \in \mathbb{R}^{n^{[l]} \times m} \tag{302}$$

Sortie : dZ (propagé aux couches précédentes), $d\gamma$ et $d\beta$ (pour la mise à jour des paramètres).

Exemple 11.3 (Vérification numérique de la rétropropagation)**.** Reprenons les valeurs de l'exemple 11.2 : un seul neurone, $m = 4$, $Z = (2, 4, 6, 8)$, $\mu = 5$, $\sigma^2 = 5$, $\gamma = 2$, $\beta = 3$, $\varepsilon = 0$.

Supposons un gradient amont uniforme $d\widetilde{Z} = (1, 1, 1, 1)$ (pour simplifier).

Étape 1 : $d\beta = 1 + 1 + 1 + 1 = 4$ et $d\gamma = 1 \times (-1.342) + 1 \times (-0.447) + 1 \times 0.447 + 1 \times 1.342 = 0$

Étape 2 : $d\hat{Z} = (1 \times 2,\ 1 \times 2,\ 1 \times 2,\ 1 \times 2) = (2, 2, 2, 2)$

Étape 3 : $d\sigma^2 = [2(-3) + 2(-1) + 2(1) + 2(3)] \times (-\frac{1}{2})(5)^{-3/2} = 0 \times (\cdots) = 0$

Étape 4 : $d\mu = (2 + 2 + 2 + 2) \times \frac{-1}{\sqrt{5}} + 0 = \frac{-8}{\sqrt{5}} \approx -3.578$

Étape 5 :

$$dZ^{(i)} = \frac{2}{\sqrt{5}} + 0 + \frac{1}{4}\left(\frac{-8}{\sqrt{5}}\right) = \frac{2}{\sqrt{5}} - \frac{2}{\sqrt{5}} = 0 \quad \text{pour tout } i$$

Interprétation : Le gradient $dZ = 0$ est logique ! Un gradient amont constant ($d\widetilde{Z} = \text{cste}$) demande de translater uniformément toutes les sorties, mais BatchNorm soustrait la moyenne : toute translation uniforme de Z est annulée. C'est précisément l'invariance par translation que confère BatchNorm.

11.6 Inférence : moyennes et variances mobiles

11.6.1 Le problème de l'inférence

Pendant l'entraînement, μ et σ^2 sont calculées sur le mini-batch courant. Mais à l'inférence (test/production), on peut traiter un seul exemple à la fois, ou des batchs de taille variable. Il y a deux problèmes fondamentaux :

Attention

[Pourquoi ne pas utiliser les statistiques du mini-batch à l'inférence]

- **Déterminisme :** La prédiction pour un exemple x dépendrait des *autres exemples* du batch. Deux batchs contenant le même x donneraient des prédictions différentes. C'est inacceptable en production.
- **Exemple unique :** Si $m = 1$, la variance est nulle ($\sigma^2 = 0$) et la normalisation $\hat{Z} = \frac{Z-Z}{0}$ est indéfinie.
- **Stabilité :** Les statistiques d'un petit batch sont bruitées (haute variance d'estimation), ce qui dégrade la qualité des prédictions.

11.6.2 Moyennes mobiles exponentielles

La solution est de maintenir, pendant l'entraînement, des **estimations globales** de la moyenne et de la variance, mises à jour à chaque mini-batch par une moyenne mobile exponentielle (exponential moving average, EMA).

Définition 11.3 (Mise à jour des statistiques d'inférence)**.** À chaque itération d'entraînement t, après avoir calculé μ_{batch} et σ^2_{batch} sur le mini-batch courant, on met à jour :

$$\mu^{(t)}_{\text{running}} = \alpha\, \mu^{(t-1)}_{\text{running}} + (1-\alpha)\, \mu_{\text{batch}} \tag{303}$$

$$\sigma^2_{\text{running}}{}^{(t)} = \alpha\, \sigma^2_{\text{running}}{}^{(t-1)} + (1-\alpha)\, \sigma^2_{\text{batch}} \tag{304}$$

où $\alpha \in [0, 1)$ est le **momentum** (typiquement $\alpha = 0.9$ ou 0.99).

Initialisation : $\mu^{(0)}_{\text{running}} = 0$ et $\sigma^2_{\text{running}}{}^{(0)} = 1$.

Justification Mathématique

La moyenne mobile exponentielle donne plus de poids aux batchs récents tout en lissant les fluctuations d'un batch à l'autre. Avec $\alpha = 0.9$, le poids d'un batch diminue d'un facteur 10 après 23 itérations ($0.9^{23} \approx 0.1$). Cela permet d'obtenir une estimation stable qui suit les variations lentes de la distribution (dues à l'apprentissage) tout en filtrant le bruit d'échantillonnage.

La forme $\alpha\, \mu_{\text{old}} + (1-\alpha)\, \mu_{\text{new}}$ est un filtre passe-bas du premier ordre : elle lisse les statistiques sur une "fenêtre" effective de $\frac{1}{1-\alpha}$ batchs. Avec $\alpha = 0.9$, c'est une fenêtre d'environ 10 batchs ; avec $\alpha = 0.99$, environ 100 batchs.

Définition 11.4 (Forward en mode inférence)**.** À l'inférence, la normalisation utilise les statistiques accumulées :

$$\widetilde{Z}^{(i)}_j = \gamma_j \cdot \frac{Z^{(i)}_j - \mu_{\text{running},j}}{\sqrt{\sigma^2_{\text{running},j} + \varepsilon}} + \beta_j \tag{305}$$

Cette formule est une transformation affine **fixe** (les paramètres $\gamma, \beta, \mu_{\text{running}}, \sigma^2_{\text{running}}$ ne dépendent pas du batch). Elle peut donc être fusionnée avec la couche linéaire précédente pour accélérer l'inférence.

Proposition 11.7 (Fusion avec la couche linéaire)**.** À l'inférence, la séquence {couche linéaire + BatchNorm} peut être remplacée par une unique couche linéaire. En posant :

$$\widetilde{W}_j = \frac{\gamma_j}{\sqrt{\sigma^2_{\text{running},j} + \varepsilon}} W_j, \qquad \widetilde{b}_j = \beta_j - \frac{\gamma_j \mu_{\text{running},j}}{\sqrt{\sigma^2_{\text{running},j} + \varepsilon}}$$

on a $\widetilde{Z}_j = \widetilde{W}_j A^{[l-1]} + \widetilde{b}_j$, soit une couche linéaire standard sans surcoût de calcul.

Démonstration. Développons l'expression de l'inférence BatchNorm :

$$\widetilde{Z}_j^{(i)} = \gamma_j \cdot \frac{Z_j^{(i)} - \mu_{\text{running},j}}{\sqrt{\sigma^2_{\text{running},j} + \varepsilon}} + \beta_j \tag{306}$$

$$= \frac{\gamma_j}{\sqrt{\sigma^2_{\text{running},j} + \varepsilon}} Z_j^{(i)} - \frac{\gamma_j \mu_{\text{running},j}}{\sqrt{\sigma^2_{\text{running},j} + \varepsilon}} + \beta_j \tag{307}$$

$$= \frac{\gamma_j}{\sqrt{\sigma^2_{\text{running},j} + \varepsilon}} \left(\sum_k W_{jk} A_k^{[l-1]} \right) + \underbrace{\beta_j - \frac{\gamma_j \mu_{\text{running},j}}{\sqrt{\sigma^2_{\text{running},j} + \varepsilon}}}_{= \widetilde{b}_j} \tag{308}$$

ce qui est bien de la forme $\widetilde{W}_j A^{[l-1]} + \widetilde{b}_j$ ✓ □

Remarque 11.4. Cette fusion est couramment utilisée dans les modèles déployés en production (mobile, embarqué). Elle élimine entièrement le surcoût de BatchNorm à l'inférence : il n'y a plus de calcul de moyenne, de variance, ni de normalisation – juste un produit matriciel avec des poids pré-calculés.

11.7 Placement dans l'architecture et bonnes pratiques

11.7.1 Avant ou après l'activation ?

Attention

[Placement de BatchNorm] La question du placement de BatchNorm par rapport à l'activation a fait l'objet de débats. Deux approches existent :
Placement original (Ioffe & Szegedy, 2015) – Avant l'activation :

$$A^{[l-1]} \xrightarrow{W^{[l]}} Z^{[l]} \xrightarrow{\text{BN}} \widetilde{Z}^{[l]} \xrightarrow{g} A^{[l]}$$

L'argument : on normalise les entrées de la non-linéarité pour qu'elles restent dans sa zone active. C'est le choix le plus courant dans la littérature et les frameworks.
Placement alternatif – Après l'activation :

$$A^{[l-1]} \xrightarrow{W^{[l]}} Z^{[l]} \xrightarrow{g} A^{[l]} \xrightarrow{\text{BN}} \widetilde{A}^{[l]}$$

Certains travaux empiriques montrent des résultats légèrement meilleurs dans ce cas, mais la différence est généralement faible.
Recommandation : Utiliser la convention Dense/Conv $\rightarrow$ BN $\rightarrow$ Activation, sauf si des expériences spécifiques montrent un avantage à l'autre placement.

11.7.2 Exigences sur la taille du mini-batch

Attention

[Taille minimale du mini-batch] Les statistiques μ et σ^2 sont estimées sur le mini-batch. Leur qualité dépend directement de m :

- $m \geq 32$ **:** Bonnes estimations, BatchNorm fonctionne bien.
- $16 \leq m < 32$ **:** Acceptable mais les estimations sont bruitées.
- $m < 16$ **:** Estimations très instables. La variance estimée peut être très éloignée de la vraie variance, ce qui dégrade les performances. Préférer LayerNorm ou GroupNorm dans ce cas.
- $m = 1$ **:** Impossible (variance nulle). BatchNorm est inutilisable.

11.7.3 Effet régularisant

Intuition

BatchNorm a un effet régularisant implicite. À chaque itération, les statistiques μ et σ^2 sont bruitées (car calculées sur un sous-ensemble aléatoire des données). Ce bruit agit comme une forme de perturbation aléatoire des activations, similaire à un Dropout léger. C'est pourquoi on observe souvent qu'en ajoutant BatchNorm, on peut réduire ou supprimer d'autres formes de régularisation (Dropout, weight decay).
Attention cependant : cet effet régularisant **diminue** quand la taille du mini-batch augmente (les estimations deviennent plus précises, donc moins bruitées). Avec de très grands batchs ($m > 1000$), l'effet régularisant de BatchNorm est négligeable.

11.8 Variantes de la normalisation

BatchNorm n'est pas la seule forme de normalisation. Selon le contexte (petits batchs, séquences de longueur variable, etc.), d'autres stratégies sont préférables. Toutes partagent la même structure : normaliser, puis appliquer une mise à l'échelle apprise γ et β.

Définition 11.5 (Layer Normalization (Ba et al., 2016))**.** Au lieu de normaliser sur le mini-batch (dimension m), on normalise sur les neurones (dimension $n^{[l]}$) pour chaque exemple séparément :

$$\mu^{(i)} = \frac{1}{n^{[l]}} \sum_{j=1}^{n^{[l]}} Z_j^{(i)}, \qquad \sigma^{2(i)} = \frac{1}{n^{[l]}} \sum_{j=1}^{n^{[l]}} (Z_j^{(i)} - \mu^{(i)})^2$$

$$\hat{Z}_j^{(i)} = \frac{Z_j^{(i)} - \mu^{(i)}}{\sqrt{\sigma^{2(i)} + \varepsilon}}, \qquad \widetilde{Z}_j^{(i)} = \gamma_j \hat{Z}_j^{(i)} + \beta_j$$

Définition 11.6 (Group Normalization (Wu & He, 2018))**.** On divise les $n^{[l]}$ neurones (ou canaux, en convolution) en G groupes de taille $n^{[l]}/G$, et on normalise au sein de chaque groupe pour chaque exemple :

$$\mu_g^{(i)} = \frac{1}{|S_g|} \sum_{j \in S_g} Z_j^{(i)}, \qquad \sigma_g^{2(i)} = \frac{1}{|S_g|} \sum_{j \in S_g} (Z_j^{(i)} - \mu_g^{(i)})^2$$

où S_g est l'ensemble des indices des neurones du groupe g.

Remarque 11.5. Les trois normalisations diffèrent par les **axes** sur lesquels on calcule la moyenne et la variance :

Méthode	**Axes de normalisation**	**Cas d'usage**
BatchNorm	Mini-batch (m)	Réseaux convolutifs, grands batchs
LayerNorm	Neurones ($n^{[l]}$)	Transformers, RNNs, m petit ou variable
GroupNorm	Sous-groupe de neurones	Détection d'objets, petits batchs

LayerNorm est l'approche standard dans les architectures Transformer (GPT, BERT, etc.) car elle ne dépend pas de la taille du batch et fonctionne naturellement avec des séquences de longueur variable.

11.9 Illustration et stabilisation de l'entraînement

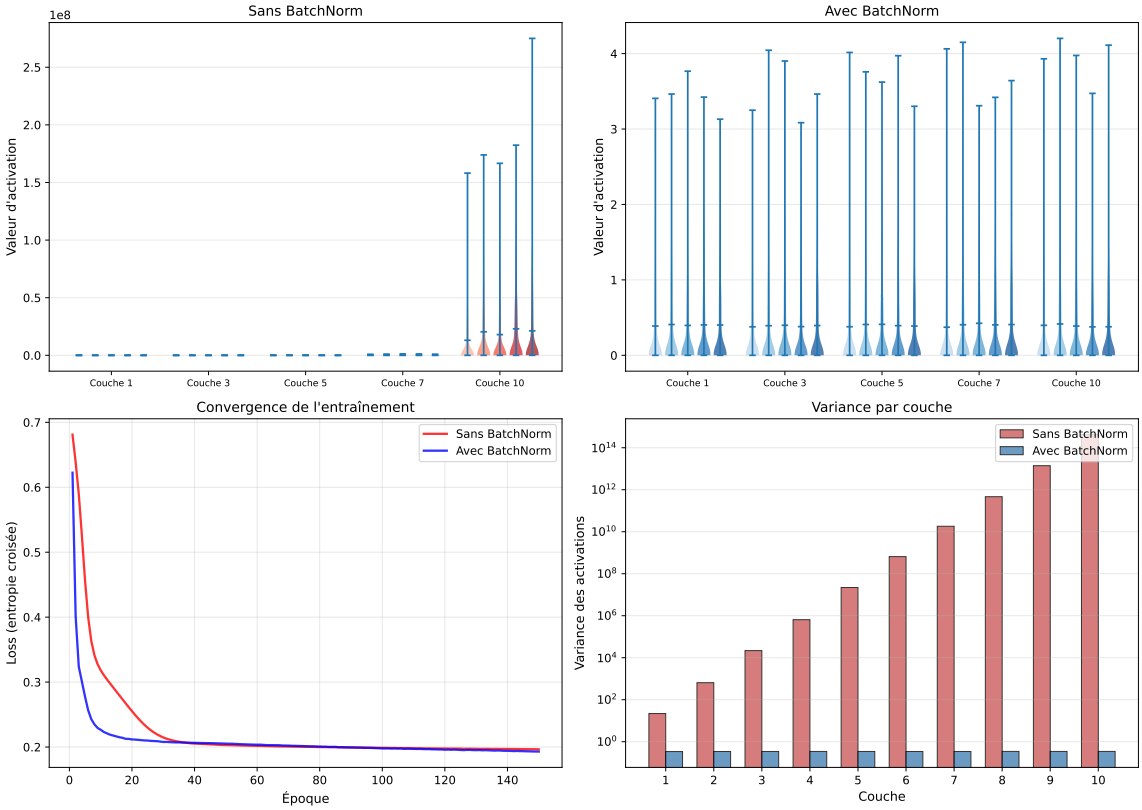

Figure 9 : **Gauche :** Évolution de la distribution des pré-activations $Z^{[3]}$ au cours de l'entraînement sur le problème des cercles concentriques, **sans** BatchNorm. La moyenne et la variance dérivent fortement. **Centre :** Avec BatchNorm, la distribution reste centrée et de variance contrôlée tout au long de l'entraînement. **Droite :** Courbes de perte comparées – BatchNorm permet d'utiliser un learning rate plus élevé et converge plus rapidement.

Intuition

La Figure 9 illustre les trois bénéfices principaux de BatchNorm :

1. **Stabilisation des distributions :** les pré-activations restent dans une plage contrôlée, ce qui maintient les fonctions d'activation dans leur zone utile.
2. **Tolérance au learning rate :** sans BatchNorm, un learning rate trop élevé fait diverger l'entraînement. Avec BatchNorm, la normalisation "absorbe" une partie des variations, permettant des pas plus grands.
3. **Convergence accélérée :** en combinant les deux effets précédents, l'entraînement atteint un meilleur coût en moins d'itérations.

11.10 Résumé : le pipeline complet d'une couche avec BatchNorm

Pipeline : couche dense avec BatchNorm

Forward (entraînement) :

1. Transformation linéaire : $Z^{[l]} = W^{[l]} A^{[l-1]}$ (pas de biais, car β le remplace)
2. Moyenne : $\mu = \frac{1}{m} Z^{[l]} \mathbf{1}_m$
3. Variance : $\sigma^2 = \frac{1}{m}(Z^{[l]} - \mu \mathbf{1}_m^T)^{\odot 2} \mathbf{1}_m$
4. Normalisation : $\hat{Z} = (Z^{[l]} - \mu \mathbf{1}_m^T) \oslash (\sqrt{\sigma^2 + \varepsilon}\, \mathbf{1}_m^T)$
5. Scale & shift : $\widetilde{Z} = \gamma \odot \hat{Z} + \beta \mathbf{1}_m^T$
6. Activation : $A^{[l]} = g(\widetilde{Z})$
7. Mise à jour des running stats : $\mu_{\text{run}} \leftarrow \alpha\, \mu_{\text{run}} + (1-\alpha)\, \mu$, idem pour σ^2

Forward (inférence) :

1. $\widetilde{Z} = \gamma \odot \frac{Z^{[l]} - \mu_{\text{run}}}{\sqrt{\sigma^2_{\text{run}} + \varepsilon}} + \beta$ (ou couche linéaire fusionnée)
2. $A^{[l]} = g(\widetilde{Z})$

Backward : $d\widetilde{Z} \to d\gamma, d\beta \mid d\hat{Z} \to d\sigma^2 \to d\mu \to dZ$
Paramètres appris : $W^{[l]}, \gamma, \beta$ (pas de $b^{[l]}$)
Hyperparamètres : ε (stabilité), α (momentum EMA)

12 Optimiseurs avancés : inertie et adaptativité

12.1 Motivation : les limites de SGD dans les vallées étroites

Dans la section 4, nous avons introduit la descente de gradient sous sa forme la plus simple :

$$\theta_{t+1} = \theta_t - \alpha \nabla L(\theta_t).$$

Avec le gradient stochastique (SGD), le gradient exact ∇L est remplacé par une estimation bruitée g_t calculée sur un mini-batch. Ce schéma fonctionne, mais il souffre de limitations sévères dès que la surface de perte est mal conditionnée. Exhibons le problème sur un exemple concret.

Exemple 12.1 (Vallée étroite en 2D)**.** Considérons la fonction quadratique

$$L(w_1, w_2) = \frac{1}{2}\left(100\, w_1^2 + w_2^2\right). \tag{309}$$

C'est une surface en forme de vallée très allongée : étroite dans la direction w_1, large dans la direction w_2. Les courbes de niveau sont des ellipses dont le demi-axe en w_1 est 10 fois plus court que celui en w_2.

Le gradient vaut :

$$\nabla L = \begin{pmatrix} 100\, w_1 \\ w_2 \end{pmatrix}.$$

Partons de $w^{(0)} = (1,1)^T$ avec un learning rate $\alpha = 0{,}01$. Calculons les premières itérations :

$$w^{(1)} = \begin{pmatrix} 1 \\ 1 \end{pmatrix} - 0{,}01 \begin{pmatrix} 100 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 0{,}99 \end{pmatrix} \tag{310}$$

$$w^{(2)} = \begin{pmatrix} 0 \\ 0{,}99 \end{pmatrix} - 0{,}01 \begin{pmatrix} 0 \\ 0{,}99 \end{pmatrix} = \begin{pmatrix} 0 \\ 0{,}9801 \end{pmatrix} \tag{311}$$

$$w^{(3)} = \begin{pmatrix} 0 \\ 0{,}9801 \end{pmatrix} - 0{,}01 \begin{pmatrix} 0 \\ 0{,}9801 \end{pmatrix} = \begin{pmatrix} 0 \\ 0{,}970299 \end{pmatrix} \tag{312}$$

La composante w_1 converge en une seule itération, mais w_2 progresse péniblement : $w_2^{(t)} = 0{,}99^t$, il faut environ $t = 459$ itérations pour atteindre $w_2 < 0{,}01$.

Augmenter α ? Si l'on prend $\alpha = 0{,}02$, la mise à jour en w_1 donne $w_1^{(1)} = 1 - 0{,}02 \times 100 = -1$: on oscille sans converger ! Pour $\alpha > 2/100 = 0{,}02$, la composante w_1 diverge.

Définition 12.1 (Matrice Hessienne et nombre de condition)**.** La matrice Hessienne de L est la matrice des dérivées secondes :

$$H = \nabla^2 L = \begin{pmatrix} \dfrac{\partial^2 L}{\partial w_1^2} & \dfrac{\partial^2 L}{\partial w_1 \partial w_2} \\ \dfrac{\partial^2 L}{\partial w_2 \partial w_1} & \dfrac{\partial^2 L}{\partial w_2^2} \end{pmatrix}.$$

Le **nombre de condition** de H est le rapport entre sa plus grande et sa plus petite valeur propre :

$$\kappa(H) = \frac{\lambda_{\max}}{\lambda_{\min}}.$$

Proposition 12.1 (Analyse spectrale de l'exemple)**.** Pour $L(w_1, w_2) = \frac{1}{2}(100\, w_1^2 + w_2^2)$, la Hessienne est :

$$H = \begin{pmatrix} 100 & 0 \\ 0 & 1 \end{pmatrix}.$$

Les valeurs propres sont $\lambda_1 = 100$ et $\lambda_2 = 1$, d'où $\kappa(H) = 100$.

Démonstration. La Hessienne est diagonale, donc ses valeurs propres sont ses coefficients diagonaux :

$$\frac{\partial^2 L}{\partial w_1^2} = 100, \qquad \frac{\partial^2 L}{\partial w_2^2} = 1, \qquad \frac{\partial^2 L}{\partial w_1 \partial w_2} = 0.$$

Le nombre de condition est $\kappa = 100/1 = 100$. □

Justification Mathématique

Pourquoi le nombre de condition est-il le cœur du problème ?
Sur une fonction quadratique $L(w) = \frac{1}{2}w^T Hw$, la descente de gradient avec pas optimal $\alpha^* = 2/(\lambda_{\max} + \lambda_{\min})$ converge avec un taux :

$$L(w^{(t)}) \leq \left(\frac{\kappa - 1}{\kappa + 1}\right)^{2t} L(w^{(0)}).$$

Application numérique. Avec $\kappa = 100$, le facteur de contraction par itération est $\left(\frac{99}{101}\right)^2 \approx 0{,}961$, soit environ 4% de réduction par itération. Il faut environ $t \approx 250$ itérations pour réduire la perte d'un facteur 10^{-4}.
Si $\kappa = 10\,000$ (typique en deep learning), le facteur vaut $\left(\frac{9999}{10001}\right)^2 \approx 0{,}9996$, et il faut $\sim 25\,000$ itérations pour le même gain.
Les optimiseurs avancés visent à ramener la convergence effective à un nombre de condition $\kappa_{\text{eff}} \approx \sqrt{\kappa}$ (momentum) ou $\kappa_{\text{eff}} \approx 1$ (méthodes adaptatives).

Attention

[Trois limites majeures de SGD]

1. **Vallées étroites.** Un learning rate assez petit pour la direction de forte courbure ($\alpha < 2/\lambda_{\max}$) est trop petit pour progresser rapidement dans les directions de faible courbure.
2. **Bruit stochastique.** Le gradient estimé sur un mini-batch fluctue, provoquant un zigzag autour du fond de la vallée qui ne diminue pas en amplitude avec les itérations (à α constant).
3. **Échelles hétérogènes des paramètres.** Un même learning rate α est appliqué uniformément à tous les paramètres, alors que les gradients de certains poids (par exemple dans les couches profondes) peuvent être 1 000 fois plus grands que d'autres.

12.2 Moyenne mobile exponentielle (EMA)

Avant de construire des optimiseurs sophistiqués, introduisons un outil fondamental qui apparaît dans chacun d'entre eux : la moyenne mobile exponentielle.

Définition 12.2 (Moyenne mobile exponentielle)**.** Étant donnée une suite de valeurs $(g_1, g_2, \ldots)$ et un coefficient $\beta \in [0, 1)$, la **moyenne mobile exponentielle** (EMA) est la suite (v_t) définie par :

$$v_0 = 0, \qquad v_t = \beta\, v_{t-1} + (1 - \beta)\, g_t, \quad t \geq 1. \tag{313}$$

Proposition 12.2 (Forme explicite de l'EMA)**.** La récurrence se déroule en une somme pondérée :

$$v_t = (1 - \beta) \sum_{k=1}^{t} \beta^{t-k}\, g_k = (1 - \beta) \sum_{j=0}^{t-1} \beta^j\, g_{t-j}. \tag{314}$$

Démonstration. Procédons par récurrence. Pour $t = 1$: $v_1 = \beta \cdot 0 + (1 - \beta)g_1 = (1 - \beta)g_1$. Cela correspond bien à la somme avec un seul terme ($k = 1$) : $(1 - \beta)\beta^{1-1}g_1 = (1 - \beta)g_1$.

Supposons la formule vraie au rang $t-1$:

$$v_{t-1} = (1-\beta)\sum_{k=1}^{t-1} \beta^{t-1-k} g_k.$$

Alors :

$$\begin{aligned}
v_t &= \beta\, v_{t-1} + (1-\beta)\, g_t && (315)\\
&= \beta(1-\beta)\sum_{k=1}^{t-1} \beta^{t-1-k} g_k + (1-\beta) g_t && (316)\\
&= (1-\beta)\sum_{k=1}^{t-1} \beta^{t-k} g_k + (1-\beta)\beta^{t-t} g_t && (317)\\
&= (1-\beta)\sum_{k=1}^{t} \beta^{t-k} g_k. \qquad \checkmark && (318)
\end{aligned}$$

□

Proposition 12.3 (Somme des poids)**.** Les coefficients de pondération dans l'EMA somment à $1-\beta^t$:

$$(1-\beta)\sum_{j=0}^{t-1} \beta^j = 1-\beta^t. \tag{319}$$

Démonstration. C'est la formule de la somme géométrique partielle :

$$\sum_{j=0}^{t-1} \beta^j = \frac{1-\beta^t}{1-\beta}.$$

En multipliant par $(1-\beta)$, on obtient $1-\beta^t$. □

Attention

[Biais aux premiers pas] Puisque $v_0 = 0$, les premiers termes de v_t sont systématiquement sous-estimés. En effet, si $g_k = \mu$ pour tout k (signal constant), alors :

$$v_t = (1-\beta)\mu\sum_{j=0}^{t-1} \beta^j = \mu(1-\beta^t).$$

Pour t petit, β^t est proche de 1 et $v_t \ll \mu$. Ce n'est que lorsque $t \to \infty$ que $v_t \to \mu$. Exemple numérique avec $\beta = 0{,}9$ et $\mu = 1$:

t	1	2	5	10	20	50
v_t	0,1	0,19	0,41	0,65	0,88	0,995

Au pas $t = 1$, l'EMA ne vaut que 10% de la vraie valeur ! La correction de biais (que nous verrons dans Adam) consiste à diviser par $1-\beta^t$ pour obtenir un estimateur non biaisé.

Intuition

L'EMA avec coefficient β moyenne approximativement les $\frac{1}{1-\beta}$ dernières valeurs. En effet, le poids du terme datant de j pas est $(1-\beta)\beta^j$, et β^j tombe en dessous de $1/e \approx 0{,}37$ pour $j \geq 1/(1-\beta)$.

β	Fenêtre effective	Usage typique
0,9	$\sim$ 10 pas	Premier moment (Adam β_1)
0,99	$\sim$ 100 pas	Lissage fort
0,999	$\sim$ 1000 pas	Second moment (Adam β_2)

12.3 SGD avec momentum

12.3.1 Définition et analogie physique

Définition 12.3 (SGD avec momentum)**.** L'algorithme SGD avec momentum maintient une **vitesse** v_t qui accumule les gradients passés :

$$v_t = \beta\, v_{t-1} + (1-\beta)\, g_t, \qquad v_0 = 0, \tag{320}$$

$$\theta_{t+1} = \theta_t - \alpha\, v_t, \tag{321}$$

où $g_t = \nabla L(\theta_t)$ est le gradient (ou son estimation stochastique), $\beta \in [0, 1)$ est le coefficient de momentum, et $\alpha > 0$ est le learning rate.

Remarque 12.1. Il existe une variante équivalente, fréquente dans PyTorch, qui écrit $v_t = \beta v_{t-1} + g_t$ et $\theta_{t+1} = \theta_t - \alpha v_t$. Cette formulation absorbe le facteur $(1-\beta)$ dans le learning rate. Les deux sont mathématiquement équivalentes à un reparamétrage de α près ; nous utilisons la convention avec $(1-\beta)$ car elle rend l'EMA explicite et unifie la présentation avec Adam.

Justification Mathématique

Le momentum provient d'une analogie avec la mécanique classique. Considérons une bille de masse m glissant sur la surface de perte L, soumise à la gravité (gradient) et à une force de frottement visqueux proportionnelle à sa vitesse :

$$m\ddot{x} = -\nabla L(x) - \gamma \dot{x}, \tag{322}$$

où $\gamma > 0$ est le coefficient de friction.
Discrétisons cette ODE avec un pas de temps Δt. Posons $\dot{x} \approx \frac{x_{t+1}-x_t}{\Delta t}$ et $\ddot{x} \approx \frac{\dot{x}_t - \dot{x}_{t-1}}{\Delta t}$.
En notant $v_t = \dot{x}_t$:

$$m\frac{v_t - v_{t-1}}{\Delta t} = -\nabla L(x_t) - \gamma v_t \tag{323}$$

$$(m + \gamma \Delta t)\, v_t = m\, v_{t-1} - \Delta t\, \nabla L(x_t) \tag{324}$$

$$v_t = \underbrace{\frac{m}{m + \gamma \Delta t}}_{\beta} v_{t-1} - \underbrace{\frac{\Delta t}{m + \gamma \Delta t}}_{1-\beta} \nabla L(x_t). \tag{325}$$

On retrouve exactement la mise à jour du momentum avec $\beta = \frac{m}{m+\gamma\Delta t}$, ce qui confirme l'analogie : β proche de 1 correspond à une masse élevée par rapport au frottement (inertie forte, bille lourde qui continue sur sa lancée), tandis que β petit correspond à un frottement dominant (la bille s'arrête vite).

Intuition

L'effet du momentum est double :

- **Accélération.** Dans une direction où le gradient pointe de manière cohérente (le fond de la vallée en w_2), les termes g_{t-k} se cumulent et la vitesse v_t croît, accélérant la descente.
- **Amortissement.** Dans une direction où le gradient oscille (les parois de la vallée en w_1), les termes positifs et négatifs s'annulent partiellement, réduisant l'amplitude des oscillations.

12.3.2 Analyse de convergence sur une quadratique

Proposition 12.4 (Convergence du momentum sur $L(w) = \frac{c}{2}w^2$)**.** Pour la fonction quadratique unidimensionnelle $L(w) = \frac{c}{2}w^2$ (avec gradient $g_t = c\, w_t$), la dynamique du momentum s'écrit comme un système linéaire d'ordre 2 :

$$\begin{pmatrix} w_{t+1} \\ v_{t+1} \end{pmatrix} = M \begin{pmatrix} w_t \\ v_t \end{pmatrix}, \qquad M = \begin{pmatrix} 1 - \alpha(1-\beta)c & -\alpha\beta \\ (1-\beta)c & \beta \end{pmatrix}. \tag{326}$$

La convergence est déterminée par le rayon spectral $\rho(M) = \max(|\lambda_1|, |\lambda_2|)$, où $\lambda_{1,2}$ sont les valeurs propres de M. La convergence est géométrique avec taux $\rho(M)^t$.

Démonstration. Écrivons les deux équations de mise à jour :

$$v_{t+1} = \beta\, v_t + (1-\beta)\, c\, w_t, \tag{327}$$

$$w_{t+1} = w_t - \alpha\, v_{t+1} = w_t - \alpha[\beta\, v_t + (1-\beta)\, c\, w_t]. \tag{328}$$

En substituant v_{t+1} dans l'expression de w_{t+1} :

$$w_{t+1} = w_t - \alpha(1-\beta)c\, w_t - \alpha\beta\, v_t = [1 - \alpha(1-\beta)c]\, w_t - \alpha\beta\, v_t, \tag{329}$$

$$v_{t+1} = (1-\beta)c\, w_t + \beta\, v_t. \tag{330}$$

D'où la matrice M annoncée. □

Exemple 12.2 (Comparaison SGD vs. Momentum sur l'exemple 2D)**.** Revenons à $L(w_1, w_2) = \frac{1}{2}(100\, w_1^2 + w_2^2)$. Comme les deux composantes sont découplées, analysons-les séparément.

SGD pur ($\beta = 0$, $\alpha = 0{,}01$) :

— Direction w_1 ($c = 100$) : $w_1^{(t+1)} = (1 - 0{,}01 \times 100)\, w_1^{(t)} = 0$. Convergence en 1 itération.

— Direction w_2 ($c = 1$) : $w_2^{(t+1)} = (1 - 0{,}01)\, w_2^{(t)} = 0{,}99\, w_2^{(t)}$. Taux $= 0{,}99$. Pour atteindre $|w_2| < 10^{-4}$: $0{,}99^t < 10^{-4}$, soit $t > 917$ itérations.

SGD + Momentum ($\beta = 0{,}9$, $\alpha = 0{,}01$) :

Direction w_2 ($c = 1$) : la matrice vaut

$$M_{w_2} = \begin{pmatrix} 1 - 0{,}001 & -0{,}009 \\ 0{,}1 & 0{,}9 \end{pmatrix} = \begin{pmatrix} 0{,}999 & -0{,}009 \\ 0{,}1 & 0{,}9 \end{pmatrix}.$$

Le polynôme caractéristique est :

$$\det(M - \lambda I) = (0{,}999 - \lambda)(0{,}9 - \lambda) - (-0{,}009)(0{,}1) \tag{331}$$

$$= \lambda^2 - 1{,}899\lambda + 0{,}8991 + 0{,}0009 \tag{332}$$

$$= \lambda^2 - 1{,}899\lambda + 0{,}9. \tag{333}$$

Le discriminant est $\Delta = 1{,}899^2 - 4 \times 0{,}9 = 3{,}606 - 3{,}6 = 0{,}006 > 0$, donc deux racines réelles :

$$\lambda = \frac{1{,}899 \pm \sqrt{0{,}006}}{2} \approx \frac{1{,}899 \pm 0{,}077}{2}.$$

D'où $\lambda_1 \approx 0{,}988$ et $\lambda_2 \approx 0{,}911$. Le rayon spectral est $\rho \approx 0{,}988$.

Pour atteindre $|w_2| < 10^{-4}$: $0{,}988^t < 10^{-4}$, soit $t > \frac{4 \ln 10}{\ln(1/0{,}988)} \approx 763$ itérations.

Le gain est modeste ici (763 vs. 917), mais sur des problèmes à κ très grand, le momentum réduit le nombre d'itérations de $O(\kappa)$ à $O(\sqrt{\kappa})$ avec un choix optimal de β.

Proposition 12.5 (Choix optimal du momentum sur une quadratique)**.** Pour la fonction $L(w) = \frac{1}{2} w^T H w$ avec H symétrique définie positive de valeurs propres $\lambda_{\min}$ et $\lambda_{\max}$, le taux de convergence optimal du momentum est atteint pour :

$$\alpha^* = \frac{4}{(\sqrt{\lambda_{\max}} + \sqrt{\lambda_{\min}})^2}, \qquad \beta^* = \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1}\right)^2,$$

avec un taux de convergence $\rho^* = \frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}$, à comparer avec $\frac{\kappa-1}{\kappa+1}$ pour SGD sans momentum.

Justification Mathématique

Pour $\kappa = 100$:

— SGD : taux $= \frac{99}{101} \approx 0{,}980$. Pour 10^{-4} de réduction : ~ 460 itérations.

— Momentum optimal : taux $= \frac{9}{11} \approx 0{,}818$. Pour 10^{-4} de réduction : ~ 46 itérations.

Le momentum permet une accélération d'un facteur ~ 10 pour $\kappa = 100$, et généralement $\sim \sqrt{\kappa}$.

12.4 Momentum de Nesterov

Définition 12.4 (Momentum de Nesterov (NAG))**.** Le momentum de Nesterov évalue le gradient non pas au point courant θ_t, mais au point **anticipé** $\theta_t - \alpha\beta\, v_{t-1}$:

$$v_t = \beta\, v_{t-1} + (1-\beta)\, \nabla L(\theta_t - \alpha\beta\, v_{t-1})\,, \tag{334}$$

$$\theta_{t+1} = \theta_t - \alpha\, v_t. \tag{335}$$

Intuition

Le momentum classique avance "à l'aveugle" : il calcule le gradient au point courant, puis fait un grand pas dans la direction accumulée. Si la surface de perte courbe brusquement (par exemple au fond d'une vallée), le momentum classique peut "dépasser" le minimum avant de corriger.
Nesterov résout ce problème en deux temps :

1. **Anticipation.** On fait d'abord un pas provisoire dans la direction du momentum : $\tilde{\theta} = \theta_t - \alpha\beta\, v_{t-1}$. C'est l'endroit où le momentum nous emmènerait si le gradient était nul.
2. **Correction.** On évalue le gradient en ce point anticipé $\nabla L(\tilde{\theta})$. Si la pente a changé de signe (on a dépassé le minimum), cette correction freine la descente *avant* qu'il ne soit trop tard.

En résumé : le momentum classique dit "je regarde mes pieds puis j'avance", tandis que Nesterov dit "j'avance d'abord, puis je regarde où j'ai atterri pour corriger".

Justification Mathématique

Sur une fonction quadratique, le momentum de Nesterov atteint le même taux de convergence optimal $\rho = \frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}$ que le momentum classique avec paramètres optimaux. Son avantage est surtout pratique : sur des fonctions non quadratiques (cas des réseaux de neurones), la correction anticipée offre une meilleure réactivité aux changements de courbure, ce qui se traduit par un entraînement plus stable.

12.5 Pas adaptatifs : RMSProp

Les méthodes à momentum accélèrent SGD dans les directions cohérentes, mais elles utilisent toujours le même learning rate α pour toutes les coordonnées. L'idée des méthodes **adaptatives** est d'attribuer un pas différent à chaque paramètre, en fonction de l'historique de ses gradients.

Définition 12.5 (RMSProp (Hinton, 2012))**.** RMSProp maintient une EMA du carré des gradients et divise la mise à jour par sa racine :

$$s_t = \rho\, s_{t-1} + (1-\rho)\, g_t^2, \qquad s_0 = 0, \tag{336}$$

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{s_t} + \varepsilon}\, g_t, \tag{337}$$

où $\rho \in [0,1)$ (typiquement $\rho = 0{,}99$), $\varepsilon > 0$ (typiquement 10^{-8}) est un terme de stabilité numérique, et g_t^2 et $\sqrt{s_t}$ s'entendent **coordonnée par coordonnée** (opérations élément par élément).

Justification Mathématique

Le terme $\sqrt{s_t}$ estime l'écart-type (RMS = Root Mean Square) des gradients récents pour chaque coordonnée. Diviser par cette quantité a deux effets :

— **Paramètres à forts gradients** (direction de forte courbure, comme w_1 dans notre exemple) : s_t est grand, le pas effectif $\alpha/\sqrt{s_t}$ est petit. On freine les oscillations.

— **Paramètres à faibles gradients** (direction de faible courbure, comme w_2) : s_t est petit, le pas effectif est grand. On accélère la progression.

Exemple 12.3 (RMSProp sur l'exemple 2D)**.** Reprenons $L(w_1, w_2) = \frac{1}{2}(100\, w_1^2 + w_2^2)$ avec $w^{(0)} = (1, 1)^T$, $\alpha = 0{,}1$, $\rho = 0{,}9$, $\varepsilon = 10^{-8}$.

Itération 1 :

$$g_1 = (100, 1)^T \tag{338}$$

$$s_1 = 0{,}1 \times g_1^2 = (0{,}1 \times 10000,\ 0{,}1 \times 1)^T = (1000, 0{,}1)^T \tag{339}$$

$$\Delta w = -\frac{0{,}1}{\sqrt{s_1}} \odot g_1 = \left(-\frac{0{,}1}{\sqrt{1000}} \times 100,\ -\frac{0{,}1}{\sqrt{0{,}1}} \times 1\right)^T \tag{340}$$

$$\approx (-0{,}316,\ -0{,}316)^T. \tag{341}$$

Observation remarquable : malgré le fait que le gradient en w_1 est 100 fois plus grand qu'en w_2, la mise à jour effective est de même amplitude dans les deux directions ! RMSProp a automatiquement normalisé les échelles.

Après cette mise à jour : $w^{(1)} \approx (0{,}684,\ 0{,}684)^T$.

Attention

RMSProp n'a pas de correction de biais pour s_t. Aux premières itérations, s_t sous-estime $\mathbb{E}[g_t^2]$, ce qui peut provoquer des pas initiaux trop grands. C'est l'une des motivations pour Adam, qui corrige ce biais.

12.6 Adam : Adaptive Moment Estimation

12.6.1 Définition complète

Définition 12.6 (Adam (Kingma & Ba, 2015))**.** Adam combine le momentum (estimation du premier moment) et RMSProp (estimation du second moment), avec correction de biais pour les deux :

Initialisation : $m_0 = 0$, $v_0 = 0$.

À chaque itération $t \geq 1$ **:**

$$m_t = \beta_1\, m_{t-1} + (1 - \beta_1)\, g_t \qquad \text{(EMA du gradient)} \tag{342}$$

$$v_t = \beta_2\, v_{t-1} + (1 - \beta_2)\, g_t^2 \qquad \text{(EMA du gradient carré)} \tag{343}$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \qquad \text{(correction de biais, 1er moment)} \tag{344}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t} \qquad \text{(correction de biais, 2nd moment)} \tag{345}$$

$$\theta_{t+1} = \theta_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon} \qquad \text{(mise à jour)} \tag{346}$$

Hyperparamètres par défaut : $\beta_1 = 0{,}9$, $\beta_2 = 0{,}999$, $\varepsilon = 10^{-8}$, $\alpha = 10^{-3}$.

12.6.2 Preuve complète de la correction de biais

Proposition 12.6 (Correction de biais du premier moment)**.** Si les gradients sont stationnaires, c'est-à-dire $\mathbb{E}[g_k] = \mu$ pour tout k, alors :

$$\mathbb{E}[m_t] = \mu\,(1 - \beta_1^t), \qquad \text{d'où} \qquad \mathbb{E}[\hat{m}_t] = \mu = \mathbb{E}[g_t].$$

Démonstration. Nous avons montré (proposition 314) que :

$$m_t = (1 - \beta_1) \sum_{k=1}^{t} \beta_1^{t-k} g_k.$$

En prenant l'espérance :

$$\begin{aligned}
\mathbb{E}[m_t] &= (1 - \beta_1) \sum_{k=1}^{t} \beta_1^{t-k} \mathbb{E}[g_k] && (347)\\
&= (1 - \beta_1)\mu \sum_{k=1}^{t} \beta_1^{t-k} && (348)\\
&= (1 - \beta_1)\mu \sum_{j=0}^{t-1} \beta_1^{j} \qquad \text{(changement } j = t - k) && (349)\\
&= (1 - \beta_1)\mu \cdot \frac{1 - \beta_1^t}{1 - \beta_1} && (350)\\
&= \mu\,(1 - \beta_1^t). && (351)
\end{aligned}$$

Donc m_t est un estimateur **biaisé** de μ : il sous-estime d'un facteur $(1 - \beta_1^t)$.
La correction est immédiate :

$$\mathbb{E}[\hat{m}_t] = \mathbb{E}\left[\frac{m_t}{1 - \beta_1^t}\right] = \frac{\mu(1 - \beta_1^t)}{1 - \beta_1^t} = \mu = \mathbb{E}[g_t]. \qquad \checkmark$$

□

Proposition 12.7 (Correction de biais du second moment)**.** De manière identique, si $\mathbb{E}[g_k^2] = \sigma^2$ pour tout k, alors :

$$\mathbb{E}[v_t] = \sigma^2(1 - \beta_2^t), \qquad \mathbb{E}[\hat{v}_t] = \sigma^2 = \mathbb{E}[g_t^2].$$

Démonstration. Le raisonnement est identique : $v_t = (1 - \beta_2) \sum_{k=1}^{t} \beta_2^{t-k} g_k^2$, d'où

$$\begin{aligned}
\mathbb{E}[v_t] &= (1 - \beta_2)\sigma^2 \sum_{j=0}^{t-1} \beta_2^{j} = \sigma^2(1 - \beta_2^t), && (352)\\
\mathbb{E}[\hat{v}_t] &= \frac{\sigma^2(1 - \beta_2^t)}{1 - \beta_2^t} = \sigma^2. \qquad \checkmark && (353)
\end{aligned}$$

□

Exemple 12.4 (Importance numérique de la correction)**.** Avec $\beta_2 = 0{,}999$ (valeur par défaut), calculons le biais aux premiers pas :

t	$1-\beta_2^t$	Facteur de correction $\frac{1}{1-\beta_2^t}$	Commentaire
1	0,001	1000	v_1 sous-estime par un facteur 1000 !
10	0,010	100	Encore un facteur 100
100	0,095	10,5	Un ordre de grandeur
1000	0,632	1,58	Biais modéré
5000	0,993	1,007	Correction négligeable

Sans correction, le second moment serait sous-estimé de 3 ordres de grandeur à la première itération, ce qui donnerait un pas effectif $\alpha/\sqrt{\hat{v}_1}$ environ $\sqrt{1000} \approx 32$ fois trop grand.

12.6.3 Adam vs. SGD+Momentum : interprétation géométrique

Intuition

Pour comprendre ce que fait Adam, réécrivons la mise à jour coordonnée par coordonnée. Pour le paramètre θ_i :

$$\theta_{i,t+1} = \theta_{i,t} - \alpha \frac{\hat{m}_{i,t}}{\sqrt{\hat{v}_{i,t}} + \varepsilon}.$$

Si le gradient g_i est constant et égal à μ_i, alors $\hat{m}_{i,t} \to \mu_i$ et $\hat{v}_{i,t} \to \mu_i^2$, d'où :

$$\Delta\theta_i \approx -\alpha \frac{\mu_i}{|\mu_i| + \varepsilon} \approx -\alpha \,\mathrm{signe}(\mu_i).$$

La mise à jour est approximativement $\pm\alpha$, **indépendamment de l'amplitude du gradient**. Adam effectue donc une forme de normalisation : chaque paramètre avance du même pas α, seule la direction (signe) compte. Cela explique pourquoi Adam est robuste aux différences d'échelle entre paramètres.

12.6.4 AdamW : découplage du weight decay

Définition 12.7 (AdamW (Loshchilov & Hutter, 2019))**.** AdamW modifie Adam en **découplant** la régularisation L2 (weight decay) de l'étape adaptative :

$$\theta_{t+1} = \theta_t - \alpha \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon} + \lambda\, \theta_t \right), \tag{354}$$

où $\lambda > 0$ est le coefficient de weight decay (typiquement $\lambda = 0{,}01$).

Justification Mathématique

Pourquoi ne pas simplement ajouter $\lambda\theta$ au gradient g_t dans Adam (ce qu'on appelle "L2 regularization in Adam") ? Montrons que les deux approches ne sont *pas* équivalentes.

Approche 1 : L2 dans Adam. On pose $\tilde{g}_t = g_t + \lambda\theta_t$ et on utilise Adam normalement avec $\tilde{g}_t$. Le terme de régularisation $\lambda\theta_t$ est alors divisé par $\sqrt{\hat{v}_t}$, qui dépend de l'historique des gradients. La force de régularisation effective sur chaque paramètre dépend donc de la taille de ses gradients passés, ce qui n'est pas souhaitable.

Approche 2 : AdamW. Le terme $\lambda\theta_t$ est appliqué *directement* à la mise à jour, sans passer par la normalisation adaptative. Chaque poids est pénalisé uniformément par un facteur λ, indépendamment de la dynamique de l'optimiseur.

En notation, pour un paramètre θ_i :

$$\text{L2 dans Adam :} \quad \Delta\theta_i = -\alpha \frac{\hat{m}_i + \lambda\theta_i/(\text{absorbé dans EMA})}{\sqrt{\hat{v}_i} + \varepsilon} \tag{355}$$

$$\text{AdamW :} \quad \Delta\theta_i = -\alpha \left(\frac{\hat{m}_i}{\sqrt{\hat{v}_i} + \varepsilon} + \lambda\theta_i \right) \tag{356}$$

La différence est subtile mais importante : dans AdamW, le weight decay λ s'applique de manière **uniforme et prévisible** à tous les poids, ce qui correspond au vrai weight decay tel qu'il est défini pour SGD.

Attention

En pratique, AdamW est **systématiquement préféré** à Adam avec L2 pour l'entraînement des réseaux modernes (Transformers, modèles de langage, etc.). C'est l'optimiseur par défaut dans les frameworks actuels (PyTorch, JAX).

12.7 Trajectoires de convergence

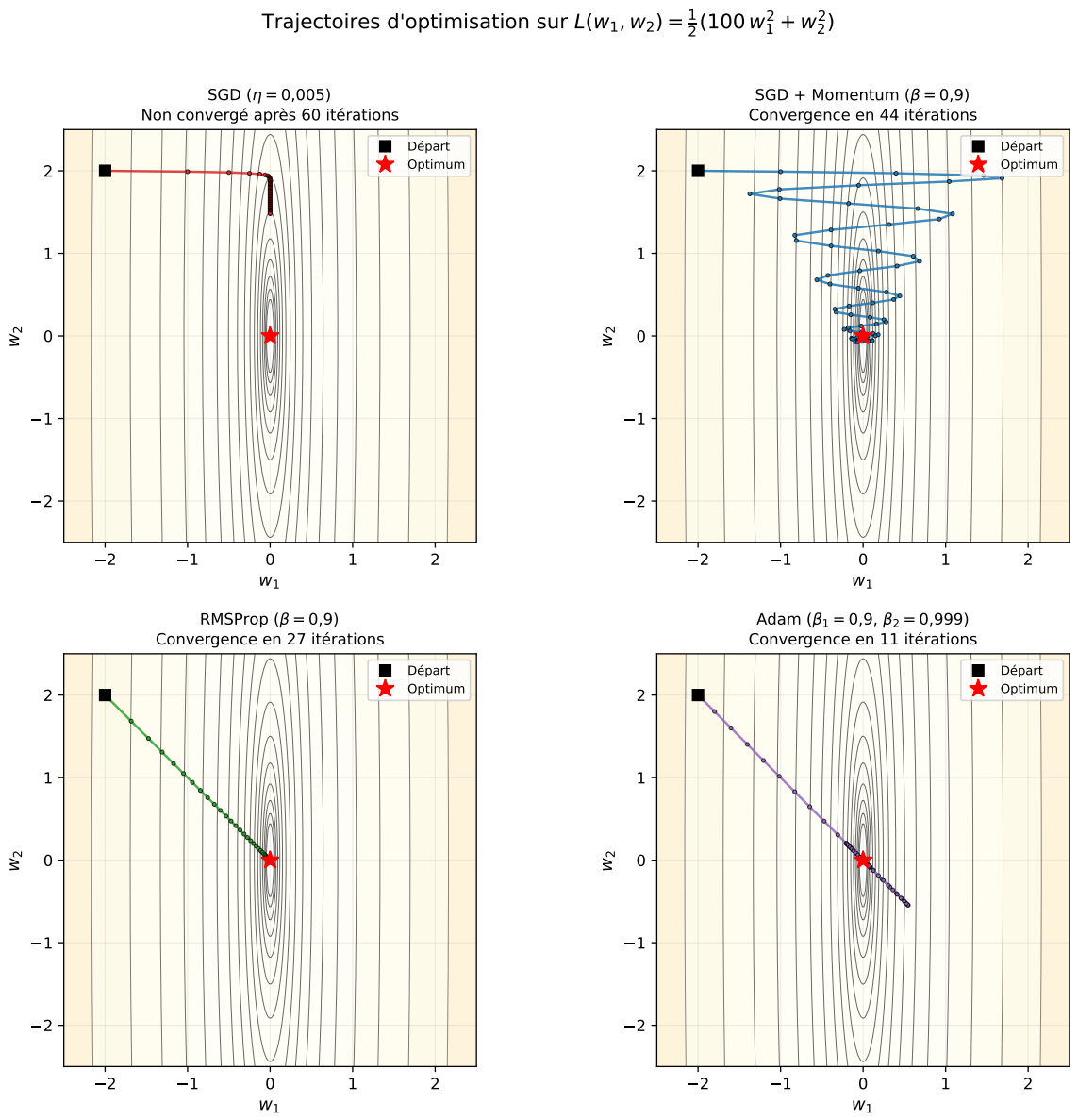

Figure 10 : Trajectoires des différents optimiseurs sur la surface $L(w_1, w_2) = \frac{1}{2}(100\, w_1^2 + w_2^2)$. SGD oscille dans la vallée étroite. Le momentum lisse les oscillations et accélère dans la direction de faible courbure. RMSProp et Adam normalisent les pas par coordonnée, éliminant presque totalement les oscillations.

12.8 Comparaison et guide pratique

Résumé des algorithmes

Optimiseur	**Momentum**	**Adaptatif**	**Correction biais**
SGD	×	×	—
SGD + Momentum	✓	×	—
Nesterov	✓(anticipé)	×	—
RMSProp	×	✓	×
Adam	✓	✓	✓
AdamW	✓	✓	✓

Hyperparamètres recommandés

Optimiseur	α	β / β_1	β_2 / ρ	Weight decay λ
SGD + Momentum	10^{-2} – 10^{-1}	0,9	—	10^{-4}
Nesterov	10^{-2} – 10^{-1}	0,9	—	10^{-4}
RMSProp	10^{-3} – 10^{-2}	—	0,99	—
AdamW	10^{-4} – 10^{-3}	0,9	0,999	10^{-2}

Que choisir en pratique ?

- **AdamW** : choix par défaut pour la grande majorité des problèmes. Particulièrement adapté aux Transformers, modèles de langage, et toute architecture où les échelles de gradients varient fortement entre couches. Robuste aux hyperparamètres : les valeurs par défaut ($\beta_1 = 0{,}9, \beta_2 = 0{,}999, \alpha = 10^{-3}$) fonctionnent souvent sans réglage.
- **SGD + Momentum (ou Nesterov)** : historiquement excellent pour les CNN en vision par ordinateur avec de gros batchs. Nécessite impérativement un *learning rate schedule* (cosine annealing, warmup, etc.) et un réglage plus fin de α, mais peut atteindre une meilleure généralisation qu'Adam dans certains cas.
- **RMSProp** : historiquement utilisé pour les réseaux récurrents (RNN, LSTM). Aujourd'hui largement supplanté par Adam/AdamW, mais reste utile comme outil pédagogique pour comprendre l'adaptativité.

Remarque 12.2. Le choix de l'optimiseur est rarement le facteur limitant en pratique. Le learning rate (et son schedule) et le weight decay ont un impact bien plus grand sur les performances finales. Une bonne stratégie est de commencer avec AdamW aux hyperparamètres par défaut, puis d'affiner le learning rate par grid search ou par des méthodes automatiques (learning rate finder).

13 Régularisation : maîtriser la complexité du modèle

13.1 Préliminaire : séparation des données et protocole d'évaluation

Intuition

Pour juger si un modèle *généralise*, il faut l'évaluer sur des données qu'il n'a **jamais vues** pendant l'entraînement. Si l'on mesure la performance uniquement sur les exemples ayant servi à ajuster les poids, on ne mesure que la capacité du modèle à mémoriser — pas à prédire. C'est pourquoi on réserve systématiquement une partie des données pour l'évaluation.

Définition 13.1 (Partition entraînement / validation / test)**.** On divise le jeu de données disponible $\mathcal{D}$ en trois sous-ensembles **disjoints** :

- **Ensemble d'entraînement** (train set, $\sim$ 60–80 %) : utilisé pour calculer les gradients et mettre à jour les paramètres du modèle.
- **Ensemble de validation** (validation set, $\sim$ 10–20 %) : utilisé pour régler les hyperparamètres (taux d'apprentissage, architecture, force de régularisation) et surveiller le sur-apprentissage au cours de l'entraînement.
- **Ensemble de test** (test set, $\sim$ 10–20 %) : utilisé **une seule fois**, à la toute fin, pour estimer la vraie performance de généralisation du modèle retenu.

Définition 13.2 (Paramètres vs hyperparamètres)**.** Il est essentiel de distinguer deux catégories de quantités :

- Les **paramètres** ($\mathbf{w}$, $\mathbf{b}$) sont les variables internes du modèle, *apprises automatiquement* par descente de gradient sur l'ensemble d'entraînement.
- Les **hyperparamètres** (α, λ, nombre de couches, taille des couches, etc.) sont des choix de conception fixés *par le praticien* avant l'entraînement. Ils sont ajustés en comparant les performances sur l'ensemble de validation.

Autrement dit, les paramètres sont optimisés par l'algorithme, tandis que les hyperparamètres sont optimisés par l'humain (ou par une recherche automatique évaluée sur la validation).

Attention

Ne jamais utiliser l'ensemble de test pour prendre des décisions sur le modèle. Si vous ajustez des hyperparamètres en observant la performance sur le test, celui-ci devient de facto un second ensemble de validation. Vous perdez alors toute estimation non biaisée de la capacité de généralisation : la performance rapportée sera optimiste, et le modèle risque de ne pas se comporter aussi bien en production. Le test ne doit servir qu'**une unique fois**, pour le verdict final.

Remarque 13.1. Lorsque les données sont rares, la **validation croisée à k plis** (*k-fold cross-validation*) constitue une alternative à la partition fixe. Le jeu de données est divisé en k sous-ensembles (plis) de taille égale. On entraîne k fois le modèle, en utilisant à chaque fois $k-1$ plis pour l'entraînement et le pli restant pour la validation. La performance finale est la moyenne des k scores obtenus, ce qui donne une estimation plus robuste lorsque m est petit. Le choix classique est $k = 5$ ou $k = 10$.

13.2 Le problème du sur-apprentissage

Nous avons maintenant tous les outils pour entraîner un réseau profond : fonctions d'activation, initialisation des poids, normalisation par lots, optimiseurs adaptatifs. Pourtant, en pratique, un réseau entraîné sur un jeu de données atteint souvent une excellente performance sur les données d'entraînement tout en échouant lamentablement sur de nouvelles données. Ce phénomène porte un nom : le **sur-apprentissage** (overfitting). Cette section est consacrée aux techniques qui permettent de le combattre.

13.2.1 Compromis biais-variance

Considérons un problème de régression où la vraie relation entre l'entrée x et la sortie est $y = f(x) + \varepsilon$, avec ε un bruit de moyenne nulle et de variance σ^2. Nous apprenons un estimateur $\hat{f}(x)$ à partir de m exemples.

Théorème 13.1 (Décomposition biais-variance)**.** Pour tout point x, l'erreur quadratique moyenne de prédiction se décompose en :

$$\mathbb{E}\left[(y - \hat{f}(x))^2\right] = \underbrace{\left(\mathbb{E}[\hat{f}(x)] - f(x)\right)^2}_{\text{biais}^2} + \underbrace{\mathbb{E}\left[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2\right]}_{\text{variance}} + \underbrace{\sigma^2}_{\text{bruit irréductible}} \tag{357}$$

où l'espérance $\mathbb{E}$ est prise sur les différents jeux d'entraînement possibles.

Démonstration. Posons $\bar{f} = \mathbb{E}[\hat{f}(x)]$ pour alléger. Développons l'erreur :

$$\mathbb{E}\left[(y - \hat{f}(x))^2\right] = \mathbb{E}\left[(f(x) + \varepsilon - \hat{f}(x))^2\right] \tag{358}$$

$$= \mathbb{E}\left[((f(x) - \hat{f}(x)) + \varepsilon)^2\right] \tag{359}$$

$$= \mathbb{E}\left[(f(x) - \hat{f}(x))^2\right] + 2\,\mathbb{E}\left[(f(x) - \hat{f}(x))\varepsilon\right] + \mathbb{E}[\varepsilon^2] \tag{360}$$

Le terme croisé vaut zéro car ε est indépendant de $\hat{f}(x)$ et de moyenne nulle, et $\mathbb{E}[\varepsilon^2] = \sigma^2$. Il reste à décomposer le premier terme. Ajoutons et retranchons $\bar{f}$:

$$\mathbb{E}\left[(f(x) - \hat{f}(x))^2\right] = \mathbb{E}\left[((f(x) - \bar{f}) + (\bar{f} - \hat{f}(x)))^2\right] \tag{361}$$

$$= (f(x) - \bar{f})^2 + 2(f(x) - \bar{f})\,\underbrace{\mathbb{E}[\bar{f} - \hat{f}(x)]}_{=0} + \mathbb{E}\left[(\hat{f}(x) - \bar{f})^2\right] \tag{362}$$

$$= \underbrace{(\bar{f} - f(x))^2}_{\text{biais}^2} + \underbrace{\mathbb{E}\left[(\hat{f}(x) - \bar{f})^2\right]}_{\text{variance}} \tag{363}$$

En recombinant, on obtient bien la décomposition annoncée. □ □

Intuition

Le biais mesure à quel point le modèle est *systématiquement* éloigné de la vraie fonction (erreur due à des hypothèses trop restrictives). La variance mesure à quel point le modèle fluctue d'un jeu d'entraînement à l'autre (sensibilité aux données particulières). Le bruit irréductible est la limite inférieure de toute méthode.
Un modèle trop simple a un biais élevé et une variance faible (sous-apprentissage). Un modèle trop complexe a un biais faible mais une variance élevée (sur-apprentissage). Toute technique de régularisation cherche à réduire la variance au prix d'une légère augmentation du biais, de sorte que l'erreur totale diminue.

13.2.2 Caractérisation formelle du sur-apprentissage

Définition 13.3 (Sur-apprentissage)**.** Soit $L_{\text{train}}(\theta)$ la perte moyenne sur l'ensemble d'entraînement et $L_{\text{test}}(\theta)$ la perte sur un ensemble de test indépendant. On dit qu'un modèle de paramètres θ est en **sur-apprentissage** si :

$$L_{\text{train}}(\theta) \ll L_{\text{test}}(\theta) \tag{364}$$

c'est-à-dire si l'écart de généralisation $L_{\text{test}}(\theta) - L_{\text{train}}(\theta)$ est anormalement grand.

Illustrons ce phénomène sur un exemple classique.

Exemple 13.1 (Régression polynomiale)**.** Considérons $m = 15$ points générés par $y = 0.5x^3 - x^2 + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, 0.3^2)$, sur l'intervalle $[-1, 2]$. Ajustons des polynômes de degrés croissants :

Degré	L_{train} (MSE)	L_{test} (MSE)	Diagnostic
1	0.85	0.92	Sous-apprentissage : biais élevé
4	0.09	0.12	Bon ajustement
15	≈ 0	4.70	Sur-apprentissage : variance élevée

Le polynôme de degré 15 interpole parfaitement les 15 points d'entraînement ($L_{\text{train}} \approx 0$), mais oscille violemment entre les points, produisant une erreur de test catastrophique.

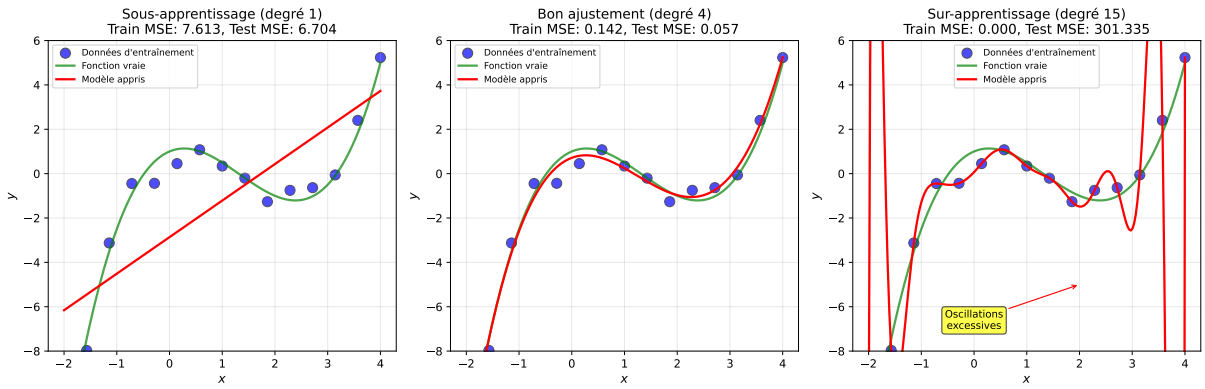

Figure 11 : Illustration du compromis biais-variance. À gauche : sous-apprentissage (modèle trop simple, biais élevé). Au centre : bon ajustement. À droite : sur-apprentissage (modèle trop complexe, variance élevée qui suit le bruit).

Attention

Comment diagnostiquer le sur-apprentissage en pratique ?

- La perte d'entraînement continue de diminuer tandis que la perte de validation remonte après un certain nombre d'époques.
- L'écart entre l'accuracy d'entraînement et celle de validation devient très grand (plus de 10–15%).
- Les poids du réseau atteignent des valeurs très élevées (signe d'une solution instable).

Le reste de cette section présente quatre familles de techniques pour combattre ce phénomène : la pénalisation des poids (L2 et L1), le dropout, l'arrêt précoce, et l'augmentation de données.

13.3 Régularisation L2 (Ridge / Weight Decay)

13.3.1 Définition formelle

L'idée fondamentale de la régularisation L2 est d'ajouter au coût un terme qui pénalise les poids de grande amplitude. Cela restreint l'espace des solutions et favorise les fonctions "lisses".

Définition 13.4 (Régularisation L2)**.** Pour un réseau de L couches avec paramètres $\theta = \{W^{[1]}, b^{[1]}, \ldots, W^{[L]}, b^{[L]}\}$, la fonction de coût régularisée s'écrit :

$$J(\theta) = L(\theta) + \frac{\lambda}{2} \sum_{l=1}^{L} \|W^{[l]}\|_F^2 \tag{365}$$

où :

- $L(\theta)$ est la perte originale (cross-entropy, MSE, etc.)
- $\|W^{[l]}\|_F^2 = \sum_{i=1}^{n^{[l]}} \sum_{j=1}^{n^{[l-1]}} (W_{ij}^{[l]})^2$ est le carré de la norme de Frobenius
- $\lambda > 0$ est l'hyperparamètre de régularisation
- Le facteur $\frac{1}{2}$ est une convention qui simplifie la dérivée

Remarque 13.2. La norme de Frobenius n'est rien d'autre que la norme euclidienne ℓ_2 de la matrice "mise à plat" en vecteur. Si $W \in \mathbb{R}^{n \times p}$, alors $\|W\|_F^2 = \mathrm{tr}(W^T W) = \sum_{i,j} W_{ij}^2$.

13.3.2 Calcul du gradient de la pénalité

Proposition 13.1 (Gradient de la pénalité L2)**.** Le gradient du terme de régularisation par rapport aux poids de la couche l est :

$$\frac{\partial}{\partial W^{[l]}} \left(\frac{\lambda}{2} \|W^{[l]}\|_F^2 \right) = \lambda W^{[l]} \tag{366}$$

Démonstration. Calculons la dérivée par rapport à un élément quelconque $W_{ij}^{[l]}$. Par définition :

$$\frac{\lambda}{2} \|W^{[l]}\|_F^2 = \frac{\lambda}{2} \sum_{k=1}^{n^{[l]}} \sum_{m=1}^{n^{[l-1]}} (W_{km}^{[l]})^2 \tag{367}$$

En dérivant par rapport à $W_{ij}^{[l]}$:

$$\frac{\partial}{\partial W_{ij}^{[l]}} \left(\frac{\lambda}{2} \sum_{k,m} (W_{km}^{[l]})^2 \right) = \frac{\lambda}{2} \sum_{k,m} \frac{\partial}{\partial W_{ij}^{[l]}} (W_{km}^{[l]})^2 \tag{368}$$

Or $\frac{\partial}{\partial W_{ij}^{[l]}} (W_{km}^{[l]})^2 = 2W_{km}^{[l]} \cdot \delta_{ik}\delta_{jm}$, où δ est le symbole de Kronecker (valant 1 si les indices sont égaux, 0 sinon). Le seul terme non nul de la somme est celui avec $k = i, m = j$:

$$\frac{\partial}{\partial W_{ij}^{[l]}} \left(\frac{\lambda}{2} \|W^{[l]}\|_F^2 \right) = \frac{\lambda}{2} \cdot 2W_{ij}^{[l]} = \lambda W_{ij}^{[l]} \tag{369}$$

En regroupant tous les éléments sous forme matricielle :

$$\frac{\partial}{\partial W^{[l]}} \left(\frac{\lambda}{2} \|W^{[l]}\|_F^2 \right) = \lambda W^{[l]} \quad \square \tag{370}$$

$\square$

13.3.3 Règle de mise à jour : weight decay

Proposition 13.2 (Mise à jour avec régularisation L2)**.** Le gradient total du coût régularisé est :

$$\frac{\partial J}{\partial W^{[l]}} = \frac{\partial L}{\partial W^{[l]}} + \lambda W^{[l]} \tag{371}$$

La règle de mise à jour par descente de gradient (avec taux d'apprentissage α) devient :

$$W^{[l]} \leftarrow W^{[l]} - \alpha \left(\frac{\partial L}{\partial W^{[l]}} + \lambda W^{[l]} \right) = (1 - \alpha\lambda)\, W^{[l]} - \alpha \frac{\partial L}{\partial W^{[l]}} \tag{372}$$

Démonstration. Développons simplement la mise à jour standard appliquée au gradient total :

$$\begin{aligned} W^{[l]} &\leftarrow W^{[l]} - \alpha \frac{\partial J}{\partial W^{[l]}} && (373)\\ &= W^{[l]} - \alpha \left(\frac{\partial L}{\partial W^{[l]}} + \lambda W^{[l]} \right) && (374)\\ &= W^{[l]} - \alpha\lambda\, W^{[l]} - \alpha \frac{\partial L}{\partial W^{[l]}} && (375)\\ &= (1 - \alpha\lambda)\, W^{[l]} - \alpha \frac{\partial L}{\partial W^{[l]}} \quad \square && (376) \end{aligned}$$

□

Intuition

Observons la formule factorisée :

$$W^{[l]} \leftarrow \underbrace{(1 - \alpha\lambda)}_{\text{facteur}<1}\, W^{[l]} - \alpha \frac{\partial L}{\partial W^{[l]}}$$

À chaque itération, *avant* d'appliquer la correction due au gradient de la perte, les poids sont multipliés par $(1 - \alpha\lambda) < 1$. Ils "décroissent" donc d'un facteur constant – d'où le nom *weight decay*. Un poids qui ne contribue pas à réduire la perte principale (c'est-à-dire dont le gradient $\frac{\partial L}{\partial W^{[l]}}$ ne compense pas la décroissance) sera progressivement ramené vers zéro.
En pratique, avec $\alpha = 0.01$ et $\lambda = 0.01$, le facteur multiplicatif vaut $1 - 10^{-4} = 0.9999$: la décroissance est subtile mais cumulée sur des milliers d'itérations, elle élimine les poids superflus.

13.3.4 Interprétation géométrique : optimisation sous contrainte

Proposition 13.3 (Lien avec l'optimisation sous contrainte)**.** Minimiser le coût régularisé

$$J(\theta) = L(\theta) + \frac{\lambda}{2} \|W\|_F^2 \tag{377}$$

est équivalent (au sens de la dualité lagrangienne) à résoudre le problème contraint :

$$\min_W L(W) \quad \text{sous la contrainte} \quad \|W\|_F^2 \leq r^2 \tag{378}$$

pour une certaine valeur de rayon $r > 0$ qui dépend de λ.

Justification Mathématique

Le lagrangien du problème contraint est :

$$\mathcal{L}(W, \mu) = L(W) + \mu \left(\|W\|_F^2 - r^2 \right) \tag{379}$$

Les conditions KKT (Karush–Kuhn–Tucker) imposent, à l'optimum :

$$\frac{\partial L}{\partial W} + 2\mu W = 0, \quad \mu \geq 0, \quad \mu \left(\|W\|_F^2 - r^2 \right) = 0 \tag{380}$$

En posant $\lambda = 2\mu$, on retrouve exactement la condition de stationnarité du coût régularisé. Géométriquement, la solution se trouve à l'intersection des courbes de niveau de $L(W)$ et de la boule $\|W\|_F \leq r$:

— Un λ grand correspond à un rayon r petit : les poids sont fortement contraints.

— Un λ petit correspond à un rayon r grand : les poids sont peu contraints.

13.3.5 Pourquoi ne pas régulariser les biais ?

Justification Mathématique

On régularise les poids $W^{[l]}$ mais pas les biais $b^{[l]}$. Pourquoi ?

1. **Rôle différent.** Les poids $W^{[l]}$ contrôlent la *sensibilité* du modèle aux entrées : des poids grands amplifient le bruit et les variations. Les biais ne font que translater les activations et n'affectent pas la complexité de la fonction apprise.
2. **Argument du nombre de paramètres.** Dans une couche avec $n^{[l]}$ neurones et $n^{[l-1]}$ entrées, il y a $n^{[l]} \times n^{[l-1]}$ poids mais seulement $n^{[l]}$ biais. Les biais représentent une fraction négligeable des paramètres et contribuent peu au sur-apprentissage.
3. **Effet néfaste.** Régulariser les biais empêcherait le modèle d'ajuster librement le niveau moyen de ses activations, réduisant l'expressivité sans bénéfice pour la généralisation.

13.3.6 Exemple numérique

Exemple 13.2 (Impact de λ sur la régression polynomiale)**.** Reprenons le polynôme de degré 12 ajusté aux données de la Figure 11. Sans régularisation, les coefficients atteignent des valeurs extrêmes pour interpoler les points bruités. Appliquons la régularisation L2 avec différentes valeurs de λ :

λ	L_{train} (MSE)	L_{test} (MSE)	$\|w\|_2$
0 (sans rég.)	0.02	2.45	1250.3
0.001	0.08	0.42	45.2
0.01	0.15	0.28	12.8
0.1	0.45	0.52	3.4
1.0	1.20	1.35	0.9

Vérifions le calcul de la mise à jour pour $\lambda = 0.01$ et $\alpha = 0.1$. Supposons qu'un poids

vaut $w = 5.0$ et que le gradient de la perte vaut $\frac{\partial L}{\partial w} = -0.3$:

$$\begin{aligned} w_{\text{nouveau}} &= (1 - \alpha\lambda)\, w - \alpha \frac{\partial L}{\partial w} && (381) \\ &= (1 - 0.1 \times 0.01) \times 5.0 - 0.1 \times (-0.3) && (382) \\ &= 0.999 \times 5.0 + 0.03 && (383) \\ &= 4.995 + 0.03 = 5.025 && (384) \end{aligned}$$

Sans régularisation, on aurait eu $w = 5.0 + 0.03 = 5.03$. La différence est faible à une itération (5.025 vs 5.03), mais cumulée sur des milliers d'itérations, le facteur $(1 - 0.001)^T$ ramène les poids inutiles vers zéro.

Observations sur le tableau :

— $\lambda = 0$: erreur d'entraînement quasi nulle mais erreur de test élevée. Les poids sont énormes ($\|w\|_2 = 1250$), signe de sur-apprentissage.

— $\lambda = 0.01$: meilleur compromis. La perte de test est minimale.

— $\lambda = 1.0$: sous-apprentissage. Les poids sont trop contraints et le modèle ne capture plus la structure des données.

13.4 Régularisation L1 (Lasso) et sparsité

13.4.1 Définition

Définition 13.5 (Régularisation L1)**.** La fonction de coût avec pénalité L1 s'écrit :

$$J(\theta) = L(\theta) + \lambda \sum_{l=1}^{L} \|W^{[l]}\|_1 \tag{385}$$

où $\|W^{[l]}\|_1 = \sum_{i,j} |W_{ij}^{[l]}|$ est la norme L1 (somme des valeurs absolues).

Remarque 13.3. Contrairement à la pénalité L2 qui utilise un facteur $\frac{1}{2}$, la convention pour L1 n'inclut pas ce facteur. La raison est que la dérivée de $|w|$ ne produit pas de facteur 2, donc la simplification n'a pas lieu d'être.

13.4.2 Sous-gradient et mise à jour

La fonction $w \mapsto |w|$ n'est pas différentiable en $w = 0$. On utilise la notion de **sous-gradient**.

Définition 13.6 (Sous-différentiel de la valeur absolue)**.** Le sous-différentiel de $|w|$ est :

$$\partial|w| = \begin{cases} \{+1\} & \text{si } w > 0 \\ \{-1\} & \text{si } w < 0 \\ [-1, +1] & \text{si } w = 0 \end{cases} \tag{386}$$

En pratique, on utilise la fonction signe :

$$\text{sign}(w) = \begin{cases} +1 & \text{si } w > 0 \\ -1 & \text{si } w < 0 \\ 0 & \text{si } w = 0 \end{cases} \tag{387}$$

Proposition 13.4 (Mise à jour avec régularisation L1)**.** Le (sous-)gradient du coût régularisé est :

$$\frac{\partial J}{\partial W^{[l]}} = \frac{\partial L}{\partial W^{[l]}} + \lambda \operatorname{sign}(W^{[l]}) \tag{388}$$

La règle de mise à jour devient :

$$W^{[l]} \leftarrow W^{[l]} - \alpha \left(\frac{\partial L}{\partial W^{[l]}} + \lambda \operatorname{sign}(W^{[l]}) \right) \tag{389}$$

Démonstration. Calculons la dérivée par rapport à un élément $W_{ij}^{[l]}$ pour $W_{ij}^{[l]} \neq 0$:

$$\frac{\partial}{\partial W_{ij}^{[l]}} \left(\lambda \sum_{k,m} |W_{km}^{[l]}| \right) = \lambda \frac{\partial}{\partial W_{ij}^{[l]}} |W_{ij}^{[l]}| = \lambda \operatorname{sign}(W_{ij}^{[l]}) \tag{390}$$

En effet, si $W_{ij}^{[l]} > 0$, alors $|W_{ij}^{[l]}| = W_{ij}^{[l]}$ et la dérivée vaut $+\lambda$. Si $W_{ij}^{[l]} < 0$, alors $|W_{ij}^{[l]}| = -W_{ij}^{[l]}$ et la dérivée vaut $-\lambda$. Sous forme matricielle, cela donne $\lambda \operatorname{sign}(W^{[l]})$. □ □

13.4.3 Pourquoi L1 produit de la sparsité

C'est la propriété la plus remarquable de la régularisation L1 : elle produit des poids **exactement nuls**, ce que L2 ne fait pas. Donnons deux arguments complémentaires.

Proposition 13.5 (Argument analytique : effet de seuil)**.** Considérons un problème unidimensionnel simplifié : minimiser $J(w) = \frac{1}{2}(w - w_0)^2 + \lambda |w|$, où w_0 est l'optimum non régularisé. La solution est :

$$w^* = \operatorname{sign}(w_0) \cdot \max(|w_0| - \lambda,\, 0) \tag{391}$$

En particulier, $w^* = 0$ dès que $|w_0| \leq \lambda$.

Démonstration. Distinguons trois cas.

Cas 1 : $w > 0$**.** Alors $|w| = w$ et $J(w) = \frac{1}{2}(w - w_0)^2 + \lambda w$. La dérivée est :

$$J'(w) = (w - w_0) + \lambda = 0 \implies w = w_0 - \lambda \tag{392}$$

Ce point est dans le domaine $w > 0$ si et seulement si $w_0 > \lambda$. Dans ce cas, $w^* = w_0 - \lambda$.

Cas 2 : $w < 0$**.** Alors $|w| = -w$ et $J(w) = \frac{1}{2}(w - w_0)^2 - \lambda w$. La dérivée est :

$$J'(w) = (w - w_0) - \lambda = 0 \implies w = w_0 + \lambda \tag{393}$$

Ce point est dans le domaine $w < 0$ si et seulement si $w_0 < -\lambda$. Dans ce cas, $w^* = w_0 + \lambda$.

Cas 3 : $|w_0| \leq \lambda$**.** Ni le cas 1 ni le cas 2 ne fournit de solution dans le bon domaine. Le minimum est atteint en $w^* = 0$.

En résumé : $w^* = \operatorname{sign}(w_0) \cdot \max(|w_0| - \lambda, 0)$. C'est l'opérateur de **seuillage doux** (soft thresholding). □ □

Intuition

Comparons avec la régularisation L2 sur le même problème simplifié : $J(w) = \frac{1}{2}(w - w_0)^2 + \frac{\lambda}{2} w^2$. La solution est $w^* = \frac{w_0}{1+\lambda}$, qui est *toujours non nulle* si $w_0 \neq 0$. La pénalité L2 *rétrécit* les poids vers zéro mais ne les y amène jamais exactement. La pénalité L1, elle, "coupe" tous les poids dont la valeur non régularisée est inférieure au seuil λ. C'est un mécanisme de **sélection automatique** : les features peu informatives sont éliminées.

Proposition 13.6 (Argument géométrique)**.** Dans l'espace des poids (w_1, w_2), la contrainte L2 $\|w\|_2^2 \leq r^2$ définit un **disque**. La contrainte L1 $\|w\|_1 \leq r$ définit un **losange** (carré tourné de 45°). Les courbes de niveau de la perte $L(w)$ sont typiquement des ellipses.

Justification Mathématique

Le point optimal sous contrainte se situe là où une courbe de niveau de L est tangente à la région admissible.

- Pour le **disque** (L2) : la tangence se produit en un point quelconque du cercle. Les deux coordonnées w_1, w_2 sont généralement non nulles.
- Pour le **losange** (L1) : les "coins" du losange se trouvent sur les axes ($w_1 = 0$ ou $w_2 = 0$). Il est géométriquement plus probable que la tangence se produise en un coin, ce qui force une coordonnée à être exactement nulle.

Ce raisonnement se généralise en dimension d : le losange a $2d$ coins, tous situés sur les axes de coordonnées, ce qui favorise fortement les solutions parcimonieuses.

Exemple 13.3 (Comparaison L1 vs L2 sur un exemple en dimension 2)**.** Soit la perte quadratique $L(w_1, w_2) = (w_1 - 3)^2 + (w_2 - 0.5)^2$, dont le minimum non contraint est $w^* = (3, 0.5)$.

Avec L2 ($\lambda = 1$) : la solution régularisée est $w^*_{\text{L2}} = \left(\frac{3}{2}, \frac{0.5}{2}\right) = (1.5, 0.25)$. Les deux composantes sont réduites mais non nulles.

Avec L1 ($\lambda = 1$) : par l'opérateur de seuillage doux, $w^*_{\text{L1}} = (\text{sign}(3) \cdot \max(3 - 1, 0), \text{sign}(0.5) \cdot \max(0.5 - 1, 0)) = (2, 0)$. La composante w_2 est exactement nulle car $|0.5| < \lambda = 1$.

13.5 Dropout : régularisation par ensemble implicite

13.5.1 Définition formelle

Définition 13.7 (Dropout)**.** Soit $A^{[l]} \in \mathbb{R}^{n^{[l]} \times m}$ les activations de la couche l. Le dropout avec taux de conservation $p \in (0, 1]$ consiste à :

1. Échantillonner un masque binaire : $D_j^{[l]} \overset{\text{i.i.d.}}{\sim} \text{Bernoulli}(p)$ pour chaque neurone j
2. Appliquer le masque : $\tilde{A}^{[l]} = D^{[l]} \odot A^{[l]}$

où $\odot$ désigne le produit de Hadamard (élément par élément).

Le taux $1 - p$ est la probabilité qu'un neurone soit "éteint". Par exemple, $p = 0.8$ signifie qu'on conserve 80% des neurones (on en éteint 20%).

Attention

Avec cette définition (dropout "standard"), l'espérance de $\tilde{A}_j^{[l]}$ est $p \cdot A_j^{[l]} \neq A_j^{[l]}$. Lors de l'inférence, il faudrait multiplier les activations par p pour compenser, ce qui complique le déploiement. C'est pourquoi on utilise en pratique l'**inverted dropout**.

Définition 13.8 (Inverted Dropout)**.** L'inverted dropout corrige le changement d'échelle *pendant l'entraînement* :

$$\tilde{A}^{[l]} = \frac{1}{p} D^{[l]} \odot A^{[l]} \quad \text{(entraînement)} \tag{394}$$

$$\tilde{A}^{[l]} = A^{[l]} \quad \text{(inférence, aucune modification)} \tag{395}$$

C'est la version implémentée dans tous les frameworks modernes (PyTorch, TensorFlow, JAX).

13.5.2 Conservation de l'espérance

Proposition 13.7 (Conservation de l'espérance avec inverted dropout)**.** Avec l'inverted dropout, l'espérance des activations est conservée :

$$\mathbb{E}[\tilde{A}_j^{[l]}] = A_j^{[l]} \tag{396}$$

Démonstration. Fixons un neurone j et un exemple. La variable $D_j^{[l]}$ suit une loi de Bernoulli de paramètre p, donc $\mathbb{E}[D_j^{[l]}] = p$. En conditionnant sur les activations $A_j^{[l]}$ (qui sont déterministes une fois les entrées fixées) :

$$\begin{aligned}
\mathbb{E}[\tilde{A}_j^{[l]}] &= \mathbb{E}\left[\frac{1}{p}\, D_j^{[l]} \cdot A_j^{[l]}\right] && (397)\\
&= \frac{1}{p}\, A_j^{[l]} \cdot \mathbb{E}[D_j^{[l]}] && (398)\\
&= \frac{1}{p}\, A_j^{[l]} \cdot p && (399)\\
&= A_j^{[l]} \quad \square && (400)
\end{aligned}$$

□

13.5.3 Effet sur la variance

Le dropout conserve l'espérance, mais il *augmente* la variance des activations. Quantifions cet effet.

Proposition 13.8 (Variance des activations avec inverted dropout)**.** Soit $A_j = A_j^{[l]}$ l'activation (déterministe) du neurone j. La variance de l'activation après inverted dropout est :

$$\mathrm{Var}(\tilde{A}_j) = \frac{1-p}{p}\, A_j^2 \tag{401}$$

Démonstration. Calculons d'abord $\mathbb{E}[\tilde{A}_j^2]$:

$$\mathbb{E}[\tilde{A}_j^2] = \mathbb{E}\left[\frac{1}{p^2}\,(D_j)^2\, A_j^2\right] = \frac{A_j^2}{p^2}\,\mathbb{E}[(D_j)^2] \tag{402}$$

Or $D_j \in \{0, 1\}$, donc $(D_j)^2 = D_j$ et $\mathbb{E}[(D_j)^2] = \mathbb{E}[D_j] = p$. D'où :

$$\mathbb{E}[\tilde{A}_j^2] = \frac{A_j^2}{p^2} \cdot p = \frac{A_j^2}{p} \tag{403}$$

La variance est donc :

$$\begin{aligned}
\mathrm{Var}(\tilde{A}_j) &= \mathbb{E}[\tilde{A}_j^2] - (\mathbb{E}[\tilde{A}_j])^2 && (404)\\
&= \frac{A_j^2}{p} - A_j^2 && (405)\\
&= A_j^2 \left(\frac{1}{p} - 1\right) && (406)\\
&= \frac{1-p}{p} A_j^2 \quad \square && (407)
\end{aligned}$$

□

Remarque 13.4. Pour $p = 0.5$ (dropout de 50%), la variance vaut A_j^2. Pour $p = 0.8$ (dropout de 20%), elle vaut $0.25\, A_j^2$. Plus on désactive de neurones (petit p), plus on injecte de bruit. Ce bruit stochastique agit comme un régulariseur, forçant le réseau à ne pas trop dépendre d'un neurone individuel.

13.5.4 Interprétation comme ensemble de sous-réseaux

Intuition

Un réseau de n neurones avec dropout a 2^n sous-réseaux possibles (chaque neurone pouvant être actif ou inactif). À chaque itération d'entraînement, un sous-réseau différent est sélectionné aléatoirement et mis à jour.
À l'inférence, utiliser le réseau complet (sans dropout) revient à calculer une **moyenne approximative** des prédictions de tous ces sous-réseaux. C'est une forme de *model averaging* : au lieu d'entraîner 2^n modèles distincts (ce qui serait impossible), le dropout réalise cette moyenne implicitement, avec le coût d'entraînement d'un seul réseau.
Cette interprétation explique pourquoi le dropout réduit la variance : la moyenne de nombreux estimateurs a une variance plus faible que chaque estimateur individuel (c'est le principe du bagging en apprentissage statistique).

Exemple 13.4 (Impact du taux de dropout)**.** Sur un problème de classification à 2 classes ("moons"), comparons différents taux de conservation p :

p (conservation)	$1-p$ (extinction)	Train Acc	Val Acc	Écart
1.0 (sans dropout)	0%	99.8%	87.2%	12.6%
0.8	20%	98.5%	92.1%	6.4%
0.5	50%	96.2%	94.3%	1.9%
0.2	80%	89.5%	88.7%	0.8%

Sans dropout : sur-apprentissage flagrant (écart de 12.6%). Avec $p = 0.5$: meilleur compromis, écart réduit à 1.9% et validation maximale. Avec $p = 0.2$: trop de neurones éteints, le réseau n'a plus assez de capacité (sous-apprentissage).

Les valeurs typiques sont $p \in [0.5, 0.8]$ pour les couches cachées, et $p \in [0.8, 1.0]$ pour la couche d'entrée (on ne veut pas perdre trop d'information brute).

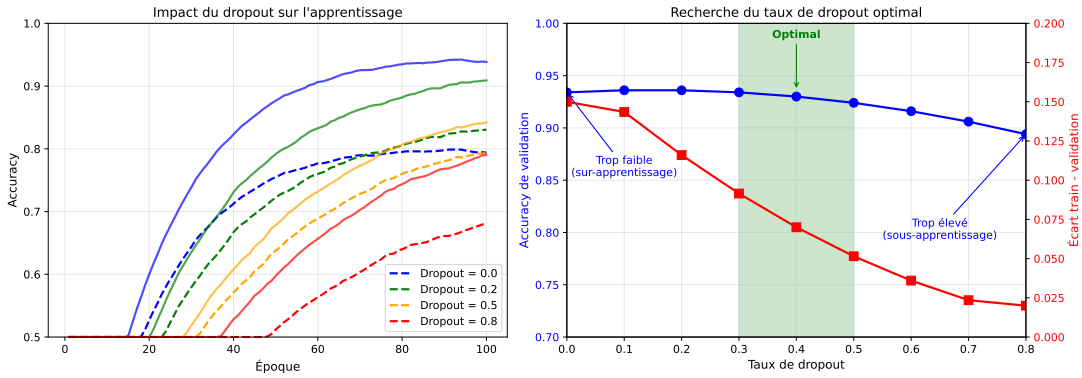

Figure 12 : Impact du taux de dropout sur l'apprentissage. À gauche : courbes d'accuracy pour différents taux. À droite : performance de validation et écart entraînement/validation en fonction du taux de dropout.

13.6 Early stopping : arrêt précoce

13.6.1 Principe et définition

Au début de l'entraînement, le réseau apprend les structures générales des données : la perte d'entraînement et la perte de validation diminuent de concert. Après un certain nombre d'époques, le réseau commence à mémoriser les particularités (le bruit) des données d'entraînement : la perte d'entraînement continue de diminuer, mais la perte de validation remonte. L'**early stopping** consiste à arrêter l'entraînement au point de divergence.

Définition 13.9 (Early stopping avec patience)**.** Soit V_t la perte sur l'ensemble de validation à l'époque t, et $V^* = \min_{\tau \leq t} V_\tau$ la meilleure perte de validation observée jusqu'à l'époque t. L'early stopping avec **patience** P arrête l'entraînement à l'époque t^* définie par :

$$t^* = \min \left\{ t \;\middle|\; V_\tau > V^*_{t-P} \;\; \forall \tau \in \{t - P + 1, \ldots, t\} \right\} \tag{408}$$

Autrement dit, on arrête si la perte de validation n'a pas amélioré V^* depuis P époques consécutives. Le modèle retenu est celui de l'époque $\arg\min_{\tau \leq t^*} V_\tau$.

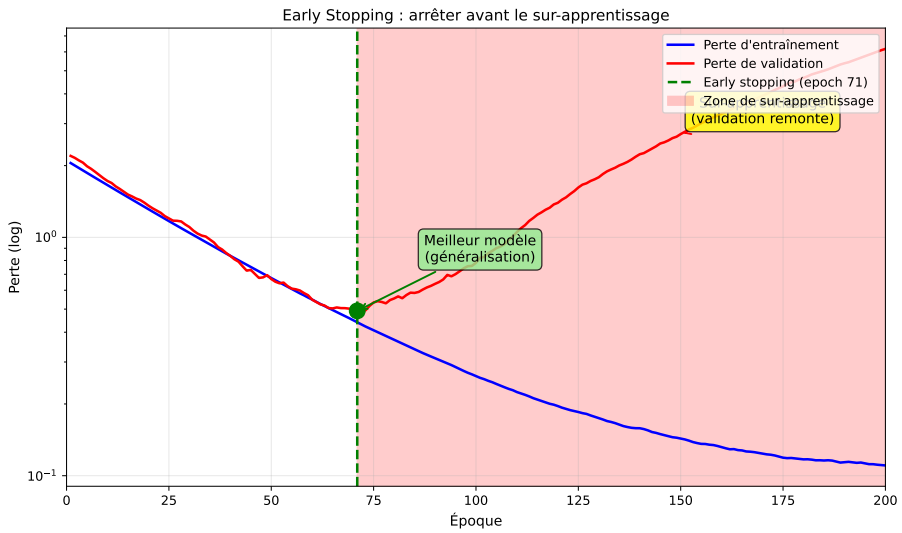

Figure 13 : Illustration de l'early stopping. La perte de validation diminue initialement, atteint un minimum, puis remonte (sur-apprentissage). Le point vert indique l'époque optimale d'arrêt.

Attention

Le choix de la patience est un compromis :

- **Patience trop faible** ($P = 2$ ou 3) : arrêt prématuré. Des fluctuations temporaires de la perte de validation peuvent déclencher l'arrêt avant la convergence.
- **Patience trop élevée** ($P = 100$) : gaspillage de calcul et risque de sur-apprentissage pendant les époques supplémentaires.
- **Valeur typique** : $P \in [10, 20]$ époques.

Il est essentiel de sauvegarder les poids du modèle à chaque amélioration de V^* (*checkpointing*), car les poids finaux (à l'arrêt) ne sont pas les meilleurs.

13.6.2 Lien avec la régularisation L2

L'early stopping n'est pas qu'une heuristique pratique : il existe un lien formel avec la régularisation L2.

Proposition 13.9 (Early stopping $\approx$ régularisation L2 (résultat de Bishop, 1995))**.** Pour un modèle linéaire avec perte quadratique $L(\theta) = \frac{1}{2}\|X\theta - y\|^2$, optimisé par descente de gradient à partir de $\theta_0 = 0$ avec taux d'apprentissage α, l'arrêt à l'itération T produit une solution qui vérifie approximativement :

$$\theta_T \approx \theta_\lambda^* \quad \text{avec} \quad \lambda \approx \frac{1}{\alpha T} \tag{409}$$

où θ_λ^* est la solution du problème régularisé L2 avec paramètre λ.

Justification Mathématique

L'idée de la démonstration repose sur la décomposition spectrale. La matrice $H = X^T X$ a des valeurs propres $\mu_1, \ldots, \mu_d$. La descente de gradient à l'itération T, partant de $\theta_0 = 0$, donne :

$$\theta_T = \sum_{k=1}^{d} \left(1 - (1 - \alpha\mu_k)^T\right) \frac{u_k^T X^T y}{\mu_k} u_k \tag{410}$$

où u_k sont les vecteurs propres de H.
La solution régularisée L2 est :

$$\theta_\lambda^* = \sum_{k=1}^{d} \frac{\mu_k}{\mu_k + \lambda} \cdot \frac{u_k^T X^T y}{\mu_k} u_k \tag{411}$$

Pour $\alpha\mu_k$ petit, on a l'approximation $(1 - \alpha\mu_k)^T \approx e^{-\alpha\mu_k T}$. Avec $\lambda = \frac{1}{\alpha T}$:

$$1 - e^{-\alpha\mu_k T} = 1 - e^{-\mu_k/\lambda} \approx \frac{\mu_k}{\mu_k + \lambda} \quad \text{pour } \mu_k/\lambda \text{ modéré} \tag{412}$$

Ainsi les deux solutions ont la même structure de rétrécissement spectral : les directions propres correspondant à de petites valeurs propres (features peu informatives) sont supprimées. Arrêter tôt (petit T) revient à avoir un grand λ, c'est-à-dire une forte régularisation.

Remarque 13.5. Ce résultat est exact pour les modèles linéaires et approximatif pour les réseaux profonds. Néanmoins, l'intuition reste valide : limiter le nombre d'itérations empêche les poids de s'éloigner trop de leur valeur initiale (proche de zéro), ce qui est exactement l'effet de la pénalité L2.

13.7 Augmentation de données et Mixup

Les techniques précédentes agissent sur le modèle (ses poids, sa structure, son entraînement). Une approche complémentaire consiste à agir sur les **données** elles-mêmes.

Définition 13.10 (Augmentation de données (Data Augmentation))**.** L'augmentation de données consiste à appliquer des transformations T aux exemples d'entraînement, telles que l'étiquette soit préservée :

$$\tilde{x} = T(x) \quad \text{avec} \quad y(\tilde{x}) = y(x) \tag{413}$$

Pour les images, les transformations classiques sont :

— **Géométriques** : rotations, translations, retournements horizontaux, recadrage aléatoire

— **Photométriques** : variations de luminosité, contraste, saturation, ajout de bruit

— **Effacement** : Cutout, RandomErasing (masquer aléatoirement une zone de l'image)

Justification Mathématique

L'augmentation de données est une forme de régularisation car elle encode des **invariances** connues dans le modèle. Si nous savons qu'un chat reste un chat après retournement horizontal, il est plus efficace de montrer cette invariance au réseau (en lui fournissant l'image retournée) que d'espérer qu'il la découvre seul. Formellement, cela équivaut à augmenter la taille effective du jeu d'entraînement, ce qui réduit la variance de l'estimateur.

Définition 13.11 (Mixup)**.** Mixup génère des exemples synthétiques par interpolation linéaire entre paires d'exemples :

$$\tilde{x} = \mu\, x_i + (1-\mu)\, x_j, \qquad \tilde{y} = \mu\, y_i + (1-\mu)\, y_j \tag{414}$$

où $\mu \sim \text{Beta}(\alpha, \alpha)$ avec $\alpha > 0$ (typiquement $\alpha = 0.2$), et (x_i, y_i), (x_j, y_j) sont deux exemples tirés aléatoirement du jeu d'entraînement.

Justification Mathématique

Mixup force le modèle à apprendre des frontières de décision **lisses** entre les classes. Au lieu de produire des probabilités abruptes (0 ou 1), le modèle doit prédire des valeurs intermédiaires dans les régions entre les exemples d'entraînement. Cela agit comme un régulariseur en réduisant la complexité effective du modèle et en améliorant sa robustesse aux perturbations adverses.
L'hyperparamètre α contrôle la force du mélange : $\alpha \to 0$ donne $\mu \to 1$ (pas de mélange), tandis que α grand donne $\mu \approx 0.5$ (mélange fort).

13.8 Synthèse et comparaison

Table 1 : Comparaison des techniques de régularisation.

Technique	**Mécanisme**	**Avantage**	**Inconvénient**	**Hyperparam.**
L2 (Ridge)	Pénalise $\|W\|_F^2$	Simple, toujours utile	Ne crée pas de sparsité	λ
L1 (Lasso)	Pénalise $\|W\|_1$	Sparsité, sélection	Non diff. en 0	λ
Dropout	Masque aléatoire	Ensemble implicite	Ralentit convergence	p
Early stopping	Arrêt précoce	Gratuit, efficace	Requiert validation	Patience P
Data augment.	Transformations	Multiplie les données	Dépend du domaine	Choix de T
Mixup	Interpolation	Frontières lisses	Moins intuitif	α

Remarque 13.6 (Lien avec le compromis biais-variance)**.** Toutes ces techniques modulent le compromis biais-variance :

- **Régularisation faible** ($\lambda \to 0$, $p \to 1$, T grand) : faible biais, forte variance (sur-apprentissage).
- **Régularisation forte** (λ grand, p petit, T petit) : fort biais, faible variance (sous-apprentissage).
- **Régularisation optimale** : le point où l'erreur totale (biais^2 + variance) est minimale. Ce point est trouvé par validation croisée.

En pratique, on combine souvent plusieurs techniques : L2 (weight decay) avec $\lambda \in [10^{-4}, 10^{-2}]$, dropout avec $p \in [0.5, 0.8]$, et early stopping avec patience $P \in [10, 20]$.

14 Réseaux Convolutifs (CNN)

Intuition

Jusqu'ici, nous avons construit des réseaux denses (MLP) couche par couche : chaque neurone d'une couche est connecté à **tous** les neurones de la couche précédente. Cette architecture est universelle, mais elle ignore totalement la **structure spatiale** des données. Pour les images, cette ignorance est catastrophique — tant en nombre de paramètres qu'en capacité à exploiter les régularités visuelles.
L'idée fondatrice des réseaux convolutifs : remplacer la multiplication matricielle dense par une **opération locale** — la convolution — qui traite chaque région de l'image avec les mêmes poids. On obtient ainsi trois propriétés cruciales :

- **Connectivité locale** : chaque neurone ne regarde qu'une petite région de l'entrée
- **Partage de poids** : les mêmes poids sont appliqués partout dans l'image
- **Équivariance à la translation** : un objet décalé produit une activation décalée

14.1 Motivation : pourquoi les MLP ne suffisent pas pour les images

14.1.1 L'explosion des paramètres

Considérons une image de taille modeste, typiquement utilisée dans les réseaux modernes de classification : 224×224 pixels en couleur (3 canaux RGB).

Exemple 14.1 (Coût paramétrique d'une couche dense sur une image)**.** L'image aplatie en vecteur a pour dimension :

$$n^{[0]} = 224 \times 224 \times 3 = 150\,528$$

Si la première couche cachée contient $n^{[1]} = 1\,000$ neurones (ce qui est modeste), la matrice de poids $W^{[1]} \in \mathbb{R}^{1000 \times 150\,528}$ contient :

$$1\,000 \times 150\,528 = 150\,528\,000 \approx 150 \text{ millions de paramètres}$$

En ajoutant les biais (1 000), on obtient 150 529 000 paramètres pour **une seule couche**. À titre de comparaison, le réseau VGG-16 complet (16 couches convolutives) n'utilise que 138 millions de paramètres au total.

Avec un réseau de 4 couches denses de cette taille, on dépasserait facilement le demi-milliard de paramètres, rendant l'entraînement impraticable (mémoire, temps de calcul, surapprentissage).

14.1.2 La perte de structure spatiale

Attention

Un MLP traite son entrée comme un vecteur **non structuré**. Si l'on permute les pixels d'une image de manière identique pour toutes les images du dataset, le MLP peut toujours apprendre à classifier (il s'adapte à la permutation). Mais cette invariance est un défaut, pas une qualité !
Dans une image, la **proximité spatiale** est porteuse d'information : les pixels voisins forment des contours, des textures, des formes. Un œil est un agencement local de pixels — le MLP ne peut pas exploiter cette localité car il connecte chaque pixel d'entrée à chaque neurone, sans notion de voisinage.

Justification Mathématique

Formellement, soit $\pi : \{1, \ldots, n\} \to \{1, \ldots, n\}$ une permutation des indices des pixels, et $P_\pi \in \mathbb{R}^{n \times n}$ la matrice de permutation correspondante. Pour un MLP avec couche dense $z = Wx + b$:

$$z' = W(P_\pi x) + b = (WP_\pi)x + b$$

Le réseau avec poids $W' = WP_\pi$ produit exactement la même sortie sur les images permutées. Autrement dit, il existe toujours un MLP qui fait aussi bien sur les images permutées que l'original sur les images non permutées. Le MLP n'a aucun **biais inductif** en faveur de la structure spatiale.
Pour les images, nous voulons au contraire un modèle qui *exploite* la localité spatiale et qui soit *incapable* de traiter des pixels distants de la même manière que des pixels voisins.

14.2 L'opération de convolution discrète

14.2.1 Définition

Définition 14.1 (Convolution discrète 2D (cross-corrélation))**.** Soit $A \in \mathbb{R}^{H \times W}$ une entrée 2D et $K \in \mathbb{R}^{k_H \times k_W}$ un noyau (kernel). La convolution discrète 2D (en pratique, une *cross-corrélation*) est définie par :

$$(A * K)(i, j) = \sum_{d_i=0}^{k_H-1} \sum_{d_j=0}^{k_W-1} K(d_i, d_j) \cdot A(i + d_i, j + d_j)$$

pour tout (i, j) tel que la fenêtre $A(i : i + k_H - 1, j : j + k_W - 1)$ soit définie.

Remarque 14.1. En traitement du signal, la vraie convolution retourne le noyau $(K(-d_i, -d_j))$. En deep learning, on utilise systématiquement la cross-corrélation (sans retournement) car le noyau est *appris* : retourner un noyau appris revient simplement à apprendre un noyau retourné. Par convention, on appelle cette opération "convolution" dans tout le domaine du deep learning, et nous suivrons cet usage.

14.2.2 Exemple numérique complet

Exemple 14.2 (Convolution 5×5 avec un noyau 3×3)**.** Soit l'entrée $A \in \mathbb{R}^{5\times 5}$ et le noyau $K \in \mathbb{R}^{3\times 3}$:

$$A = \begin{pmatrix} 1 & 0 & 2 & 1 & 0 \\ 0 & 1 & 3 & 0 & 1 \\ 2 & 1 & 0 & 2 & 1 \\ 1 & 0 & 1 & 1 & 0 \\ 0 & 2 & 1 & 0 & 1 \end{pmatrix}, \qquad K = \begin{pmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{pmatrix}$$

Ce noyau est un détecteur de bords verticaux : il calcule la différence entre la colonne de gauche et la colonne de droite de chaque fenêtre 3×3.

Calculons $(A * K)(0, 0)$ — la fenêtre en haut à gauche :

$$\begin{aligned}(A * K)(0,0) &= 1 \cdot 1 + 0 \cdot 0 + 2 \cdot (-1) + 0 \cdot 1 + 1 \cdot 0 + 3 \cdot (-1) + 2 \cdot 1 + 1 \cdot 0 + 0 \cdot (-1) \\ &= 1 + 0 - 2 + 0 + 0 - 3 + 2 + 0 + 0 = -2 \end{aligned} \tag{415}$$

Calculons $(A * K)(0, 1)$ — fenêtre décalée d'un pixel à droite :

$$\begin{aligned}(A * K)(0,1) &= 0 \cdot 1 + 2 \cdot 0 + 1 \cdot (-1) + 1 \cdot 1 + 3 \cdot 0 + 0 \cdot (-1) + 1 \cdot 1 + 0 \cdot 0 + 2 \cdot (-1) \\ &= 0 + 0 - 1 + 1 + 0 + 0 + 1 + 0 - 2 = -1 \end{aligned} \tag{416}$$

En poursuivant pour toutes les positions valides, la sortie $S = A * K \in \mathbb{R}^{3\times 3}$ est :

$$S = \begin{pmatrix} -2 & -1 & 2 \\ -1 & -1 & 1 \\ 2 & 0 & -1 \end{pmatrix}$$

On observe que les valeurs fortement positives ou négatives correspondent aux zones où il y a un fort contraste horizontal (bord vertical).

14.2.3 Formule de la taille de sortie

Proposition 14.1 (Taille de sortie d'une convolution)**.** Soit une entrée de taille n, un noyau de taille k, un padding p (nombre de zéros ajoutés de chaque côté) et un stride s (pas de déplacement). La taille de sortie est :

$$n_{\text{out}} = \left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1$$

Cette formule s'applique indépendamment à chaque dimension spatiale (H et W).

Démonstration. Avec un padding p de chaque côté, la taille effective de l'entrée est $n + 2p$. Le noyau de taille k peut être placé à la première position ($i = 0$) si $k \leq n + 2p$, ce qui est toujours le cas en pratique.

La dernière position valide $i_{\max}$ vérifie $i_{\max} + k - 1 \leq n + 2p - 1$, soit $i_{\max} \leq n + 2p - k$.

Avec un stride s, les positions sont $i \in \{0, s, 2s, \ldots\}$. La plus grande valeur de t telle que $ts \leq n + 2p - k$ est :

$$t_{\max} = \left\lfloor \frac{n + 2p - k}{s} \right\rfloor$$

Le nombre de positions est $t_{\max} + 1$ (de $t = 0$ à $t = t_{\max}$), d'où :

$$n_{\text{out}} = \left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1$$

□

Exemple 14.3 (Vérification sur l'exemple précédent)**.** Dans l'exemple 14.2 : $n = 5$, $k = 3$, $p = 0$, $s = 1$.

$$n_{\text{out}} = \left\lfloor \frac{5 + 0 - 3}{1} \right\rfloor + 1 = 2 + 1 = 3 \quad \checkmark$$

14.2.4 Propriétés fondamentales

Proposition 14.2 (Linéarité de la convolution)**.** L'opération de convolution est linéaire par rapport à l'entrée :

$$(\alpha A_1 + \beta A_2) * K = \alpha(A_1 * K) + \beta(A_2 * K)$$

et par rapport au noyau :

$$A * (\alpha K_1 + \beta K_2) = \alpha(A * K_1) + \beta(A * K_2)$$

pour tous scalaires $\alpha, \beta \in \mathbb{R}$.

Démonstration. Cela découle directement de la linéarité de la somme dans la définition :

$$\begin{aligned}
((\alpha A_1 + \beta A_2) * K)(i, j) &= \sum_{d_i, d_j} K(d_i, d_j) \cdot (\alpha A_1 + \beta A_2)(i + d_i, j + d_j) \\
&= \alpha \sum_{d_i, d_j} K(d_i, d_j) \cdot A_1(i + d_i, j + d_j) + \beta \sum_{d_i, d_j} K(d_i, d_j) \cdot A_2(i + d_i, j + d_j) \\
&= \alpha(A_1 * K)(i, j) + \beta(A_2 * K)(i, j) \qquad (417)
\end{aligned}$$

La preuve est identique pour la linéarité par rapport au noyau. □

Proposition 14.3 (Équivariance à la translation)**.** Soit T_τ l'opérateur de translation : $(T_\tau A)(i, j) = A(i - \tau_i, j - \tau_j)$. Alors :

$$T_\tau(A) * K = T_\tau(A * K)$$

Autrement dit, translater l'entrée puis convoluer donne le même résultat que convoluer puis translater la sortie.

Démonstration. Calculons $(T_\tau(A) * K)(i, j)$:

$$\begin{aligned}
(T_\tau(A) * K)(i, j) &= \sum_{d_i, d_j} K(d_i, d_j) \cdot (T_\tau A)(i + d_i, j + d_j) \\
&= \sum_{d_i, d_j} K(d_i, d_j) \cdot A(i + d_i - \tau_i, j + d_j - \tau_j) \\
&= (A * K)(i - \tau_i, j - \tau_j) \\
&= T_\tau(A * K)(i, j) \qquad (418)
\end{aligned}$$

L'étape clé est que le décalage τ passe à travers la somme sans être affecté, car les indices de sommation d_i, d_j parcourent le noyau qui est le même en toute position. □

Intuition

L'équivariance à la translation signifie que le réseau n'a pas besoin d'apprendre à détecter un chat en haut à gauche *et* un chat en bas à droite séparément. Un seul filtre suffit : s'il détecte un motif à une position, il le détectera aussi à toute autre position. C'est un biais inductif extrêmement puissant pour les images.

14.3 D'une convolution à une couche convolutive

L'opération de convolution définie ci-dessus porte sur une entrée 2D avec un seul noyau. En pratique, les entrées ont plusieurs *canaux* (ex : RGB) et on applique plusieurs filtres pour extraire différentes caractéristiques.

14.3.1 Entrée multi-canaux et filtres multiples

Définition 14.2 (Couche convolutive)**.** Soit une entrée $A \in \mathbb{R}^{C_{\text{in}} \times H \times W}$ à C_{in} canaux. Une couche convolutive avec C_{out} filtres de taille $k \times k$ est définie par :

— C_{out} noyaux $W_c \in \mathbb{R}^{C_{\text{in}} \times k \times k}$ pour $c = 1, \ldots, C_{\text{out}}$

— C_{out} biais $b_c \in \mathbb{R}$ pour $c = 1, \ldots, C_{\text{out}}$

La sortie $Z \in \mathbb{R}^{C_{\text{out}} \times H' \times W'}$ est calculée par :

$$Z_c(i,j) = \sum_{c'=1}^{C_{\text{in}}} \sum_{d_i=0}^{k-1} \sum_{d_j=0}^{k-1} W_c(c', d_i, d_j) \cdot A(c', i \cdot s + d_i, j \cdot s + d_j) + b_c \tag{419}$$

pour $c = 1, \ldots, C_{\text{out}}$ et (i,j) parcourant les positions spatiales valides.

Intuition

Chaque filtre W_c est un "détecteur de motif" 3D : il examine simultanément tous les canaux d'entrée dans une fenêtre $k \times k$, et produit **un seul scalaire** par position spatiale. Appliquer C_{out} filtres produit C_{out} cartes d'activation (feature maps), chacune spécialisée dans la détection d'un motif particulier (bord horizontal, texture, coin, etc.).

14.3.2 Comptage des paramètres

Proposition 14.4 (Nombre de paramètres d'une couche convolutive)**.** Une couche convolutive avec C_{in} canaux d'entrée, C_{out} filtres de taille $k \times k$ contient :

$$\text{Paramètres} = C_{\text{out}} \times (C_{\text{in}} \times k \times k + 1)$$

Le terme $+1$ provient du biais par filtre.

Exemple 14.4 (Comparaison dense vs convolutif)**.** Première couche d'un réseau sur des images $224 \times 224 \times 3$:

Couche dense avec 1000 neurones (comme calculé précédemment) :

$$\text{Paramètres} = 1\,000 \times (224 \times 224 \times 3 + 1) = 150\,529\,000$$

Couche convolutive avec $C_{\text{out}} = 64$ filtres de taille 3×3 :

$$\text{Paramètres} = 64 \times (3 \times 3 \times 3 + 1) = 64 \times 28 = 1\,792$$

Le ratio est spectaculaire : la couche convolutive utilise **84 000 fois moins** de paramètres que la couche dense, tout en produisant une sortie riche de $64 \times 224 \times 224 \approx 3.2$ millions de valeurs (avec padding "same").

Cette économie provient directement du **partage de poids** : les mêmes 27 poids d'un filtre $3 \times 3 \times 3$ sont appliqués à chacune des $224 \times 224 = 50\,176$ positions spatiales.

Remarque 14.2. Le nombre de paramètres d'une couche convolutive est **indépendant** de la taille spatiale de l'entrée. Un même filtre 3×3 peut s'appliquer à une image 32×32 ou 1024×1024 sans changer le nombre de poids. C'est un avantage considérable pour la généralisation.

14.4 Padding et Stride

14.4.1 Padding

Définition 14.3 (Modes de padding)**.** Soit une entrée de taille n et un noyau de taille k :

— **Valid** ($p = 0$) : aucun zéro n'est ajouté. La sortie est strictement plus petite :

$$n_{\text{out}} = n - k + 1$$

— **Same** ($p = \lfloor k/2 \rfloor$, avec $s = 1$) : on ajoute suffisamment de zéros pour que la sortie ait la même taille que l'entrée :

$$n_{\text{out}} = n$$

Justification Mathématique

Pourquoi le padding "same" est-il si courant ?
Sans padding, chaque couche convolutive réduit la taille spatiale de $k-1$ pixels. Avec un noyau 3×3, on perd 2 pixels par dimension par couche. Après 50 couches (courant dans les réseaux modernes), on aurait perdu 100 pixels : une image 224×224 se réduirait à 124×124 avant même toute opération de pooling. De plus, les pixels en bordure participent à moins de calculs que les pixels centraux, ce qui crée un biais spatial.
Le padding "same" résout ces deux problèmes : la taille spatiale est préservée et chaque pixel contribue de manière plus uniforme.

Exemple 14.5 (Padding same pour un noyau 3×3)**.** Avec $k = 3$, on a $p = \lfloor 3/2 \rfloor = 1$. Vérifions avec la formule de la proposition 14.1 :

$$n_{\text{out}} = \left\lfloor \frac{n + 2 \cdot 1 - 3}{1} \right\rfloor + 1 = n - 1 + 1 = n \quad \checkmark$$

14.4.2 Stride

Définition 14.4 (Stride (pas))**.** Le stride s est le nombre de pixels dont le noyau se déplace entre deux positions consécutives. Avec $s > 1$, la sortie est sous-échantillonnée d'un facteur s dans chaque dimension :

$$n_{\text{out}} = \left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1$$

Exemple 14.6 (Effet du stride)**.** Entrée $n = 8$, noyau $k = 3$, padding $p = 1$:

— $s = 1$: $n_{\text{out}} = \lfloor (8 + 2 - 3)/1 \rfloor + 1 = 8$ (taille préservée)

— $s = 2$: $n_{\text{out}} = \lfloor (8 + 2 - 3)/2 \rfloor + 1 = 4$ (taille divisée par 2)

— $s = 4$: $n_{\text{out}} = \lfloor (8 + 2 - 3)/4 \rfloor + 1 = 2$ (taille divisée par 4)

Intuition

Un stride $s = 2$ joue un rôle similaire au pooling (section 14.5) : il réduit la résolution spatiale. Mais contrairement au pooling qui applique une opération fixe (max, moyenne), le stride effectue un **sous-échantillonnage appris** car les poids du noyau sont optimisés. Dans les architectures modernes, le stride $s = 2$ remplace souvent le pooling pour la réduction spatiale.

14.5 Pooling : réduction spatiale

Définition 14.5 (Max pooling et average pooling)**.** Soit $A \in \mathbb{R}^{C \times H \times W}$ une feature map. Le pooling avec une fenêtre $k_p \times k_p$ et stride s_p opère **indépendamment** sur chaque canal c :

— **Max pooling** :

$$P_c(i,j) = \max_{0 \le d_i < k_p,\ 0 \le d_j < k_p} A(c, i \cdot s_p + d_i, j \cdot s_p + d_j)$$

— **Average pooling** :

$$P_c(i,j) = \frac{1}{k_p^2} \sum_{d_i=0}^{k_p-1} \sum_{d_j=0}^{k_p-1} A(c, i \cdot s_p + d_i, j \cdot s_p + d_j)$$

En pratique, le choix le plus courant est $k_p = 2$, $s_p = 2$, qui divise chaque dimension spatiale par 2.

Remarque 14.3. Le pooling n'a **aucun paramètre appris**. Il ne change pas le nombre de canaux : C canaux en entrée donnent C canaux en sortie. Seules les dimensions spatiales sont réduites.

Intuition

Pourquoi réduire la résolution spatiale ?

1. **Invariance locale** : le max pooling retourne la même valeur que l'activation maximale soit à la position $(0,0)$ ou $(1,1)$ dans la fenêtre 2×2. Cela confère une certaine tolérance aux petites translations et déformations.
2. **Réduction de la dimension** : avec $k_p = s_p = 2$, chaque opération de pooling divise le nombre de positions spatiales par 4. Après 3 poolings successifs, une feature map 224×224 devient 28×28 : $64\times$ moins de valeurs à traiter.
3. **Agrandissement du champ récepteur** : après pooling, chaque pixel de la feature map "voit" une zone plus large de l'image originale. Les couches convolutives suivantes capturent ainsi des motifs à plus grande échelle.

14.5.1 Rétropropagation à travers le max pooling

Proposition 14.5 (Gradient du max pooling)**.** Lors du forward pass, le max pooling sélectionne pour chaque fenêtre la position (d_i^*, d_j^*) de l'élément maximal. Lors du backward pass, le gradient est routé **uniquement** vers cet élément :

$$\frac{\partial P_c(i,j)}{\partial A(c, i \cdot s_p + d_i, j \cdot s_p + d_j)} = \begin{cases} 1 & \text{si } (d_i, d_j) = (d_i^*, d_j^*) \\ 0 & \text{sinon} \end{cases}$$

En d'autres termes, le gradient amont $\frac{\partial \mathcal{L}}{\partial P_c(i,j)}$ est copié intégralement à la position du maximum, et les positions non maximales reçoivent un gradient nul.

Remarque 14.4. C'est pourquoi l'implémentation du max pooling stocke les indices des éléments maximaux ("switches" ou "indices") pendant le forward pass : ils sont nécessaires pour le backward pass. Cette opération est analogue à ReLU (section 9) qui route le gradient vers les neurones actifs et bloque les neurones inactifs.

14.6 Rétropropagation dans les couches convolutives

Nous avons vu à la section 7 comment la rétropropagation calcule les gradients couche par couche en remontant le réseau. Pour une couche dense, le gradient $dW^{[l]} = \frac{1}{m} dZ^{[l]}(A^{[l-1]})^T$ est un produit matriciel. Que se passe-t-il pour une couche convolutive ?

14.6.1 Notations

Soit une couche convolutive avec entrée $A \in \mathbb{R}^{C_{\text{in}} \times H \times W}$, noyaux $W \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times k \times k}$, biais $b \in \mathbb{R}^{C_{\text{out}}}$, et sortie $Z \in \mathbb{R}^{C_{\text{out}} \times H' \times W'}$ (stride $s = 1$, padding $p = 0$ pour simplifier). On connaît $\frac{\partial \mathcal{L}}{\partial Z} \in \mathbb{R}^{C_{\text{out}} \times H' \times W'}$ par la couche suivante.

14.6.2 Gradient par rapport aux poids

Proposition 14.6 (Gradient dW : une convolution)**.** Le gradient de la loss par rapport au poids $W_c(c', d_i, d_j)$ est :

$$\frac{\partial \mathcal{L}}{\partial W_c(c', d_i, d_j)} = \sum_{i=0}^{H'-1} \sum_{j=0}^{W'-1} \frac{\partial \mathcal{L}}{\partial Z_c(i, j)} \cdot A(c', i + d_i, j + d_j) \tag{420}$$

Démonstration. Par la règle de la chaîne, puisque $W_c(c', d_i, d_j)$ intervient dans le calcul de $Z_c(i, j)$ pour **toutes** les positions (i, j) (c'est le partage de poids) :

$$\frac{\partial \mathcal{L}}{\partial W_c(c', d_i, d_j)} = \sum_{i,j} \frac{\partial \mathcal{L}}{\partial Z_c(i, j)} \cdot \frac{\partial Z_c(i, j)}{\partial W_c(c', d_i, d_j)} \tag{421}$$

Or, d'après l'équation (419) :

$$Z_c(i, j) = \sum_{c''=1}^{C_{\text{in}}} \sum_{d'_i=0}^{k-1} \sum_{d'_j=0}^{k-1} W_c(c'', d'_i, d'_j) \cdot A(c'', i + d'_i, j + d'_j) + b_c$$

Le seul terme dépendant de $W_c(c', d_i, d_j)$ est celui où $c'' = c'$, $d'_i = d_i$, $d'_j = d_j$, d'où :

$$\frac{\partial Z_c(i, j)}{\partial W_c(c', d_i, d_j)} = A(c', i + d_i, j + d_j)$$

En substituant, on obtient bien l'équation (420). □

Intuition

L'équation (420) a une interprétation élégante : le gradient $dW_c(c', \cdot, \cdot)$ est lui-même une **convolution** entre le canal c' de l'entrée A et la carte de gradient $\frac{\partial \mathcal{L}}{\partial Z_c}$. Le partage de poids signifie que toutes les positions spatiales contribuent au gradient d'un même poids.

14.6.3 Gradient par rapport à l'entrée

Proposition 14.7 (Gradient dA : une convolution "full")**.** Le gradient de la loss par rapport à l'entrée $A(c', i_0, j_0)$ est :

$$\frac{\partial \mathcal{L}}{\partial A(c', i_0, j_0)} = \sum_{c=1}^{C_{\text{out}}} \sum_{d_i=0}^{k-1} \sum_{d_j=0}^{k-1} \frac{\partial \mathcal{L}}{\partial Z_c(i_0 - d_i, j_0 - d_j)} \cdot W_c(c', d_i, d_j) \tag{422}$$

où la somme est restreinte aux indices $(i_0 - d_i, j_0 - d_j)$ valides (dans $\{0, \ldots, H' - 1\} \times \{0, \ldots, W' - 1\}$).

Démonstration. La valeur $A(c', i_0, j_0)$ apparaît dans $Z_c(i, j)$ pour toutes les positions (i, j) et tous les filtres c tels qu'il existe (d_i, d_j) avec $i + d_i = i_0$ et $j + d_j = j_0$, c'est-à-dire $i = i_0 - d_i$ et $j = j_0 - d_j$.

Par la règle de la chaîne :

$$\frac{\partial \mathcal{L}}{\partial A(c', i_0, j_0)} = \sum_{c=1}^{C_{\text{out}}} \sum_{i,j} \frac{\partial \mathcal{L}}{\partial Z_c(i, j)} \cdot \frac{\partial Z_c(i, j)}{\partial A(c', i_0, j_0)} \tag{423}$$

Or, $\frac{\partial Z_c(i,j)}{\partial A(c', i_0, j_0)} = W_c(c', i_0 - i, j_0 - j)$ si $(i_0 - i, j_0 - j) \in \{0, \ldots, k - 1\}^2$, et 0 sinon. En posant $d_i = i_0 - i$, $d_j = j_0 - j$:

$$\frac{\partial \mathcal{L}}{\partial A(c', i_0, j_0)} = \sum_{c=1}^{C_{\text{out}}} \sum_{d_i=0}^{k-1} \sum_{d_j=0}^{k-1} \frac{\partial \mathcal{L}}{\partial Z_c(i_0 - d_i, j_0 - d_j)} \cdot W_c(c', d_i, d_j)$$

□

Justification Mathématique

L'équation (422) est une convolution de $\frac{\partial \mathcal{L}}{\partial Z_c}$ avec le noyau $W_c(c', \cdot, \cdot)$ **retourné** (indices $k - 1 - d_i$, $k - 1 - d_j$ au lieu de d_i, d_j). De plus, il faut un padding de $k - 1$ sur $\frac{\partial \mathcal{L}}{\partial Z}$ pour que toutes les positions de A reçoivent un gradient. C'est ce qu'on appelle la convolution "full".

Résultat remarquable : **la rétropropagation à travers une convolution est elle-même une convolution**. Plus précisément :

- dW est obtenu par convolution de A avec dZ (corrélation croisée)
- dA est obtenu par convolution "full" de dZ avec le noyau retourné $\text{rot}_{180}(W)$

Cette élégance structurelle est l'une des raisons pour lesquelles les CNN sont si efficaces computationnellement : le forward pass et le backward pass utilisent le même type d'opération.

14.6.4 Gradient par rapport au biais

Proposition 14.8 (Gradient db)**.** Le gradient par rapport au biais b_c est simplement :

$$\frac{\partial \mathcal{L}}{\partial b_c} = \sum_{i=0}^{H'-1} \sum_{j=0}^{W'-1} \frac{\partial \mathcal{L}}{\partial Z_c(i, j)}$$

C'est la somme spatiale du gradient amont pour le canal c.

Démonstration. Le biais b_c apparaît additivement dans $Z_c(i, j)$ pour toute position (i, j), donc $\frac{\partial Z_c(i,j)}{\partial b_c} = 1$ pour tout (i, j), et le résultat suit par la règle de la chaîne. □

14.7 Architecture type : de LeNet à la conception moderne

14.7.1 LeNet-5 : l'architecture fondatrice

LeNet-5 (LeCun et al., 1998) est le premier réseau convolutif ayant démontré l'efficacité des CNN sur un problème réel (reconnaissance de chiffres manuscrits, images $32 \times 32 \times 1$).

Exemple 14.7 (Architecture et paramètres de LeNet-5)**.**

Couche	Opération	Taille sortie	Paramètres	Calcul
Entrée	—	$1 \times 32 \times 32$	0	—
Conv1	6 filtres 5×5, $s = 1$	$6 \times 28 \times 28$	$6 \times (1 \times 25 + 1) = 156$	$\lfloor \frac{32-5}{1} \rfloor + 1 = 28$
Pool1	Avg pool 2×2, $s = 2$	$6 \times 14 \times 14$	0	$28/2 = 14$
Conv2	16 filtres 5×5, $s = 1$	$16 \times 10 \times 10$	$16 \times (6 \times 25 + 1) = 2\,416$	$\lfloor \frac{14-5}{1} \rfloor + 1 = 10$
Pool2	Avg pool 2×2, $s = 2$	$16 \times 5 \times 5$	0	$10/2 = 5$
Dense1	120 neurones	120	$400 \times 120 + 120 = 48\,120$	$16 \times 25 = 400$ entrées
Dense2	84 neurones	84	$120 \times 84 + 84 = 10\,164$	—
Sortie	10 neurones (softmax)	10	$84 \times 10 + 10 = 850$	—
		Total	**61 706**	

Points à remarquer :

— Les couches convolutives ne contiennent que $156 + 2\,416 = 2\,572$ paramètres (4% du total).

— Les couches denses dominent avec 59 134 paramètres (96%). Ce déséquilibre s'accentue avec des images plus grandes, ce qui a motivé les architectures modernes à remplacer les couches denses par du *global average pooling*.

— Le réseau suit le schéma classique : la résolution spatiale diminue ($32 \to 28 \to 14 \to 10 \to 5$) tandis que le nombre de canaux augmente ($1 \to 6 \to 16$).

14.7.2 Patterns de conception modernes

Définition 14.6 (Bloc Conv-BN-ReLU)**.** Le bloc élémentaire des architectures modernes enchaîne trois opérations :

1. **Convolution** : transformation linéaire apprise (section 14.2)
2. **Batch Normalization** : normalisation des activations (section 11)
3. **ReLU** : non-linéarité $\max(0, z)$ (section 9)

On note ce bloc $\text{Conv}(C_{\text{in}}, C_{\text{out}}, k) \to \text{BN} \to \text{ReLU}$.

Intuition

L'ordre Conv-BN-ReLU n'est pas arbitraire :

— La BN est placée *après* la convolution car elle normalise les pré-activations $Z^{[l]}$, contrôlant leur distribution avant la non-linéarité (cf. section 11).

— ReLU est en dernier car la BN a centré les activations autour de zéro : environ la moitié des neurones sont actifs, ce qui est le régime optimal pour ReLU (cf. section 9 sur les activations et section 10 sur l'initialisation He).

Principes de conception d'un CNN moderne

1. **Sous-échantillonnage progressif** : la résolution spatiale est divisée par 2 régulièrement (par stride $s = 2$ ou pooling), typiquement $224 \to 112 \to 56 \to 28 \to 14 \to 7$.
2. **Augmentation des canaux** : le nombre de canaux est doublé à chaque sous-échantillonnage, typiquement $64 \to 128 \to 256 \to 512$. La quantité totale d'information ($C \times H \times W$) reste ainsi du même ordre.
3. **Global Average Pooling (GAP)** : au lieu d'aplatir la dernière feature map en vecteur (ce qui crée une couche dense massive), on calcule la moyenne spatiale de chaque canal : $\mathrm{GAP}(Z) \in \mathbb{R}^C$ avec $\mathrm{GAP}(Z)_c = \frac{1}{H \cdot W} \sum_{i,j} Z_c(i,j)$. Cela élimine les couches denses intermédiaires.
4. **Connexions résiduelles** : pour les réseaux très profonds, on ajoute des raccourcis $A^{[l+2]} = \mathrm{ReLU}(Z^{[l+2]} + A^{[l]})$. Cela facilite la propagation du gradient (cf. les problèmes de gradient évanescent discutés à la section 10).

14.8 Équivariance et invariance

14.8.1 Théorème d'équivariance

Nous avons démontré à la proposition 14.3 que la convolution commute avec la translation. Formalisons ce résultat dans le contexte d'un réseau complet.

Théorème 14.1 (Équivariance des couches convolutives)**.** Soit f_{conv} la fonction réalisée par une couche convolutive (convolution + biais + activation point-wise). Soit T_τ l'opérateur de translation spatiale par $\tau = (\tau_i, \tau_j)$. Alors :

$$f_{\text{conv}}(T_\tau(A)) = T_\tau(f_{\text{conv}}(A))$$

pour toute entrée A et toute translation τ (en supposant un padding suffisant pour que les bords ne causent pas d'effets de troncature).

Démonstration. Décomposons f_{conv} en ses trois étapes et montrons que chacune commute avec T_τ.

Étape 1 : Convolution. Nous l'avons prouvé à la proposition 14.3 :

$$(T_\tau A) * W + b = T_\tau(A * W) + b$$

Le biais b_c est un scalaire ajouté à toutes les positions, donc $T_\tau(Z + b) = T_\tau(Z) + b$.

Combiné : $(T_\tau A) * W + b = T_\tau(A * W + b)$.

Étape 2 : Activation point-wise. Soit σ une fonction d'activation appliquée élément par élément ($\sigma(Z)(c,i,j) = \sigma(Z(c,i,j))$). Alors :

$$\sigma(T_\tau(Z))(c,i,j) = \sigma(Z(c, i - \tau_i, j - \tau_j)) = (T_\tau(\sigma(Z)))(c,i,j)$$

car σ agit sur chaque valeur indépendamment de sa position spatiale.

Conclusion. En composant les deux :

$$f_{\text{conv}}(T_\tau(A)) = \sigma((T_\tau A) * W + b) = \sigma(T_\tau(A * W + b)) = T_\tau(\sigma(A * W + b)) = T_\tau(f_{\text{conv}}(A)) \tag{424}$$

□

Attention

L'équivariance n'est pas l'invariance !

— **Équivariance** : la sortie se translate en même temps que l'entrée. Si un chat se déplace de 10 pixels à droite, les feature maps se déplacent de 10 pixels à droite.

— **Invariance** : la sortie ne change *pas* quand l'entrée est translatée. Pour la classification ("est-ce un chat ?"), c'est l'invariance qui est désirée.

Les couches convolutives seules fournissent l'équivariance. L'invariance est construite par des mécanismes supplémentaires.

14.8.2 De l'équivariance à l'invariance

Proposition 14.9 (Le Global Average Pooling est invariant à la translation)**.** Soit $\mathrm{GAP} : \mathbb{R}^{C\times H\times W} \to \mathbb{R}^C$ défini par $\mathrm{GAP}(Z)_c = \frac{1}{HW}\sum_{i,j} Z_c(i,j)$. Pour toute translation τ telle que le contenu translaté reste dans les limites spatiales (translations circulaires ou padding infini) :

$$\mathrm{GAP}(T_\tau(Z)) = \mathrm{GAP}(Z)$$

Démonstration. La somme sur toutes les positions spatiales est invariante à la permutation de ses termes. La translation ne fait que réorganiser les termes de la somme :

$$\sum_{i,j}(T_\tau Z)_c(i,j) = \sum_{i,j} Z_c(i-\tau_i, j-\tau_j) = \sum_{i',j'} Z_c(i',j')$$

par le changement de variables $i' = i - \tau_i$, $j' = j - \tau_j$ (les indices parcourent les mêmes valeurs). □

Intuition

La chaîne complète d'un CNN pour la classification construit l'invariance progressivement :

1. Les couches convolutives fournissent l'**équivariance** : les features se déplacent avec l'objet.
2. Le pooling avec stride réduit la résolution et introduit une **invariance locale** : de petites translations (de l'ordre de la fenêtre de pooling) sont absorbées.
3. Les empilements successifs de convolutions et pooling agrandissent le **champ récepteur** : chaque neurone des couches profondes "voit" une grande partie de l'image, et les petites translations sont absorbées par les poolings intermédiaires.
4. Le Global Average Pooling final élimine toute information de position, fournissant une **invariance complète** à la translation.

On passe donc graduellement de l'équivariance (couches précoces, résolution fine) à l'invariance (couche finale, résolution 1×1). C'est cette transition progressive qui permet au réseau de détecter *où* sont les features (pour les localiser) tout en étant capable de classifier *indépendamment* de leur position.

Remarque 14.5. En pratique, l'invariance n'est qu'approximative. Les effets de bord (padding par des zéros), les strides qui ne divisent pas exactement la taille spatiale, et le nombre fini de couches font que l'invariance est brisée pour les grandes translations. L'augmentation de données (translations, rotations, retournements aléatoires pendant l'entraînement) reste nécessaire pour renforcer cette invariance au-delà de ce que l'architecture garantit.

15 Modèles séquentiels : RNN et LSTM

15.1 Motivation : les limites des architectures fixes

Les réseaux denses (sections 6–7) et convolutifs (section 14) partagent une hypothèse silencieuse : la taille de l'entrée est fixée à l'avance. Un MLP attend un vecteur $x \in \mathbb{R}^n$ de dimension constante ; un CNN attend une image de taille $H \times W \times C$ déterminée lors de la conception. Pourtant, de nombreuses données du monde réel sont **séquentielles** et de longueur variable : phrases en langue naturelle, séries temporelles financières, signaux audio, séquences d'ADN.

Exemple 15.1 (Prédire le caractère suivant)**.** Considérons la tâche de prédire le prochain caractère dans une séquence. Étant donné les caractères `b`, `o`, `n`, `j`, `o`, `u`, le modèle doit prédire `r`. Mais la séquence `b`, `o`, `n` pourrait tout aussi bien précéder `n`, `e` (pour *bonne*). Le contexte — c'est-à-dire l'historique complet des éléments déjà vus — est crucial. Un MLP qui recevrait uniquement le dernier caractère serait impuissant.

Pire encore, les séquences n'ont pas toutes la même longueur. Comment nourrir un réseau dense qui attend exactement n entrées avec des séquences de 5, 50 ou 500 éléments ?

Attention

[Solutions naïves et leurs défauts] On pourrait imaginer des contournements :

- **Padding** à une longueur maximale $T_{\max}$: gaspillage de calcul et perte de la structure temporelle.
- **Fenêtre glissante** de taille fixe : on perd toute information au-delà de la fenêtre.
- **Aplatir** toute la séquence en un vecteur : le nombre de paramètres explose avec $T_{\max}$, et le modèle ne généralise pas à des longueurs inédites.

Aucune de ces approches ne capture élégamment la nature récurrente de l'information séquentielle.

Intuition

L'idée fondamentale est de doter le réseau d'une **mémoire interne** — un vecteur d'état h_t — qui évolue à chaque pas de temps. À l'instant t, le réseau reçoit l'élément courant x_t et son propre état précédent h_{t-1}, puis produit un nouvel état h_t. Ainsi, les mêmes paramètres sont réutilisés à chaque pas (partage temporel, analogue au partage spatial des CNN vu en section 14), et la longueur de la séquence n'a pas besoin d'être fixée à l'avance.

15.2 Des mots aux vecteurs : tokenisation et embeddings

Intuition

Tout l'outillage mathématique développé jusqu'ici — produits matriciels, fonctions d'activation, descente de gradient — opère sur des **vecteurs numériques** $x_t \in \mathbb{R}^d$. Or, les séquences du monde réel (texte en langue naturelle, brins d'ADN, suites d'événements catégoriels) sont composées de **symboles discrets** issus d'un alphabet fini. Avant de pouvoir appliquer un RNN ou tout autre modèle, il faut donc transformer ces symboles en vecteurs : c'est le rôle conjoint de la *tokenisation* et des *embeddings*.

Définition 15.1 (Tokenisation)**.** La **tokenisation** est le processus qui découpe une entrée brute (par exemple une phrase) en unités discrètes appelées **tokens**. Trois granularités sont courantes :

- **Mot** (*word-level*) : chaque mot est un token. Simple mais le vocabulaire explose avec les formes fléchies et les mots rares.
- **Sous-mot** (*subword*, e.g. BPE — *Byte Pair Encoding*) : on fusionne itérativement les paires de caractères les plus fréquentes. Bon compromis entre taille du vocabulaire et couverture.
- **Caractère** (*character-level*) : chaque caractère est un token. Vocabulaire très petit, mais les séquences deviennent longues.

Le résultat est une séquence d'indices entiers $(k_1, k_2, \ldots, k_T)$ où chaque $k_t \in \{1, \ldots, |V|\}$ réfère à une entrée dans un vocabulaire V de taille $|V|$.

Définition 15.2 (Encodage one-hot)**.** Chaque indice de token $k \in \{1, \ldots, |V|\}$ peut être représenté par un vecteur **one-hot** $e_k \in \mathbb{R}^{|V|}$ dont toutes les composantes sont nulles sauf la k-ième, égale à 1 :

$$(e_k)_j = \begin{cases} 1 & \text{si } j = k, \\ 0 & \text{sinon.} \end{cases}$$

Attention

[Limites du one-hot] Les vecteurs one-hot posent deux problèmes majeurs :

- **Dimension excessive** : $e_k \in \mathbb{R}^{|V|}$ avec $|V|$ typiquement entre 30 000 et 50 000. Stocker et multiplier des matrices de cette taille est coûteux.
- **Absence de similarité** : pour tous tokens distincts $k \neq k'$, on a $e_k^\top e_{k'} = 0$. Le mot *chat* est aussi distant de *chien* que de *galaxie*, ce qui est sémantiquement absurde.

Définition 15.3 (Embedding (plongement lexical))**.** Un **embedding** est une matrice apprénable $E \in \mathbb{R}^{d \times |V|}$ qui associe à chaque token d'indice k un vecteur dense :

$$x_t = E\, e_k = E_{:,k} \in \mathbb{R}^d,$$

où $d \ll |V|$ est la **dimension d'embedding** (typiquement $d \in [64, 768]$). En pratique, le produit $E\, e_k$ revient simplement à extraire la k-ième colonne de E (un simple accès mémoire, sans multiplication). La matrice E est apprise conjointement avec le reste du réseau par rétropropagation.

Justification Mathématique

Au fil de l'entraînement, la matrice E apprend à placer les tokens sémantiquement proches **près les uns des autres** dans $\mathbb{R}^d$. Par exemple, les vecteurs associés à *roi* et *reine* se retrouvent dans des régions voisines de l'espace. L'embedding réalise ainsi une **réduction de dimensionnalité** — de $|V|$ à d — dans laquelle le réseau découvre sa propre représentation des entrées discrètes, adaptée à la tâche supervisée.

Remarque 15.1. Cette couche d'embedding constitue le **point d'entrée** de toutes les architectures modernes de traitement du langage : RNN, LSTM et Transformer. C'est elle qui permet d'appliquer l'outillage mathématique développé dans ce cours (sections 6–14) à des données textuelles. Dans l'architecture Transformer (section 16), des *encodages positionnels* sont ajoutés à ces embeddings pour injecter l'information d'ordre dans la séquence, le mécanisme d'attention étant par nature invariant à la permutation.

15.3 Le réseau récurrent simple (RNN)

15.3.1 Définition formelle

Définition 15.4 (Réseau récurrent simple (Elman RNN))**.** Soit une séquence d'entrées $(x_1, x_2, \ldots, x_T)$ avec $x_t \in \mathbb{R}^d$. Un RNN simple est défini par un état caché $h_t \in \mathbb{R}^{n_h}$ et une sortie $y_t \in \mathbb{R}^{n_y}$, mis à jour selon :

$$h_t = \tanh(W_{hh}\, h_{t-1} + W_{xh}\, x_t + b_h)\,, \tag{425}$$

$$y_t = W_{hy}\, h_t + b_y, \tag{426}$$

où les paramètres sont :

- $W_{hh} \in \mathbb{R}^{n_h \times n_h}$: poids de récurrence (état vers état),
- $W_{xh} \in \mathbb{R}^{n_h \times d}$: poids d'entrée (entrée vers état),
- $b_h \in \mathbb{R}^{n_h}$: biais de l'état caché,
- $W_{hy} \in \mathbb{R}^{n_y \times n_h}$: poids de sortie (état vers sortie),
- $b_y \in \mathbb{R}^{n_y}$: biais de sortie.

L'état initial est typiquement $h_0 = 0$.

Justification Mathématique

Le choix de tanh comme activation est historique et motivé par son centrage autour de zéro (rappelons, section 9, que $\tanh(z) = 2\sigma(2z) - 1$), ce qui permet à l'état caché de prendre des valeurs positives et négatives. L'image bornée $(-1, 1)$ limite l'explosion des activations, au prix du risque de saturation analysé plus loin.

15.3.2 Dépliement temporel et partage de paramètres

L'équation (425) définit un système dynamique discret. Pour l'entraîner par rétropropagation, on **déplie** (*unrolls*) le RNN sur T pas de temps, obtenant un réseau feedforward profond de T couches partageant les mêmes poids W_{hh}, W_{xh}, b_h.

Remarque 15.2. Ce partage de paramètres est l'analogue temporel du partage spatial dans les CNN (section 14). Là où un filtre convolutif est appliqué à chaque position spatiale, les matrices du RNN sont appliquées à chaque position temporelle. Les avantages sont les mêmes : réduction drastique du nombre de paramètres et capacité de généralisation à des longueurs (ou tailles) non vues à l'entraînement.

Proposition 15.1 (Nombre de paramètres d'un RNN simple)**.** Le nombre total de paramètres apprenables est :

$$|\theta|_{\text{RNN}} = n_h(n_h + d + 1) + n_y(n_h + 1),$$

indépendant de la longueur T de la séquence.

Démonstration. On dénombre :

— W_{hh} : $n_h \times n_h$ paramètres,
— W_{xh} : $n_h \times d$ paramètres,
— b_h : n_h paramètres,
— W_{hy} : $n_y \times n_h$ paramètres,
— b_y : n_y paramètres.

Le total est $n_h^2 + n_h d + n_h + n_y n_h + n_y = n_h(n_h + d + 1) + n_y(n_h + 1)$. □

15.3.3 Exemple numérique complet

Exemple 15.2 (RNN sur 3 pas de temps)**.** Prenons $d = 2$, $n_h = 2$, $n_y = 1$, et les paramètres :

$$W_{xh} = \begin{pmatrix} 0{,}5 & 0{,}3 \\ 0{,}1 & 0{,}7 \end{pmatrix}, \quad W_{hh} = \begin{pmatrix} 0{,}2 & -0{,}1 \\ 0{,}4 & 0{,}3 \end{pmatrix}, \quad b_h = \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \quad h_0 = \begin{pmatrix} 0 \\ 0 \end{pmatrix}.$$

Séquence d'entrée : $x_1 = (1, 0)^\top$, $x_2 = (0, 1)^\top$, $x_3 = (1, 1)^\top$.

Pas $t = 1$:

$$z_1 = W_{hh}\, h_0 + W_{xh}\, x_1 + b_h = \begin{pmatrix} 0 \\ 0 \end{pmatrix} + \begin{pmatrix} 0{,}5 \\ 0{,}1 \end{pmatrix} = \begin{pmatrix} 0{,}5 \\ 0{,}1 \end{pmatrix},$$

$$h_1 = \tanh(z_1) = \begin{pmatrix} \tanh(0{,}5) \\ \tanh(0{,}1) \end{pmatrix} \approx \begin{pmatrix} 0{,}462 \\ 0{,}100 \end{pmatrix}.$$

Pas $t = 2$:

$$\begin{aligned} z_2 = W_{hh}\, h_1 + W_{xh}\, x_2 + b_h &= \begin{pmatrix} 0{,}2 \times 0{,}462 + (-0{,}1) \times 0{,}100 \\ 0{,}4 \times 0{,}462 + 0{,}3 \times 0{,}100 \end{pmatrix} + \begin{pmatrix} 0{,}3 \\ 0{,}7 \end{pmatrix} \\ &= \begin{pmatrix} 0{,}082 \\ 0{,}215 \end{pmatrix} + \begin{pmatrix} 0{,}3 \\ 0{,}7 \end{pmatrix} = \begin{pmatrix} 0{,}382 \\ 0{,}915 \end{pmatrix}, \end{aligned}$$

$$h_2 = \tanh(z_2) \approx \begin{pmatrix} 0{,}364 \\ 0{,}724 \end{pmatrix}.$$

Pas $t = 3$:

$$\begin{aligned} z_3 = W_{hh}\, h_2 + W_{xh}\, x_3 + b_h &= \begin{pmatrix} 0{,}2 \times 0{,}364 + (-0{,}1) \times 0{,}724 \\ 0{,}4 \times 0{,}364 + 0{,}3 \times 0{,}724 \end{pmatrix} + \begin{pmatrix} 0{,}8 \\ 0{,}8 \end{pmatrix} \\ &= \begin{pmatrix} 0{,}001 \\ 0{,}363 \end{pmatrix} + \begin{pmatrix} 0{,}8 \\ 0{,}8 \end{pmatrix} = \begin{pmatrix} 0{,}801 \\ 1{,}163 \end{pmatrix}, \end{aligned}$$

$$h_3 = \tanh(z_3) \approx \begin{pmatrix} 0{,}665 \\ 0{,}822 \end{pmatrix}.$$

On observe comment l'état h_t accumule progressivement l'information des entrées passées. L'état h_3 dépend de x_1, x_2 et x_3, bien que seul x_3 soit l'entrée courante.

15.4 Rétropropagation à travers le temps (BPTT)

15.4.1 Formulation du gradient

On considère une perte totale $L = \sum_{t=1}^{T} L_t$, où L_t est la perte au pas t (par exemple, l'entropie croisée entre y_t et la cible). Sur le graphe déplié, la rétropropagation standard (section 4) s'applique ; on l'appelle **Backpropagation Through Time** (BPTT).

Proposition 15.2 (Gradient par rapport aux poids de récurrence)**.** Le gradient de la perte totale par rapport à W_{hh} est :

$$\frac{\partial L}{\partial W_{hh}} = \sum_{t=1}^{T} \frac{\partial L_t}{\partial W_{hh}} = \sum_{t=1}^{T} \sum_{k=1}^{t} \frac{\partial L_t}{\partial h_t} \left(\prod_{j=k+1}^{t} \frac{\partial h_j}{\partial h_{j-1}} \right) \frac{\partial h_k}{\partial W_{hh}} \bigg|_{\text{direct}}, \tag{427}$$

où $\frac{\partial h_k}{\partial W_{hh}}\big|_{\text{direct}}$ désigne la dérivée en ne considérant que la dépendance directe de h_k en W_{hh} (et non via h_{k-1}).

Dérivation complète. Fixons t et calculons $\frac{\partial L_t}{\partial W_{hh}}$. Puisque L_t dépend de W_{hh} à travers h_t, h_{t-1}, ..., h_1, la règle de la chaîne donne :

$$\frac{\partial L_t}{\partial W_{hh}} = \sum_{k=1}^{t} \frac{\partial L_t}{\partial h_k} \frac{\partial h_k}{\partial W_{hh}} \bigg|_{\text{direct}}.$$

Or, $\frac{\partial L_t}{\partial h_k}$ se décompose en chaîne :

$$\frac{\partial L_t}{\partial h_k} = \frac{\partial L_t}{\partial h_t} \cdot \frac{\partial h_t}{\partial h_{t-1}} \cdot \frac{\partial h_{t-1}}{\partial h_{t-2}} \cdots \frac{\partial h_{k+1}}{\partial h_k} = \frac{\partial L_t}{\partial h_t} \prod_{j=k+1}^{t} \frac{\partial h_j}{\partial h_{j-1}}.$$

En sommant sur t de 1 à T, on obtient l'équation (427).

Explicitons le Jacobien de chaque pas. D'après (425) :

$$\frac{\partial h_t}{\partial h_{t-1}} = \operatorname{diag}\big(1 - h_t^2\big)\, W_{hh},$$

où $\operatorname{diag}(1 - h_t^2)$ est la matrice diagonale dont les entrées sont les dérivées de tanh évaluées en z_t, et h_t^2 désigne le carré composante par composante. □

15.4.2 BPTT tronquée

Définition 15.5 (BPTT tronquée)**.** La BPTT tronquée à τ pas consiste à limiter la somme interne à $k \in \{t - \tau + 1, \ldots, t\}$ au lieu de $k \in \{1, \ldots, t\}$. Cela revient à couper le gradient au-delà de τ pas dans le passé :

$$\frac{\partial L_t}{\partial W_{hh}} \approx \sum_{k=\max(1,\, t-\tau+1)}^{t} \frac{\partial L_t}{\partial h_t} \left(\prod_{j=k+1}^{t} \frac{\partial h_j}{\partial h_{j-1}} \right) \frac{\partial h_k}{\partial W_{hh}} \bigg|_{\text{direct}}.$$

Justification Mathématique

En pratique, les séquences peuvent contenir des milliers de pas de temps. Déplier le graphe sur T entier est coûteux en mémoire (il faut stocker tous les h_t) et en calcul. La troncature à τ pas introduit un biais dans le gradient mais réduit le coût à $\mathcal{O}(\tau)$ par pas. C'est un compromis entre fidélité du gradient et faisabilité computationnelle. Le choix de τ est un hyperparamètre (typiquement entre 20 et 200).

15.5 Le problème des gradients qui disparaissent (et explosent)

15.5.1 Analyse formelle

Le produit de Jacobiens dans l'équation (427) est au cœur du problème. Analysons son comportement.

Théorème 15.1 (Condition d'explosion et de disparition des gradients)**.** Posons $J_t = \frac{\partial h_t}{\partial h_{t-1}} = \operatorname{diag}(1 - h_t^2)\, W_{hh}$. Le gradient rétropropagé de l'instant t à l'instant k est :

$$\prod_{j=k+1}^{t} J_j.$$

Soit $\lambda_{\max}$ la plus grande valeur propre en module de W_{hh}. Alors :

1. Si $\|J_j\| < 1$ pour tout j (ce qui est favorisé quand $|\lambda_{\max}| < 1$), le produit décroît exponentiellement :

$$\left\| \prod_{j=k+1}^{t} J_j \right\| \leq \prod_{j=k+1}^{t} \|J_j\| \leq \gamma^{t-k}, \quad \gamma < 1.$$

   Les gradients **disparaissent**.

2. Si $|\lambda_{\max}| > 1$ et les activations ne saturent pas, le produit croît exponentiellement. Les gradients **explosent**.

Démonstration. Commençons par borner la norme du Jacobien. Puisque $\tanh'(z) = 1 - \tanh^2(z) \in (0, 1]$, on a $\|\operatorname{diag}(1 - h_t^2)\| \leq 1$ (en norme spectrale). Par sous-multiplicativité :

$$\|J_t\| = \|\operatorname{diag}(1 - h_t^2)\, W_{hh}\| \leq \|\operatorname{diag}(1 - h_t^2)\| \cdot \|W_{hh}\| \leq \|W_{hh}\|.$$

Donc si $\|W_{hh}\| < 1$ (ce qui équivaut à $|\lambda_{\max}| < 1$ pour une matrice symétrique, et en est une condition suffisante en général), alors $\|J_t\| \leq \gamma < 1$ pour tout t, et :

$$\left\| \prod_{j=k+1}^{t} J_j \right\| \leq \gamma^{t-k} \xrightarrow{t-k\to\infty} 0.$$

Réciproquement, si $|\lambda_{\max}| > 1$ et que les dérivées de tanh restent proches de 1 (activations non saturées, ce qui arrive quand les pré-activations sont proches de 0), alors $\|J_t\| > 1$ et le produit croît exponentiellement. □

15.5.2 Exemple numérique

Exemple 15.3 (Disparition du gradient sur 20 pas)**.** Considérons $n_h = 1$ (cas scalaire pour simplifier). Alors $J_t = (1 - h_t^2) \cdot w_{hh}$. Supposons $w_{hh} = 0{,}8$ et que les activations restent petites ($h_t \approx 0{,}3$ en moyenne, donc $1 - h_t^2 \approx 0{,}91$). Le facteur d'atténuation par pas est :

$$|J_t| \approx 0{,}91 \times 0{,}8 = 0{,}728.$$

Après $t - k = 20$ pas :

$$\left| \prod_{j=k+1}^{t} J_j \right| \approx 0{,}728^{20} \approx 0{,}0018.$$

Le gradient provenant de 20 pas dans le passé est atténué d'un facteur ~ 550. Après 50 pas, l'atténuation atteint $0{,}728^{50} \approx 3{,}5 \times 10^{-7}$: le gradient est numériquement nul.

À l'inverse, avec $w_{hh} = 1{,}5$ et des activations non saturées ($1 - h_t^2 \approx 0{,}91$), le facteur par pas est $|J_t| \approx 1{,}365$ et après 20 pas :

$$1{,}365^{20} \approx 580.$$

Le gradient explose.

Remarque 15.3. Ce phénomène est une manifestation temporelle du problème discuté en section 9 (saturation des activations) et en section 10 (propagation de la variance). La différence cruciale est qu'ici la profondeur est dictée par la longueur de la séquence T, qui peut être bien plus grande que le nombre de couches d'un réseau dense classique.

15.5.3 Gradient clipping

Définition 15.6 (Gradient clipping par norme)**.** Le gradient clipping consiste à restreindre la norme du gradient global avant la mise à jour des paramètres :

$$g \leftarrow \begin{cases} g & \text{si } \|g\| \leq \theta, \\ \theta \cdot \dfrac{g}{\|g\|} & \text{si } \|g\| > \theta, \end{cases}$$

où $g = \nabla_\theta L$ est le gradient total et $\theta > 0$ est le seuil de clipping.

Justification Mathématique

Le clipping ne résout pas la disparition du gradient (il n'amplifie pas les gradients faibles), mais il empêche l'explosion. Lorsque le gradient dépasse le seuil θ, on conserve sa direction mais on réduit son amplitude. C'est une heuristique simple et efficace, quasi universellement utilisée pour l'entraînement des RNN. Elle préserve la direction de descente tout en bornant la taille du pas.

Attention

[Limites du gradient clipping] Le clipping traite le symptôme (explosion) mais pas la cause (produit de Jacobiens > 1). Il ne résout pas non plus la disparition des gradients, qui empêche le réseau d'apprendre des dépendances à long terme. C'est cette limitation fondamentale qui motive les architectures LSTM et GRU.

15.6 LSTM : Long Short-Term Memory

15.6.1 Motivation : une autoroute pour le gradient

Le problème central du RNN simple est que l'information doit traverser une succession de multiplications matricielles et de non-linéarités saturantes. L'idée du LSTM (Hochreiter & Schmidhuber, 1997) est d'introduire un second vecteur d'état — l'**état de cellule** c_t — dont la dynamique est *additive* plutôt que multiplicative. Cet état de cellule agit comme une autoroute : l'information peut y circuler sur de longues distances sans être déformée à chaque pas.

15.6.2 Définition formelle

Définition 15.7 (LSTM)**.** Un LSTM maintient deux états : l'état de cellule $c_t \in \mathbb{R}^{n_h}$ et l'état caché $h_t \in \mathbb{R}^{n_h}$. Notons $[\cdot\,,\,\cdot]$ la concaténation de vecteurs. Les équations du LSTM sont :

Porte d'oubli (*forget gate*) :

$$f_t = \sigma(W_f\,[h_{t-1},\,x_t] + b_f) \tag{428}$$

Porte d'entrée (*input gate*) :

$$i_t = \sigma(W_i\,[h_{t-1},\,x_t] + b_i) \tag{429}$$

Cellule candidate :

$$\tilde{c}_t = \tanh(W_c\,[h_{t-1},\,x_t] + b_c) \tag{430}$$

Mise à jour de l'état de cellule :

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \tag{431}$$

Porte de sortie (*output gate*) :

$$o_t = \sigma(W_o\,[h_{t-1},\,x_t] + b_o) \tag{432}$$

État caché :

$$h_t = o_t \odot \tanh(c_t) \tag{433}$$

où σ est la sigmoïde (section 9), $\odot$ le produit de Hadamard (composante par composante), et les matrices $W_f, W_i, W_c, W_o \in \mathbb{R}^{n_h \times (n_h+d)}$.

Intuition

Chaque porte est un vecteur de valeurs dans $(0, 1)$ qui module un flux d'information :

- f_t contrôle quelle fraction de l'ancien état de cellule c_{t-1} est conservée. Si $f_t \approx 1$, la mémoire est intégralement transmise.
- i_t contrôle quelle fraction de la nouvelle information $\tilde{c}_t$ est incorporée.
- o_t contrôle quelle fraction de l'état de cellule est exposée comme état caché h_t.

L'équation (431) est la clé de l'architecture : c'est une mise à jour **additive** pondérée, pas une transformation non-linéaire complète comme dans le RNN simple.

15.6.3 Flux du gradient dans le LSTM

Théorème 15.2 (Gradient à travers l'état de cellule)**.** Le gradient de c_t par rapport à c_{t-1} est :

$$\frac{\partial c_t}{\partial c_{t-1}} = \mathrm{diag}(f_t) + \text{termes en } \frac{\partial f_t}{\partial c_{t-1}}, \frac{\partial i_t}{\partial c_{t-1}}, \frac{\partial \tilde{c}_t}{\partial c_{t-1}}. \tag{434}$$

Si l'on néglige les dépendances indirectes de f_t, i_t et $\tilde{c}_t$ par rapport à c_{t-1} (ces termes passent par $h_{t-1} = o_{t-1} \odot \tanh(c_{t-1})$ et sont de second ordre), on obtient l'approximation fondamentale :

$$\frac{\partial c_t}{\partial c_{t-1}} \approx \mathrm{diag}(f_t). \tag{435}$$

Démonstration. D'après l'équation (431), $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$. En dérivant composante par composante par rapport à c_{t-1} :

$$\frac{\partial (c_t)_j}{\partial (c_{t-1})_j} = (f_t)_j + (c_{t-1})_j \frac{\partial (f_t)_j}{\partial (c_{t-1})_j} + (\tilde{c}_t)_j \frac{\partial (i_t)_j}{\partial (c_{t-1})_j} + (i_t)_j \frac{\partial (\tilde{c}_t)_j}{\partial (c_{t-1})_j}.$$

Le premier terme, $(f_t)_j$, est la contribution directe. Les trois autres termes impliquent les dérivées des portes par rapport à c_{t-1}, qui transitent par h_{t-1} : la dépendance est $c_{t-1} \to \tanh(c_{t-1}) \to h_{t-1} = o_{t-1} \odot \tanh(c_{t-1}) \to f_t, i_t, \tilde{c}_t$. En pratique, ces termes sont petits devant $(f_t)_j$ car ils impliquent des produits de dérivées de sigmoïdes. L'approximation (435) est donc valide au premier ordre. □

Attention

[Voici le résultat central de cette section] L'équation (435) est capitale. Comparons avec le RNN simple :

- **RNN simple** : $\frac{\partial h_t}{\partial h_{t-1}} = \mathrm{diag}(1 - h_t^2)\, W_{hh}$ — une multiplication matricielle à chaque pas.
- **LSTM** : $\frac{\partial c_t}{\partial c_{t-1}} \approx \mathrm{diag}(f_t)$ — une simple multiplication composante par composante, **pas de multiplication matricielle**.

Le gradient sur T pas est approximativement $\prod_{j=k+1}^{t} \mathrm{diag}(f_j)$, c'est-à-dire un produit de nombres dans $(0, 1)$ pour chaque composante. Si $f_j \approx 1$ (la porte d'oubli laisse passer), le gradient est préservé quasi intégralement. La porte d'oubli agit comme une **vanne** qui contrôle le flux du gradient.

Justification Mathématique

C'est pourquoi le biais b_f de la porte d'oubli est souvent initialisé à une valeur positive (typiquement 1 ou 2) : cela pousse f_t vers 1 au début de l'entraînement, assurant que le gradient circule librement et que le réseau peut apprendre des dépendances à long terme avant de décider quoi oublier.

15.6.4 Exemple numérique

Exemple 15.4 (Un pas LSTM avec $d = 2$, $n_h = 2$)**.** Prenons $h_0 = (0, 0)^\top$, $c_0 = (0, 0)^\top$, $x_1 = (1, 0{,}5)^\top$. La concaténation est $[h_0, x_1] = (0,\ 0,\ 1,\ 0{,}5)^\top \in \mathbb{R}^4$. Utilisons des poids simplifiés où chaque matrice $W \in \mathbb{R}^{2\times 4}$:

$$W_f = \begin{pmatrix} 0{,}1 & 0{,}2 & 0{,}3 & 0{,}1 \\ 0{,}2 & 0{,}1 & 0{,}1 & 0{,}3 \end{pmatrix}, \quad b_f = \begin{pmatrix} 1 \\ 1 \end{pmatrix} \quad \text{(biais positif)}$$

Porte d'oubli :

$$W_f[h_0, x_1] + b_f = \begin{pmatrix} 0{,}3 \times 1 + 0{,}1 \times 0{,}5 \\ 0{,}1 \times 1 + 0{,}3 \times 0{,}5 \end{pmatrix} + \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 1{,}35 \\ 1{,}25 \end{pmatrix},$$

$$f_1 = \sigma(1{,}35,\ 1{,}25) \approx (0{,}794,\ 0{,}777)^\top.$$

Supposons que les calculs analogues donnent $i_1 \approx (0{,}622,\ 0{,}598)^\top$, $\tilde{c}_1 \approx (0{,}380,\ 0{,}291)^\top$ et $o_1 \approx (0{,}645,\ 0{,}611)^\top$.

État de cellule :

$$c_1 = f_1 \odot c_0 + i_1 \odot \tilde{c}_1 = (0, 0)^\top + (0{,}622 \times 0{,}380,\ 0{,}598 \times 0{,}291)^\top = (0{,}236,\ 0{,}174)^\top.$$

État caché :

$$h_1 = o_1 \odot \tanh(c_1) = (0{,}645,\ 0{,}611)^\top \odot (0{,}232,\ 0{,}172)^\top \approx (0{,}150,\ 0{,}105)^\top.$$

Notons que $f_1 \approx 0{,}79$: la porte d'oubli est largement ouverte (grâce au biais $b_f = 1$), le gradient rétropropagé à travers c serait atténué d'un facteur $\sim 0{,}79$ par pas au lieu de $\sim 0{,}73$ dans le RNN simple, et surtout *sans multiplication matricielle.*

15.6.5 Nombre de paramètres

Proposition 15.3 (Paramètres d'un LSTM)**.** Le nombre total de paramètres d'une couche LSTM est :

$$|\theta|_{\text{LSTM}} = 4\, n_h\, (n_h + d + 1).$$

Démonstration. Le LSTM comporte 4 ensembles de poids et biais (un par porte plus le candidat). Chaque ensemble contient une matrice $W \in \mathbb{R}^{n_h \times (n_h + d)}$ et un biais $b \in \mathbb{R}^{n_h}$, soit $n_h(n_h + d) + n_h = n_h(n_h + d + 1)$ paramètres. Au total : $4\, n_h(n_h + d + 1)$. □

Remarque 15.4. Comparé au RNN simple qui a $n_h(n_h + d + 1)$ paramètres pour l'état caché (plus les paramètres de sortie), le LSTM est environ 4 fois plus coûteux en paramètres pour la partie récurrente. Ce surcoût est le prix de la capacité à modéliser les dépendances à long terme.

15.7 GRU : une simplification efficace

Définition 15.8 (Gated Recurrent Unit (GRU))**.** Le GRU (Cho et al., 2014) simplifie le LSTM en fusionnant l'état de cellule et l'état caché en un seul vecteur h_t, et en n'utilisant que deux portes :

Porte de mise à jour (*update gate*) :

$$z_t = \sigma(W_z\, [h_{t-1},\, x_t] + b_z) \tag{436}$$

Porte de réinitialisation (*reset gate*) :

$$r_t = \sigma(W_r\, [h_{t-1},\, x_t] + b_r) \tag{437}$$

Candidat :

$$\tilde{h}_t = \tanh(W_h\, [r_t \odot h_{t-1},\, x_t] + b_h) \tag{438}$$

Mise à jour de l'état :

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \tag{439}$$

Intuition

La porte de mise à jour z_t joue simultanément le rôle de la porte d'oubli et de la porte d'entrée du LSTM. Si $z_t \approx 0$, l'état précédent est conservé intact ($h_t \approx h_{t-1}$) ; si $z_t \approx 1$, l'état est entièrement remplacé par le candidat $\tilde{h}_t$. La contrainte $z_t + (1 - z_t) = 1$ impose que ce qui est oublié est exactement compensé par ce qui est ajouté, ce qui n'est pas le cas dans le LSTM où f_t et i_t sont indépendants.
La porte de réinitialisation r_t permet au candidat de "repartir de zéro" quand $r_t \approx 0$, en ignorant l'état précédent.

Proposition 15.4 (Paramètres d'un GRU)**.** Le nombre total de paramètres d'une couche GRU est :

$$|\theta|_{\text{GRU}} = 3\, n_h\, (n_h + d + 1).$$

Démonstration. Le GRU comporte 3 ensembles de poids et biais (deux portes plus le candidat). Chaque ensemble : $n_h(n_h+d)+n_h = n_h(n_h+d+1)$. Total : $3\, n_h(n_h+d+1)$. □

Exemple 15.5 (Comparaison du nombre de paramètres)**.** Avec $d = 256$ (dimension d'entrée) et $n_h = 512$ (dimension cachée) :

Architecture	**Paramètres (partie récurrente)**
RNN simple	$512 \times (512 + 256 + 1) = 393\,728$
GRU	$3 \times 393\,728 = 1\,181\,184$
LSTM	$4 \times 393\,728 = 1\,574\,912$

Le GRU offre $\sim 75\%$ des paramètres du LSTM. En pratique, les performances du GRU et du LSTM sont souvent comparables, le GRU étant parfois préféré pour des raisons de rapidité d'entraînement sur des jeux de données modestes.

15.8 Architectures et applications

15.8.1 Configurations entrée-sortie

Les RNN (et par extension LSTM/GRU) se déclinent en plusieurs configurations selon la tâche.

Définition 15.9 (Configurations séquentielles)**.** Soit $(x_1, \ldots, x_T)$ une séquence d'entrée et $(y_1, \ldots, y_{T'})$ une séquence de sortie.

— **Many-to-one** ($T' = 1$) : on utilise uniquement l'état final h_T pour produire une sortie unique $y = W_{hy}\, h_T + b_y$. Applications : classification de séquences (analyse de sentiment, détection d'anomalies dans une série temporelle).

— **Many-to-many synchrone** ($T' = T$) : on produit une sortie y_t à chaque pas de temps. Applications : étiquetage de séquences (POS tagging, reconnaissance d'entités nommées).

— **Many-to-many asynchrone** ($T' \neq T$) : un encodeur lit la séquence d'entrée et résume l'information dans un vecteur contexte h_T, puis un décodeur génère la séquence de sortie autoregressivement. Applications : traduction automatique, résumé de texte.

— **One-to-many** ($T = 1$) : à partir d'une entrée unique (par exemple un vecteur de description), le réseau génère une séquence. Applications : génération de légendes d'images, génération de musique.

15.8.2 RNN bidirectionnels

Définition 15.10 (RNN bidirectionnel)**.** Un RNN bidirectionnel combine deux RNN : un RNN *avant* qui lit la séquence de gauche à droite ($\overrightarrow{h}_t$), et un RNN *arrière* qui la lit de droite à gauche ($\overleftarrow{h}_t$). L'état à chaque position est la concaténation :

$$h_t = [\overrightarrow{h}_t,\ \overleftarrow{h}_t] \in \mathbb{R}^{2n_h}.$$

Les deux RNN possèdent des paramètres distincts et sont entraînés conjointement.

Justification Mathématique

Dans de nombreuses tâches, le contexte futur est aussi informatif que le contexte passé. Par exemple, pour déterminer la fonction grammaticale du mot *"avocat"* dans une phrase, il faut connaître les mots qui le suivent. Le RNN bidirectionnel donne à chaque position un accès à l'ensemble de la séquence.

Attention

[Limitation] Le RNN bidirectionnel nécessite d'avoir accès à la séquence complète avant de produire une sortie. Il ne convient donc pas aux tâches autorégressives (génération mot par mot), où le futur n'est pas encore connu au moment de la prédiction.

15.8.3 RNN profonds (empilés)

Définition 15.11 (RNN profond à L couches)**.** Un RNN profond empile L couches récurrentes. Notons $h_t^{[l]}$ l'état caché de la couche l au temps t. L'entrée de la couche l au temps t est la sortie $h_t^{[l-1]}$ de la couche inférieure :

$$h_t^{[1]} = \mathrm{RNNCell}^{[1]}\left(h_{t-1}^{[1]},\ x_t\right), \tag{440}$$

$$h_t^{[l]} = \mathrm{RNNCell}^{[l]}\left(h_{t-1}^{[l]},\ h_t^{[l-1]}\right), \quad l = 2, \ldots, L, \tag{441}$$

où RNNCell peut être un RNN simple, un LSTM ou un GRU. Chaque couche possède ses propres paramètres.

Justification Mathématique

L'empilement de couches récurrentes permet une hiérarchisation des représentations temporelles, de la même manière que l'empilement de couches convolutives capture des caractéristiques visuelles de plus en plus abstraites (section 14). La couche 1 peut capturer des motifs locaux (syllabes, fluctuations rapides), tandis que les couches supérieures synthétisent des motifs à plus grande échelle (structures de phrases, tendances de long terme). En pratique, 2 à 4 couches sont typiques ; au-delà, les bénéfices diminuent et le coût d'entraînement augmente significativement.

Remarque 15.5. Dans un RNN profond, le gradient doit circuler dans deux directions : à travers le temps (via les connexions récurrentes) et à travers les couches (via les connexions verticales). L'utilisation de cellules LSTM ou GRU atténue le premier problème ; des techniques comme les connexions résiduelles (ajout de $h_t^{[l-1]}$ à $h_t^{[l]}$, analogue au skip connection des réseaux résiduels) atténuent le second.

16 Mécanisme d'Attention et Transformers

16.1 Motivation : les limites des RNN pour les longues séquences

À la section précédente, nous avons étudié les réseaux récurrents (RNN) et leurs variantes à portes (LSTM, GRU). Ces architectures traitent les séquences de manière *séquentielle* : l'information au pas t ne peut atteindre le pas $t+k$ qu'en traversant k étapes de récurrence, chacune médiatisée par un état caché $h_t \in \mathbb{R}^d$.

Intuition

Imaginez un jeu de téléphone arabe avec 100 participants. Le premier chuchote un message au deuxième, qui le transmet au troisième, et ainsi de suite. Peu importe la qualité de la mémoire de chaque participant, le message original sera déformé, voire perdu, en arrivant au centième. C'est exactement le problème d'un RNN : toute l'information de la séquence doit transiter par un vecteur de taille fixe h_t.

Attention

Même les LSTM, conçus pour atténuer le problème du gradient évanescent (section 15), souffrent de deux limitations fondamentales pour les longues séquences :

1. **Le goulot d'étranglement informationnel.** L'état caché $h_t \in \mathbb{R}^d$ doit encoder *toute* l'information pertinente de la séquence vue jusqu'au pas t. Pour une séquence de 500 mots, c'est un résumé extrêmement compressé.
2. **Le traitement séquentiel.** Le calcul de h_t dépend de h_{t-1}, qui dépend de h_{t-2}, etc. Cette dépendance séquentielle interdit toute parallélisation sur la longueur de la séquence, ce qui rend l'entraînement sur GPU très inefficace.

L'idée clé qui résout ces deux problèmes est d'une simplicité remarquable : au lieu de forcer l'information à transiter pas à pas à travers des états cachés, **pourquoi ne pas permettre à chaque position de la séquence de consulter directement toutes les autres positions ?** C'est le mécanisme d'**attention**.

16.2 Attention : le mécanisme fondamental

16.2.1 Requêtes, clés et valeurs

Le mécanisme d'attention repose sur une analogie avec un système de recherche d'information. Étant donné une *requête* (query), on cherche parmi un ensemble de *clés* (keys) celles qui sont les plus pertinentes, puis on retourne une combinaison pondérée des *valeurs* (values) associées.

Définition 16.1 (Mécanisme d'attention (forme générale))**.** Soient $q \in \mathbb{R}^{d_k}$ un vecteur requête, $\{k_1, \ldots, k_m\} \subset \mathbb{R}^{d_k}$ un ensemble de m vecteurs clés, et $\{v_1, \ldots, v_m\} \subset \mathbb{R}^{d_v}$ les vecteurs valeurs associés. Le mécanisme d'attention calcule :

1. **Scores de compatibilité :**

$$s_i = \text{score}(q, k_i) = q^\top k_i \quad \text{pour } i = 1, \ldots, m \tag{442}$$

2. **Poids d'attention** (normalisation par softmax) :

$$\alpha_i = \frac{\exp(s_i)}{\sum_{j=1}^{m} \exp(s_j)} = \mathrm{softmax}(s)_i \tag{443}$$

3. **Sortie :** combinaison pondérée des valeurs :

$$\mathrm{out} = \sum_{i=1}^{m} \alpha_i \, v_i \in \mathbb{R}^{d_v} \tag{444}$$

Remarque 16.1. Les poids d'attention α_i vérifient $\alpha_i \geq 0$ et $\sum_i \alpha_i = 1$ (propriétés du softmax, cf. section 8). La sortie est donc un **barycentre** des valeurs, où les poids reflètent la pertinence de chaque clé par rapport à la requête. Si une clé k_j est très alignée avec q (produit scalaire élevé), le poids α_j sera proche de 1 et la sortie sera dominée par v_j.

16.2.2 Exemple numérique complet

Exemple 16.1 (Calcul d'attention pas à pas)**.** Considérons une séquence de $m = 4$ positions avec des vecteurs de dimension $d_k = d_v = 3$. Supposons que la requête est celle de la position 1, et que les clés et valeurs de toutes les positions sont :

$$q = \begin{pmatrix}1\\0\\1\end{pmatrix}, \quad k_1 = \begin{pmatrix}1\\1\\0\end{pmatrix}, \quad k_2 = \begin{pmatrix}0\\1\\1\end{pmatrix}, \quad k_3 = \begin{pmatrix}1\\0\\1\end{pmatrix}, \quad k_4 = \begin{pmatrix}0\\0\\0\end{pmatrix}$$

$$v_1 = \begin{pmatrix}1\\0\\0\end{pmatrix}, \quad v_2 = \begin{pmatrix}0\\1\\0\end{pmatrix}, \quad v_3 = \begin{pmatrix}0\\0\\1\end{pmatrix}, \quad v_4 = \begin{pmatrix}1\\1\\1\end{pmatrix}$$

Étape 1 : Scores de compatibilité.

$$s_1 = q^\top k_1 = 1 \cdot 1 + 0 \cdot 1 + 1 \cdot 0 = 1 \tag{445}$$
$$s_2 = q^\top k_2 = 1 \cdot 0 + 0 \cdot 1 + 1 \cdot 1 = 1 \tag{446}$$
$$s_3 = q^\top k_3 = 1 \cdot 1 + 0 \cdot 0 + 1 \cdot 1 = 2 \tag{447}$$
$$s_4 = q^\top k_4 = 0 \tag{448}$$

Étape 2 : Poids d'attention.

$$\alpha_1 = \frac{e^1}{e^1 + e^1 + e^2 + e^0} = \frac{2.718}{2.718 + 2.718 + 7.389 + 1.000} = \frac{2.718}{13.825} \approx 0.197 \tag{449}$$
$$\alpha_2 \approx 0.197 \tag{450}$$
$$\alpha_3 = \frac{7.389}{13.825} \approx 0.534 \tag{451}$$
$$\alpha_4 = \frac{1.000}{13.825} \approx 0.072 \tag{452}$$

On vérifie : $0.197 + 0.197 + 0.534 + 0.072 = 1.000$.

Étape 3 : Sortie.

$$\text{out} = 0.197 \begin{pmatrix}1\\0\\0\end{pmatrix} + 0.197 \begin{pmatrix}0\\1\\0\end{pmatrix} + 0.534 \begin{pmatrix}0\\0\\1\end{pmatrix} + 0.072 \begin{pmatrix}1\\1\\1\end{pmatrix} \tag{453}$$

$$= \begin{pmatrix}0.197 + 0.072\\0.197 + 0.072\\0.534 + 0.072\end{pmatrix} = \begin{pmatrix}0.269\\0.269\\0.606\end{pmatrix} \tag{454}$$

La sortie est dominée par la troisième composante, car k_3 (la clé la plus alignée avec q) a obtenu le plus grand poids d'attention (0.534), et $v_3 = (0, 0, 1)^\top$.

16.3 Scaled Dot-Product Attention

16.3.1 Formulation matricielle

En pratique, on ne calcule pas l'attention pour une seule requête mais pour toutes les positions simultanément, ce qui permet une implémentation efficace par produits matriciels.

Définition 16.2 (Scaled Dot-Product Attention)**.** Soient $Q \in \mathbb{R}^{n \times d_k}$ la matrice des requêtes (n requêtes de dimension d_k), $K \in \mathbb{R}^{m \times d_k}$ la matrice des clés, et $V \in \mathbb{R}^{m \times d_v}$ la matrice des valeurs. L'attention par produit scalaire mis à l'échelle est :

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \tag{455}$$

où le softmax est appliqué *ligne par ligne* (chaque ligne correspond à une requête), et la division par $\sqrt{d_k}$ est le facteur de mise à l'échelle.

Remarque 16.2 (Analyse dimensionnelle)**.** Vérifions les dimensions à chaque étape :

- $QK^\top$: $(n \times d_k) \cdot (d_k \times m) = n \times m$ — c'est la matrice des scores, une valeur par paire (requête, clé).
- $\text{softmax}(\cdot)$: $n \times m$ — le softmax normalise chaque ligne (somme à 1 sur les m clés).
- $\text{softmax}(\cdot) \cdot V$: $(n \times m) \cdot (m \times d_v) = n \times d_v$ — la sortie a n vecteurs de dimension d_v.

La sortie a donc la même dimension que les requêtes (une sortie d_v par requête), ce qui est cohérent.

16.3.2 Justification du facteur $\sqrt{d_k}$

Le facteur de mise à l'échelle $1/\sqrt{d_k}$ dans l'équation (455) n'est pas un choix arbitraire. Sans lui, l'attention devient pratiquement inutile pour de grandes dimensions.

Proposition 16.1 (Variance du produit scalaire)**.** Soient $q, k \in \mathbb{R}^{d_k}$ deux vecteurs aléatoires dont les composantes sont i.i.d. de moyenne nulle et variance unitaire : $q_i, k_i \sim (0, 1)$ indépendants. Alors :

$$\mathbb{E}[q^\top k] = 0, \qquad \text{Var}(q^\top k) = d_k \tag{456}$$

Démonstration. Le produit scalaire s'écrit $q^\top k = \sum_{i=1}^{d_k} q_i k_i$. C'est une somme de d_k termes indépendants.

Espérance. Par linéarité et indépendance :

$$\mathbb{E}[q^\top k] = \sum_{i=1}^{d_k} \mathbb{E}[q_i]\,\mathbb{E}[k_i] = \sum_{i=1}^{d_k} 0 \cdot 0 = 0 \tag{457}$$

Variance. Comme les termes $q_i k_i$ sont indépendants (car les composantes sont indépendantes entre elles) :

$$\mathrm{Var}(q^\top k) = \sum_{i=1}^{d_k} \mathrm{Var}(q_i k_i) \tag{458}$$

Pour chaque terme, puisque q_i et k_i sont indépendants et de moyenne nulle :

$$\begin{aligned} \mathrm{Var}(q_i k_i) &= \mathbb{E}[q_i^2 k_i^2] - (\mathbb{E}[q_i k_i])^2 && (459)\\ &= \mathbb{E}[q_i^2]\,\mathbb{E}[k_i^2] - 0 && (460)\\ &= \mathrm{Var}(q_i) \cdot \mathrm{Var}(k_i) = 1 \cdot 1 = 1 && (461) \end{aligned}$$

Donc $\mathrm{Var}(q^\top k) = \sum_{i=1}^{d_k} 1 = d_k$. □ □

Justification Mathématique

Cette proposition a des conséquences critiques pour le softmax. L'écart-type du produit scalaire est $\sqrt{d_k}$, donc pour $d_k = 512$ (dimension typique dans un Transformer), les scores $q^\top k$ ont un écart-type de $\sqrt{512} \approx 22.6$.
Or, le softmax $\sigma(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$ est très sensible à l'amplitude de ses entrées. Si un score vaut $+22$ tandis que les autres valent -22, on a :

$$\alpha_{\max} = \frac{e^{22}}{e^{22} + (m-1)e^{-22}} \approx 1 - (m-1)e^{-44} \approx 1 \tag{462}$$

Le softmax sature : presque tout le poids est concentré sur un seul vecteur. C'est une distribution quasi déterministe ("one-hot"). Le gradient du softmax dans cette zone est quasiment nul (cf. section 8), ce qui bloque l'apprentissage.
En divisant par $\sqrt{d_k}$, on obtient des scores de variance $d_k/d_k = 1$, donc d'écart-type 1. Le softmax reste dans une zone où il produit des distributions douces, avec des gradients informatifs. C'est la même logique que l'initialisation He (section 10) : contrôler la variance pour éviter la saturation.

16.4 Multi-Head Attention

L'attention à une seule tête calcule une unique moyenne pondérée des valeurs. Cela force le modèle à compresser toute l'information relationnelle dans un seul ensemble de poids d'attention. L'attention multi-têtes résout ce problème en effectuant plusieurs calculs d'attention en parallèle.

Définition 16.3 (Multi-Head Attention)**.** Soient $Q \in \mathbb{R}^{n \times d_{\text{model}}}$, $K \in \mathbb{R}^{m \times d_{\text{model}}}$, $V \in \mathbb{R}^{m \times d_{\text{model}}}$. L'attention multi-têtes avec h têtes est définie par :

$$\mathrm{MultiHead}(Q, K, V) = \mathrm{Concat}(\mathrm{head}_1, \ldots, \mathrm{head}_h)\, W^O \tag{463}$$

où chaque tête effectue un calcul d'attention dans un sous-espace appris :

$$\text{head}_i = \text{Attention}(QW_i^Q,\ KW_i^K,\ VW_i^V) \tag{464}$$

avec les matrices de projection :

$$W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v} \tag{465}$$

et la projection de sortie $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$.

Typiquement, on pose $d_k = d_v = d_{\text{model}}/h$, de sorte que chaque tête opère dans un sous-espace de dimension réduite.

Remarque 16.3 (Analyse dimensionnelle)**.** Détaillons les dimensions pour des valeurs concrètes. Prenons $d_{\text{model}} = 512$, $h = 8$, donc $d_k = d_v = 64$.

— Projection des requêtes pour la tête i : QW_i^Q a la dimension $(n \times 512) \cdot (512 \times 64) = n \times 64$.

— Chaque head_i a la dimension $n \times 64$ (sortie de l'attention).

— $\text{Concat}(\text{head}_1, \ldots, \text{head}_8)$ a la dimension $n \times 512$.

— Après multiplication par $W^O \in \mathbb{R}^{512 \times 512}$: la sortie est $n \times 512$.

La sortie a donc la même dimension que l'entrée ($n \times d_{\text{model}}$), ce qui permet d'empiler les blocs.

16.4.1 Coût de calcul : multi-têtes vs. tête unique

Proposition 16.2 (Équivalence du coût de calcul)**.** Avec h têtes de dimension $d_k = d_{\text{model}}/h$, le coût total de calcul de l'attention multi-têtes est du même ordre que celui d'une attention à tête unique de dimension d_{model}.

Démonstration. **Attention à tête unique** de dimension d_{model} : le produit $QK^\top$ coûte $O(n \cdot m \cdot d_{\text{model}})$ opérations, et la multiplication par V coûte $O(n \cdot m \cdot d_{\text{model}})$. Coût total de l'attention proprement dite : $O(n \cdot m \cdot d_{\text{model}})$.

Attention multi-têtes avec h têtes de dimension $d_k = d_{\text{model}}/h$:

— Pour chaque tête, le produit QW_i^Q coûte $O(n \cdot d_{\text{model}} \cdot d_k)$. Idem pour KW_i^K et VW_i^V. Coût des projections pour h têtes : $3h \cdot O(n \cdot d_{\text{model}} \cdot d_k) = O(n \cdot d_{\text{model}}^2)$ (car $h \cdot d_k = d_{\text{model}}$).

— Pour chaque tête, l'attention coûte $O(n \cdot m \cdot d_k)$. Pour h têtes : $h \cdot O(n \cdot m \cdot d_k) = O(n \cdot m \cdot d_{\text{model}})$.

— La projection finale W^O coûte $O(n \cdot d_{\text{model}}^2)$.

Le coût total est $O(n \cdot d_{\text{model}}^2 + n \cdot m \cdot d_{\text{model}})$, qui est du même ordre que l'attention à tête unique augmentée des projections linéaires. □ □

Intuition

Si l'attention à tête unique est un seul projecteur qui éclaire la scène sous un angle fixe, l'attention multi-têtes est un ensemble de projecteurs orientés différemment. Chaque tête peut capturer un type de relation distinct :

- Une tête peut apprendre les relations **syntaxiques** (sujet-verbe, article-nom).
- Une autre peut capturer les relations **sémantiques** (synonymes, coréférences).
- Une troisième peut se spécialiser sur la **proximité positionnelle** (mots adjacents).

Le tout pour un coût de calcul équivalent, grâce à la réduction proportionnelle de la dimension par tête.

16.5 Encodage positionnel

16.5.1 Le problème de la permutation

L'attention, telle que définie, est une opération *équivariante par permutation* : si l'on permute les positions de la séquence d'entrée, les sorties sont permutées de la même manière, et les poids d'attention sont identiques.

Proposition 16.3 (Équivariance par permutation)**.** Soit π une permutation de $\{1, \ldots, n\}$. Notons X_π la matrice obtenue en permutant les lignes de X selon π. En auto-attention ($Q = K = V = X$) :

$$\text{Attention}(X_\pi, X_\pi, X_\pi) = [\text{Attention}(X, X, X)]_\pi \tag{466}$$

Démonstration. Notons $A = \text{softmax}(XX^\top/\sqrt{d_k})$. Pour l'entrée permutée :

$$X_\pi X_\pi^\top = P_\pi X (P_\pi X)^\top = P_\pi X X^\top P_\pi^\top \tag{467}$$

où P_π est la matrice de permutation. Le softmax étant appliqué ligne par ligne, et P_π ne faisant que réordonner les lignes et colonnes :

$$\text{softmax}\left(\frac{P_\pi X X^\top P_\pi^\top}{\sqrt{d_k}}\right) = P_\pi \,\text{softmax}\left(\frac{XX^\top}{\sqrt{d_k}}\right) P_\pi^\top = P_\pi A P_\pi^\top \tag{468}$$

La sortie est donc :

$$P_\pi A P_\pi^\top \cdot P_\pi X = P_\pi A (P_\pi^\top P_\pi) X = P_\pi A X = P_\pi \cdot \text{Attention}(X, X, X) \quad \square \tag{469}$$

$\square$

Attention

Cette propriété signifie que l'attention seule ne distingue pas l'ordre des mots. Les phrases "le chat mange la souris" et "la souris mange le chat" produiraient les mêmes représentations (à permutation près). Pour le traitement du langage, c'est inacceptable : il faut injecter explicitement l'information de position.

16.5.2 Encodage positionnel sinusoïdal

Définition 16.4 (Encodage positionnel sinusoïdal)**.** Pour une position $\text{pos} \in \{0, 1, \ldots, n-1\}$ dans la séquence et une dimension $i \in \{0, 1, \ldots, d_{\text{model}}/2 - 1\}$ du vecteur d'encodage, l'encodage positionnel est :

$$\text{PE}(\text{pos}, 2i) = \sin\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right) \tag{470}$$

$$\text{PE}(\text{pos}, 2i+1) = \cos\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right) \tag{471}$$

Le vecteur $\text{PE}(\text{pos}) \in \mathbb{R}^{d_{\text{model}}}$ est *ajouté* (et non concaténé) à l'embedding de l'entrée :

$$x_{\text{pos}} \leftarrow x_{\text{pos}} + \text{PE}(\text{pos}) \tag{472}$$

Remarque 16.4. Chaque dimension de l'encodage positionnel correspond à une sinusoïde de fréquence différente. La dimension $2i$ utilise une fréquence $\omega_i = 1/10000^{2i/d_{\text{model}}}$. Les premières dimensions (i petit) oscillent rapidement (haute fréquence, période courte), tandis que les dernières dimensions (i grand) oscillent lentement (basse fréquence, période très longue). Cela donne une représentation multi-échelle de la position, similaire à une décomposition en séries de Fourier.

16.5.3 Propriété fondamentale : expressibilité des positions relatives

La raison principale du choix sinusoïdal est que le modèle peut apprendre à détecter les positions *relatives* par combinaison linéaire.

Théorème 16.1 (Linéarité des décalages positionnels)**.** Pour tout décalage k fixé, il existe une matrice $M_k \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$ (ne dépendant pas de pos) telle que :

$$\text{PE}(\text{pos} + k) = M_k \cdot \text{PE}(\text{pos}) \tag{473}$$

Démonstration. Considérons une paire de dimensions $(2i, 2i+1)$. Les valeurs de l'encodage pour la position pos sont $(\sin(\omega_i \cdot \text{pos}),\ \cos(\omega_i \cdot \text{pos}))$ où $\omega_i = 1/10000^{2i/d_{\text{model}}}$.

Pour la position $\text{pos} + k$, les formules d'addition trigonométrique donnent :

$$\sin(\omega_i(\text{pos} + k)) = \sin(\omega_i \cdot \text{pos})\cos(\omega_i k) + \cos(\omega_i \cdot \text{pos})\sin(\omega_i k) \tag{474}$$

$$\cos(\omega_i(\text{pos} + k)) = \cos(\omega_i \cdot \text{pos})\cos(\omega_i k) - \sin(\omega_i \cdot \text{pos})\sin(\omega_i k) \tag{475}$$

En notation matricielle, pour cette paire de dimensions :

$$\begin{pmatrix} \text{PE}(\text{pos}+k, 2i) \\ \text{PE}(\text{pos}+k, 2i+1) \end{pmatrix} = \underbrace{\begin{pmatrix} \cos(\omega_i k) & \sin(\omega_i k) \\ -\sin(\omega_i k) & \cos(\omega_i k) \end{pmatrix}}_{R_i(k)\ :\ \text{matrice de rotation}} \begin{pmatrix} \text{PE}(\text{pos}, 2i) \\ \text{PE}(\text{pos}, 2i+1) \end{pmatrix} \tag{476}$$

Comme les paires de dimensions sont indépendantes, la matrice M_k est diagonale par blocs :

$$M_k = \text{diag}(R_0(k),\ R_1(k),\ \ldots,\ R_{d_{\text{model}}/2-1}(k)) \tag{477}$$

Cette matrice ne dépend que de k (le décalage) et non de pos. Elle est de plus orthogonale ($M_k^\top M_k = I$), car chaque bloc $R_i(k)$ est une matrice de rotation. □ □

Intuition

Cette propriété est essentielle : elle signifie que le produit scalaire $\mathrm{PE}(\mathrm{pos})^\top W^\top W\,\mathrm{PE}(\mathrm{pos}+k)$, qui apparaît dans les scores d'attention, peut capturer une fonction de la distance relative k sans avoir besoin d'encoder explicitement chaque paire de positions absolues. Le modèle peut ainsi *généraliser à des longueurs de séquence non vues pendant l'entraînement.*

16.5.4 Exemple numérique

Exemple 16.2 (Calcul de l'encodage positionnel)**.** Prenons $d_{\text{model}} = 4$ (pour simplifier). Les fréquences sont :

$$\omega_0 = \frac{1}{10000^{0/4}} = 1, \qquad \omega_1 = \frac{1}{10000^{2/4}} = \frac{1}{100} = 0.01$$

Les encodages pour les positions 0, 1, 2, 3 :

pos	$\sin(\omega_0 \cdot \mathrm{pos})$	$\cos(\omega_0 \cdot \mathrm{pos})$	$\sin(\omega_1 \cdot \mathrm{pos})$	$\cos(\omega_1 \cdot \mathrm{pos})$
0	$\sin(0) = 0$	$\cos(0) = 1$	$\sin(0) = 0$	$\cos(0) = 1$
1	$\sin(1) = 0.841$	$\cos(1) = 0.540$	$\sin(0.01) = 0.010$	$\cos(0.01) = 1.000$
2	$\sin(2) = 0.909$	$\cos(2) = -0.416$	$\sin(0.02) = 0.020$	$\cos(0.02) = 1.000$
3	$\sin(3) = 0.141$	$\cos(3) = -0.990$	$\sin(0.03) = 0.030$	$\cos(0.03) = 1.000$

On observe que les deux premières composantes (haute fréquence) varient rapidement et distinguent bien les positions proches. Les deux dernières composantes (basse fréquence) varient très lentement : elles distinguent les positions éloignées.

16.6 Le bloc Transformer

Nous avons maintenant toutes les briques élémentaires. Un bloc Transformer les assemble en une unité modulaire qui sera empilée N fois pour former l'architecture complète.

16.6.1 Architecture d'un bloc encodeur

Définition 16.5 (Bloc Transformer (encodeur))**.** Un bloc Transformer encodeur transforme une séquence $X \in \mathbb{R}^{n \times d_{\text{model}}}$ par la composition suivante :

$$\text{Étape 1 :} \quad X' = \mathrm{LayerNorm}(X + \mathrm{MultiHead}(X, X, X)) \tag{478}$$

$$\text{Étape 2 :} \quad X'' = \mathrm{LayerNorm}(X' + \mathrm{FFN}(X')) \tag{479}$$

où FFN est un réseau feed-forward appliqué position par position :

$$\mathrm{FFN}(x) = W_2\,\sigma(W_1 x + b_1) + b_2 \tag{480}$$

avec $W_1 \in \mathbb{R}^{d_{ff} \times d_{\text{model}}}$, $W_2 \in \mathbb{R}^{d_{\text{model}} \times d_{ff}}$, et σ est une activation non linéaire (ReLU dans le Transformer original, GELU dans les variantes modernes, cf. section 9).

Remarque 16.5. Typiquement, $d_{ff} = 4\,d_{\text{model}}$. Pour $d_{\text{model}} = 512$, on a $d_{ff} = 2048$. Le FFN projette d'abord dans un espace de dimension supérieure ("expansion"), applique la non-linéarité, puis ramène dans la dimension du modèle ("contraction"). C'est un réseau à deux couches au sens de la section 7.

16.6.2 Connexions résiduelles

Les termes "$X + \ldots$" dans les équations (478)–(479) sont des **connexions résiduelles** (skip connections), identiques à celles étudiées en section 10 dans le contexte de la propagation du gradient dans les réseaux profonds.

Justification Mathématique

Rappelons le problème central : dans un réseau de N blocs empilés, le gradient doit traverser N couches lors de la rétropropagation. Sans connexion résiduelle, le gradient de la sortie $X^{(N)}$ par rapport à l'entrée $X^{(0)}$ est un produit de N jacobiennes :

$$\frac{\partial X^{(N)}}{\partial X^{(0)}} = \prod_{l=1}^{N} \frac{\partial X^{(l)}}{\partial X^{(l-1)}} \tag{481}$$

Ce produit tend à exploser ou s'évanouir exponentiellement (cf. section 10). Avec la connexion résiduelle $X^{(l)} = X^{(l-1)} + F^{(l)}(X^{(l-1)})$, le gradient devient :

$$\frac{\partial X^{(l)}}{\partial X^{(l-1)}} = I + \frac{\partial F^{(l)}}{\partial X^{(l-1)}} \tag{482}$$

Le terme identité I garantit qu'il existe toujours un "chemin direct" pour le gradient, même si $\frac{\partial F^{(l)}}{\partial X^{(l-1)}}$ est petit. Cela stabilise l'entraînement de Transformers avec $N = 12$, 24, voire 96 blocs.

16.6.3 Layer Normalization

Définition 16.6 (Layer Normalization)**.** Contrairement à la Batch Normalization (section 11) qui normalise sur la dimension du mini-batch, la Layer Normalization normalise sur la dimension des features *pour chaque position indépendamment*. Pour un vecteur $x \in \mathbb{R}^{d_{\text{model}}}$:

$$\text{LayerNorm}(x) = \gamma \odot \frac{x - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \beta \tag{483}$$

où $\mu = \frac{1}{d_{\text{model}}} \sum_{j=1}^{d_{\text{model}}} x_j$, $\sigma^2 = \frac{1}{d_{\text{model}}} \sum_{j=1}^{d_{\text{model}}} (x_j - \mu)^2$, et $\gamma, \beta \in \mathbb{R}^{d_{\text{model}}}$ sont des paramètres apprenables.

Justification Mathématique

Pourquoi Layer Normalization plutôt que Batch Normalization dans les Transformers ?

1. **Indépendance de la taille du batch.** La LayerNorm calcule les statistiques sur les features d'un seul exemple. Elle fonctionne donc identiquement en entraînement et en inférence, sans nécessiter de moyenne mobile (contrairement à la BatchNorm, cf. section 11).
2. **Séquences de longueurs variables.** Les séquences de texte ont des longueurs différentes. La BatchNorm nécessiterait de normaliser sur des positions correspondantes entre les exemples du batch, ce qui est mal défini pour les positions au-delà de la longueur la plus courte.
3. **Cohérence avec l'auto-régression.** En génération (décodeur), les tokens futurs ne sont pas disponibles. La LayerNorm, qui ne regarde que le vecteur courant, est naturellement compatible avec cette contrainte.

16.6.4 Décompte des paramètres

Proposition 16.4 (Nombre de paramètres d'un bloc Transformer)**.** Un bloc Transformer encodeur avec $d_{\text{model}} = d$, h têtes, et $d_{ff} = 4d$ contient :

$$N_{\text{params}} = 4d^2 + 8d^2 + 4d = 12d^2 + 4d \approx 12d^2 \tag{484}$$

Démonstration. Détaillons chaque composante :

Multi-Head Attention :

— Projections W_i^Q, W_i^K, W_i^V pour h têtes : chacune est $d \times d_k$ avec $d_k = d/h$. Au total pour les h têtes : $h \times (d \times d/h) = d^2$ par matrice. Trois matrices : $3d^2$ paramètres (plus $3d$ biais).

— Projection de sortie $W^O \in \mathbb{R}^{d\times d}$: d^2 paramètres (plus d biais).

— Total attention : $4d^2 + 4d$.

Feed-Forward Network :

— $W_1 \in \mathbb{R}^{4d\times d}$: $4d^2$ paramètres, $b_1 \in \mathbb{R}^{4d}$: $4d$ biais.

— $W_2 \in \mathbb{R}^{d\times 4d}$: $4d^2$ paramètres, $b_2 \in \mathbb{R}^d$: d biais.

— Total FFN : $8d^2 + 5d$.

Layer Normalization : deux couches de LayerNorm, chacune avec $\gamma, \beta \in \mathbb{R}^d$: $4d$ paramètres.

Total : $4d^2 + 4d + 8d^2 + 5d + 4d = 12d^2 + 13d$. Pour $d = 512$: $12 \times 512^2 + 13 \times 512 = 3\,145\,728 + 6\,656 \approx 3.15$ millions de paramètres par bloc. Le terme en d est négligeable devant d^2, d'où l'approximation $12d^2$. □ □

Algorithme : Passe avant d'un bloc Transformer encodeur

Entrée : $X \in \mathbb{R}^{n \times d}$ (séquence de n tokens de dimension d)
Paramètres : W_i^Q, W_i^K, W_i^V pour $i = 1, \ldots, h$; W^O ; W_1, b_1, W_2, b_2 ; $\gamma_1, \beta_1, \gamma_2, \beta_2$

1. **Multi-Head Self-Attention :**
   (a) Pour chaque tête $i = 1, \ldots, h$:
   — $Q_i = XW_i^Q, \quad K_i = XW_i^K, \quad V_i = XW_i^V$
   — $\text{head}_i = \text{softmax}\left(\frac{Q_i K_i^\top}{\sqrt{d_k}}\right) V_i$
   (b) $\text{MHA}(X) = [\text{head}_1; \ldots; \text{head}_h]\, W^O$
2. **Résiduel + LayerNorm :** $X' = \text{LayerNorm}(X + \text{MHA}(X))$
3. **Feed-Forward :** $\text{FFN}(X') = \sigma(X' W_1^\top + b_1)\, W_2^\top + b_2 \quad$ (position par position)
4. **Résiduel + LayerNorm :** $X'' = \text{LayerNorm}(X' + \text{FFN}(X'))$

Sortie : $X'' \in \mathbb{R}^{n \times d}$

16.7 Masquage causal pour la génération

Dans un modèle de langage, la tâche est de prédire le prochain token étant donné les précédents : $P(x_{t+1} \mid x_1, \ldots, x_t)$. Il est donc **interdit** de consulter les positions futures. L'attention standard, qui permet à chaque position de voir toutes les autres, doit être modifiée.

Définition 16.7 (Masque causal)**.** Le masque causal est une matrice $M \in \{0, -\infty\}^{n \times n}$ définie par :

$$M_{ij} = \begin{cases} 0 & \text{si } j \leq i \quad \text{(position passée ou présente)} \\ -\infty & \text{si } j > i \quad \text{(position future)} \end{cases} \tag{485}$$

L'attention causale (masked self-attention) est :

$$\text{CausalAttention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V \tag{486}$$

Proposition 16.5 (Annulation des poids futurs)**.** Avec le masque causal, les poids d'attention pour les positions futures sont exactement nuls : $\alpha_{ij} = 0$ pour tout $j > i$.

Démonstration. Soit $s_{ij} = (QK^\top)_{ij}/\sqrt{d_k}$ le score non masqué. Après ajout du masque, le score effectif est :

$$\tilde{s}_{ij} = s_{ij} + M_{ij} = \begin{cases} s_{ij} & \text{si } j \leq i \\ s_{ij} - \infty = -\infty & \text{si } j > i \end{cases} \tag{487}$$

Le poids d'attention pour la position i regardant la position $j > i$ est :

$$\alpha_{ij} = \frac{\exp(\tilde{s}_{ij})}{\sum_{k=1}^n \exp(\tilde{s}_{ik})} = \frac{\exp(-\infty)}{\sum_{k=1}^n \exp(\tilde{s}_{ik})} = \frac{0}{\sum_{k=1}^n \exp(\tilde{s}_{ik})} = 0 \tag{488}$$

car $\exp(-\infty) = \lim_{x \to -\infty} e^x = 0$. Le dénominateur est strictement positif car il contient au moins le terme $\exp(\tilde{s}_{ii}) = \exp(s_{ii}) > 0$. $\square$ $\square$

Remarque 16.6. En pratique, on n'utilise pas la valeur $-\infty$ mais une très grande valeur négative, par exemple -10^9 en virgule flottante simple précision. Le résultat est numériquement identique car $\exp(-10^9) \approx 0$ en précision machine.

Exemple 16.3 (Matrice d'attention causale)**.** Pour une séquence de 4 tokens, la matrice des poids d'attention avant normalisation (après masquage) a la forme :

$$\frac{QK^\top}{\sqrt{d_k}} + M = \begin{pmatrix} s_{11} & -\infty & -\infty & -\infty \\ s_{21} & s_{22} & -\infty & -\infty \\ s_{31} & s_{32} & s_{33} & -\infty \\ s_{41} & s_{42} & s_{43} & s_{44} \end{pmatrix} \tag{489}$$

Après softmax ligne par ligne :

$$\alpha = \begin{pmatrix} 1 & 0 & 0 & 0 \\ \alpha_{21} & \alpha_{22} & 0 & 0 \\ \alpha_{31} & \alpha_{32} & \alpha_{33} & 0 \\ \alpha_{41} & \alpha_{42} & \alpha_{43} & \alpha_{44} \end{pmatrix} \tag{490}$$

C'est une matrice **triangulaire inférieure**. Le premier token ne peut regarder que lui-même, le deuxième peut regarder les positions 1 et 2, et ainsi de suite. C'est exactement la propriété auto-régressive souhaitée.

16.8 L'architecture Transformer complète

16.8.1 Encodeur-décodeur (Vaswani et al., 2017)

L'architecture Transformer originale, introduite dans l'article "Attention Is All You Need", suit un schéma encodeur-décodeur.

Définition 16.8 (Architecture Transformer encodeur-décodeur)**.** L'architecture complète est composée de :

1. **Encodeur** : une pile de N blocs identiques. Chaque bloc contient :
   — Auto-attention multi-têtes (self-attention) + résiduel + LayerNorm
   — FFN + résiduel + LayerNorm

   L'entrée de l'encodeur est la séquence source augmentée de l'encodage positionnel.
2. **Décodeur** : une pile de N blocs. Chaque bloc contient :
   — Auto-attention **masquée** (causale) + résiduel + LayerNorm
   — Attention **croisée** (cross-attention) : les requêtes viennent du décodeur, les clés et valeurs viennent de la sortie de l'encodeur. Résiduel + LayerNorm.
   — FFN + résiduel + LayerNorm

La sortie du dernier bloc du décodeur est projetée vers le vocabulaire par une couche linéaire suivie d'un softmax (cf. section 8) pour obtenir une distribution de probabilité sur le token suivant.

Remarque 16.7 (Attention croisée)**.** L'attention croisée (cross-attention) est la pièce maîtresse du lien entre encodeur et décodeur. Elle permet au décodeur de "consulter" la représentation de la séquence source. Formellement, si $H_{\text{enc}} \in \mathbb{R}^{n_s \times d}$ est la sortie de l'encodeur et $H_{\text{dec}} \in \mathbb{R}^{n_t \times d}$ est la sortie de la sous-couche d'auto-attention du décodeur :

$$\text{CrossAttention} = \text{MultiHead}(Q = H_{\text{dec}},\ K = H_{\text{enc}},\ V = H_{\text{enc}}) \tag{491}$$

Les requêtes sont issues du décodeur ("que cherche-t-on ?") tandis que les clés et valeurs proviennent de l'encodeur ("où et quoi regarder dans la source ?").

16.8.2 Variantes modernes

L'architecture originale a donné naissance à trois grandes familles :

1. **Encodeur seul (type BERT).** On n'utilise que la pile d'encodeurs, avec une auto-attention *bidirectionnelle* (chaque position voit toutes les autres). L'entraînement se fait par prédiction de tokens masqués (masked language modeling). Ces modèles excellent en compréhension de texte : classification, extraction d'entités, réponse à des questions.
2. **Décodeur seul (type GPT).** On n'utilise que la pile de décodeurs, avec attention causale. L'entraînement est auto-régressif : prédire le prochain token. Pas besoin d'attention croisée puisqu'il n'y a pas d'encodeur. Ces modèles sont les plus puissants pour la génération de texte et forment la base des grands modèles de langage (LLM).
3. **Encodeur-décodeur (type T5, BART).** L'architecture complète, idéale pour les tâches séquence-à-séquence : traduction, résumé, etc.

Intuition

Le succès des Transformers repose sur une conjonction de facteurs :

— **Parallélisation massive.** Contrairement aux RNN, tous les tokens sont traités simultanément. L'entraînement exploite pleinement les GPU modernes.

— **Modélisation directe des dépendances.** Deux tokens séparés de 500 positions sont reliés en une seule couche d'attention, contre 500 étapes de récurrence pour un RNN.

— **Lois d'échelle (scaling laws).** Empiriquement, la performance des Transformers s'améliore de manière prévisible avec le nombre de paramètres, la taille des données, et le budget de calcul. C'est cette propriété qui a motivé le développement de modèles de plus en plus grands (des centaines de milliards de paramètres).

16.9 Analyse de complexité

Analysons le coût de calcul d'un bloc Transformer pour une séquence de longueur n et une dimension de modèle d.

Proposition 16.6 (Complexité d'un bloc Transformer)**.** Les deux composantes principales d'un bloc Transformer ont les complexités suivantes :

1. **Auto-attention :** $O(n^2 d)$
2. **Feed-forward :** $O(nd^2)$

Démonstration. **Auto-attention.** Les projections $Q = XW^Q$, $K = XW^K$, $V = XW^V$ coûtent chacune $O(nd^2)$ (ignoré ici car absorbé dans le FFN). Le calcul critique est le produit $QK^\top \in \mathbb{R}^{n\times n}$:

— $QK^\top$: multiplication de $(n \times d)$ par $(d \times n) = O(n^2 d)$.

— Softmax : $O(n^2)$ (négligeable).

— Multiplication par V : $(n \times n) \cdot (n \times d) = O(n^2 d)$.

Complexité totale de l'attention : $O(n^2 d)$.

Feed-forward. Le FFN applique deux transformations linéaires à chacun des n vecteurs de dimension d :

— $W_1 x + b_1$: multiplication de $(4d \times d)$ par un vecteur de dimension d, pour n positions : $O(n \cdot 4d \cdot d) = O(nd^2)$.

— $W_2 z + b_2$: idem, $O(nd^2)$.

Complexité totale du FFN : $O(nd^2)$. □ □

Attention

Le terme $O(n^2 d)$ de l'auto-attention est **quadratique en la longueur de la séquence**. Pour une séquence de $n = 1\,000$ tokens avec $d = 512$, la matrice d'attention $QK^\top$ contient 10^6 éléments — c'est gérable. Mais pour $n = 100\,000$, cette matrice contient 10^{10} éléments, ce qui est prohibitif en mémoire et en calcul. C'est le principal goulot d'étranglement des Transformers pour les longues séquences. Plusieurs approches ont été proposées pour réduire cette complexité :

— **Attention creuse (sparse attention)** : ne calculer l'attention que pour un sous-ensemble de paires de positions, par exemple les positions proches et quelques positions éloignées sélectionnées.

— **Attention linéaire** : reformuler le softmax pour obtenir un noyau qui permet de calculer l'attention en $O(nd^2)$.

— **Fenêtre glissante (sliding window)** : limiter l'attention à une fenêtre locale de taille $w \ll n$, ramenant le coût à $O(nwd)$.

16.9.1 Comparaison avec les RNN

Table 2 : Comparaison de la complexité par couche entre RNN et Transformer.

Architecture	**Complexité par couche**	**Opérations séquentielles**	**Chemin max.**
RNN	$O(nd^2)$	$O(n)$	$O(n)$
Transformer	$O(n^2 d)$	$O(1)$	$O(1)$

Justification Mathématique

Interprétons chaque colonne :

1. **Complexité par couche.** Le RNN effectue n multiplications matrice-vecteur de taille $d \times d$: coût $O(nd^2)$. Le Transformer effectue un produit matriciel $QK^\top$ de coût $O(n^2 d)$. Pour $n < d$ (cas courant : $d = 512$, $n = 128$), le Transformer est plus rapide. Pour $n \gg d$, le RNN a l'avantage théorique.
2. **Opérations séquentielles.** C'est la différence fondamentale. Le RNN impose $O(n)$ étapes séquentielles (h_t dépend de h_{t-1}), ce qui limite la parallélisation sur GPU. Le Transformer calcule toutes les positions en parallèle : $O(1)$ étapes séquentielles. Cette parallélisation est la raison principale de la supériorité pratique des Transformers.
3. **Chemin maximal.** C'est la longueur du plus court chemin dans le graphe de calcul entre deux positions quelconques. Pour un RNN, l'information du premier token doit traverser n étapes pour atteindre le dernier : chemin $O(n)$. Pour un Transformer, chaque position a un accès direct à toutes les autres via l'attention : chemin $O(1)$. Cela facilite considérablement l'apprentissage des dépendances à longue distance.

17 Synthèse finale : du neurone unique au Transformer

17.1 Le parcours accompli

Ce cours a construit, brique par brique, les fondements mathématiques du deep learning. Chaque section a répondu à une question précise, et chaque réponse a ouvert la voie à la suivante. Retraçons cette logique.

Nous sommes partis d'un **neurone unique** (section 2) : un simple produit scalaire suivi d'une sigmoïde, capable de tracer une frontière linéaire entre deux classes. Pour entraîner ce neurone, il a fallu définir ce que signifie "bien prédire" — d'où la **fonction de coût** (section 3), dérivée rigoureusement du maximum de vraisemblance. Puis, pour minimiser cette perte, nous avons introduit la **descente de gradient** (section 4) et la notion de backpropagation.

Le passage à l'échelle a exigé la **vectorisation** (section 5) : reformuler tous les calculs en opérations matricielles. Cela nous a permis de généraliser naturellement à une **couche de neurones** (section 6), puis à des **réseaux profonds** (section 7) où la composition de non-linéarités confère au modèle une expressivité arbitraire. La **classification multi-classes** (section 8) a étendu le cadre binaire via la softmax et la cross-entropie catégorielle.

Les sections suivantes ont stabilisé et raffiné cette architecture. Le choix des **fonctions d'activation** (section 9) conditionne la capacité du réseau à propager l'information. L'**initialisation** (section 10) garantit que les activations et les gradients conservent une variance raisonnable dès le premier forward pass. La **Batch Normalization** (section 11) maintient cette stabilité tout au long de l'entraînement. Les **optimiseurs avancés** (section 12) accélèrent la convergence grâce au momentum et à l'adaptativité. La **régularisation** (section 13) contrôle la complexité pour garantir la généralisation.

Enfin, nous avons exploré les trois grandes familles d'architectures spécialisées. Les **réseaux convolutifs** (section 14) exploitent la structure spatiale des images grâce au partage de poids et aux champs récepteurs locaux. Les **réseaux récurrents** (section 15)

traitent les séquences en maintenant un état caché évolutif, avec les LSTM pour résoudre le problème des gradients qui disparaissent. Le **mécanisme d'attention et les Transformers** (section 16) remplacent la récurrence par un accès direct à toutes les positions, atteignant aujourd'hui l'état de l'art dans la quasi-totalité des domaines.

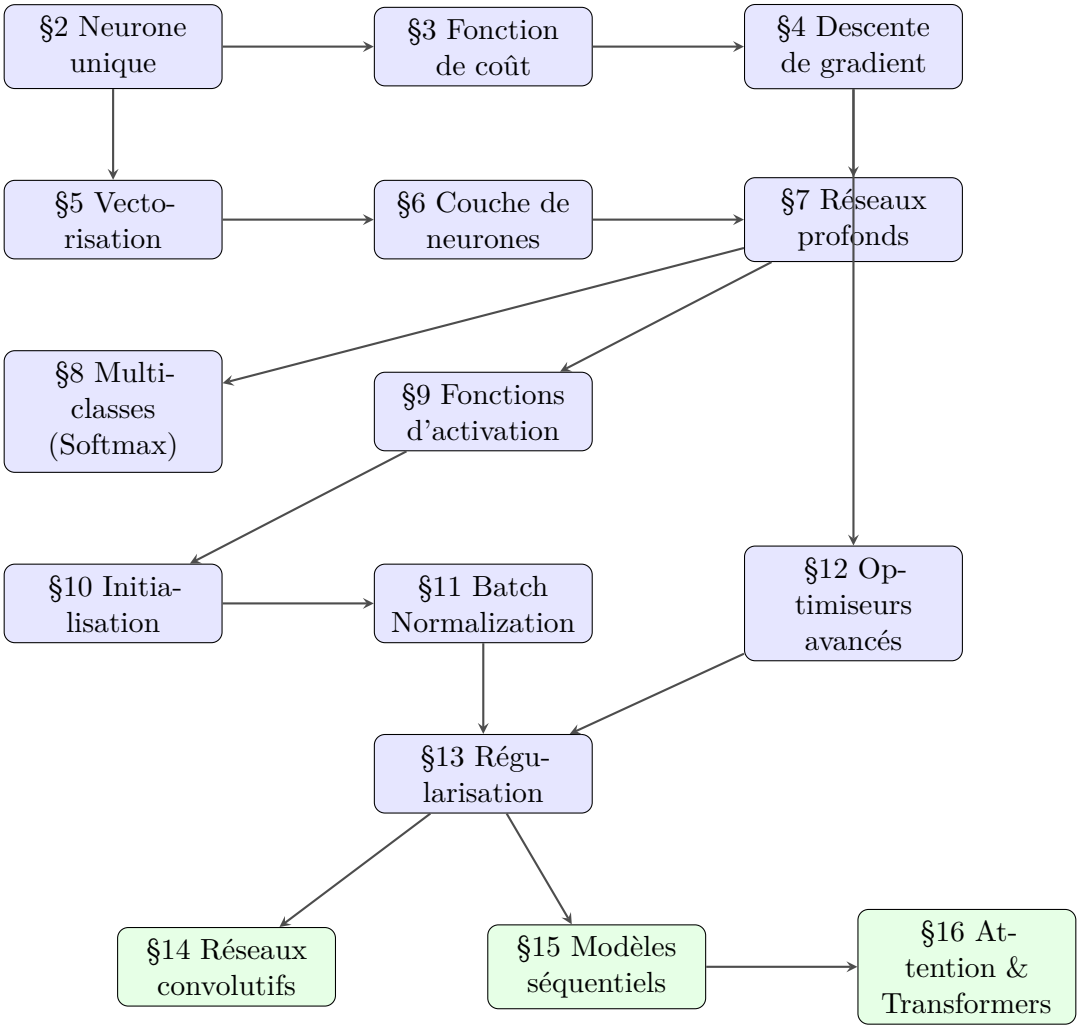

Figure 14 : Dépendances logiques entre les sections du cours. Les sections vertes représentent les architectures spécialisées qui s'appuient sur les fondements (sections bleues).

Remarque 17.1. Ce parcours n'est pas linéaire par hasard. Chaque section résout un problème soulevé par la précédente : la sigmoïde appelle une perte, la perte appelle un optimiseur, la profondeur appelle l'initialisation, et ainsi de suite. Les architectures spécialisées (CNN, RNN, Transformers) ne sont pas des techniques isolées : elles réutilisent *toutes* les briques fondamentales — backpropagation, BatchNorm, Adam, dropout — et n'ajoutent qu'une structure adaptée au type de données.

17.2 Les trois piliers du deep learning

Intuition

[Architecture conceptuelle] Toutes les sections du cours s'organisent autour de trois piliers :

1. **Modélisation** — construire une fonction paramétrique expressive (sections 2, 5, 6, 7, 8, 9, 14, 15, 16)
2. **Optimisation** — trouver les meilleurs paramètres efficacement (sections 3, 4, 12)
3. **Généralisation** — s'assurer que le modèle fonctionne sur de nouvelles données (sections 10, 11, 13)

Un bon praticien du deep learning maîtrise les trois simultanément.

17.2.1 Pilier 1 : Modélisation

La modélisation consiste à construire une fonction $f_\theta : \mathbb{R}^{n_x} \to \mathbb{R}^{n_y}$ capable d'approximer la relation entrée-sortie. Notre construction a suivi une progression en quatre étapes, puis trois spécialisations.

Définition 17.1 (Du neurone au réseau profond)**. Neurone unique** (section 2) — frontière de décision linéaire :

$$a = \sigma(w^T x + b), \quad \sigma(z) = \frac{1}{1 + e^{-z}} \tag{492}$$

Réseau profond (section 7) — composition de couches :

$$A^{[l]} = g^{[l]}\left(W^{[l]} A^{[l-1]} + b^{[l]}\right), \quad l = 1, \ldots, L \tag{493}$$

Couche convolutive (section 14) — partage de poids et localité :

$$Z_c(i,j) = \sum_{c'=0}^{C_{\text{in}}-1} \sum_{\delta_i,\delta_j} W_{c,c'}(\delta_i, \delta_j)\, A_{c'}(i \cdot s + \delta_i,\ j \cdot s + \delta_j) + b_c \tag{494}$$

Attention (section 16) — accès direct à toute la séquence :

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \tag{495}$$

17.2.2 Pilier 2 : Optimisation

L'optimisation cherche $\theta^* = \arg\min_\theta J(\theta)$ par des mises à jour itératives basées sur le gradient.

Définition 17.2 (De la descente de gradient à Adam)**. Perte** (section 3) — critère issu du maximum de vraisemblance :

$$J = -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \log a^{(i)} + (1 - y^{(i)}) \log(1 - a^{(i)}) \right] \tag{496}$$

Descente de gradient (section 4) — mise à jour de base :

$$\theta \leftarrow \theta - \alpha \, \nabla_\theta J \tag{497}$$

Backpropagation (sections 4, 7) — la règle de la chaîne appliquée couche par couche donne un calcul des gradients en $O(m \cdot |\theta|)$. Ce même principe s'applique à la BPTT pour les RNN (section 15) et aux couches convolutives (section 14).

Adam (section 12) — momentum + taux adaptatifs :

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \tag{498}$$

$$\theta \leftarrow \theta - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}, \quad \hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t} \tag{499}$$

17.2.3 Pilier 3 : Généralisation

Sans précautions, un réseau profond sur-apprend. Trois familles de techniques y remédient.

Définition 17.3 (Initialisation, normalisation et régularisation)**. Initialisation He** (section 10) — préserver la variance :

$$W_{ij}^{[l]} \sim \mathcal{N}\left(0, \; \frac{2}{n^{[l-1]}}\right) \tag{500}$$

Normalisation (section 11) — stabiliser les distributions internes :

$$\hat{z} = \frac{z - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \varepsilon}}, \qquad y = \gamma \hat{z} + \beta \tag{501}$$

BatchNorm pour les CNN (section 14), LayerNorm pour les Transformers (section 16).

Régularisation (section 13) — pénaliser les poids excessifs et injecter du bruit :

$$J_{\text{reg}} = J + \frac{\lambda}{2} \sum_{l=1}^{L} \left\| W^{[l]} \right\|_F^2, \qquad \tilde{A}^{[l]} = \frac{1}{p} D^{[l]} \odot A^{[l]} \tag{502}$$

17.3 Algorithme complet d'entraînement moderne

L'algorithme ci-dessous intègre *toutes* les techniques vues dans ce cours. C'est la procédure de référence pour entraîner un réseau profond en pratique.

Algorithme complet : entraînement d'un réseau de neurones moderne

Phase 0 — Initialisation

1. Pour chaque couche $l = 1, \ldots, L$:
   — $W^{[l]} \sim \mathcal{N}\Big(0, \sqrt{2/n^{[l-1]}}\Big)$ (He, section 10)
   — $b^{[l]} = 0, \quad \gamma^{[l]} = 1, \quad \beta^{[l]} = 0$ (paramètres de normalisation)
2. Initialiser Adam : $m_0 = 0$, $v_0 = 0$, $t = 0$
3. $L_{\text{best}} = +\infty$, compteur de patience $= 0$

Phase 1 — Boucle d'entraînement
Pour chaque époque $= 1, \ldots, T_{\max}$:
Pour chaque mini-batch $(X_{\mathcal{B}}, Y_{\mathcal{B}})$:

1. Forward propagation (sections 7, 9, 11, 14, 16) :

$$Z^{[l]} = W^{[l]} \tilde{A}^{[l-1]} + b^{[l]} \qquad \text{pré-activation (dense ou conv.)} \tag{503}$$

$$\hat{Z}^{[l]} = \gamma^{[l]} \frac{Z^{[l]} - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \varepsilon}} + \beta^{[l]} \qquad \text{normalisation} \tag{504}$$

$$A^{[l]} = g^{[l]}(\hat{Z}^{[l]}) \qquad \text{activation (ReLU, GELU, etc.)} \tag{505}$$

$$\tilde{A}^{[l]} = \frac{1}{p} D^{[l]} \odot A^{[l]} \qquad \text{dropout (entraînement)} \tag{506}$$

2. Calcul de la perte (sections 3, 8, 13) :

$$J = \underbrace{-\frac{1}{m} \sum_{i=1}^{m} \sum_{k=1}^{K} y_k^{(i)} \log a_k^{(i)}}_{\text{cross-entropie}} + \underbrace{\frac{\lambda}{2} \sum_{l=1}^{L} \left\| W^{[l]} \right\|_F^2}_{\text{régularisation L2}} \tag{507}$$

3. Backward propagation (sections 4, 7) :

$$dZ^{[L]} = A^{[L]} - Y \tag{508}$$

$$dW^{[l]} = \frac{1}{m} dZ^{[l]} \left(A^{[l-1]}\right)^T + \lambda W^{[l]} \tag{509}$$

$$db^{[l]} = \frac{1}{m} \sum_i dZ^{[l](i)} \tag{510}$$

$$dA^{[l-1]} = \left(W^{[l]}\right)^T dZ^{[l]} \tag{511}$$

4. Mise à jour Adam (section 12) : $t \leftarrow t + 1$

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla J, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla J)^2 \tag{512}$$

$$\theta \leftarrow \theta - \alpha \frac{m_t / (1 - \beta_1^t)}{\sqrt{v_t / (1 - \beta_2^t)} + \varepsilon} \tag{513}$$

Phase 2 — Early stopping (section 13)

— Calculer L_{val} (sans dropout, avec statistiques mobiles de normalisation)
— Si $L_{\text{val}} < L_{\text{best}}$: sauvegarder le modèle, $L_{\text{best}} \leftarrow L_{\text{val}}$, patience $\leftarrow 0$
— Sinon : patience $\leftarrow$ patience $+1$
— Si patience $>$ seuil : **arrêter** et restaurer le meilleur modèle

Attention

Lors de l'inférence, deux changements critiques :

— Le **dropout est désactivé** : on utilise tous les neurones.

— La **normalisation utilise les statistiques mobiles** accumulées pendant l'entraînement.

Pour les Transformers (section 16), le masquage causal reste actif à l'inférence pour la génération auto-régressive.

17.4 Diagnostic pratique

Attention

[Guide de dépannage : mon réseau ne converge pas] **1. La perte stagne (pas d'apprentissage)**

- ✓ Learning rate trop petit → multiplier par 3–10
- ✓ Gradients nuls (vanishing) → vérifier l'initialisation (§10), passer à ReLU (§9)
- ✓ Labels mal encodés → vérifier la correspondance one-hot / indices
- ✓ Bug dans le forward → tester sur un seul batch (doit atteindre perte ≈ 0)

2. La perte explose (divergence)

- ✓ Learning rate trop grand → diviser par 10
- ✓ Initialisation trop forte → vérifier $\mathrm{Var}(Z^{[l]}) \approx 1$ (§10)
- ✓ Gradients explosifs → gradient clipping (§15)
- ✓ Données non normalisées → centrer-réduire les entrées

3. Sur-apprentissage (train $\ll$ val)

- ✓ Ajouter régularisation L2 ($\lambda \approx 10^{-3}$ à 10^{-4}) (§13)
- ✓ Ajouter dropout ($p = 0.5$ pour les couches denses) (§13)
- ✓ Augmenter la taille du dataset (data augmentation, section 13)
- ✓ Réduire la capacité du modèle (moins de couches ou de neurones)
- ✓ Activer l'early stopping avec patience $\approx$ 10–20 époques

4. Sous-apprentissage (train et val hauts)

- ✓ Augmenter la capacité (plus de couches, plus de neurones)
- ✓ Réduire la régularisation (λ, dropout)
- ✓ Envisager une architecture adaptée au type de données (CNN pour images, Transformer pour séquences)
- ✓ Augmenter le nombre d'époques ou le learning rate

17.5 Récapitulatif des formules clés

Table 3 : Formules essentielles du cours, avec renvoi aux sections.

Concept	**Formule**	**Section**
Sigmoïde	$\sigma(z) = \dfrac{1}{1+e^{-z}}$	2
Softmax	$a_k = \dfrac{e^{z_k}}{\sum_j e^{z_j}}$	8
Cross-entropie (binaire)	$J = -\dfrac{1}{m}\sum_i \left[y^{(i)}\log a^{(i)} + (1-y^{(i)})\log(1-a^{(i)})\right]$	3
Cross-entropie (multi-classes)	$J = -\dfrac{1}{m}\sum_i \sum_k y_k^{(i)} \log a_k^{(i)}$	8
Gradient de sortie	$dZ^{[L]} = A^{[L]} - Y$	4, 7
Gradient des poids	$dW^{[l]} = \dfrac{1}{m}\, dZ^{[l]}(A^{[l-1]})^T$	4, 7
Initialisation He	$W_{ij}^{[l]} \sim \mathcal{N}\left(0, \dfrac{2}{n^{[l-1]}}\right)$	10
Batch Normalization	$\hat{z} = \dfrac{z-\mu}{\sqrt{\sigma^2+\varepsilon}}, \quad y = \gamma\hat{z} + \beta$	11
Adam	$\theta \leftarrow \theta - \alpha\, \dfrac{\hat{m}_t}{\sqrt{\hat{v}_t}+\varepsilon}$	12
Régularisation L2	$J_{\text{reg}} = J + \dfrac{\lambda}{2}\lVert W\rVert_F^2$	13
Dropout	$\tilde{A} = \dfrac{1}{p}\, D \odot A, \quad D_j \sim \text{Bernoulli}(p)$	13
Convolution 2D	$Z_c(i,j) = \sum_{c',\delta} W_{c,c'}(\delta)\, A_{c'}(i\cdot s+\delta_i, j\cdot s+\delta_j) + b_c$	14
LSTM (cellule)	$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$	15
Attention	$\text{Att}(Q,K,V) = \text{softmax}\left(\dfrac{QK^T}{\sqrt{d_k}}\right) V$	16

Conclusion

Nous sommes partis d'un seul neurone traçant une droite entre deux nuages de points, et nous avons abouti aux Transformers qui alimentent les modèles de langage les plus avancés. Chaque étape a été motivée par un problème concret et résolue par un outil mathématique précis.

Ce qu'il faut retenir

Le deep learning n'est pas une boîte noire. C'est une discipline technique fondée sur trois principes clairs :

— La **modélisation probabiliste** donne un sens précis à la perte et aux prédictions.

— L'**optimisation par gradient** fournit un algorithme efficace pour ajuster les paramètres.

— Le **contrôle de la complexité** garantit que les performances se transfèrent aux données nouvelles.

Les architectures varient — couches denses, convolutions, récurrence, attention — mais ces trois piliers restent invariants. Les fondements mathématiques de ce cours vous permettront non seulement d'utiliser les architectures existantes, mais surtout de les *comprendre*, de les *diagnostiquer*, et de les *améliorer*.